
CISCO CERTIFIED NETWORK ASSOCIATE

CCNA 200-301 VERSION 2.0

Notes, Configurations and Labs

WRITTEN FROM ZERO

No prior networking assumed. Protocol theory and device configuration taught together, because the exam tests both and mostly tests the second.

 25% Infrastructure and Connectivity	 25% Switching and Network Access	 20% IP Routing	 20% Services and Security	 10% AI, Operations and Management
--	---	--	--	--

Fifty-five per cent of this exam is hands-on. Of the twenty-nine topics Cisco publishes, eight begin *Troubleshoot* or *Diagnose* and eight begin *Configure*. Only six begin *Describe*. Under the previous blueprint, v1.1, the word “troubleshoot” appeared in no topic statement at all. You cannot pass this exam by reading.

Aligned to blueprint version 2.0, live from 3 February 2027 • August 27, 2026

The Exam, in One Page

Exam	Implementing and Administering Cisco Solutions, 200-301 CCNA, blueprint version 2.0
Length	120 minutes
Live from	3 February 2027. Version 1.1 retires on 2 February 2027; the exam number does not change, so check which blueprint a book or course is written against before you trust it
Domains	Five: Network Infrastructure and Connectivity (25%), Switching and Network Access (25%), IP Routing (20%), Network Services and Security (20%), AI and Network Operations and Management (10%)
Topics	Twenty-nine, numbered 1.1 to 5.6. Every chapter of this volume is tagged with the ones it discharges, and Appendix D proves all twenty-nine are covered
The shape of it	Eight topics ask you to <i>troubleshoot</i> or <i>diagnose</i> ; eight ask you to <i>configure</i> ; six ask you to <i>describe</i> ; the rest ask you to interpret, validate, manage, select or use. Roughly a third of the exam is fixing something broken at a command line, under time pressure
Agentic AI	Cisco’s own exam description warns that you “may be required to evaluate output and recommendations from agentic AI and digital network assistants” — and not only in domain 5. See Chapter 32

What changed from version 1.1, and why it matters to you

If you have an older book, it is teaching a different exam. QoS, NTP, TFTP and FTP, the OSI and TCP/IP model descriptions, TCP versus UDP comparison, device-role and topology-architecture theory, WLC GUI configuration, REST APIs, JSON and Terraform have all gone. OSPFv3, hypervisors and containers, SFTP and SCP, DNS record diagnosis, storm control, RA guard, PoE configuration, DHCP server configuration and switch virtual interfaces have arrived — along with a whole domain on AI in network operations. Above all, troubleshooting went from nothing to about a third of the paper.

How to Use This Volume

This book assumes you have never configured a network device and have not studied networking before. It teaches the theory and the configuration together, in the order they can actually be learned, and every chapter states which exam topics it discharges.

The furniture in every chapter

Each chapter is built from the same parts, always in the same order, so you always know where you are.

Element	What it is for
EXAM TOPICS 2.1	The blueprint topics this chapter discharges, colour-coded by domain. If you are revising against the blueprint, navigate by these
🎯 Learning Outcomes	What you will be able to <i>do</i> after the chapter, written as actions
🔗 Before You Start	The specific earlier chapters this one leans on
🔑 Key Terms	Vocabulary introduced here. All terms also appear in Appendix F
📖 Definition	A precise statement of a term
💡 Concept	The intuition behind it — why it works this way
📊 Worked Example	A calculation done in full, one step at a time. Cover the answer and try it first; these are the places the exam expects speed
➤_ Configure	The actual commands, in the order you type them. Read these with a device or simulator open, not in an armchair
🔍 Verification	The <code>show</code> command that proves the configuration worked, and the output you should see. Learning to read this output is learning to troubleshoot
➤_ Host command	The same, but run on a Windows, macOS or Linux machine rather than on IOS. The operating system is named in the title, because <code>ipconfig</code> and <code>ip addr</code> are not interchangeable
📄 Playbook	An Ansible YAML file, named by its filename. Chapters 31 onward
⚠ Common Pitfalls	Mistakes made every cohort
🔧 Break & Fix	A broken device to diagnose. This is the 28% of the exam that is troubleshooting, practised in every chapter rather than revised once at the end
🧪 Lab	A hands-on exercise with a topology, an addressing table, numbered tasks and a verification step. Labs are numbered and indexed in Appendix G
🎓 On the exam	How this material is actually tested, and the traps in the question wording
❓ Checkpoint	Quick self-tests. Answers in Appendix E
📋 Chapter Summary	The chapter compressed to what must be retained

Element	What it is for
 Review Questions	Exam-style questions for practice

How to study with it

1. **Type every configuration.** Not copy and paste — type it. The exam asks you to produce commands from memory under time pressure, and muscle memory is the only thing that survives the pressure.
2. **Never skip the verification block.** Half of troubleshooting is knowing what healthy output looks like, and you only learn that by looking at it repeatedly when nothing is wrong.
3. **Do the Break & Fix before reading the answer.** Sit with the symptom. The exam will not hand you a diagnosis.
4. **Build the labs, do not read them.** A lab you have read is worth nothing on an exam that asks you to configure.
5. **Use Appendix D to revise.** It maps every one of the twenty-nine exam topics to the chapter that covers it, so you can check your own coverage rather than hoping.

What you need

- **Parts I to III: Cisco Packet Tracer.** Free with a Cisco Networking Academy account, runs on a modest laptop, and covers everything in the first three parts — VLANs, trunking, EtherChannel, spanning tree, static routing and OSPF — perfectly well.
- **Parts IV onwards: CML or GNS3.** From Part IV the exam names real platform behaviour — topic [4.3](#) says *IOS XE* explicitly, and SFTP, SCP and Ansible need a real control plane. Packet Tracer approximates these; it does not reproduce them.
- **Somewhere to write.** A test centre supplies an erasable whiteboard, not paper, and you may not bring your own — so practise subnetting under the same constraint, and without a calculator.

About the configurations in this book

Every configuration here is written to be syntactically correct for Cisco IOS and IOS XE, and every one follows the command order a device actually requires. They have **not** been verified on live hardware. Platform, image and version differences are real: a command that works on one switch may be unavailable or spelled differently on another. Before issuing a lab to students, run it once yourself on whatever platform your department has. That advice is worth more than a claim of validation this book cannot honestly make.

Syllabus

Course description

A first course in computer networking, taught to certification standard. It develops the ability to design, configure, verify and troubleshoot a small enterprise network — switching, routing, addressing, services and security — and prepares the student for the Cisco CCNA 200-301 v2.0 examination. No prior networking study is assumed. The course is built around laboratory work, because the certification it targets is more than half practical.

Course learning outcomes

On completion, a student will be able to:

CLO	Outcome	Exam domain
CLO-1	Explain how data moves across a switched and routed network, and use addressing, subnetting and the transport layer correctly. (C2, C3)	1.3 1.4
CLO-2	Configure switching for a small enterprise: VLANs, trunks, EtherChannel, spanning tree and edge-host connectivity. (C3)	2.1 2.2 2.5
CLO-3	Configure and interpret IP routing — static routes, single-area OSPF for IPv4 and IPv6, and first hop redundancy. (C3, C4)	3.1 3.2 3.3 3.4
CLO-4	Configure network services and security: DHCP, NAT, access control lists, device access control and layer 2 protection. (C3)	4.1 4.3 4.6 4.7
CLO-5	Diagnose and repair a broken network from symptoms, using <code>show</code> output, ping, traceroute and packet capture. (C4, C5)	2.4 1.6 1.7
CLO-6	Describe modern network operations — management approaches, SNMP, syslog, automation with Ansible, and the role of agentic AI — and select an effective prompt for an operations task. (C2, C3)	5.1 5.2 5.3 5.4 5.5 5.6
CLO-7	Work safely and professionally on shared infrastructure, documenting changes and respecting the blast radius of a configuration. (A3)	—

CLO-5 carries the heaviest assessment weight deliberately: it is the outcome the exam weights most heavily and the one students are least prepared for.

Assessment

Component	Weight	Notes
Laboratory portfolio	30%	One submission per lab: topology, running configuration, and the <code>show</code> output proving it works

Component	Weight	Notes
Break & Fix assessments	25%	Timed. A pre-broken topology; find and fix the fault, and write down what it was. Mirrors the exam's heaviest verb
Configuration midterm	15%	At a device or simulator, not on paper
Theory examination	20%	Written, covering the <i>describe</i> and <i>interpret</i> topics
Professional practice	10%	Documentation quality, change discipline, teamwork in the lab

A department running this course inside a degree can map the laboratory portfolio and the Break & Fix assessments onto a practical component, and the remaining 35% onto the written component, without changing the weights.

Contents

The Exam, in One Page	ii
How to Use This Volume	iii
Syllabus	v
List of Figures	xv
I Foundations	1
1 Networks, Models and This Exam	3
1.1 What a network has to solve	3
1.2 Layers, and why they survive	4
1.3 The devices, and what each one decides	5
1.4 This exam, and this book	6
2 The Physical Layer and Cable Faults	8
2.1 Choosing the medium	8
2.2 The counters that name the fault	10
3 Ethernet, Frames and Switching	13
3.1 Reading hexadecimal first	13
3.2 The frame	14
3.3 How a switch actually works	14
4 IPv4 Addressing and Subnetting	19
4.1 Counting in binary, once	19
4.2 The address and the mask	20
4.3 The method: block size, or “magic number”	22
4.4 Subnetting a block to a requirement	23
4.5 Configuring and verifying an address	23
5 IPv6 Addressing	26
5.1 Writing the address	26
5.2 The types you must recognise	27
5.3 Modified EUI-64	27
5.4 Configuring IPv6	28
6 TCP, UDP and the Diagnostic Toolkit	31
6.1 Ports, and what they identify	31
6.2 Ping, and how to ping deliberately	32
6.3 Traceroute	33
6.4 Reading a capture	33

7 The IOS Command Line	36
7.1 Modes, and the prompt that tells you where you are	36
7.2 Two configurations, and only one survives a reboot	37
7.3 Getting help, and typing less	37
7.4 A first-time setup, end to end	38
 II Switching and Network Access	 41
8 VLANs and Access Ports	43
8.1 What a VLAN actually does	43
8.2 Creating VLANs and assigning ports	44
8.3 Verifying, three ways	45
9 Edge-Host Connectivity: PoE, Voice, APs and IoT	48
9.1 Power over Ethernet	48
9.2 Voice VLANs: two VLANs, one cable	49
9.3 Access points, standalone and controller-based	50
9.4 Printers, IoT appliances and virtualised hosts	51
10 Trunking, 802.1Q and SVIs	54
10.1 The tag, and the one VLAN that has none	54
10.2 DTP, and why to turn it off	55
10.3 Routing between VLANs	56
11 EtherChannel and LACP	60
11.1 Why bundling needs a protocol	60
11.2 The negotiation protocols	61
11.3 Configuring a layer 2 EtherChannel	61
11.4 A layer 3 EtherChannel	62
12 Rapid PVST+ and the Protection Features	65
12.1 Why a layer 2 loop is fatal	65
12.2 How the tree is chosen	66
12.3 Configuring Rapid PVST+	67
12.4 The protection features	68
13 CDP, LLDP and Documenting a Network	72
13.1 Two protocols, one job	72
13.2 Reading the output	73
13.3 Validating a diagram	74
14 Troubleshooting Layer 2	77
14.1 Scope first, then layers	77
14.2 The first three commands	78
14.3 The six faults, and what each looks like	78

III	IP Routing	82
15	The Routing Table and How a Router Decides	84
15.1	What a router is actually doing	84
15.2	Reading the table	85
15.3	Rule one: longest prefix match	86
15.4	Rule two: administrative distance	86
15.5	Three ways a packet dies	86
16	Static Routing, IPv4 and IPv6	90
16.1	The four kinds	90
16.2	Next hop, exit interface, or both	91
16.3	Static routing for IPv6	91
16.4	Floating statics, properly	92
17	OSPFv2, Single Area	95
17.1	What OSPF does that a static route cannot	95
17.2	Adjacencies, and the five things that must match	96
17.3	Configuring single-area OSPFv2	96
17.4	DR and BDR on broadcast links	97
17.5	Verifying	98
18	OSPFv3 for IPv6	101
18.1	What stays the same	101
18.2	What changes	101
18.3	Configuring it	102
18.4	Verifying	103
19	First Hop Redundancy: HSRP and VRRP	106
19.1	The problem	106
19.2	Configuring HSRP	107
19.3	Interpreting the status	108
IV	Network Services and Security	111
20	DHCP: Client, Server and Relay	113
20.1	What DHCP is solving	113
20.2	Role 1: IOS as a client	114
20.3	Role 2: IOS as a server	115
20.4	Role 3: IOS as a relay agent	116
20.5	Troubleshooting: three symptoms, three places to look	117

21 DNS, and Diagnosing Records	121
21.1 How a name becomes an address	121
21.2 The six record types	122
21.3 Querying: the two tools	123
21.4 DNS on IOS itself	124
21.5 Diagnosing the three named failures	124
22 NAT and PAT on IOS XE	128
22.1 The problem NAT solves, and the one it does not	128
22.2 The four terms	129
22.3 Three kinds of NAT	129
22.4 Verifying	131
22.5 Order of operations, and why it catches people	131
23 Device Access, AAA and Secure File Transfer	135
23.1 Local access done properly	135
23.2 AAA: three separate questions	136
23.3 TACACS+ against RADIUS	138
23.4 Moving files securely	138
24 Access Control Lists	143
24.1 What an ACL does	143
24.2 Wildcard masks	144
24.3 Standard ACLs	144
24.4 Extended ACLs	145
24.5 Named ACLs, and why they are what you should write	146
24.6 Applying and verifying	146
25 Layer 2 Security	151
25.1 Port security — limiting who may appear on a port	152
25.2 DHCP snooping — stopping rogue servers	153
25.3 Dynamic ARP inspection — stopping ARP lies	154
25.4 Storm control — limiting flooding	155
25.5 RA guard — the IPv6 equivalent	156
26 IPsec VPNs	160
26.1 The problem	160
26.2 Two deployment models	161
26.3 The protocols	162
26.4 Transport mode and tunnel mode	162
26.5 IPsec and NAT	164
26.6 When things do not work	164

V	Wireless, Virtualisation and Operations	168
27	Wireless Principles	170
27.1	Why wireless is different	170
27.2	Bands and channels	171
27.3	RF characteristics	172
27.4	Security protocols	173
27.5	Causes of interference	174
28	Troubleshooting Client Connectivity	179
28.1	Start at the host	179
28.2	The same job on three operating systems	180
28.3	Step 1: reading the IP configuration	181
28.4	Steps 2 to 4: the reachability ladder	182
28.5	Wireless security parameters	182
29	Hypervisors, Virtual Machines and Containers	188
29.1	Why any of this exists	188
29.2	What a virtual machine is	189
29.3	Containers	190
29.4	What this does to your network	191
30	Management Approaches, SNMP and Syslog	196
30.1	Five ways to manage a network	196
30.2	SNMP	197
30.3	Syslog, and how to read a message	199
31	Ansible and Configuration Management	205
31.1	The problem	205
31.2	The pieces	206
31.3	Inventory: which devices	207
31.4	Executing commands	207
32	AI in Network Operations	213
32.1	Generative and agentic	213
32.2	Where it is actually used	214
32.3	Selecting a prompt	215
VI	Sitting the Exam	221
33	A Troubleshooting Method for the 28 Per Cent	223
33.1	Why method beats knowledge under pressure	223
33.2	The cycle	224
33.3	Five structured methods	224
33.4	Scope before you touch	225

33.5 Evidence discipline	225
33.6 Symptom index	226
33.7 Handing it over	226
34 Exam Strategy	230
34.1 The constraint that shapes everything	230
34.2 Question types	231
34.3 Budgeting 120 minutes	231
34.4 Reading the question	232
34.5 The last month	233
34.6 On the day	233
Appendices	237
A Command Reference	238
A.1 Getting on the box (Chapter 7, Chapter 23)	238
A.2 Interfaces (Chapter 2, Chapter 10)	238
A.3 VLANs, trunks and edge ports (Chapter 8–Chapter 10)	239
A.4 EtherChannel and spanning tree (Chapter 11, Chapter 12)	239
A.5 Routing (Chapter 16–Chapter 19)	240
A.6 Services (Chapter 20, Chapter 22)	240
A.7 Security (Chapter 24, Chapter 25, Chapter 23)	241
A.8 Management and files (Chapter 30, Chapter 23)	242
B The show Command Cookbook	243
B.1 Is the interface healthy? (Chapter 2)	243
B.2 Is layer 2 right? (Chapter 8–Chapter 12)	243
B.3 What is attached? (Chapter 13)	244
B.4 Where does traffic go? (Chapter 15–Chapter 19)	244
B.5 Are the services working? (Chapter 20, Chapter 22, Chapter 24)	245
B.6 What is the security state? (Chapter 25)	245
B.7 What happened? (Chapter 30)	246
B.8 The ten to know cold	246
C Subnetting and IPv6 Drills	248
C.1 The method	248
C.2 The two tables to memorise	248
C.3 Drill 1 — network, broadcast and range	249
C.4 Drill 2 — sizing a subnet	250
C.5 Drill 3 — IPv6 compression	250
C.6 Drill 4 — EUI-64	251
C.7 Answers	251
D Blueprint Coverage Matrix	253

E Checkpoint Answers	255
Part I — Foundations	255
Part II — Switching and Network Access	256
Part III — IP Routing	258
Part IV — Network Services and Security	259
Part V — Wireless, Virtualisation and Operations	262
Part VI — Sitting the Exam	264
F Glossary	266
G Schedule, Lab Index and Exam Facts	271
G.1 A sixteen-week schedule	271
G.2 Lab index	272
G.3 Exam facts, on one page	273

List of Figures

P1.1	Part I in order. Each chapter is a prerequisite for the next, and the last two are the ones everything in Parts II to V depends on.	2
1.1	A small enterprise network. Access switches connect end devices; a distribution switch aggregates them and routes between VLANs; the router reaches other networks; the firewall polices what leaves. Every chapter in Parts II to IV configures one piece of this picture.	5
3.1	The whole switching algorithm. Flooding is not a failure — it is how the table gets built, and the second frame between the same two hosts is already efficient.	15
P2.1	Part II. Each chapter adds one capability to the same switched network, and the last two are about proving it works and finding out why it does not.	42
8.1	The whole idea, in one picture. Four ports on one switch become two broadcast domains that cannot reach each other — and nothing physical changed.	44
9.1	One cable, two VLANs, and no trunk. The phone tags its own traffic and passes the PC's through untouched.	49
10.1	A trunk is one cable carrying many VLANs. Every VLAN but one is labelled; the unlabelled one is the native VLAN, and a disagreement about it joins two networks silently.	54
11.1	Bundling is not about speed for one conversation — it is about spanning tree seeing one link instead of four, and therefore blocking none of them.	60
12.1	Spanning tree does not remove the cable. It blocks one port, so the physical redundancy is still there and can be brought back automatically when a link fails.	66
P3.1	Part III. Static routing and OSPF are two answers to the same question; first Chapter 15 establishes how the router picks between whatever answers it has.	83
17.1	Link state in one picture. The database is shared and identical; the routing table each router builds from it is its own.	96
19.1	The host is configured once and never reconfigured. Failover moves the virtual IP <i>and</i> the virtual MAC, which is why nothing on the LAN has to re-learn anything.	107
P4.1	Part IV. Three chapters make the network usable; four decide who is allowed to use it. Chapter 20 belongs to domain 1 and the rest to domain 4.	112
20.1	DORA. Three of the four messages are broadcasts, which is why a router between client and server stops the process dead unless it is configured to relay.	114
21.1	Only step 1 is recursive. The host asks one question and waits; the resolver does the walking and caches what it learns for the TTL.	122
22.1	The same two hosts, four names. <i>Inside</i> and <i>outside</i> name the host; <i>local</i> and <i>global</i> name the viewpoint.	129

24.1	Why placement is part of the design. The same ACL is correct in one place and an outage in the other.	145
25.1	Five features, five abuses, one port. The uplink is trusted; everything facing a user is not.	151
26.1	Where the tunnel starts decides the model. Site-to-site protects gateway to gateway; remote access protects all the way to the user's device.	161
26.2	Transport mode keeps the original IP header. Tunnel mode encrypts it and adds a new one, which is what lets private addresses cross a public network.	164
P5.1	Part V. The top row completes domain 1; the bottom row is the whole of domain 5, and every topic in it is new to v2.0.	169
27.1	The 2.4 GHz band. Channels 1, 6 and 11 are the only trio that does not overlap; anything between them interferes with two networks instead of sharing one.	171
28.1	The reachability ladder. Each rung that passes eliminates everything below it; the first rung that fails names the area to investigate.	180
29.1	The difference is one layer. A type 1 hypervisor <i>is</i> the operating system; a type 2 hypervisor is an application running on one.	189
29.2	VM 1 talking to VM 2 never reaches SW1. Everything else does, over a trunk — a single physical port carrying three VLANs.	190
29.3	One layer removed. Containers share the host's kernel, which is why they are small and fast — and why the isolation between them is weaker.	190
30.1	SNMP works in both directions: the manager polls on port 161, and agents report events unprompted to port 162.	198
30.2	Every IOS syslog message has the same four parts after the timestamp. The severity is the single digit between the two hyphens.	199
31.1	Everything lives on the control node. The devices need only the SSH access they already have.	206
32.1	The difference is the loop and who closes it. A generative system hands you an answer; an agentic one acts and checks its own work.	214
P6.1	The twenty-nine topics by their opening verb. Only a fifth of the blueprint asks you to recall anything; the rest asks you to do something or to read something and decide.	222
33.1	The loop is between 5 and 6, and it is the point. A hypothesis that does not predict correctly is discarded, not patched.	224

PART I

Foundations

Before you can configure a network you have to know what is moving through it, how it is addressed, and how to talk to a device at all. Part I builds that foundation from nothing.

It is also where two of the exam's heaviest habits are formed. The first is **subnetting on paper**, quickly and without a calculator — topic [1.3](#) asks you to troubleshoot addressing, and you cannot troubleshoot what you cannot compute. The second is **reading device output**, which is what topic [2.4](#) tests and what every troubleshooting question in the paper depends on.

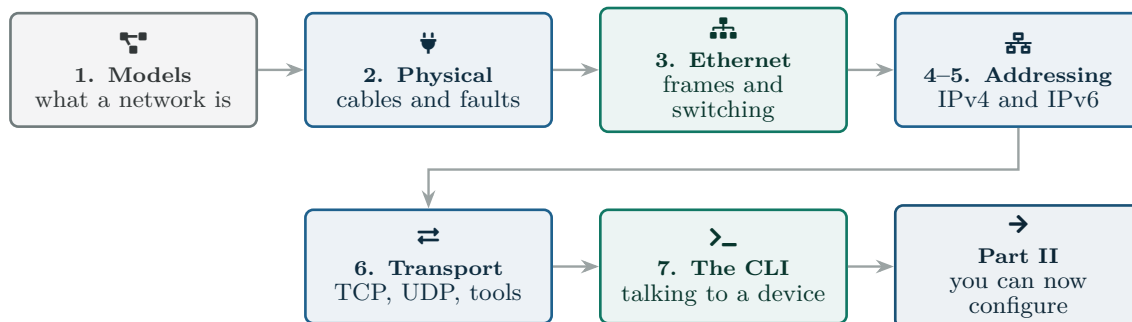


Figure P1.1: Part I in order. Each chapter is a prerequisite for the next, and the last two are the ones everything in Parts II to V depends on.

Networks, Models and This Exam

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Describe** what a network does, in terms of the four things every network must solve: naming, addressing, forwarding and error handling.
- **Explain** the layered model well enough to use it as a troubleshooting tool, which is the only reason this exam still cares about it.
- **Identify** the devices in a small enterprise network and say what each one decides.
- **Navigate** the CCNA v2.0 blueprint and this book against it.

🔗 Before You Start

Nothing. This is the first chapter, and the course assumes no prior networking.

📖 Key Terms in This Chapter

host • node • link • frame • packet • segment • encapsulation • layer • protocol • switch • router • access layer • distribution layer • broadcast domain • collision domain

1.1 What a network has to solve

Two computers in the same room, joined by a cable, can exchange bytes. Everything else in networking exists because that arrangement does not scale. Add a third machine and you need a way to say *which* machine you mean. Add a building and you need a way to find a machine you cannot see. Add the internet and you need a way to reach a machine whose owner you have never met, over equipment owned by people you will never meet either.

💡 The four problems, and the four answers

Every network technology in this book is an answer to one of four questions.

- **Who am I talking to?** → addressing. MAC addresses locally (Chapter 3), IP addresses globally (Chapter 4).
- **How do I find them?** → forwarding. Switches learn (Chapter 3); routers decide (Chapter 15).
- **How do we not talk over each other?** → media access and loop prevention (Chapter 3, Chapter 12).
- **What if it goes wrong?** → error detection, acknowledgement and retransmission (Chapter 6).

When you meet a new protocol, ask which of the four it is answering. It will be one of them.

📄 Protocol

An agreed set of rules for a conversation: what may be said, in what order, in what format, and what to do when the expected reply does not arrive.

Two people who both know to say “hello”, wait, then speak, have a protocol. Two computers need theirs written down to the bit, because neither can improvise. Almost every capitalised

acronym in this book — ARP, DHCP, OSPF, HSRP — is one such agreement, and each exists because somebody had one of the four problems above.

1.2 Layers, and why they survive

A network stack is split into layers so that each layer can be changed without rewriting the others. Wi-Fi replaced the cable without anyone rewriting the web. That is the engineering argument, and it is real, but it is not why the layered model matters to you this year.

💡 The model is a troubleshooting tool

When something does not work, the layered model tells you *what to test next*. If a cable is unplugged, checking DNS is a waste of a minute you do not have in the exam. Work up: is the link up? Does it have an address? Can it reach its gateway? Can it resolve a name? Every troubleshooting question in this exam — and there are eight topics' worth — is answered by walking the layers in order.

The v2.0 blueprint no longer asks you to *describe* the OSI model, as v1.1 did. It assumes you use it.

#	Layer	What it decides	What breaks, and the symptom
7–5	Application	What the data means	Wrong URL, expired certificate; “the site is down but ping works”
4	Transport	Which program on the host, and whether loss is repaired	Blocked port; “it pings but the application times out”
3	Network	Which network, and the path between networks	Wrong IP, wrong mask, no route; “I can reach my own subnet but nothing else”
2	Data link	Which device on this link	Wrong VLAN, trunk down, spanning tree blocking; “the port is up but nothing passes”
1	Physical	Bits on the wire	Cable, transceiver, duplex or speed mismatch; “the port is down”, or worse, up with errors climbing

The bottom two rows are where this exam starts: topic [1.1](#) is entirely physical-layer diagnosis, and it is examined in Chapter 2.

📦 Encapsulation, and the names of things

As data moves down the stack each layer wraps it in a header. The wrapped unit gets a different name at each layer, and using the right one is how you show you know which layer you are discussing.

- Application data becomes a **segment** at layer 4 (TCP) or a **datagram** (UDP), when a port number is attached.
- That becomes a **packet** at layer 3, when IP addresses are attached.
- That becomes a **frame** at layer 2, when MAC addresses are attached.

- That becomes **bits** at layer 1.

Switches forward frames. Routers forward packets. That single sentence explains most of the difference between the two devices.

1.3 The devices, and what each one decides

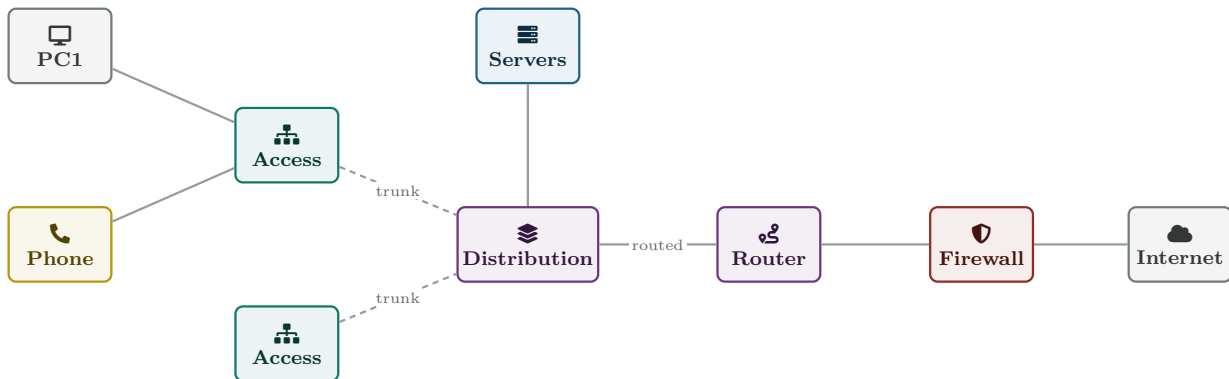


Figure 1.1: A small enterprise network. Access switches connect end devices; a distribution switch aggregates them and routes between VLANs; the router reaches other networks; the firewall polices what leaves. Every chapter in Parts II to IV configures one piece of this picture.

Device	The decision it makes	Where in this book
Switch (layer 2)	“Which port leads to this MAC address?” Learns by watching source addresses; floods when it does not know	Chapter 3, then all of Part II
Router	“Which interface leads to this IP network?” Never floods; drops what it cannot route	Part III
Multilayer switch	Both. Switches within a VLAN, routes between them using an SVI	Chapter 10, Chapter 15
Access point	Bridges wireless clients onto a wired VLAN	Chapter 27
Firewall	“Is this flow permitted?” Stateful, unlike an access list	Chapter 24, Chapter 26

Two domains worth keeping straight

A **collision domain** is the set of ports that could transmit at the same instant and interfere. Every switch port is its own collision domain, which is why modern switched networks do not collide — and why a collisions counter that is climbing means something is badly wrong, usually a duplex mismatch (Chapter 2).

A **broadcast domain** is the set of devices that receive each other’s broadcasts. One VLAN is one broadcast domain. Splitting broadcast domains is the entire purpose of Chapter 8, and joining them again is the purpose of Chapter 15.

1.4 This exam, and this book

How the blueprint is built, and how to read it

Cisco publishes twenty-nine numbered topics in five domains. The number is the whole contract: if a technique is not in a topic statement, it is not directly examinable, and if it is, the *verb* tells you the depth expected.

- *Describe* — you must be able to explain it. Six topics.
- *Interpret, Validate, Select, Use, Manage* — you must be able to read output or make a choice. Seven topics.
- *Configure* — you must produce the commands. Eight topics.
- *Troubleshoot* or *Diagnose* — you must find and fix a fault. Eight topics.

Sixteen of twenty-nine are configure-or-fix. That is the single most useful fact about this exam, and it should change how you study: time at a command line beats time with a highlighter, by a wide margin.

Every chapter of this book carries its topic numbers under the title. If you want to revise by blueprint rather than by chapter, Appendix D maps all twenty-nine topics to the chapters that cover them.

Common Pitfalls

- **Studying a v1.1 book.** The exam number, 200-301, did not change when the blueprint did. A book that teaches QoS, NTP, REST APIs, JSON or Terraform in a CCNA context is written for the old blueprint. Check the version before you trust the contents.
- **Learning the OSI layers as a list to recite.** Nobody will ask you to name layer 6. They will show you a broken network and expect you to work upwards.
- **Assuming “switch” means layer 2.** Multilayer switches route. Most of the distribution layer in a real network is switches doing routing, and topic [2.1](#) explicitly includes switch virtual interfaces.

Break & Fix — Three reports, three different layers, and no commands yet.

You have not configured anything and cannot yet read a `show` command. That is deliberate: this first Break & Fix is about *placing* a fault, not fixing one, because deciding which layer to investigate is the decision that all the later chapters depend on.

Three reports arrive within an hour.

- A. “The whole of the second floor is offline.” Nobody there can reach anything, including each other.
- B. “I can get to the file server but not the intranet site.” This user’s colleague, at the next desk, has no trouble with either.
- C. “Everything is slow this afternoon.” It was fine this morning. Several people on different floors agree.

Place each one before reading on. Which layer would you start at, and what single question would you ask first?

A — start at layers 1 and 2. A whole floor failing together, including traffic *between* hosts on that floor, means the fault is common to all of them and below the network layer — an uplink, a switch, or power. Nothing about this is an addressing or routing problem, because hosts on the same segment do not need routing to reach each other. First question: does anything on that floor reach anything else on that floor?

B — start at layer 7, then 3. One user, one service, while a neighbour on the same segment is fine. That rules out the cable, the switch and the VLAN immediately — they are shared, and they are working. Suspect name resolution or the client's own configuration. First question: does the intranet site work for them *by IP address*? That one test splits “name resolution” from “everything else” (Chapter 21).

C — do not start anywhere yet. “Slow” is not a fault report, it is a feeling, and it is the one report that most often wastes an afternoon. It could be a duplex mismatch (layer 1), a loop (layer 2), a saturated uplink, or a slow server that has nothing to do with the network. First question: slow doing *what*, from *where*, compared with *when*? Get one measurable, repeatable case before touching anything.

The point. Reports A and B contain enough information to place the fault within a layer or two before you log in to anything. Report C does not, and the correct first action is to go and get that information rather than to start guessing. Chapter 33 builds this into a method; for now, notice that the useful question was different in all three cases, and that in none of them was it “what was broken last time”.

Checkpoint — Test Yourself

1. A user reports that a web page will not load. Ping to the server's IP address succeeds. Which layers have you just ruled out, and which is the first one you should now suspect?
2. A switch port shows as up, but the collisions counter is increasing steadily. Which layer is at fault, and what is the most likely single cause?
3. Why does a router drop a packet for an unknown network when a switch floods a frame for an unknown MAC address?

Chapter Summary

- Networks solve four problems: addressing, forwarding, media access and error handling. Every protocol you meet answers one of them.
- Layers matter because they tell you what to test next. Work upwards from the physical layer; do not start at DNS.
- Switches forward frames using MAC addresses and flood when they do not know. Routers forward packets using IP addresses and drop when they do not know.
- One VLAN is one broadcast domain. Every switch port is its own collision domain.
- CCNA v2.0 has twenty-nine topics. Sixteen of them ask you to configure or to fix. Study accordingly.

Review Questions

1. (MCQ) Which unit of data carries MAC addresses? (a) segment (b) packet (c) frame (d) datagram
2. (MCQ) A device receives a frame for a destination MAC address it has never seen. What does it do? (a) drops it (b) floods it out all ports in the VLAN except the one it arrived on (c) returns it to the sender (d) routes it
3. (Short) Give one reason the v2.0 blueprint dropped the requirement to describe the OSI model, and say what it expects instead.
4. (Short) Explain the difference between a collision domain and a broadcast domain, and give the device that bounds each.
5. (Applied) A colleague says “we do not need to learn troubleshooting, it is only one section of the exam”. Using the verb counts in the blueprint, explain why that is wrong.

The Physical Layer and Cable Faults

EXAM TOPICS 1.1

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Select** the right medium — copper, multimode fibre, single-mode fibre — for a given distance and speed.
- **Read** the interface counters that expose a physical fault, and say which counter implicates which cause.
- **Diagnose** the classic faults: duplex mismatch, speed mismatch, wrong cable type, bad pin-out, dirty or over-long fibre, and a transceiver operating outside its optical budget.
- **Distinguish** a port that is down from a port that is up and damaging traffic — the second is far more dangerous.

🔗 Before You Start

Chapter 1 for the layered model and the idea of a collision domain.

📖 Key Terms in This Chapter

twisted pair • Cat5e/Cat6 • multimode • single-mode • SFP • attenuation • auto-negotiation • duplex mismatch • CRC error • runt • giant • late collision • input error • pin-out • straight-through • crossover • Auto-MDIX

🎓 Read the topic wording

Topic 1.1 says *diagnose* interface and cable issues, and then names them: “collisions, errors, mismatched duplex, speed, distance, interface, signal levels, pin out, and cable types”. That list is a syllabus in itself. You will not be asked to describe a cable; you will be shown counters and asked what is wrong.

2.1 Choosing the medium

📄 Bits per second, and why it is not bytes

Link speeds are always quoted in **bits** per second: 1 Gbps is one thousand million *bits* each second. File sizes are quoted in **bytes**, and a byte is eight bits.

So a 1 Gbps link moves at most about 125 megabytes per second, not 1000 — divide by eight. The lower-case **b** means bits and the upper-case **B** means bytes, which is the entire difference between Mbps and MBps.

Two further distinctions worth having straight from the start:

- **Bandwidth** is the rated capacity of the link — what it could carry.
- **Throughput** is what you actually got, after overhead, contention, errors and whatever the far end could manage.

Throughput is always lower, and a user reporting “slow” is reporting throughput. Nothing in this chapter changes bandwidth; several things in it quietly destroy throughput.

Duplex, and attenuation

Duplex is who may transmit, and when.

- **Half duplex** — one end at a time, like a walkie-talkie. If both transmit at once the signals overlap and the frames are destroyed; that overlap is a **collision**, and each end is expected to detect it, back off and retry.
- **Full duplex** — both ends at once, on separate pairs of wires, like a telephone call. **Collisions are impossible**, so a full-duplex port that reports collisions is telling you something is badly wrong. That is the single most useful sentence in this chapter.

Modern switched links are full duplex, and both ends normally agree on it through **auto-negotiation** — a brief exchange when the link comes up in which each end states what it can do and both pick the best they share.

Attenuation is separate and simpler: a signal loses strength with distance and through material. Every distance limit in the table below is really an attenuation limit — the point at which the far end can no longer reliably tell a 1 from a 0.

Medium	Typical reach	Speed	Use it when
Cat5e twisted pair	100 m	1 Gbps	Desk to access switch. The default, and the distance limit is a hard 100 m including patch leads
Cat6 / Cat6a	100 m (55 m for 10G on Cat6)	1–10 Gbps	Uplinks within a building, or any run you expect to upgrade
Multimode fibre (OM3/OM4)	300–400 m	1–10 Gbps	Between floors or between buildings on one campus. Cheaper optics, shorter reach
Single-mode fibre	Kilometres	1–100 Gbps	Between sites. More expensive optics, effectively unlimited campus reach
Direct-attach copper	1–10 m	10 Gbps+	Switch to switch inside one rack

Distance is a fault, not a preference

The exam names *distance* as a diagnosable issue for a reason. Exceed the reach of a medium and the link does not fail cleanly — it comes up and then corrupts frames, which shows as climbing CRC errors rather than a down port. A 95 m run that works in winter and fails in summer is a real phenomenon, and the answer is fibre, not another patch lead.

2.1.1 Copper pin-outs, and why you can mostly forget them

A **straight-through** cable connects pin 1 to pin 1; a **crossover** swaps the transmit and receive pairs. Historically you needed a crossover between two devices of the same kind — switch to switch, PC to router — and a straight-through between unlike devices.

Auto-MDIX makes the switch detect and correct this itself, and it is on by default on every modern platform. It is still examinable because it fails in two situations: when a port has been hard-coded to a speed and duplex (Auto-MDIX requires auto-negotiation to be on), and on older or simulated gear. If a link between two switches will not come up and both ends are hard-coded, the pin-out is a live suspect.

2.2 The counters that name the fault

Q SW1# show interfaces GigabitEthernet0/1

```
GigabitEthernet0/1 is up, line protocol is up (connected)
Hardware is Gigabit Ethernet, address is 00d0.58a1.0201
MTU 1500 bytes, BW 1000000 Kbit, DLY 10 usec
Full-duplex, 1000Mb/s, media type is 10/100/1000BaseTX
5 minute input rate 41000 bits/sec, 12 packets/sec
  284517 packets input, 39281044 bytes, 0 no buffer
  Received 1204 broadcasts (0 multicasts)
  0 runs, 0 giants, 0 throttles
  0 input errors, 0 CRC, 0 frame, 0 overrun, 0 ignored
  271033 packets output, 33771201 bytes, 0 underruns
  0 output errors, 0 collisions, 0 interface resets
  0 late collision, 0 deferred
```

That is a healthy port: every error counter zero, full duplex, and the speed matching the media. Learn what this looks like, because you diagnose by difference.

Counter climbing	What it means	Most likely cause
CRC and input errors	Frames arrived damaged	Bad cable, exceeded distance, dirty fibre connector, electrical interference. If collisions is zero, suspect the cable before the duplex
collisions on a full-duplex link	Should be impossible	Duplex mismatch. One end is half, the other full
late collision	A collision after the first 64 bytes	Duplex mismatch, or a segment longer than the standard allows
runts	Frames shorter than 64 bytes	Collisions, or a failing NIC
giants	Frames longer than the MTU	MTU mismatch, or a misconfigured jumbo-frame setting on one side
input errors with CRC zero	Damaged, but not by corruption	Often an MTU or framing problem rather than the cable
output drops	The device could not transmit fast enough	Congestion, not a cable fault. Do not replace the cable

💡 The duplex mismatch, in one paragraph

Auto-negotiation lets two ports agree on speed and duplex. If one end is set to duplex full by hand and the other is left on auto, the auto end cannot detect duplex, falls back to half, and you have a mismatch. The half-duplex end listens before transmitting and treats simultaneous transmission as a collision; the full-duplex end transmits whenever it likes. The link stays *up*. It just loses a percentage of every conversation, and the symptom the user reports is “the network is slow”, not “the network is down”.

The fix is to set both ends the same way. The rule of thumb is: leave both on auto, or hard-code both. Never one of each.

➤ Configure — a hard-coded access port, both ends matching

```
interface GigabitEthernet0/1
description Link to SW2 - speed and duplex fixed at both ends
speed 1000
duplex full
no shutdown
```

2.2.1 Fibre and signal levels

Topic 1.1 names *signal levels*. On a fibre link the transceiver reports how much light it is transmitting and receiving, in dBm, and a receive level below the optic's sensitivity produces exactly the same symptom as a bad copper cable — a link that is up with climbing CRC errors.

```
SW1# show interfaces GigabitEthernet0/25 transceiver
```

Port	Temperature (Celsius)	Voltage (Volts)	Optical	Optical
			Tx Power (dBm)	Rx Power (dBm)
Gi0/25	33.2	3.31	-2.4	-28.9

A receive power of -28.9 dBm against a typical sensitivity of about -23 dBm means the link is starved of light. Causes, in the order worth checking: a dirty or unseated connector, a bend or break in the fibre, too many patch panels, a run longer than the optic supports, or a single-mode optic on multimode fibre.

! Common Pitfalls

- **Replacing the cable first.** It is the most expensive check in time and the least likely single cause on a link that has been working. Read the counters first: they tell you which of six causes it is.
- **Treating “up/up” as healthy.** A port can be up, passing traffic, and corrupting five per cent of it. The exam will show you exactly this and expect you not to declare it fine.
- **Hard-coding one end.** The commonest way to create a duplex mismatch is to fix one side “to be safe”.
- **Reading raw totals instead of a rate.** Counters accumulate since the last clear. Ten thousand CRC errors over two years is noise; ten thousand since yesterday is a fault. Use `clear counters` and watch.

🔧 Break & Fix — “The file server is slow, but it is not down”.

A user copies a large file and gets a fraction of the expected throughput. Ping succeeds with no loss. You look at the access port:

```
SW1# show interfaces GigabitEthernet0/7 | include duplex|collision|CRC
```

```
Half-duplex, 100Mb/s, media type is 10/100/1000BaseTX
14432 input errors, 118 CRC, 0 frame, 0 overrun, 0 ignored
0 output errors, 89211 collisions, 0 interface resets
2244 late collision, 0 deferred
```

Diagnosis. The port is half duplex at 100 Mbps on gigabit-capable media, with a very large collision count and — decisively — *late* collisions. Late collisions on a modern switched link are the signature of a duplex mismatch: the far end is transmitting full duplex while this end still listens before sending. The small CRC count is a consequence of the collisions, not a separate cable fault.

Fix. Find the server NIC setting. It has almost certainly been hard-coded to full duplex while the switch port was left on auto. Set both ends to auto, or both to `speed 1000 / duplex full`, then `clear counters` and confirm the errors stop climbing. If they do not, *now* suspect the cable.

🔧 Lab 2.1 — Reading a broken link (*Packet Tracer*)

Topology. Two 2960 switches connected by one link; a PC on each.

Task 1. Cable SW1 Gi0/1 to SW2 Gi0/1. Confirm both PCs can ping each other, then run `show interfaces Gi0/1` on both and record the speed, duplex and every error counter while the link is healthy.

Task 2. On SW1 only, set speed 100 and duplex half. Leave SW2 on auto.

Task 3. Generate traffic between the PCs. Record what changes on each side. Which counter moved first?

Task 4. Repeat with SW1 at duplex full and SW2 on auto — the classic mismatch. Note that the link stays up.

Task 5. Extension. Without looking at your own configuration, hand the topology to a partner, have them introduce one fault from this chapter, and diagnose it from `show output` alone in under three minutes.

Verification. You should be able to state, from counters only, whether a link is healthy, mismatched, or physically damaged.

? Checkpoint — Test Yourself

1. A gigabit link is up. `collisions` is increasing and the interface reports full duplex. What single conclusion can you draw, and why is it certain?
2. A fibre link shows Rx power of -31 dBm against a sensitivity of -23 dBm. Give three causes in the order you would check them.
3. Why is a port that is administratively down easier to deal with than one that is up with rising CRC errors?

✓ Chapter Summary

- Copper is 100 m, full stop. Multimode reaches a campus; single-mode reaches a city. Exceeding a medium's reach corrupts frames rather than dropping the link.
- Diagnose from counters, not from intuition. CRC without collisions points at the cable; collisions on a full-duplex link point at duplex; late collisions are close to proof of a duplex mismatch.
- Set both ends of a link the same way — both auto, or both fixed.
- “Up/up” does not mean healthy. The dangerous fault is the one that passes most traffic and damages the rest.
- On fibre, read `show interfaces ... transceiver`. Light levels are a diagnosable fault the blueprint names explicitly.

✍ Review Questions

1. (MCQ) Which counter most strongly indicates a duplex mismatch? (a) giants (b) late collision (c) output drops (d) no buffer
2. (MCQ) A 10 Gbps link is required over 2 km between two buildings. Which medium? (a) Cat6a (b) multimode OM4 (c) single-mode fibre (d) direct-attach copper
3. (Short) Explain why hard-coding duplex on one end of a link and leaving the other on auto produces a mismatch rather than a failure to link.
4. (Short) A port shows 8,000 CRC errors and zero collisions. State the two most likely causes and the command you would run next.
5. (Applied) You inherit a network where every access port has been hard-coded to `speed 100` and `duplex full`. Describe the risk this creates, and the safest order in which to migrate the estate back to auto-negotiation.

Ethernet, Frames and Switching

EXAM TOPICS **1.1**

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Describe** the Ethernet frame and the purpose of each field you will actually meet on a device.
- **Explain** how a switch learns, forwards, floods and ages MAC addresses — the algorithm behind every switching chapter that follows.
- **Read** the MAC address table and use it to prove where a device is physically connected.
- **Predict** what a switch does with a unicast, broadcast or unknown destination.

🔗 Before You Start

Chapter 1 for frames and broadcast domains; Chapter 2 for the link underneath.

📖 Key Terms in This Chapter

Ethernet • MAC address • OUI • unicast • broadcast • multicast • CAM table • MAC address table • learning • flooding • filtering • ageing • unknown unicast • MTU • FCS

3.1 Reading hexadecimal first

Everything in this chapter — and every MAC address, every IPv6 address and every frame type for the rest of the book — is written in **hexadecimal**. It is worth two minutes now, because nothing later makes sense without it.

📄 Hexadecimal, and why it exists

Ordinary decimal counts in tens using ten symbols, 0 to 9. **Hex** counts in sixteens using sixteen symbols: 0 to 9, then a b c d e f for ten to fifteen.

It exists because **one hex digit is exactly four bits**, so two hex digits are exactly one byte. Writing a 48-bit MAC address in decimal would take fifteen digits with no relationship to the underlying bits; in hex it takes twelve, and each one maps to a nibble you can reason about.

Hex	Dec	Binary	Hex	Dec	Binary	Hex	Dec	Binary
0	0	0000	6	6	0110	c	12	1100
1	1	0001	7	7	0111	d	13	1101
2	2	0010	8	8	1000	e	14	1110
3	3	0011	9	9	1001	f	15	1111
4	4	0100	a	10	1010			
5	5	0101	b	11	1011			

💡 The three things you actually have to do with it

- **Recognise it.** A leading 0x means “what follows is hex” — 0x0800 is the number 2048, not eight hundred. Cisco also writes hex with no prefix at all, as in 00d0.58a1.0201.
- **Expand one digit to four bits,** using the table above. This is all that EUI-64 (Chapter 5) and the MAC address bits below require.
- **Know that f is all ones.** So ffff.ffff.ffff is forty-eight 1 bits — every bit set — which is exactly why that is the broadcast address.

You will never be asked to convert a long hex number to decimal in your head. You will regularly be asked what one digit means in bits.

📊 Worked Example: What is the first byte of 00d0.58a1.0201 in bits?

The first byte is the first *two* hex digits, 00.

From the table, 0 is 0000. So 00 is 0000 0000 — eight zero bits.

That matters immediately: the conceptbox after the frame explains that the low-order bit of the first byte says unicast or multicast, and the next bit up says whether the address was assigned by a manufacturer. Both are zero here, so this is a unicast address burned in by its vendor. In Chapter 28 you will meet an address where the second bit is *one*, and that single bit is the whole diagnosis.

3.2 The frame

📖 The Ethernet frame, field by field

- **Preamble** — clock synchronisation. You will never see it.
- **Destination MAC** (6 bytes) — who this frame is for. The switch reads this to decide where to send it.
- **Source MAC** (6 bytes) — who sent it. The switch reads this to *learn*.
- **Type/Length** (2 bytes) — what is inside. 0x0800 is IPv4, 0x86DD is IPv6, 0x8100 means a VLAN tag follows (Chapter 10).
- **Payload** — 46 to 1500 bytes. Below 46 the frame is padded; this is why a frame under 64 bytes total is a **runt**.
- **FCS** (4 bytes) — a checksum. If it fails, the frame is counted as a CRC error and discarded (Chapter 2).

A **MAC address** is 48 bits, written as twelve hex digits. The first half is the **OUI**, assigned to the manufacturer; the second half is assigned by that manufacturer. Cisco writes them as 00d0.58a1.0201; almost everyone else writes 00:d0:58:a1:02:01. They are the same address, and you should be able to read both.

💡 Three kinds of destination

- **Unicast** — one specific device. The low-order bit of the first byte is 0.
- **Broadcast** — ffff.ffff.ffff, every device in the VLAN. This is what defines a broadcast domain.
- **Multicast** — a group. The low-order bit of the first byte is 1. OSPF (Chapter 17), HSRP and VRRP (Chapter 19) all use multicast.

3.3 How a switch actually works

A switch runs one small algorithm, and everything in Part II is a modification of it.

💡 Learn, then forward or flood

For every frame that arrives on a port:

1. **Learn.** Record the *source* MAC address against the port it arrived on, in this VLAN.
Reset the ageing timer if it is already there.
2. **Decide on the destination.**
 - Known unicast → **forward** out that one port.
 - Unknown unicast → **flood** out every other port in the VLAN.
 - Broadcast or multicast → **flood**.
 - Destination is on the same port it arrived on → **filter**, that is, discard it.

A switch never modifies the addresses in a frame it forwards, and it never consults an IP address. Both of those are the router's job.

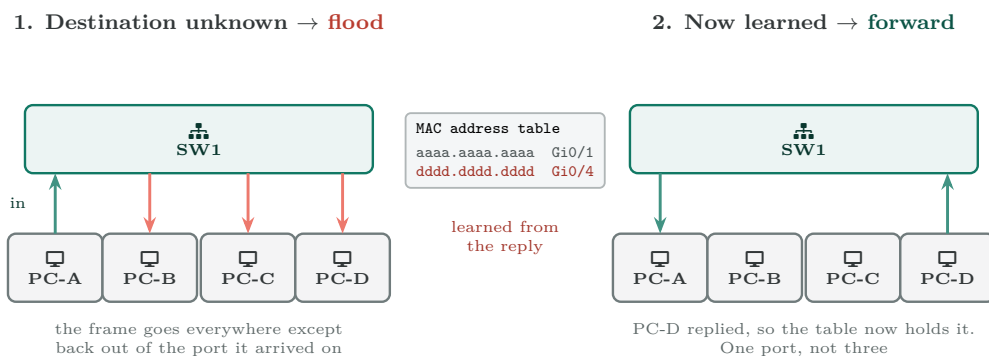


Figure 3.1: The whole switching algorithm. Flooding is not a failure — it is how the table gets built, and the second frame between the same two hosts is already efficient.

⚠️ The consequence that matters: unknown unicast flooding

A switch floods a unicast frame when it does not know the destination. That is not a fault — it is how the network keeps working while the table is learned. But it means a full or wiped MAC table causes traffic to be sprayed to every port, which is both a performance problem and a security one. It is also the mechanism a MAC flooding attack abuses, which is why Chapter 25 exists and why topic 4.7 names port security.

Entries age out after five minutes of silence by default. A device that has been quiet longer than that has to be re-learned, and the first frame to it is flooded.

SW1# show mac address-table

Mac Address Table			
Vlan	Mac Address	Type	Ports
1	00d0.58a1.0201	DYNAMIC	Gi0/1
1	0060.5c2b.19a4	DYNAMIC	Gi0/2
10	000a.f36e.7712	DYNAMIC	Gi0/5
10	0002.1751.4c33	STATIC	Gi0/6
20	00e0.b05f.2210	DYNAMIC	Gi0/8
Total Mac Addresses for this criterion: 5			

This table is the most useful single output on a switch. It answers “where is this device physically plugged in?”, and it does so by evidence rather than by documentation — which is exactly what topic 2.3 asks you to do when it says *validate the accuracy of network documentation*.

>_ Configure — a static MAC entry, and a shorter ageing time

```
! Pin a known device to a known port. Rarely needed, but it survives
! ageing and cannot be moved by a spoofed frame.
mac address-table static 0002.1751.4c33 vlan 10 interface GigabitEthernet0/6
!
! Age entries out faster than the 300-second default.
mac address-table aging-time 120
```

📊 Worked Example: Following a frame across two switches

PC-A (00d0.58a1.0201) on SW1 Gi0/1 sends to PC-B (00e0.b05f.2210) on SW2 Gi0/3. Both switches have empty tables.

1. The frame arrives at SW1 Gi0/1. SW1 *learns* 00d0.58a1.0201 on Gi0/1.
 2. SW1 does not know PC-B, so it *floods* — out its uplink and every other access port.
 3. SW2 receives it on its uplink and learns 00d0.58a1.0201 on *that uplink port*, not on Gi0/1. A switch only ever knows the direction, not the distance.
 4. SW2 does not know PC-B either, so it floods. PC-B receives it.
 5. PC-B replies. Now SW2 learns PC-B on Gi0/3, and because it already knows PC-A, it *forwards* rather than flooding.
 6. SW1 learns PC-B on its uplink. From here on the conversation is unicast in both directions.
- Two floods to start a conversation, then none. That is the whole design.

! Common Pitfalls

- **Expecting a switch to know a distance.** The table says which port, never how many hops. Five hundred devices can appear behind one uplink port and that is entirely normal.
- **Reading a stale table.** An entry can be up to five minutes old. If you have just moved a cable, the table may still show the old port.
- **Confusing the MAC table with the ARP table.** The MAC table maps MAC → port and lives on a switch. The ARP table maps IP → MAC and lives on any device with an IP address. The exam will test that you know which to look at.
- **Forgetting the VLAN column.** The same MAC address can legitimately appear in two VLANs. Learning is always per VLAN.

🔧 Break & Fix — Two ports, one MAC address, and a flapping log.

Users on VLAN 10 report intermittent drops. The log shows:

```
SW1# show logging | include MACFLAP
```

```
%SW_MATM-4-MACFLAP_NOTIF: Host 000a.f36e.7712 in vlan 10 is flapping
between port Gi0/5 and port Gi0/12
```

Diagnosis. The switch is learning the same source MAC on two ports in quick succession. A device cannot be in two places, so one of three things is true: there is a **physical loop** joining those ports (the same frame is arriving by two paths — see Chapter 12), a device is **spoofing** that address, or two devices have been given the **same MAC** by a bad clone or a virtualisation misconfiguration.

Fix. Check spanning tree first, because a loop is both the most likely cause and the most damaging: `show spanning-tree vlan 10` and look for a port that should be blocking and is not. If the topology is loop-free, shut Gi0/12, confirm the flapping stops, and trace what is attached to it.

Lab 3.1 — Watching a switch learn (*Packet Tracer*)

Topology. Two switches, uplinked; two PCs on SW1, one on SW2. All in VLAN 1 for now.

Task 1. Power everything on and immediately run `show mac address-table` on both switches. Record what is there before any traffic.

Task 2. Ping from PC-A to PC-C. Re-run the command on both switches and record which addresses each has learned, and against which port.

Task 3. Explain, from your own capture, why SW2 shows PC-A against its uplink rather than against a host port.

Task 4. Run `clear mac address-table dynamic`, then ping again and watch the table rebuild.

Task 5. Set `mac address-table aging-time 15`. Leave a PC idle for thirty seconds and confirm its entry disappears.

Task 6. Extension. Add a second cable between the two switches to create a loop. Observe what happens to the MAC table and the log before spanning tree settles — then read Chapter 12 to find out why the network survived.

Checkpoint — Test Yourself

1. A switch receives a frame whose destination MAC is already in its table, against the same port the frame arrived on. What does it do, and what is that behaviour called?
2. Why does the first ping between two devices often show a longer round trip than the ones after it?
3. You need to prove which wall port a particular laptop is using. Which table do you consult, on which device, and what do you need to know first?

Chapter Summary

- A switch learns from the *source* address and decides on the *destination* address. Everything else follows.
- Known unicast is forwarded, unknown unicast and broadcast are flooded, and a frame is never sent back out of the port it came in on.
- The MAC address table maps MAC to port, per VLAN, and ages entries out after five minutes by default.
- MAC table is not ARP table. One lives on a switch and maps to ports; the other maps IP to MAC and lives on anything with an IP address.
- A MAC address flapping between two ports means a loop, a spoof, or a duplicate — and you check for the loop first.

Review Questions

1. (MCQ) A switch receives a frame destined for `ffff.ffff.ffff`. What does it do? (a) forwards it to the router (b) floods it out all ports in the VLAN except the ingress port (c) drops it (d) sends it to the CPU
2. (MCQ) Which field does a switch use to populate its MAC address table? (a) destination MAC (b) source MAC (c) source IP (d) Type/Length
3. (Short) Explain why a switch shows hundreds of MAC addresses against a single uplink port, and why this is not a fault.
4. (Short) Give two distinct causes of a MAC flap message and say which you would investigate first, with a reason.

5. (**Applied**) A security auditor asks you to prove that no unauthorised device is connected to the finance VLAN. Describe, with commands, how you would produce that evidence and what its limitations are.

IPv4 Addressing and Subnetting

EXAM TOPICS **1.3**

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Convert** between dotted decimal, binary and prefix length fast enough to do it in an exam without a calculator.
- **Compute** the network address, broadcast address, usable range and host count of any subnet, in under thirty seconds.
- **Subnet** a given block to meet a stated requirement, and apply variable-length masks without overlapping.
- **Diagnose** the four addressing faults that account for almost every broken host: wrong mask, wrong gateway, address outside the subnet, and duplicate address.

🔗 Before You Start

Chapter 1 and Chapter 3. You need to be comfortable that a switch works on MAC addresses and a router works on IP addresses.

📖 Key Terms in This Chapter

dotted decimal • prefix length • subnet mask • network address • broadcast address • usable range • block size • magic number • RFC 1918 • private address • public address • VLSM • default gateway • APIPA

🎓 This topic is worth more time than any other in Part I

Topic 1.3 says *troubleshoot* IPv4 addressing, assignment and subnetting, “public and private”. You will be given a host that cannot reach something and expected to spot that its mask is wrong. Every second you spend computing a subnet boundary is a second not spent reasoning, so the arithmetic has to be automatic. Learn the method in §4.3 until it is boring.

4.1 Counting in binary, once

Everything in this chapter is a consequence of one fact: an IPv4 address is **32 bits**, and we write it as four decimal numbers only for our own convenience. The shortcut in Section 4.3 lets you subnet without ever writing a bit down — but it will be a magic trick you can forget rather than a method you can rebuild unless you look at the bits properly once. This is that once.

📄 Bits, bytes and octets

A **bit** is a single 0 or 1. Eight bits make a **byte**, which in networking is almost always called an **octet** — the words mean the same thing here.

Each position in an octet is worth twice the one to its right:

Position value	128	64	32	16	8	4	2	1
192 is	1	1	0	0	0	0	0	0

Position value	128	64	32	16	8	4	2	1
255 is	1	1	1	1	1	1	1	1
77 is	0	1	0	0	1	1	0	1

💡 Both directions, in one line each

Binary to decimal: add the position values wherever there is a 1. $64 + 8 + 4 + 1 = 77$.

Decimal to binary: walk left to right, and at each position ask “does this fit in what is left?”. For 77: 128 no (0), 64 yes (1, leaving 13), 32 no (0), 16 no (0), 8 yes (1, leaving 5), 4 yes (1, leaving 1), 2 no (0), 1 yes (1, leaving 0). That gives 01001101.

Two facts fall straight out and are worth holding on to. **255 is all eight bits set** — which is why it is the largest value an octet can hold, and why a mask octet of 255 means “all of this octet is network”. And **0 is all eight clear**, meaning “none of it is”.

📊 Worked Example: Where the block size comes from

Take 192.168.10.77 with mask 255.255.255.192, and write only the last octet of each in bits:

address 77	01 001101
mask 192	11 000000

The mask has **two** 1 bits in this octet, so two bits of it are network and the remaining **six** are host. Set those six host bits to zero and you have the network address; set them all to one and you have the broadcast address:

network	01 000000	= 64
broadcast	01 111111	= 127

So the subnet is 192.168.10.64 to 192.168.10.127 — and it is **64 addresses** wide, because six free host bits give $2^6 = 64$ combinations.

Now notice: $256 - 192 = 64$ as well. That is not a coincidence and it is not a trick. Subtracting the mask value from 256 is simply a fast way of asking “how many combinations do the remaining host bits have?” **That is the magic number**, and Section 4.3 does the whole job with it.

💡 Powers of two, which you should know on sight

Host bits decide subnet size, so these eleven numbers cover almost every question you will be asked.

2^1	2^2	2^3	2^4	2^5	2^6	2^7	2^8	2^9	2^{10}	2^{11}
2	4	8	16	32	64	128	256	512	1024	2048

Usable hosts is always two fewer than the total, because the all-zeros address names the network itself and the all-ones address is the broadcast. Neither can be given to a host. So six host bits give 64 addresses and **62** usable ones.

4.2 The address and the mask

An IPv4 address is 32 bits, written as four decimal bytes. It carries two pieces of information at once, and the mask is what separates them.

Network part and host part

The **subnet mask** marks, with 1 bits, which leading bits of the address identify the **network**. The remaining 0 bits identify the **host** within it. Written as a count of 1 bits, the mask is a **prefix length**: 255.255.255.0 and /24 are the same statement.

Two addresses are on the same subnet if and only if their network parts are identical. That single sentence is the source of most addressing faults: a host with the wrong mask computes a different network part, decides its neighbour is remote, and sends the traffic to a gateway that may not want it.

Default gateway

The address a host sends traffic to when the destination is **not** on its own subnet. Almost always a router or a layer 3 switch interface on that subnet.

Every host makes exactly one decision for every packet it sends, and it makes it with the mask:

- **Same network part as mine?** Then the destination is local. ARP for it directly and send the frame straight there. The gateway is not involved at all.
- **Different?** Then it is remote. ARP for the *gateway*, and hand the frame to it — with the destination's IP address still inside, but the gateway's MAC address on the outside.

Three consequences that account for a great many faults, and that Chapter 28 will make you test for directly:

- **A host with no gateway can reach its own subnet and nothing else.** Everything local works, so the user reports “the internet is down” rather than “my machine is misconfigured”.
- **The gateway must be inside the host's own subnet.** A host will not ARP for something it believes is remote, so a gateway outside the subnet is unreachable by definition — which is the fault in this chapter's Break & Fix.
- **Get the mask wrong and the host makes the local-or-remote decision wrongly,** for some destinations and not others. That is why a wrong mask presents as “some things work”, which is far harder to read than “nothing works”.

Prefix	Mask	Block	Usable	Typical use
/24	255.255.255.0	256	254	A standard user VLAN
/25	255.255.255.128	128	126	Half a /24
/26	255.255.255.192	64	62	A small office
/27	255.255.255.224	32	30	A rack or a server segment
/28	255.255.255.240	16	14	A tiny segment
/29	255.255.255.248	8	6	Four routers and room to grow
/30	255.255.255.252	4	2	A point-to-point link between routers
/31	255.255.255.254	2	2	A point-to-point link, no waste (RFC 3021)
/32	255.255.255.255	1	1	A host route or a loopback

Usable is block size minus two, because the first address in a block is the network address and the last is the broadcast address. The exception is /31, where both are usable because a point-to-point link has no need for a broadcast.

Private addresses — RFC 1918

Three ranges are reserved for internal use and are never routed on the internet. The exam expects you to recognise them instantly.

- 10.0.0.0/8 — one very large block
- 172.16.0.0/12 — that is 172.16.0.0 to 172.31.255.255, and the upper bound catches people out
- 192.168.0.0/16

Also worth knowing: 169.254.0.0/16 is **APIPA**, which a host assigns itself when DHCP fails. A host with a 169.254 address has not been given one — see Chapter 20.

4.3 The method: block size, or “magic number”

You have now seen where this comes from, so it is a shortcut rather than a spell. In the exam you will not have time to write out binary — and after this section you will not need to.

Four steps, every time

1. **Find the interesting octet** — the one where the mask is neither 255 nor 0.
2. **Block size** = 256 – (the mask value in that octet). This is the magic number.
3. **Count in blocks** from zero in that octet until you pass the address. The multiple you last passed is the **network address**.
4. **Broadcast** = next network address – 1. Usable range is everything between the two, exclusive.

Worked Example: Where does 172.16.83.190/26 live?

1. Mask for /26 is 255.255.255.192. The interesting octet is the fourth.
2. Block size = 256 – 192 = 64.
3. Count in 64s in the fourth octet: 0, 64, 128, **192**. Our host is 190, which sits after 128 and before 192. Network address is 172.16.83.128.
4. Next network is 172.16.83.192, so broadcast is 172.16.83.191. Usable range is 172.16.83.129 to 172.16.83.190.

So .190 is the *last usable address* in its subnet — a perfectly valid host address, and a favourite exam distractor because it looks like a broadcast.

Worked Example: The same method in the third octet: 10.44.150.7/21

1. /21 is 255.255.248.0. Interesting octet is the third.
2. Block size = 256 – 248 = 8.
3. Count in 8s in the third octet: 0, 8, ..., 144, **152**. Our 150 sits after 144. Network is 10.44.144.0.
4. Next network is 10.44.152.0, so broadcast is 10.44.151.255. Usable is 10.44.144.1 to 10.44.151.254 — 2046 hosts.

Note that the fourth octet plays no part in finding the boundary. Only the interesting octet does.

4.4 Subnetting a block to a requirement

Worked Example: Split 192.168.20.0/24 for four sites of 50, 25, 10 and 2 hosts

Work *largest first*, or you will strand addresses.

1. 50 hosts needs 62 usable → /26, block 64. 192.168.20.0/26 — usable .1–.62.
2. 25 hosts needs 30 usable → /27, block 32. Next free address is .64: 192.168.20.64/27 — usable .65–.94.
3. 10 hosts needs 14 usable → /28, block 16. Next free is .96: 192.168.20.96/28 — usable .97–.110.
4. 2 hosts (a router link) → /30, block 4. Next free is .112: 192.168.20.112/30 — usable .113–.114.

Everything from .116 upwards is still free. This is **VLSM**: different masks inside one block, sized to the requirement.

The two rules VLSM must obey

- **A subnet must start on a multiple of its own block size.** A /27 can start at .64 or .96, never at .70.
- **Subnets must not overlap.** If you allocate smallest-first you will eventually place a large subnet where it does not fit, and have to start again. Largest first avoids this.

4.5 Configuring and verifying an address

Configure — an interface address and a default gateway

```
! On a router interface -- the mask is written in full.
interface GigabitEthernet0/0
  description User VLAN 10 gateway
  ip address 192.168.20.1 255.255.255.192
  no shutdown
!
! On a layer 2 switch, the management address lives on an SVI,
! and the switch needs a default gateway because it does not route.
interface Vlan10
  ip address 192.168.20.2 255.255.255.192
  no shutdown
!
ip default-gateway 192.168.20.1
```

R1# show ip interface brief

Interface	IP-Address	OK?	Method	Status	Protocol
GigabitEthernet0/0	192.168.20.1	YES	manual	up	up
GigabitEthernet0/1	unassigned	YES	unset	administratively down	down
Vlan1	unassigned	YES	unset	down	down

Reading the two status columns

The pair of columns is the single most useful diagnostic on a router, and the combinations mean different things.

- **up / up** — working.
- **up / down** — the cable and electrical link are fine, but the protocol is not. Usually an encapsulation or keepalive mismatch, or the far end is not configured.

- **down / down** — a physical problem. Cable, transceiver, or the other end is shut. Go back to Chapter 2.
- **administratively down / down** — somebody typed `shutdown`. The fix is `no shutdown`.

! Common Pitfalls

- **Forgetting no shutdown.** Router interfaces are shut by default. Switch ports are not. This asymmetry catches everyone once.
- **Assuming 172.16.0.0/12 stops at 172.16.** It runs to 172.31.255.255. 172.32.0.0 is public.
- **Giving the gateway an address outside the subnet.** A host will accept the configuration and then fail silently for everything off-net.
- **Using the network or broadcast address on a host.** .0 and .255 are only network and broadcast in a /24. In a /26 the boundaries move, and 192.168.20.64 is a network address even though it does not end in zero.

🔧 Break & Fix — “I can reach my own floor but nothing else”.

A host is configured 192.168.20.100 mask 255.255.255.192, gateway 192.168.20.1. It can ping 192.168.20.90 but not its gateway and nothing beyond.

Diagnosis. Work the mask. /26 gives block size 64, so the boundaries are .0, .64, .128, .192. The host at .100 is in the 192.168.20.64/26 subnet, usable .65–.126. Its gateway at .1 is in the *first* subnet, 192.168.20.0/26. The host and its gateway are on different subnets, so the host cannot ARP for it and has no way off its own segment. It can still reach .90 because that address is in the same /26.

Fix. Either move the host into the subnet its gateway serves, or — far more likely what was intended — correct the gateway to 192.168.20.65, the first usable address of the subnet the host is actually in. Then confirm with a ping to the gateway before testing anything remote.

🔧 Lab 4.1 — Subnet, configure, then break it (*Packet Tracer*)

Topology. One router, two switches, four PCs — two per VLAN.

Task 1. Subnet 172.16.5.0/24 into two subnets of at least 100 hosts each. Write down, before touching a device: network address, first and last usable address, and broadcast address for both.

Task 2. Configure the router with a gateway address in each subnet and the PCs with addresses from your plan. Verify with `show ip interface brief` and a ping within each subnet.

Task 3. Prove routing works between the two subnets by pinging across.

Task 4. Now change one PC’s mask from your planned value to 255.255.255.0. Predict what will break *before* you test, then test it. Explain the result in terms of the network part.

Task 5. Change it back, then set one PC’s gateway to an address in the other subnet. Predict and test again.

Task 6. Extension. Re-plan the same /24 using VLSM for four segments of 100, 50, 20 and 2 hosts, and configure it without overlaps.

? Checkpoint — Test Yourself

1. For 10.19.200.37/27: give the network address, broadcast address, and first and last usable address.

2. A host is 192.168.1.130/25 and its gateway is 192.168.1.126. Will it work? Show your reasoning.
3. How many usable addresses does a /30 provide, and why is a /31 sometimes used instead on a router-to-router link?

Chapter Summary

- Address plus mask equals two facts: which network, and which host. Devices are on the same subnet only if their network parts match.
- Block size = 256 – mask octet. Count in blocks. That method answers every subnetting question you will be asked.
- Usable = block – 2, except /31 and /32.
- Private ranges are 10/8, 172.16/12 (to 172.31) and 192.168/16. A 169.254 address means DHCP failed.
- Plan VLSM largest first, and start every subnet on a multiple of its own block size.
- Four faults cover most cases: wrong mask, wrong gateway, address outside the subnet, duplicate address.

Review Questions

1. **(MCQ)** A host is addressed 192.168.90.200/28. What is the broadcast address of its subnet?
(a) 192.168.90.207 (b) 192.168.90.255 (c) 192.168.90.191 (d) 192.168.90.215
2. **(MCQ)** Which is a valid host address in 10.0.8.0/22? (a) 10.0.8.0 (b) 10.0.11.255
(c) 10.0.11.254 (d) 10.0.12.1
3. **(Short)** Explain why a host with the correct address but a mask that is too short may still reach some destinations and not others.
4. **(Short)** A host has address 169.254.14.9. What has happened, and what would you check first?
5. **(Applied)** You are given 192.168.40.0/24 and asked for six subnets: 60, 60, 28, 12, 12 and 2 hosts. Produce a VLSM plan with no overlaps, showing network address, mask and usable range for each, and state how much of the block remains free.

IPv6 Addressing

EXAM TOPICS **1.4**

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Abbreviate** and expand an IPv6 address correctly, applying both compression rules without ambiguity.
- **Identify** the address types the exam names: global unicast, link local, unique local, multicast, and the solicited-node address.
- **Construct** an interface identifier by modified EUI-64, which the blueprint calls out explicitly.
- **Size** a prefix, and **diagnose** the assignment faults that stop an IPv6 host communicating.

🔗 Before You Start

Chapter 4 — the ideas of network part, host part and prefix length carry over unchanged. Chapter 3 for MAC addresses, which EUI-64 is built from.

📖 Key Terms in This Chapter

global unicast • link local • unique local • multicast • solicited-node • anycast • modified EUI-64 • SLAAC • router advertisement • neighbour discovery • dual stack • prefix

🎓 What v2.0 asks, and what it dropped

Topic **1.4** says *troubleshoot* IPv6 address configuration, assignment and prefix sizing, and narrows the scope to “unicast and modified EUI 64”. The previous blueprint asked you to compare every address type in detail; this one does not. Concentrate on getting an address onto an interface, understanding where the link-local one came from, and spotting a wrong prefix.

5.1 Writing the address

An IPv6 address is 128 bits, written as eight groups of four hex digits separated by colons. If hexadecimal is not yet automatic, the two-minute primer in Chapter 3 is the one thing to re-read before going on — this whole chapter is written in it, and modified EUI-64 below cannot be followed without being able to expand one hex digit into four bits.

Two rules compress it.

💡 The two compression rules

1. **Drop leading zeros** in any group. 0db8 becomes db8; 0000 becomes 0. Trailing zeros are never dropped.
2. **Replace one run of all-zero groups with ::**. You may do this **once only**, because a second :: would make the address ambiguous — there would be no way to know how many zeros each stood for.

Worked Example: Compressing an address

Start with the full form:

2001:0db8:0000:0000:0000:ff00:0042:8329

1. Drop leading zeros in each group: 2001:db8:0:0:0:ff00:42:8329
2. Replace the longest run of zero groups with :: 2001:db8::ff00:42:8329

Expanding is the same in reverse. Given 2001:db8::1, count the groups present — two before the :: and one after, so three — and the :: must therefore stand for five zero groups: 2001:0db8:0000:0000:0000:0000:0000:0001.

5.2 The types you must recognise

Type	Prefix	What it is for
Global unicast	2000::/3	Routable on the internet. In practice you meet 2001:db8::/32, which is reserved for documentation and used throughout this book
Link local	fe80::/10	Valid on one link only, never routed. Every IPv6 interface has one automatically, and routing protocols use it as the next hop
Unique local	fc00::/7	The rough equivalent of RFC 1918 private space. In practice fd00::/8
Multicast	ff00::/8	Groups. ff02::1 is all nodes on the link, ff02::2 is all routers, ff02::5 is OSPFv3
Solicited-node	ff02::1:ff00:0/104	Used by neighbour discovery in place of ARP. Formed from the last 24 bits of the unicast address
Loopback	::1	The equivalent of 127.0.0.1
Unspecified	::	“No address yet”, used as a source during autoconfiguration

There is no broadcast in IPv6

IPv6 has no broadcast address at all. Where IPv4 would broadcast, IPv6 uses a multicast group — usually ff02::1, all nodes. This is why ARP does not exist in IPv6: its job is done by neighbour discovery using the solicited-node multicast address, which reaches only the handful of hosts whose addresses end the same way rather than every device on the segment.

5.3 Modified EUI-64

The blueprint names this explicitly, so it is examinable arithmetic.

Turning a 48-bit MAC into a 64-bit interface ID

1. Split the MAC in half.
2. Insert fffe in the middle.
3. **Flip the seventh bit** of the first byte — the universal/local bit. In hex this means adding or subtracting 2 from the second hex digit.

 Worked Example: EUI-64 from 00d0.58a1.0201

1. Split: 00d0.58 and a1.0201.
2. Insert fffe: 00d0:58ff:fea1:0201.
3. Flip the seventh bit of 00. In binary 00 is 0000 0000; flipping the seventh bit gives 0000 0010, which is 02.
4. Interface ID is 02d0:58ff:fea1:0201. With a prefix of 2001:db8:acad:10::/64 the full address is 2001:db8:acad:10:2d0:58ff:fea1:201.

The quick check: an address containing ff:fe in the middle of its host portion was almost certainly built by EUI-64.

5.4 Configuring IPv6

>_ Configure — IPv6 on a router interface, three ways

```
! Routing must be enabled globally first -- this is the step most often
! forgotten, and without it the router will not forward or advertise.
ipv6 unicast-routing
!
interface GigabitEthernet0/0
description User VLAN 10 gateway
! 1. A fully specified address.
ipv6 address 2001:db8:acad:10::1/64
! 2. Or let the device build the host part by EUI-64.
ipv6 address 2001:db8:acad:20::/64 eui-64
! 3. A link-local address is created automatically, but you can pin it
!    to something memorable -- worth doing, because routing protocols
!    show it as the next hop.
ipv6 address fe80::1 link-local
no shutdown
```

R1# show ipv6 interface brief

```
GigabitEthernet0/0    [up/up]
FE80::1
2001:DB8:ACAD:10::1
2001:DB8:ACAD:20:2D0:58FF:FEA1:201
GigabitEthernet0/1    [administratively down/down]
unassigned
```

Note that one interface holds several addresses at once, and that this is normal rather than a fault. An IPv6 interface essentially always has a link-local address plus whatever global addresses it has been given.

How a host gets an address: SLAAC versus DHCPv6

A host sends a router solicitation to ff02::2; a router replies with a **router advertisement** carrying the prefix. Then either:

- **SLAAC** — the host builds its own address from the advertised prefix plus an interface ID it generates. No server involved.
- **DHCPv6** — the advertisement tells the host to ask a server, which supplies the address and other options.

A host with only a fe80:: address and no global one has heard no router advertisement. That is the IPv6 equivalent of an APIPA address, and it points at the router, not the host.

! Common Pitfalls

- **Omitting ipv6 unicast-routing.** Interfaces come up and addresses look right, but the router neither forwards nor advertises.
- **Using :: twice.** It is not merely discouraged, it is invalid, because the address can no longer be expanded uniquely.
- **Dropping trailing zeros.** 2001:db8:1000:: cannot be shortened to 2001:db8:1::. Only *leading* zeros go.
- **Assuming a prefix other than /64 for a LAN.** SLAAC requires a 64-bit interface ID, so a LAN prefix longer than /64 breaks autoconfiguration. Point-to-point links are the usual exception.

🔧 Break & Fix — Dual-stack host reaches IPv4 destinations only.

A workstation has a working IPv4 address and, for IPv6, only fe80::2d0:58ff:fea1:201. It cannot reach any IPv6 destination, on or off the segment.

Diagnosis. A link-local address is generated by the host itself and proves nothing except that the interface is up. The absence of a global address means no router advertisement is being received. Either no router is configured for IPv6 on that VLAN, or `ipv6 unicast-routing` is missing, or the router interface has no global prefix to advertise.

Fix. On the gateway, confirm `ipv6 unicast-routing` is present in the running configuration, then check `show ipv6 interface GigabitEthernet0/0` shows a global address and that router advertisements are not suppressed. Bring the prefix back and the host will build its own address within seconds — no change is needed on the host at all.

🔧 Lab 5.1 — Dual-stack a small network (Packet Tracer)

Topology. One router, one switch, two PCs, addressed for both protocols.

- Task 1.** Enable `ipv6 unicast-routing` and give Gi0/0 the address 2001:db8:acad:10::1/64 and link-local fe80::1.
- Task 2.** Set both PCs to obtain IPv6 automatically. Record the addresses they build and confirm they contain ff:fe.
- Task 3.** Compute by hand what each PC's EUI-64 address should be from its MAC, and check your answer against the device.
- Task 4.** Ping between the PCs using IPv6, then ping the router's link-local address from a PC. Note that the link-local ping needs no global addressing at all.
- Task 5.** Remove `ipv6 unicast-routing` and observe what happens to the PCs after their addresses expire.
- Task 6. Extension.** Configure a second VLAN with prefix 2001:db8:acad:20::/64 and route between the two.

? Checkpoint — Test Yourself

1. Compress 2001:0db8:0000:0012:0000:0000:0000:00ab fully.
2. A host shows only an fe80:: address. What is missing, and on which device would you look?
3. Build the modified EUI-64 interface identifier for MAC aabb.cc00.0100.

📋 Chapter Summary

- Drop leading zeros; use :: once, for the longest zero run.

- 2000::/3 global, fe80::/10 link local, fd00::/8 unique local, ff00::/8 multicast. There is no broadcast.
- EUI-64: split the MAC, insert fffe, flip the seventh bit.
- ipv6 unicast-routing is a separate, easily forgotten step.
- LANs are /64 because SLAAC needs a 64-bit interface ID.
- Only a link-local address means no router advertisement was heard — investigate the router, not the host.

Review Questions

1. (MCQ) Which address is a valid compression of 2001:0db8:0000:0000:1000:0000:0000:0001?
(a) 2001:db8::1000::1 (b) 2001:db8:0:0:1::1 (c) 2001:db8::1000:0:0:1 (d) 2001:db8:1000::1
2. (MCQ) Which multicast group do all IPv6 routers on a link join? (a) ff02::1 (b) ff02::2 (c) ff02::5 (d) ff02::9
3. (Short) Explain why :: may appear only once in an address.
4. (Short) A LAN is configured with a /112 prefix and hosts are set to autoconfigure. What will happen, and why?
5. (Applied) You are dual-stacking an existing IPv4 network. Describe the order in which you would enable IPv6 on the router, verify each step, and what evidence you would collect before declaring the segment working.

TCP, UDP and the Diagnostic Toolkit

EXAM TOPICS **2.4**

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** what a port number identifies, and recognise the well-known ports that appear in access lists and in fault reports.
- **Choose** correctly between TCP and UDP behaviour when reasoning about a symptom.
- **Use** ping, extended ping and traceroute deliberately, varying source, size and count to test a specific hypothesis.
- **Read** packet capture output well enough to say which direction failed.

🔗 Before You Start

Chapter 4 for addressing, Chapter 3 for what a switch does with a frame.

📖 Key Terms in This Chapter

port • socket • TCP • UDP • three-way handshake • acknowledgement • retransmission • well-known port • ephemeral port • ICMP • echo request • TTL • extended ping • traceroute

🎓 The toolkit is the exam

Topic **2.4** names the tools by name: “show commands (including show logs), ping, extended ping, trace route, and packet capture output”. It is one of the eight *troubleshoot* topics, and the tools in it are how you will answer the other seven. Learn them as instruments, not as trivia.

6.1 Ports, and what they identify

An IP address identifies a host. A **port number** identifies which program on that host. Together they form a **socket**, and a conversation is defined by four things: source address, source port, destination address, destination port.

Port	Service	Protocol	Why it matters here
22	SSH	TCP	Device management. Chapter 23
23	Telnet	TCP	Device management, unencrypted. Disable it
53	DNS	UDP, TCP	Name resolution. Chapter 21
67, 68	DHCP	UDP	Address assignment. Chapter 20
69	TFTP	UDP	Legacy file transfer. Replaced by SFTP/SCP in v2.0
80, 443	HTTP, HTTPS	TCP	The traffic users notice
161, 162	SNMP	UDP	Monitoring and traps. Chapter 30
514	Syslog	UDP	Log delivery. Chapter 30

Client programs use a high-numbered **ephemeral** port as their source. This is why a return packet is not simply “port 80” — it is addressed to whatever ephemeral port the client opened, and why stateless access lists need care (Chapter 24).

💡 TCP or UDP, and what each failure looks like

TCP establishes a connection with a three-way handshake — SYN, SYN-ACK, ACK — numbers every byte, acknowledges what arrives and retransmits what does not. It costs latency and gains reliability.

UDP sends and forgets. No handshake, no acknowledgement, no retransmission. It is right for anything where a late packet is worse than a lost one, and for short exchanges where a handshake would double the cost — which is why DNS and DHCP use it.

The diagnostic value is in how they fail. A blocked TCP port produces a *connection timeout or refusal* — something definite. A blocked UDP port often produces *nothing at all*, because there was never going to be an acknowledgement. “It just hangs” points at UDP or at a one-way path; “connection refused” means something answered and said no.

6.2 Ping, and how to ping deliberately

Ping sends an ICMP echo request and waits for a reply. It proves that a path exists in *both* directions — a point worth dwelling on, because a successful ping rules out far more than a failed one confirms.

Q R1# ping 192.168.20.10

```
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 192.168.20.10, timeout is 2 seconds:
!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 1/2/4 ms
```

Symbol	Meaning
!	Reply received. The path works in both directions
.	Timed out. Either no path, no return path, or something is dropping silently
U	Destination unreachable — a router explicitly told you it has no route. This is far more informative than a timeout
Q	Source quench: congestion
M	Could not fragment. An MTU problem

💡 The extended ping is the one that finds faults

A plain ping from a router sources the packet from the exit interface. That is often not the address you care about. **Extended ping** lets you set the source, and this single option answers the commonest routing question: “does the far end have a route *back* to this subnet?” Run ping with no arguments to enter the extended dialogue.

Q R1# ping

```
Protocol [ip]:
Target IP address: 192.168.30.10
Repeat count [5]: 100
Datagram size [100]: 1500
Timeout in seconds [2]:
Extended commands [n]: y
Source address or interface: 192.168.20.1
Type of service [0]:
```

```
Set DF bit in IP header? [no]: yes
Sweep range of sizes [n]:
Sending 100, 1500-byte ICMP Echos to 192.168.30.10, timeout is 2 seconds:
Packet sent with a source address of 192.168.20.1
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
Success rate is 100 percent (100/100), round-trip min/avg/max = 1/3/12 ms
```

- **Source address** — test the return path for a specific subnet.
- **Repeat count** — 100 pings exposes intermittent loss that five will miss.
- **Datagram size with DF bit set** — finds MTU problems. If 1500 fails and 1400 succeeds, something on the path cannot carry a full-size frame.

6.3 Traceroute

Traceroute sends packets with deliberately small TTL values. The first router decrements TTL to zero, discards the packet and reports back; the second does the same for TTL 2, and so on. The result is a list of hops.

```
R1# traceroute 192.168.30.10
```

```
Type escape sequence to abort.
Tracing the route to 192.168.30.10
```

```
 0 10.0.12.2 4 msec 1 msec 1 msec
 1 10.0.23.3 8 msec 5 msec 4 msec
 2 * * *
 3 * * *
```

! Where a traceroute stops is not always where the fault is

Asterisks mean no reply from that hop. That can mean the fault is there — or that the hop is configured not to reply, or that the *return* path from that hop is broken while the forward path is fine. A traceroute that dies at hop 3 narrows the search to hop 3 onwards; it does not convict hop 3. Confirm with an extended ping sourced from an address the far end should know.

6.4 Reading a capture

```
SW1# show capture BUF
```

```
 1 0.000000 192.168.20.10 -> 192.168.30.10 ICMP Echo request id=1 seq=1
 2 2.001113 192.168.20.10 -> 192.168.30.10 ICMP Echo request id=1 seq=2
 3 4.002240 192.168.20.10 -> 192.168.30.10 ICMP Echo request id=1 seq=3
```

Three requests, no replies. That is a one-way path: the packets are leaving and arriving, but nothing is coming back. The fault is on the return path or at the destination host, and it is emphatically *not* the local switch — which is exactly the conclusion the capture is there to let you draw.

! Common Pitfalls

- **Treating a failed ping as proof of a broken forward path.** Ping needs both directions. Half the time the forward path is fine.
- **Pinging from the wrong source.** A router pinging from its exit interface tests a path the users never take.
- **Concluding from five packets.** Intermittent loss needs a hundred.
- **Forgetting that ICMP may be filtered.** Plenty of hosts and firewalls drop echo requests by policy. “It does not ping” is not the same as “it is down”.

Break & Fix — Web server reachable by IP, refuses connections by name and port.

Users cannot load an internal site. From a workstation, `ping 10.20.5.40` succeeds with 0% loss. Browsing to the address times out.

Diagnosis. Ping proves layer 3 connectivity in both directions, so addressing, masks, routing and the physical path are all sound. The failure is above layer 3 — either the service is not listening, or something is filtering TCP 443 specifically. A timeout rather than a refusal points at a filter: a closed port on a live host normally answers with a reset.

Fix. Test the port rather than the host. From a router on the path, an extended ping cannot help — it is ICMP only — so use `telnet 10.20.5.40 443`: a blank screen means the port is open, a refusal means closed, and a hang means filtered. Then walk the access lists on the path (Chapter 24). Do not touch addressing; the ping already cleared it.

Lab 6.1 — Diagnose with intent (*Packet Tracer*)

Topology. Two routers back to back, one LAN each, one PC per LAN.

Task 1. Address everything and confirm end-to-end ping between the PCs.

Task 2. From R1, run a plain ping to the far PC, then an extended ping sourced from the LAN interface. Explain why the second is the more useful test.

Task 3. Remove the return route on R2. Predict what a ping from PC-A will show, and what a traceroute will show, *before* testing.

Task 4. Restore the route. Now set the far LAN interface MTU to 1400, and use an extended ping with size 1500 and the DF bit set. Record the result.

Task 5. Run a hundred-packet ping and record whether any loss appears.

Task 6. Extension. Have a partner break one thing — an address, a mask, a route or an interface — and identify which, using only ping, extended ping and traceroute, in under five minutes.

Checkpoint — Test Yourself

1. A ping returns U.U.U. What has happened, and why is that more useful than five dots?
2. Why does an extended ping with a specified source often succeed when a plain ping from the same router fails?
3. A traceroute shows three hops then asterisks. List three different explanations.

Chapter Summary

- A port identifies the program; address plus port is a socket.
- TCP handshakes, acknowledges and retransmits; UDP does none of it. Their failure modes differ, and the difference is diagnostic.
- Ping tests both directions at once. A success rules out a great deal; a failure convicts nothing on its own.
- Extended ping sets the source, size and DF bit — it is the tool that finds return-path and MTU faults.
- Traceroute narrows the search; it does not name the culprit.
- A capture showing requests and no replies is a one-way path.

Review Questions

1. (MCQ) Which pair of ping output characters indicates that a router explicitly reported it had no route? (a) (b) UUUUU (c) !!!!! (d) MMMMM

2. **(MCQ)** Which protocol would you expect a DNS query to use by default? (a) TCP 53 (b) UDP 53 (c) UDP 67 (d) TCP 22
3. **(Short)** Explain why setting the source address on a ping is the single most useful extended option when troubleshooting routing.
4. **(Short)** A service “hangs” rather than being refused. What does that suggest, and what would you check?
5. **(Applied)** A user reports a file transfer that succeeds for small files and fails for large ones. Describe, with the exact ping options you would use, how you would confirm or rule out an MTU problem.

The IOS Command Line

EXAM TOPICS **2.4**

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Move** between IOS command modes without hesitating, and recognise which mode a prompt indicates.
- **Distinguish** the running configuration from the startup configuration, and save deliberately rather than by habit.
- **Perform** a first-time device setup: hostname, addressing, passwords, SSH and a banner.
- **Filter** show output, which is what makes long output usable under exam time pressure.

🔗 Before You Start

Chapter 4 for addressing and Chapter 6 for the diagnostic commands you will now be typing for real.

📖 Key Terms in This Chapter

user EXEC • privileged EXEC • global configuration • interface configuration • line configuration • running-config • startup-config • NVRAM • context-sensitive help • pipe • banner • SSH

7.1 Modes, and the prompt that tells you where you are

Mode	Prompt	What you can do, and how you got there
User EXEC	SW1>	Look, barely. Enter with <code>enable</code>
Privileged EXEC	SW1#	All <code>show</code> commands, <code>copy</code> , <code>reload</code> , <code>debug</code> . Enter configuration with <code>configure terminal</code>
Global configuration	SW1(config)#	Device-wide settings: <code>hostname</code> , <code>users</code> , <code>routing</code>
Interface configuration	SW1(config-if)#	One interface. Entered with <code>interface Gi0/1</code>
Line configuration	SW1(config-line)#	Console and VTY access. Entered with <code>line vty 0 15</code>
VLAN configuration	SW1(config-vlan)#	Naming a VLAN. Chapter 8
Router configuration	R1(config-router)#	A routing protocol. Chapter 17

💡 Getting out, and the two ways to do it

`exit` steps back one level. `end`, or `Ctrl-Z`, jumps straight to privileged EXEC from anywhere. When an exam question shows you a prompt, the prompt is telling you which commands are currently legal — a global command typed in interface mode is rejected, and that is a favourite trap.

One shortcut worth knowing: `do` runs a privileged EXEC command without leaving configuration mode, so `do show ip interface brief` works from `SW1(config-if)#`.

7.2 Two configurations, and only one survives a reboot

📖 running-config and startup-config

The **running configuration** is in RAM. It is what the device is doing right now, and every command you type takes effect there *immediately*.

The **startup configuration** is in NVRAM. It is what the device will load when it next boots. Nothing you type reaches it until you copy it there.

- `copy running-config startup-config` — save. Abbreviated `wr` or `write memory` on most platforms.
- `show running-config` — what is in force.
- `show startup-config` — what will be in force after a reload.

⚠️ Not saving is a technique, not only a mistake

Because configuration takes effect immediately and is lost on reload, a deliberately unsaved change is a safety net. If you are about to modify the access list or the interface you are connected through, and you get it wrong, a power cycle restores the last saved configuration and your access with it.

The professional habit is: make the risky change, verify you still have access, *then* save. Reversing that order is how people lock themselves out of devices in another building.

7.3 Getting help, and typing less

Keystroke	Effect
<code>?</code>	List every command valid here
<code>sh ?</code>	List commands starting with “sh”
<code>show ip ?</code>	List the next keyword. Note the space — <code>show ip?</code> means something different
<code>Tab</code>	Complete the current keyword
Abbreviation	Type the shortest unambiguous form. <code>sh ip int br</code> is <code>show ip interface brief</code>
<code>Ctrl-A / Ctrl-E</code>	Jump to start / end of line
<code>Ctrl-C</code>	Abandon this command
<code>Ctrl-Shift-6</code>	Abort a running ping or traceroute
Up arrow	Previous command

💡 Filtering output is a survival skill

A full `show running-config` on a production switch runs to hundreds of lines. The pipe cuts it to what you need, and in an exam it is the difference between answering and scrolling.

- `| include vlan` — only lines containing “vlan”
- `| exclude ^!` — drop the `!` comment lines
- `| begin interface` — start output at the first match
- `| section interface Gi0/1` — that interface’s whole block

🔍 SW1# `show running-config | section interface GigabitEthernet0/1`

```
interface GigabitEthernet0/1
description Uplink to SW2
switchport trunk encapsulation dot1q
switchport mode trunk
switchport trunk allowed vlan 10,20,30
```

7.4 A first-time setup, end to end

➤ Configure — a switch from the box to remotely manageable

```
enable
configure terminal
!
! 1. Identify the device. The hostname appears in every prompt and log.
hostname SW1
!
! 2. Protect privileged mode. "secret" is hashed; "password" is not --
!     never use "enable password".
enable secret Str0ng-Enable-Secret
!
! 3. A local user for SSH, with full privilege.
username netadmin privilege 15 secret Str0ng-User-Secret
!
! 4. A management address. A layer 2 switch has no routed ports, so
!     this lives on an SVI, and it needs a gateway to be reachable
!     from another subnet.
interface Vlan1
ip address 10.0.0.11 255.255.255.0
no shutdown
exit
ip default-gateway 10.0.0.1
!
! 5. SSH needs a domain name and a key. 2048-bit minimum; version 2 only.
ip domain-name example.local
crypto key generate rsa modulus 2048
ip ssh version 2
!
! 6. Console: require login, and stop output interrupting your typing.
line console 0
login local
logging synchronous
exec-timeout 10 0
exit
!
! 7. Remote access: SSH only, authenticated against local users.
line vty 0 15
login local
transport input ssh
exec-timeout 10 0
```

```

exit
!
! 8. A legal banner. Never write "welcome" -- it has been argued in court.
banner motd # Authorised access only. Activity is logged. #
!
end
copy running-config startup-config

```

Q SW1# show ip ssh

```

SSH Enabled - version 2.0
Authentication timeout: 120 secs; Authentication retries: 3

```

! Common Pitfalls

- **Using enable password.** It is stored in clear text. `enable secret` is hashed, and where both exist the secret wins.
- **Forgetting transport input ssh.** Leave it at the default and Telnet stays available, which undoes the point of configuring SSH.
- **Generating a key without a hostname and domain name.** The key is named after them, and the command fails or produces a useless key if they are still at defaults.
- **Saving before verifying.** See the warning above. Verify access, then save.
- **Omitting logging synchronous.** Without it a log message arrives in the middle of the line you are typing, and you cannot tell what you have actually entered.

🔧 Break & Fix — The change worked yesterday and the device came back without it.

A colleague configured a new VLAN and confirmed it working. After a power cut overnight, the VLAN is gone and the configuration looks like it did last week.

Diagnosis. The change was made in the running configuration and never copied to the startup configuration. The reload loaded NVRAM, which did not have it. Nothing is broken — the device did exactly what it was told.

Confirm it by comparing the two before doing anything else:

Q SW1# show archive config differences nvram:startup-config system:running-config

```

!Contextual Config Diffs:
+vlan 30
+ name Guest
+interface GigabitEthernet0/9
+ switchport access vlan 30

```

Any output at all means the two differ and a reload will lose the difference.

Fix. Re-apply the change, verify, then `copy running-config startup-config`. As a habit, run that comparison before you leave any device.

🔧 Lab 7.1 — Bring up a switch you can reach from your desk (Packet Tracer)

Topology. One switch, one PC on the same VLAN, connected by console cable and by an Ethernet cable.

Task 1. Connect by console. Move through all five modes in turn and write down the prompt at each. Use `end` to jump back from the deepest.

Task 2. Set hostname, `enable secret` and a local user.

Task 3. Give `Vlan1` an address in the PC's subnet and a default gateway. Ping the switch from the PC to prove it.

Task 4. Enable SSH properly: domain name, 2048-bit key, version 2, and `transport input ssh` on the VTY lines.

Task 5. SSH from the PC. Then try Telnet and confirm it is refused.

Task 6. Make a small change, do *not* save, reload, and observe the change vanish. Repeat with a save.

Task 7. Extension. Practise `| include`, `| section` and `| begin` on `show running-config` until you can extract a single interface block in one command without thinking.

Checkpoint — Test Yourself

1. You are at `SW1(config-if)#` and need to check the routing table without losing your place. What do you type?
2. Which configuration does a device load on boot, and which one are you editing when you type a command?
3. Why must a hostname and domain name be set before generating an RSA key?

Chapter Summary

- The prompt tells you the mode, and the mode decides which commands are legal. `end` returns to privileged EXEC from anywhere; `do` runs one EXEC command without leaving.
- Commands take effect in the running configuration immediately and are lost on reload until copied to the startup configuration.
- Verify access first, save second. An unsaved risky change is a safety net.
- `enable secret`, not `enable password`. SSH version 2 with a 2048-bit key, and `transport input ssh` to shut off Telnet.
- Filtering with `| include`, `| section` and `| begin` is what makes device output usable at speed.

Review Questions

1. (MCQ) Which prompt indicates global configuration mode? (a) `SW1>` (b) `SW1#` (c) `SW1(config)#` (d) `SW1(config-if)#`
2. (MCQ) Which command shows only the lines of the running configuration that mention a trunk? (a) `show run | begin trunk` (b) `show run | include trunk` (c) `show run | section trunk` (d) `show run trunk`
3. (Short) Explain the difference between `| include` and `| section`, with an example where the answers differ.
4. (Short) Give the professional argument for *not* saving a configuration immediately after making a change to a remote device.
5. (Applied) Write the complete configuration to take a factory-default switch to the point where you can administer it by SSH from another subnet, and state which single omission would leave Telnet available.

PART II

Switching and Network Access

This part and the next are half the exam between them. Domain 2, Switching and Network Access, is 25% on its own, and it is the domain where the blueprint stops asking you to explain things and starts asking you to *configure* them: three of its five topics begin with that word.

It is also the part where a HEC-trained student is furthest from the exam. No published HEC networking course outline in any edition names a VLAN, a trunk, an EtherChannel or a spanning tree. Everything in these seven chapters is new ground, and all of it is typed rather than read.

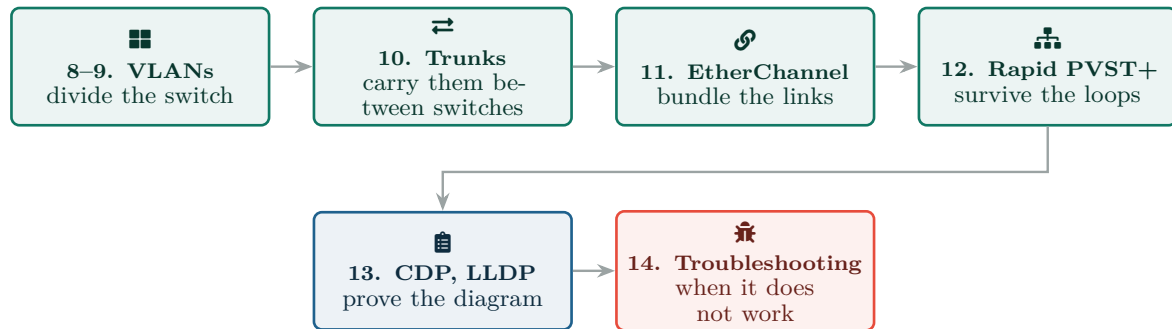


Figure P2.1: Part II. Each chapter adds one capability to the same switched network, and the last two are about proving it works and finding out why it does not.

Where the marks are in domain 2

Topic 2.1 (switch-to-switch and switch-to-router connectivity) and 2.2 (edge-host port attributes) are both *Configure* topics and between them cover trunks, EtherChannel, SVIs, VLANs and PoE. Topic 2.5 is *Configure* as well — Rapid PVST+ with PortFast and the guards. Topic 2.3 asks you to *validate* documentation with CDP and LLDP, and 2.4 to *troubleshoot* with `show` commands, ping, traceroute and capture output.

Four of the five are hands-on. Read this part with a simulator open.

VLANs and Access Ports

EXAM TOPICS **2.2**

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** what a VLAN changes about a switch, in terms of the learning-and-flooding algorithm from Chapter 3.
- **Create** VLANs and assign access ports to them.
- **Verify** VLAN membership from three different angles, and know which one to trust when they disagree.
- **Diagnose** the classic VLAN faults: a port in the wrong VLAN, a VLAN that does not exist, and a VLAN that is shut down.

🔗 Before You Start

Chapter 3 for the MAC table and flooding; Chapter 7 for moving around the command line.

📖 Key Terms in This Chapter

VLAN • broadcast domain • access port • VLAN database • default VLAN • native VLAN • management VLAN • SVI • VLAN 1 • shutdown VLAN

8.1 What a VLAN actually does

📖 VLAN

A **VLAN** is a broadcast domain implemented in software on a switch. Ports in VLAN 10 behave as though they were on a physically separate switch from ports in VLAN 20: a broadcast from one never reaches the other, and a frame cannot cross between them without passing through something that routes.

💡 One line changes in the switching algorithm

Recall the algorithm from Chapter 3: learn the source, then forward, flood, or filter. VLANs change exactly one thing — **every step is now scoped to a VLAN**.

- Learning is per VLAN. The same MAC address may legitimately appear in two VLANs on the same switch.
- Flooding is per VLAN. An unknown unicast in VLAN 10 goes out every *VLAN 10* port and no others.
- A frame never leaves the VLAN it arrived in. Not once, not for broadcast, not for anything.

That is the whole mechanism. Everything else about VLANs — trunking, inter-VLAN routing, voice VLANs — is machinery for coping with the consequences of that one rule.

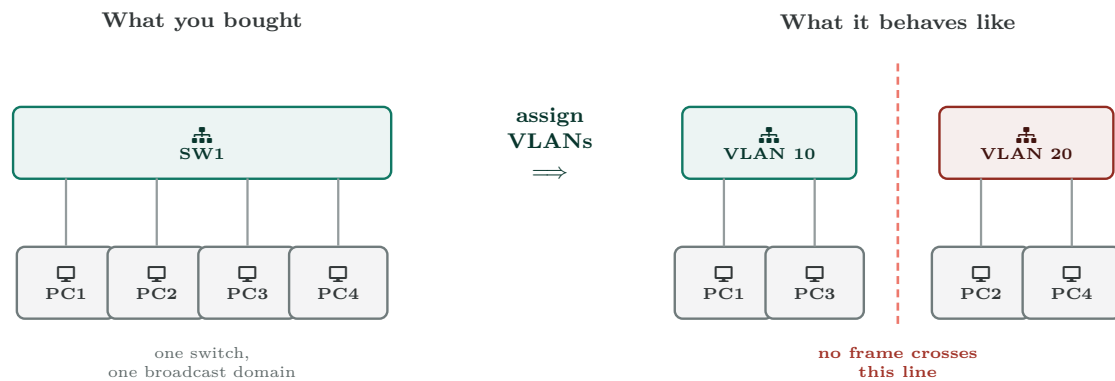


Figure 8.1: The whole idea, in one picture. Four ports on one switch become two broadcast domains that cannot reach each other — and nothing physical changed.

⚠ The consequence people forget

If two devices are in different VLANs they cannot communicate, *even if they are in the same IP subnet, plugged into adjacent ports on the same switch*. The switch will not carry the frame. Conversely two devices in the same VLAN but different subnets cannot communicate either, because IP will not deliver it.

VLAN and subnet are separate ideas that must be kept aligned by design. The convention that saves everyone trouble is one VLAN, one subnet, and a number that matches: VLAN 10 is 192.168.10.0/24, VLAN 20 is 192.168.20.0/24.

8.2 Creating VLANs and assigning ports

➤ Configure — three VLANs and the access ports that use them

```
! Create the VLANs and name them. A name is documentation that
! survives on the device -- use it.
```

```
vlan 10
  name Staff
vlan 20
  name Guest
vlan 30
  name Voice
exit
```

```
!
! Assign a single access port.
```

```
interface GigabitEthernet0/5
  description Staff desk 5
  switchport mode access
  switchport access vlan 10
  no shutdown
```

```
!
! Assign a block of ports in one go.
```

```
interface range GigabitEthernet0/6 - 12
  description Staff desks 6-12
  switchport mode access
  switchport access vlan 10
```

```
!
interface range GigabitEthernet0/13 - 18
  description Guest desks
  switchport mode access
  switchport access vlan 20
```

💡 Why switchport mode access matters

Without it the port is left in dynamic auto mode, which means it will negotiate and may become a *trunk* if the device on the other end asks for one. On an access port that is both a fault waiting to happen and a security hole — an attacker who can negotiate a trunk can reach every VLAN.

Set it explicitly. It is one line, and it is the difference between a port that does what you configured and a port that does what the far end suggests.

8.3 Verifying, three ways

🔍 SW1# show vlan brief

VLAN Name	Status	Ports
1 default	active	Gi0/1, Gi0/2, Gi0/3, Gi0/4
10 Staff	active	Gi0/5, Gi0/6, Gi0/7, Gi0/8 Gi0/9, Gi0/10, Gi0/11, Gi0/12
20 Guest	active	Gi0/13, Gi0/14, Gi0/15
30 Voice	active	
1002 fddi-default	act/unsup	

This is the view by VLAN. Note that VLAN 30 exists but has no ports — it was created and never used, which is a common and harmless state, and also a common symptom of a half-finished change.

🔍 SW1# show interfaces GigabitEthernet0/5 switchport

```
Name: Gi0/5
Switchport: Enabled
Administrative Mode: static access
Operational Mode: static access
Administrative Trunking Encapsulation: dot1q
Access Mode VLAN: 10 (Staff)
Trunking Native Mode VLAN: 1 (default)
Voice VLAN: none
```

This is the view by port, and it is the one to trust when something is odd. **Administrative** mode is what you configured; **operational** mode is what the port has actually become. When those two differ, you have found your fault.

🔍 SW1# show mac address-table vlan 10

Mac Address Table			
Vlan	Mac Address	Type	Ports
10	000a.f36e.7712	DYNAMIC	Gi0/5
10	0060.5c2b.19a4	DYNAMIC	Gi0/7

This is the view by evidence: which devices are actually present in the VLAN. If a port is in VLAN 10 and no MAC address ever appears against it, nothing is plugged in or the device is silent — and that distinction is the start of most access-layer troubleshooting.

❗ Common Pitfalls

- **Assigning a port to a VLAN that does not exist.** On many platforms the switch creates it silently; on others the port simply stops forwarding. Either way `show vlan brief` tells you within seconds.
- **Leaving everything in VLAN 1.** VLAN 1 cannot be deleted and carries control traffic. Do not put users on it, and do not use it as the management VLAN.

- **Forgetting that a VLAN can be shut down.** shutdown inside vlan 10 stops the whole VLAN forwarding on that switch while every port still shows as correctly assigned. It is an unusual fault and therefore a nasty one.
- **Deleting a VLAN and leaving ports assigned to it.** Those ports become inactive. show vlan brief will not list them at all, which is confusing until you know to look for their absence.

Break & Fix — A new desk cannot reach anything, and the port looks fine.

A user on Gi0/9 has no connectivity. The port is up, the cable is good, and the configuration reads exactly like the working port next to it.

```
SW1# show vlan brief | include Staff|Gi0/9
```

```
10   Staff                                active    Gi0/5, Gi0/6, Gi0/7, Gi0/8
```

Gi0/9 is not listed against VLAN 10 — nor against any other VLAN.

```
SW1# show interfaces GigabitEthernet0/9 switchport
```

```
Name: Gi0/9
Switchport: Enabled
Administrative Mode: static access
Operational Mode: static access
Access Mode VLAN: 15 (Inactive)
```

Diagnosis. The port was assigned to VLAN 15, which does not exist on this switch — most likely a typing slip for 10, or a VLAN that was deleted after the port was configured. The word (Inactive) is the whole diagnosis: the port is administratively correct and operationally orphaned.

Fix. Either create VLAN 15 if it was meant to exist, or reassign the port: switchport access vlan 10. Confirm with show vlan brief that Gi0/9 now appears in the Staff list, then confirm a MAC address learns against it.

Lab 8.1 — Divide one switch into three networks (Packet Tracer)

Topology. One switch, six PCs — two per VLAN — all in the same IP subnet to begin with.

Task 1. With every port in VLAN 1, confirm all six PCs can ping each other. Record the MAC address table.

Task 2. Create VLANs 10, 20 and 30 with names, and move two PCs into each.

Task 3. Re-test. Which pings now succeed and which fail? Explain each result in terms of broadcast domains, not IP.

Task 4. Run all three verification commands from this chapter and reconcile them.

Task 5. Assign Gi0/11 to VLAN 99, which you have not created. Record what show vlan brief and show interfaces switchport each say.

Task 6. Enter vlan 20 and type shutdown. Confirm the ports still look correctly assigned while nothing passes. Undo it.

Task 7. Extension. Re-address the PCs so each VLAN is its own subnet. Note that inter-VLAN pings still fail — and that fixing it needs Chapter 10.

Checkpoint — Test Yourself

1. Two PCs are in the same IP subnet, plugged into the same switch, in VLANs 10 and 20. Can they ping each other? Justify your answer at layer 2.

2. Which command distinguishes what you configured on a port from what the port has actually become?
3. A port shows `Access Mode VLAN: 15 (Inactive)`. State the two possible causes.

Chapter Summary

- A VLAN is a broadcast domain in software. Learning, flooding and forwarding all become per-VLAN operations.
- A frame never leaves its VLAN without being routed.
- Keep VLAN and subnet aligned one-to-one; mismatches produce faults that look like everything and nothing.
- Always set `switchport mode access` explicitly, or the port may negotiate a trunk.
- Verify three ways: `show vlan brief` by VLAN, `show interfaces switchport` by port, and the MAC table by evidence. Administrative versus operational mode is where faults show.

Review Questions

1. **(MCQ)** Which command assigns an access port to VLAN 10? (a) `vlan 10` (b) `switchport access vlan 10` (c) `switchport trunk vlan 10` (d) `ip vlan 10`
2. **(MCQ)** A port reports `Administrative Mode: dynamic auto` and `Operational Mode: trunk`. What has happened? (a) it was configured as a trunk (b) it negotiated a trunk with the far end (c) the VLAN is inactive (d) the port is errdisabled
3. **(Short)** Explain why leaving a port in dynamic auto is a security problem as well as an operational one.
4. **(Short)** A VLAN exists, ports are assigned, and nothing forwards. Name a cause that `show vlan brief` would reveal and one it would not.
5. **(Applied)** You are asked to segment a 24-port switch for staff, guests and printers, with guests unable to reach the other two. Write the configuration, and state what additional device or feature you would need for the isolation requirement to hold once routing is added.

Edge-Host Connectivity: PoE, Voice, APs and IoT

EXAM TOPICS 2.2

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Configure** an access port correctly for each of the five device types the blueprint names.
- **Explain** how a data VLAN and a voice VLAN share one cable, and why that is not a trunk in the ordinary sense.
- **Configure** and verify Power over Ethernet, including budget and per-port limits.
- **Diagnose** the faults peculiar to edge ports: a phone that gets no power, an AP that sees only one VLAN, and a device in the wrong VLAN because it was plugged into the wrong socket.

🔗 Before You Start

Chapter 8 for VLANs and access ports. Chapter 7 for interface ranges.

📖 Key Terms in This Chapter

Power over Ethernet • PoE, PoE+, UPoE • power budget • voice VLAN • data VLAN • standalone AP • controller-based AP • LWAPP/CAPWAP • IoT appliance • errdisable

📖 Read topic 2.2 carefully — it lists the devices

Topic 2.2 is “configure Layer 2 switch port attributes for edge-host connectivity (VLAN, PoE, port channel, and LACP)” and then names five categories: desktops, printers and IoT appliances; wireless access points, standalone and controller-based; VoIP phones; virtualised hosts; and network appliances. The exam expects a different port configuration for several of them, so learn them as five cases rather than one.

9.1 Power over Ethernet

📖 PoE

Power over Ethernet delivers electrical power over the same twisted pair that carries data, so a phone, camera or access point needs one cable rather than two. The switch negotiates with the device and supplies only what it asks for.

Standard	Per port	Name	Typically powers
802.3af	15.4 W	PoE	IP phone, basic AP, door sensor
802.3at	30 W	PoE+	Modern AP, PTZ camera
802.3bt	60–90 W	UPoE, PoE++	Multi-radio AP, display, thin client

⚠ The budget is the constraint, not the port

A switch has a total power budget shared across all ports — often far less than the number of ports times the maximum per port. A 24-port PoE+ switch with a 370 W budget cannot deliver 30 W to all 24. When the budget is exhausted the switch stops powering *new* devices, so the symptom is “the phone I plugged in this morning works, the one I plugged in this afternoon does not”, and nothing in the port configuration looks wrong. Check `show power inline` before you check anything else.

➤ Configure — PoE on an access port

```
interface GigabitEthernet0/10
  description AP in the east corridor
  switchport mode access
  switchport access vlan 10
  ! auto is the default: negotiate and give the device what it asks for.
  power inline auto
  ! Cap a port that must never take more than a phone's worth.
  ! power inline auto max 15400
  no shutdown
!
! Never power this port -- a data-only port that must not energise.
interface GigabitEthernet0/24
  power inline never
```

🔍 SW1# show power inline

Available:370.0(w) Used:71.5(w) Remaining:298.5(w)

Interface	Admin	Oper	Power (Watts)	Device	Class	Max
Gi0/1	auto	on	6.3	IP Phone 7960	2	15.4
Gi0/2	auto	on	6.3	IP Phone 7960	2	15.4
Gi0/10	auto	on	28.6	AIR-AP2802I	4	30.0
Gi0/11	auto	off	0.0	n/a	n/a	30.0
Gi0/24	never	off	0.0	n/a	n/a	0.0

Read the top line first. **Remaining** is what decides whether the next device you plug in will power up at all.

9.2 Voice VLANs: two VLANs, one cable

An IP phone has a small switch inside it: the cable from the wall goes into the phone, and the PC plugs into the back of the phone. Both need network access, and they should not be in the same VLAN — voice needs its own subnet for quality and for security.

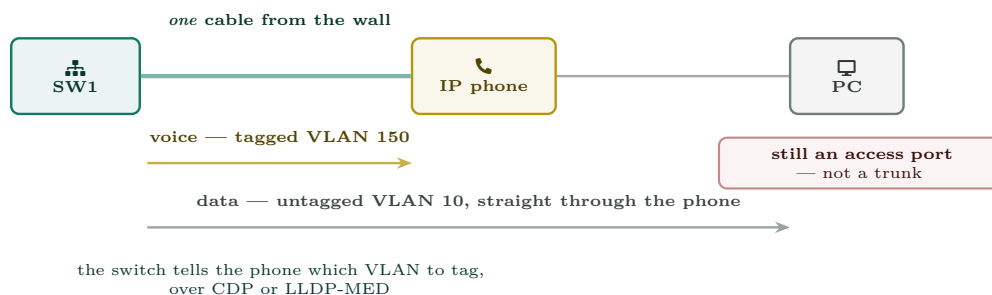


Figure 9.1: One cable, two VLANs, and no trunk. The phone tags its own traffic and passes the PC's through untouched.

💡 How one access port carries two VLANs

The port is configured with a **data VLAN** and a **voice VLAN**. The switch tells the phone, over CDP or LLDP-MED, which VLAN ID to tag its own traffic with. The phone tags voice frames and passes the PC's frames through untagged.

So the switch sees tagged voice traffic and untagged data traffic on one access port. Strictly this is a trunk carrying two VLANs, but it is configured as an access port and Cisco calls it a **multi-VLAN access port**. Do not configure `switchport mode trunk` here — the voice VLAN command does what is needed, and a real trunk would change how the PC's untagged traffic is handled.

➤ Configure — a desk port for a phone with a PC behind it

```
interface GigabitEthernet0/1
description Desk 1 - phone with PC daisy-chained
switchport mode access
switchport access vlan 10
switchport voice vlan 30
! The PC should forward immediately rather than waiting for
! spanning tree -- see chapter on Rapid PVST+.
spanning-tree portfast
power inline auto
no shutdown
```

🔍 SW1# show interfaces GigabitEthernet0/1 switchport

```
Name: Gi0/1
Switchport: Enabled
Administrative Mode: static access
Operational Mode: static access
Access Mode VLAN: 10 (Staff)
Voice VLAN: 30 (Voice)
```

9.3 Access points, standalone and controller-based

The blueprint distinguishes the two, and the distinction changes the port configuration completely.

	Standalone (autonomous)	Controller-based (lightweight)
Where the config lives	On the AP itself	On the wireless LAN controller
What crosses the wire	Each SSID's traffic in its own VLAN	All client traffic tunnelled to the controller in CAPWAP
Switch port	Trunk , carrying every SSID's VLAN	Access port , in the AP management VLAN
Scales to	A handful of APs	Hundreds

⚠ The most common AP misconfiguration

Configuring a lightweight AP's port as a trunk, or a standalone AP's port as an access port. Both leave the AP reachable — it will get an address and appear healthy — while clients on some or all SSIDs get nothing. If wireless users on one SSID work and another SSID does not, look at the switch port before you touch the wireless configuration.

➤ Configure — both kinds of access point

```
! Lightweight AP: one access port in the AP management VLAN.
! Client traffic is tunnelled to the controller, so no client
```

```

! VLANs need to exist on this switch at all.
interface GigabitEthernet0/10
  description Lightweight AP - CAPWAP to WLC
  switchport mode access
  switchport access vlan 50
  power inline auto
  spanning-tree portfast
!
! Standalone AP: a trunk, because the AP bridges each SSID
! into a different VLAN and tags them itself.
interface GigabitEthernet0/11
  description Standalone AP - SSIDs in VLANs 10 and 20
  switchport trunk encapsulation dot1q
  switchport mode trunk
  switchport trunk native vlan 50
  switchport trunk allowed vlan 10,20,50
  power inline auto

```

9.4 Printers, IoT appliances and virtualised hosts

- **Printers and IoT appliances** — ordinary access ports. Put them in their own VLAN: they are rarely patched, often speak plaintext protocols, and are a favourite foothold. Add port security (Chapter 25) because they never move.
- **Virtualised hosts** — a server running virtual machines usually needs a *trunk*, because each VM may sit in a different VLAN and the hypervisor’s virtual switch tags accordingly. This is the one “end device” that is not an access port.
- **Network appliances** — firewalls, load balancers and the like. Read the vendor’s requirement; most want a trunk, some want several access ports.

! Common Pitfalls

- **PortFast on a port that leads to a switch.** PortFast skips the spanning tree listening and learning states. On an edge port that is correct; on a port leading to another switch it can create a loop. Use BPDU guard alongside it (Chapter 12).
- **Assuming the phone’s PC port is in the voice VLAN.** It is not — untagged traffic from the PC lands in the data VLAN, which is the whole point.
- **Ignoring the power class.** A class 4 device on a switch with only 802.3af ports will not power up, however correct the configuration looks.
- **Trunking to a lightweight AP.** See the warning above.

🔧 Break & Fix — Half the wireless users work; the other half get no address.

Users on the Staff SSID are fine. Users on Guest associate to the wireless network but never receive an IP address. The AP is reachable and its configuration has not changed.

Q SW1# show interfaces GigabitEthernet0/11 trunk

Port	Mode	Encapsulation	Status	Native vlan
Gi0/11	on	802.1q	trunking	50
Port	Vlans allowed on trunk			
Gi0/11	10,50			

Diagnosis. This is a standalone AP, correctly trunked, but the allowed VLAN list carries 10 and 50 only. VLAN 20 — the Guest SSID’s VLAN — is not permitted on the trunk, so Guest clients associate at the radio (which the AP handles alone) and then have no path to

the DHCP server. Association succeeding while addressing fails is the signature of a VLAN that does not reach the wire.

Fix. Add the missing VLAN to the trunk: `switchport trunk allowed vlan add 20`. Note `add` — without it you replace the whole list and drop VLANs 10 and 50, taking down the users who were working. Confirm with `show interfaces trunk`, then have a Guest client renew.

Lab 9.1 — Wire an edge properly (*Packet Tracer*)

Topology. One PoE switch; an IP phone with a PC behind it; a lightweight AP; a printer.

Task 1. Create VLANs 10 (Staff), 20 (Guest), 30 (Voice), 50 (AP-Mgmt) and 60 (Printers).

Task 2. Configure the phone port with data VLAN 10, voice VLAN 30, PortFast and PoE. Verify both VLANs appear on the one port.

Task 3. Configure the AP port as an access port in VLAN 50 with PoE.

Task 4. Configure the printer port in VLAN 60 with PortFast.

Task 5. Run `show power inline` and record the budget, the used figure and the class of each powered device.

Task 6. Set `power inline never` on the printer port and observe what happens to a PoE device moved there.

Task 7. Extension. Change the AP to standalone: reconfigure its port as a trunk carrying VLANs 10, 20 and 50, and explain in one sentence why the two AP types need opposite port configurations.

Checkpoint — Test Yourself

1. An IP phone powers up and works; a PC plugged into the back of it gets an address in VLAN 10. Which VLAN carries the phone's own traffic, and how did the phone learn to use it?
2. A new AP will not power on. `show power inline` shows the port `auto/off` and `Remaining:4.2(w)`. What is wrong?
3. Why does a lightweight AP need only one VLAN on its switch port when it serves four SSIDs?

Chapter Summary

- PoE classes: 15.4 W af, 30 W at, up to 90 W bt. The shared *budget* is what actually runs out, so read `show power inline` first.
- A voice VLAN plus a data VLAN on one access port lets a phone and a PC share a cable: the phone tags, the PC does not.
- Lightweight APs take an access port in the management VLAN; standalone APs take a trunk carrying every SSID's VLAN. Getting this backwards breaks some SSIDs and not others.
- Printers and IoT devices belong in their own VLAN with port security. Virtualised hosts usually need a trunk.
- Use `switchport trunk allowed vlan add`, never bare `allowed vlan`, on a live trunk.

Review Questions

1. (MCQ) Which port configuration suits a controller-based access point? (a) trunk with all client VLANs (b) access port in the AP management VLAN (c) routed port (d) access port in VLAN 1

2. **(MCQ)** Which command stops a port ever supplying power? (a) `power inline auto` (b) `power inline never` (c) `no power inline` (d) `shutdown`
3. **(Short)** Explain why a port with a voice VLAN is not configured as a trunk, even though it carries two VLANs.
4. **(Short)** A device on a PoE port shows `Oper: off` while other ports are powering normally. Give two causes and the command that distinguishes them.
5. **(Applied)** You must connect twelve IP phones with PCs behind them, four PoE+ access points and six printers to one 24-port switch with a 370 W budget. Estimate whether the budget suffices, state your assumptions, and write the configuration for one port of each type.

Trunking, 802.1Q and SVIs

EXAM TOPICS **2.1**

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** what an 802.1Q tag is and why the native VLAN is the exception that causes most trunk faults.
- **Configure** a trunk between two switches and between a switch and a router, and prune it to the VLANs it should carry.
- **Configure** switch virtual interfaces so that a multilayer switch routes between VLANs.
- **Diagnose** the four trunk faults: mode mismatch, encapsulation mismatch, native VLAN mismatch and a missing VLAN in the allowed list.

🔗 Before You Start

Chapter 8 for VLANs and access ports; Chapter 4 for addressing, which the SVI section needs.

📖 Key Terms in This Chapter

trunk • 802.1Q • tag • native VLAN • allowed VLAN list • DTP • dynamic auto • dynamic desirable • SVI • inter-VLAN routing • router on a stick • subinterface

10.1 The tag, and the one VLAN that has none

A single link between two switches must carry traffic for every VLAN, and the receiving switch has to know which VLAN each frame belongs to. **802.1Q** solves this by inserting a four-byte tag into the frame carrying the VLAN ID.

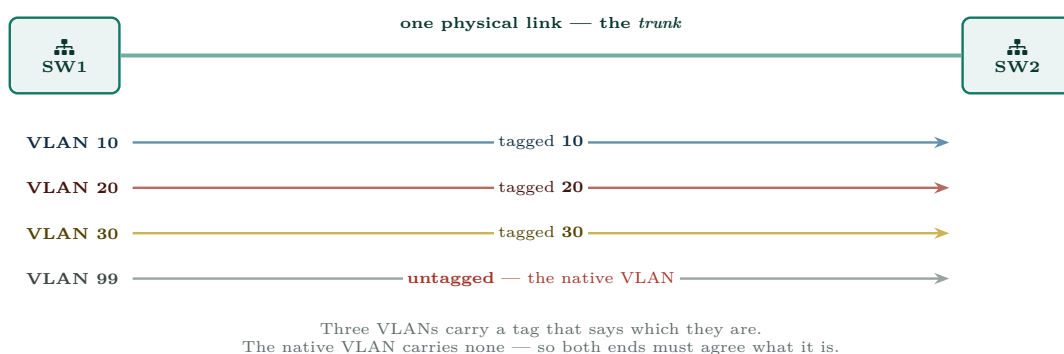


Figure 10.1: A trunk is one cable carrying many VLANs. Every VLAN but one is labelled; the unlabelled one is the native VLAN, and a disagreement about it joins two networks silently.

💡 The native VLAN is untagged, and that is the whole problem

One VLAN on every trunk is designated the **native VLAN**, and its frames cross the trunk **untagged**. The receiving switch assumes any untagged frame it sees on a trunk belongs to *its own* native VLAN.

If the two ends disagree about which VLAN is native, traffic silently leaks between VLANs: frames sent untagged in VLAN 1 by one switch are received into VLAN 99 by the other. Nothing goes down, nothing logs an error at layer 2, and two VLANs that should be isolated are now joined. CDP will complain, which is one of several reasons to leave it on (Chapter 13). Set the native VLAN explicitly and identically at both ends, and make it an unused VLAN — never VLAN 1, and never a VLAN carrying users.

>_ Configure — a trunk between two switches, configured deliberately

```
! On BOTH ends, identically.
interface GigabitEthernet0/1
  description Uplink to SW2
  ! Some platforms require the encapsulation before the mode; on
  ! switches that only support dot1q the command is rejected as
  ! unavailable, which is not an error.
  switchport trunk encapsulation dot1q
  switchport mode trunk
  ! An unused VLAN as native, matched at the far end.
  switchport trunk native vlan 999
  ! Carry only what this link needs. Pruning limits broadcast
  ! flooding and limits the blast radius of a mistake.
  switchport trunk allowed vlan 10,20,30
  ! Do not negotiate -- we have decided what this port is.
  switchport nonegotiate
  no shutdown
```

Q SW1# show interfaces trunk

Port	Mode	Encapsulation	Status	Native vlan
Gi0/1	on	802.1q	trunking	999
Port	Vlans allowed on trunk			
Gi0/1	10,20,30			
Port	Vlans allowed and active in management domain			
Gi0/1	10,20,30			
Port	Vlans in spanning tree forwarding state and not pruned			
Gi0/1	10,20,30			

Four blocks, and they narrow as they go. A VLAN can be *allowed* but not *active* (it does not exist on this switch), or active but not *forwarding* (spanning tree is blocking it). When a VLAN is missing from the last block but present in the first, you have found the layer at which it is being lost.

⚠ allowed vlan replaces; allowed vlan add appends

Typing `switchport trunk allowed vlan 40` on a live trunk does not add VLAN 40 — it makes 40 the *only* allowed VLAN and instantly drops every other VLAN on that link. On a production uplink this is an outage.

Use `switchport trunk allowed vlan add 40`. The equivalents are `remove`, `except` and `all`.

10.2 DTP, and why to turn it off

Dynamic Trunking Protocol lets two Cisco switches negotiate whether a link becomes a trunk. It is convenient and it is a liability.

Local mode	vs trunk	vs dynamic desirable	vs dynamic auto / access
trunk	trunk	trunk	trunk / mismatch
dynamic desirable	trunk	trunk	trunk / access
dynamic auto	trunk	trunk	access / access
access	mismatch	access	access / access

The two entries worth memorising are the failures. `dynamic auto` on both ends gives you an **access port**, not a trunk — each side waits for the other to ask. And `trunk` facing `access` is a genuine mismatch that passes only the access VLAN.

💡 Configure both ends; negotiate nothing

Set `switchport mode trunk` and `switchport nonegotiate` on uplinks, and `switchport mode access` on edge ports. Then a port is what you said it is, and an attacker who plugs a laptop into a wall socket cannot negotiate himself a trunk into every VLAN.

10.3 Routing between VLANs

A VLAN is a broadcast domain and traffic cannot leave it without being routed. There are two ways to provide that, and the exam names the modern one.

10.3.1 Switch virtual interfaces — the modern way

An **SVI** is a virtual layer 3 interface inside a multilayer switch, representing one VLAN. Give each VLAN an SVI with an address and the switch routes between them at hardware speed.

>_ Configure — a multilayer switch routing three VLANs

```
! Routing must be switched on -- easily forgotten, and without it the
! SVIs come up and nothing is routed between them.
ip routing
!
interface Vlan10
  description Staff gateway
  ip address 192.168.10.1 255.255.255.0
  no shutdown
!
interface Vlan20
  description Guest gateway
  ip address 192.168.20.1 255.255.255.0
  no shutdown
!
interface Vlan30
  description Voice gateway
  ip address 192.168.30.1 255.255.255.0
  no shutdown
```

⚠️ What it takes to bring an SVI up

An SVI shows up/up only when *all* of the following hold:

- the VLAN exists in the VLAN database and is not shut down;
- at least one access port in that VLAN is up, **or** the VLAN is allowed and forwarding on a trunk;
- the SVI itself is not shut down.

An SVI that stays down with a perfectly good address almost always means the second condition: no live port in that VLAN. It is not an addressing fault.

Q SW1# show ip interface brief | include Vlan

Vlan10	192.168.10.1	YES	manual	up	up
Vlan20	192.168.20.1	YES	manual	up	up
Vlan30	192.168.30.1	YES	manual	up	down

VLAN 30's SVI is up administratively and down at protocol level — no active port in VLAN 30. Plug in a phone, or allow VLAN 30 on a trunk, and it comes up.

10.3.2 Router on a stick — the older way

Where the switch cannot route, one router interface carries every VLAN over a trunk, divided into **subinterfaces**.

>_ Configure — router on a stick

```
interface GigabitEthernet0/0
  description Trunk to SW1
  no ip address
  no shutdown
!
interface GigabitEthernet0/0.10
  description Staff
  encapsulation dot1Q 10
  ip address 192.168.10.1 255.255.255.0
!
interface GigabitEthernet0/0.20
  description Guest
  encapsulation dot1Q 20
  ip address 192.168.20.1 255.255.255.0
!
! The native VLAN's subinterface is marked, not tagged.
interface GigabitEthernet0/0.999
  encapsulation dot1Q 999 native
```

Every VLAN's traffic crosses the one physical link twice — in and out — so this scales badly. It appears in the exam because it still appears in small networks.

! Common Pitfalls

- **Native VLAN mismatch.** The two ends disagree, VLANs leak, and nothing goes down. CDP logs it; look for %CDP-4-NATIVE_VLAN_MISMATCH.
- **Bare allowed vlan on a live trunk.** Replaces the list. Always add.
- **Forgetting ip routing on a multilayer switch.** The SVIs come up and traffic still will not cross between VLANs.
- **Expecting an SVI to be up with no active port in its VLAN.** It will not be, and no amount of re-addressing changes that.
- **Leaving VLAN 1 as native.** It carries control traffic and it is every attacker's first guess.

🔧 Break & Fix — One VLAN crosses the uplink; the other three do not.

After a maintenance window, users in VLAN 10 are fine and users in 20, 30 and 40 cannot reach the servers. Both uplink ports show as trunking.

Q SW2# show interfaces trunk

Port	Mode	Encapsulation	Status	Native vlan
Gi0/1	on	802.1q	trunking	999
Port	Vlans allowed on trunk			
Gi0/1	10			

Diagnosis. The trunk is healthy; the allowed list has been reduced to VLAN 10 alone. Somebody ran `switchport trunk allowed vlan 10` intending to add VLAN 10 to the existing list, and replaced it instead. The link stayed up throughout, which is why nothing alerted.

Fix. Restore the full list explicitly rather than adding one VLAN at a time: `switchport trunk allowed vlan 10,20,30,40`. Verify with `show interfaces trunk` on *both* ends — the allowed list is configured per side, and the effective set is the intersection, so a matching mistake at the far end would leave the fault half-fixed and twice as confusing.

Lab 10.1 — Trunk two switches and route between VLANs (*Packet Tracer*)

Topology. Two switches trunked together; a multilayer switch or router providing gateways; two PCs per VLAN across both switches.

Task 1. Create VLANs 10, 20 and 999 on both switches. Put PCs in 10 and 20 on each switch.

Task 2. Before configuring the trunk, try to ping between the two switches' VLAN 10 PCs. Record the failure and explain it.

Task 3. Configure the trunk on both ends with native VLAN 999 and an allowed list of 10, 20 and 999. Re-test.

Task 4. Change the native VLAN on one end only to 998. Record what still works, what breaks, and what the log says.

Task 5. Restore it, then run `switchport trunk allowed vlan 20` on one end and observe the outage you have just caused. Fix it with `add`.

Task 6. Enable `ip routing` and configure SVIs for VLANs 10 and 20. Prove inter-VLAN routing works.

Task 7. Extension. Shut every access port in VLAN 20 and watch the SVI go down. Explain why in terms of the three conditions above.

Checkpoint — Test Yourself

1. Both ends of a link are set to `dynamic auto`. What does the link become, and why?
2. An SVI has a correct address and shows `up/down`. What is the most likely cause and how would you confirm it?
3. Why is a native VLAN mismatch more dangerous than a trunk that fails to come up at all?

Chapter Summary

- 802.1Q tags every VLAN except the native one, which crosses untagged. Mismatched native VLANs leak traffic between VLANs silently.
- Configure both ends explicitly and add `switchport nonegotiate`. `dynamic auto` on both ends yields an access port, not a trunk.
- `allowed vlan` replaces the list; `allowed vlan add` appends. This distinction causes real outages.

- `show interfaces trunk` narrows from allowed to active to forwarding; the block where a VLAN disappears tells you the layer at fault.
- An SVI needs `ip routing`, an existing unshut VLAN, and at least one live port in that VLAN.

Review Questions

1. **(MCQ)** Which command adds VLAN 40 to a trunk without disturbing the VLANs already carried? (a) `switchport trunk allowed vlan 40` (b) `switchport trunk allowed vlan add 40` (c) `switchport trunk vlan 40` (d) `switchport access vlan 40`
2. **(MCQ)** A trunk's two ends have different native VLANs. What happens? (a) the trunk fails to form (b) both ends shut down (c) untagged traffic is bridged between two VLANs (d) all VLANs are pruned
3. **(Short)** Explain why an SVI can show up/down when its address and mask are correct.
4. **(Short)** Give two reasons to set the native VLAN to an unused VLAN rather than leaving it as VLAN 1.
5. **(Applied)** A colleague reports that after adding a new VLAN, three existing VLANs stopped working across an uplink. State what they almost certainly typed, what they should have typed, and how you would verify the repair on both ends.

EtherChannel and LACP

EXAM TOPICS 2.1

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** why bundling links needs a protocol rather than just more cables, in terms of what spanning tree would otherwise do.
- **Configure** a layer 2 EtherChannel with LACP, and a layer 3 EtherChannel between routed ports.
- **Verify** a bundle and read the flags in `show etherchannel summary`.
- **Diagnose** the parameter mismatches that stop a bundle forming — the single most common EtherChannel fault.

🔗 Before You Start

Chapter 10 for trunks and allowed VLAN lists; Chapter 8 for access ports.

📖 Key Terms in This Chapter

EtherChannel • port channel • LACP • PAgP • active • passive • on • load balancing • bundle • suspended port • layer 3 EtherChannel

11.1 Why bundling needs a protocol

Run two cables between a pair of switches and you do not get twice the bandwidth. You get a loop, and spanning tree (Chapter 12) blocks one of them — so you have paid for two links and can use one.

📄 EtherChannel

An **EtherChannel** bundles two to eight physical links into one logical interface, the **port channel**. Spanning tree sees a single link and does not block any member, so all of the bandwidth is usable, and if one member fails the others carry on without a topology change.

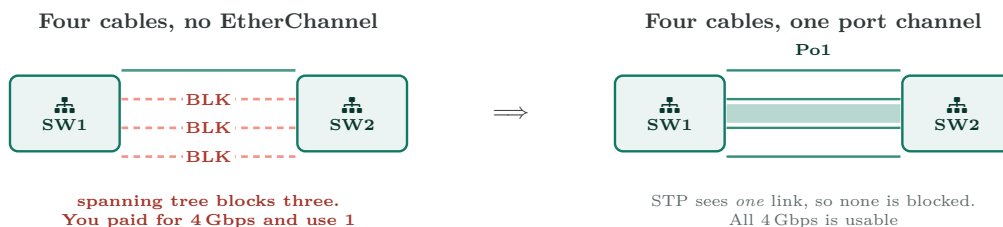


Figure 11.1: Bundling is not about speed for one conversation — it is about spanning tree seeing one link instead of four, and therefore blocking none of them.

💡 Load balancing is per conversation, not per packet

Frames are distributed across members by hashing fields from the frame — source MAC, destination MAC, source and destination IP, or a combination. Every frame in one conversation hashes the same way, so a conversation always takes one member link.

The consequence catches people out: a single large file transfer between two hosts gets *one member's worth* of bandwidth, not the whole bundle. Four 1 Gbps links do not give one server a 4 Gbps transfer. They give you 4 Gbps of aggregate capacity across many conversations, which is a different thing. When somebody complains that a new EtherChannel “did not make the backup faster”, this is why.

11.2 The negotiation protocols

Protocol	Modes	Forms a bundle with	Notes
LACP 802.3ad	active, passive	active–active, active–passive. Not passive–passive	Open standard. Use this. Passive waits to be asked; two waiters never speak
PAgP	desirable, auto	desirable–desirable, desirable–auto. Not auto–auto	Cisco proprietary. Same trap: two auto ends form nothing
Static	on	on–on only	No negotiation and no safety net. A mismatch here creates a loop rather than failing cleanly

⚠️ Never mix on with a negotiating mode

on at one end and active at the other does not form a bundle, and worse, the on end starts forwarding on every member immediately while the far end still treats them as individual links. That is a loop, built by hand. Either both ends negotiate, or both ends are on. LACP active at both ends is the right default.

11.3 Configuring a layer 2 EtherChannel

💡 The rule that governs everything: members must match

Every member port in a bundle, *at both ends*, must agree on:

- speed and duplex;
- switch mode — all access, or all trunk;
- if trunking: the same native VLAN and the same allowed VLAN list;
- if access: the same access VLAN.

One port that disagrees is suspended and stays out of the bundle. Configure the port channel interface and let the settings inherit downward, rather than configuring each member separately — that is how you avoid the mismatch in the first place.

➤ Configure — a two-member LACP trunk bundle, both ends

```
! Put the physical members into the channel group first. Creating
! the group creates the Port-channel interface automatically.
interface range GigabitEthernet0/1 - 2
description Member of Po1 - uplink to SW2
shutdown
```

```

channel-group 1 mode active
no shutdown
exit
!
! Now configure the logical interface. Settings applied here
! propagate to every member, which is what keeps them matching.
interface Port-channel1
description Uplink bundle to SW2
switchport trunk encapsulation dot1q
switchport mode trunk
switchport trunk native vlan 999
switchport trunk allowed vlan 10,20,30
!
! Hash on both source and destination IP for a better spread than
! the default source-MAC-only on many platforms.
port-channel load-balance src-dst-ip

```

Q SW1# show etherchannel summary

```

Flags:  D - down          P - bundled in port-channel
        I - stand-alone  s - suspended
        H - Hot-standby  (LACP only)
        R - Layer3       S - Layer2
        U - in use       f - failed to allocate aggregator
        M - not in use, minimum links not met
        u - unsuitable for bundling
        w - waiting to be aggregated
        d - default port

```

```

Number of channel-groups in use: 1
Number of aggregators:          1

```

Group	Port-channel	Protocol	Ports
1	Po1(SU)	LACP	Gi0/1(P) Gi0/2(P)

Read the flags. Po1(SU) means layer **S**witched and **U**sed — healthy. Each member showing (P) is bundled. The ones to worry about are (I) stand-alone, (s) suspended and (D) down.

Flag	What it is telling you
(P)	Bundled and forwarding. What you want
(I)	Stand-alone — the port is up but is not in the bundle. Usually the far end is not running a compatible mode
(s)	Suspended — the port's parameters do not match the others. Compare speed, duplex, mode, native VLAN and allowed list
(D)	Down. A physical problem; go back to Chapter 2
(SU)	On the port channel: layer 2, in use
(RU)	On the port channel: layer 3, in use

11.4 A layer 3 EtherChannel

Between two multilayer switches you may want the bundle routed rather than switched.

>_ Configure — a routed bundle

```

interface Port-channel2
description Routed bundle to Core2
! no switchport makes this a routed interface -- do it BEFORE
! adding an address, or the address command is rejected.

```

```

no switchport
ip address 10.255.0.1 255.255.255.252
!
interface range GigabitEthernet0/23 - 24
no switchport
channel-group 2 mode active
no shutdown

```

! Common Pitfalls

- **Passive–passive or auto–auto.** Neither end initiates and no bundle forms. The ports stay up as individual links, which spanning tree then blocks — so you get a working network and half the bandwidth you paid for.
- **Mismatched allowed VLAN lists.** A favourite: the bundle forms, one VLAN is missing from one end, and only that VLAN breaks.
- **Configuring members individually.** Configure interface Port-channel1; let it inherit.
- **Adding no switchport after the IP address.** On a layer 3 bundle the order matters.
- **Expecting one flow to use the whole bundle.** It never will.

🔧 Break & Fix — The bundle formed with one member instead of four.

A new four-link bundle shows only Gi0/1 bundled; the other three are suspended.

Q SW1# show etherchannel summary | begin Group

Group	Port-channel	Protocol	Ports
1	Po1(SU)	LACP	Gi0/1(P) Gi0/2(s) Gi0/3(s) Gi0/4(s)

Q SW1# show interfaces GigabitEthernet0/2 switchport | include Access|Trunking|Mode

```

Administrative Mode: static access
Operational Mode: static access
Access Mode VLAN: 1 (default)
Trunking Native Mode VLAN: 1 (default)

```

Diagnosis. Gi0/1 was configured as a trunk before being added to the group, and the others were left as access ports in VLAN 1. Members must agree on switch mode, so the three that disagree with the first are suspended. The (s) flag is the whole diagnosis; the second command identifies which parameter differs.

Fix. Do not fix the members one by one — that invites the same fault again. Apply the trunk configuration to interface Port-channel1 and let it propagate, then confirm all four show (P). If a member still refuses, compare speed and duplex next.

🔧 Lab 11.1 — Bundle four links and break them (*Packet Tracer*)

Topology. Two switches joined by four Ethernet links; one PC on each.

Task 1. Cable all four links with no EtherChannel configured. Run `show spanning-tree` and record how many are forwarding and how many are blocking. Explain the result.

Task 2. Configure Po1 with LACP active at both ends, trunked, native VLAN 999, allowed 10 and 20. Verify with `show etherchannel summary`.

Task 3. Re-run `show spanning-tree`. How many logical links does it now see?

Task 4. Shut one member. Confirm traffic continues and that spanning tree does not reconverge.

- Task 5.** Set one member's access VLAN to 20 by configuring the physical port directly. Record the flag it earns and explain it.
- Task 6.** Change one end to `passive` while the other is also `passive`. Record what the bundle does.
- Task 7. Extension.** Build a layer 3 EtherChannel between two multilayer switches with a /30 and prove you can ping across it.

🔍 Checkpoint — Test Yourself

1. Two switches are configured `channel-group 1 mode passive` at both ends. What happens, and what does spanning tree do about it?
2. A member port shows (s). Name three parameters you would compare and the command that shows them.
3. Why does adding a second member to a bundle not double the speed of a single file copy?

📋 Chapter Summary

- An EtherChannel makes several links look like one, so spanning tree blocks none of them.
- LACP `active` at both ends is the sensible default. `Passive-passive` and `auto-auto` form nothing; `on` mixed with a negotiating mode builds a loop.
- Members must match on speed, duplex, switch mode, native VLAN and allowed list. Configure the port channel interface and let it inherit.
- Read `show etherchannel summary` by its flags: (P) bundled, (s) suspended for a mismatch, (I) stand-alone, (D) down.
- Load balancing is per conversation. One flow uses one member.

🔧 Review Questions

1. (MCQ) Which LACP mode combination forms a bundle? (a) `passive/passive` (b) `active/passive` (c) `auto/auto` (d) `on/active`
2. (MCQ) A member shows the (s) flag. What does it mean? (a) it is bundled (b) it is suspended because its parameters differ (c) it is down (d) it is a layer 3 port
3. (Short) Explain why mixing `on` with `active` is more dangerous than a combination that simply fails to bundle.
4. (Short) A backup job runs at 1 Gbps across a four-member 1 Gbps bundle. Explain to the person who ordered the extra links why this is expected behaviour and what would actually help.
5. (Applied) Write the full configuration, both ends, for a three-member LACP trunk carrying VLANs 10, 20 and 30 with native VLAN 999, and list the four `show` outputs you would capture to prove it is healthy.

Rapid PVST+ and the Protection Features

EXAM TOPICS 2.5

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** what a layer 2 loop does to a network, and why it is worse than any layer 3 failure.
- **Predict** the root bridge, root ports and designated ports for a given topology, and **influence** the outcome deliberately.
- **Configure** Rapid PVST+, root primary and secondary, PortFast, BPDU guard, root guard and loop guard.
- **Diagnose** an unexpected root bridge, an errdisabled port and a port that will not leave blocking.

🔗 Before You Start

Chapter 3 for flooding — which is what makes a loop catastrophic — and Chapter 10 for trunks between switches.

📖 Key Terms in This Chapter

BPDU • root bridge • bridge ID • bridge priority • root port • designated port • alternate port • path cost • Rapid PVST+ • PortFast • BPDU guard • root guard • loop guard • errdisable

12.1 Why a layer 2 loop is fatal

💡 Three things happen at once, and none of them stops

An IP packet carries a TTL that decrements at each router, so a routing loop eventually discards the packet. **An Ethernet frame has no TTL.** A frame caught in a switching loop circulates until somebody unplugs something.

- **Broadcast storm.** A single broadcast is flooded, arrives back, and is flooded again — multiplying without limit until the links saturate.
- **MAC table instability.** The same source address arrives on two ports repeatedly, so the table is rewritten constantly and unicast forwarding becomes unreliable (see the flap message in Chapter 3).
- **Duplicate frame delivery.** Hosts receive multiple copies, which breaks protocols that assume they will not.

The network does not degrade gracefully; it stops, usually within seconds. This is why spanning tree exists and why it is on by default.

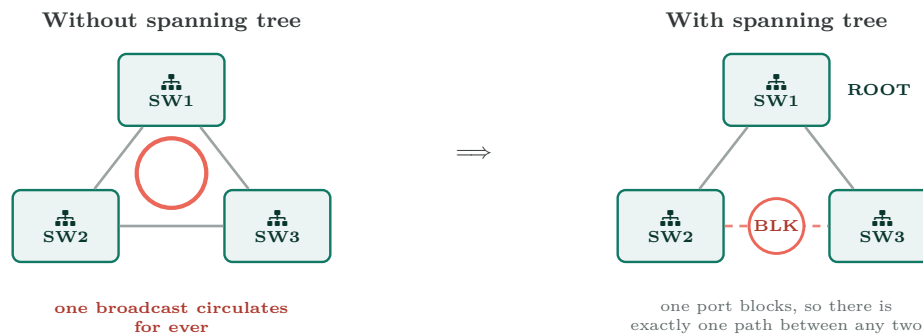


Figure 12.1: Spanning tree does not remove the cable. It blocks one port, so the physical redundancy is still there and can be brought back automatically when a link fails.

12.2 How the tree is chosen

Bridge ID and root election

Every switch has a **bridge ID**: a 4-bit priority, plus a 12-bit extended system ID carrying the VLAN number, plus its MAC address. The switch with the **lowest bridge ID wins** and becomes the **root bridge**.

Default priority is 32768. Because the extended system ID adds the VLAN number, the effective priority in VLAN 10 is $32768 + 10 = 32778$. Priorities are configured in steps of 4096.

Worked Example: Which of these three is the root, in VLAN 10?

Three switches, all left at defaults:

SW1	priority 32768	MAC 0011.2233.0100
SW2	priority 32768	MAC 0011.2233.0050
SW3	priority 32768	MAC 0011.2233.0200

Step 1 — compare priority. All three are 32768, and all three are in VLAN 10, so all three have the same effective priority of $32768 + 10 = 32778$. No winner yet.

Step 2 — the tie-break is the MAC address, and lowest wins. Compare them left to right, as you would compare words alphabetically. The first eight digits are identical, so the decision comes down to the last four: 0100, 0050, 0200.

In hex, 0050 is the smallest of the three (Chapter 3 if the hex is not yet automatic — 0050 is 80 in decimal, 0100 is 256).

SW2 is the root bridge.

Now notice what just happened: **nobody chose this**. The winner is whichever switch happens to have the lowest MAC address, which correlates with nothing except manufacturing date. That is the point of the alert below.

⚠ Left alone, the oldest switch wins

If every switch keeps the default priority, the election is decided by the lowest MAC address — which usually means the oldest device in the building becomes the root of your network. That is almost never where you want traffic to converge. **Always set the root deliberately.**

💡 Then every port takes a role

Once the root is known:

- Each non-root switch picks its **root port** — the one port with the lowest cumulative path cost to the root. It forwards.
- Each segment picks a **designated port** — the port on that segment closest to the root. It forwards.
- Everything else becomes an **alternate** (or backup) port and **blocks**. That is the loop being broken.

Ties are broken by lowest sender bridge ID, then lowest sender port ID.

Link speed	Cost	Note
10 Mbps	100	Legacy
100 Mbps	19	
1 Gbps	4	
10 Gbps	2	Costs are cumulative along the path to the root

12.3 Configuring Rapid PVST+

Rapid PVST+ runs one rapid spanning tree instance per VLAN. It converges in seconds rather than the 30–50 of the original standard, and it is what the blueprint names.

>_ Configure — Rapid PVST+ with a deliberate root

```
! Run rapid per-VLAN spanning tree.
spanning-tree mode rapid-pvst
!
! On the switch that SHOULD be root for these VLANs:
spanning-tree vlan 10,20,30 root primary
!
! On the switch that should take over if it fails:
!   spanning-tree vlan 10,20,30 root secondary
!
! The macro above simply sets a priority. You can be explicit:
!   spanning-tree vlan 10 priority 4096
```

🔍 SW1# show spanning-tree vlan 10

```
VLAN0010
  Spanning tree enabled protocol rstp
  Root ID    Priority    4106
             Address     00d0.58a1.0200
             This bridge is the root
             Hello Time 2 sec  Max Age 20 sec  Forward Delay 15 sec

  Bridge ID  Priority    4106  (priority 4096 sys-id-ext 10)
             Address     00d0.58a1.0200

Interface    Role Sts Cost      Prio.Nbr Type
-----
Gi0/1        Desg FWD 4        128.1    P2p
Gi0/2        Desg FWD 4        128.2    P2p
```

Two things to read first: **This bridge is the root** — is it the switch you intended? — and the priority, 4106, which is $4096 + 10$ and confirms the extended system ID is doing what it should.

Q SW2# show spanning-tree vlan 10

```
VLAN0010
Spanning tree enabled protocol rstp
Root ID    Priority    4106
           Address    00d0.58a1.0200
           Cost       4
           Port       1 (GigabitEthernet0/1)

Interface      Role Sts Cost      Prio.Nbr Type
-----
Gi0/1          Root FWD 4         128.1   P2p
Gi0/2          Altn BLK 4         128.2   P2p
```

Gi0/2 is Altn BLK — alternate, blocking. That is the loop being broken, and it is a healthy state, not a fault.

12.4 The protection features

Topic 2.5 names PortFast, root guard, loop guard and BPDU guard explicitly.

Feature	What it does	Put it where
PortFast	Skips listening and learning; the port forwards immediately	Access ports with an end device. Saves 30 s on boot and stops DHCP timing out
BPDU guard	Errdisables the port if a BPDU arrives	The same access ports. A BPDU there means somebody plugged in a switch
Root guard	Ignores superior BPDUs; puts the port in root-inconsistent state	Ports facing downstream switches you do not want becoming root
Loop guard	Blocks a port that stops receiving BPDUs rather than letting it forward	Root and alternate ports on trunks, where a unidirectional link would otherwise create a loop

⚠ PortFast and BPDU guard belong together

PortFast on its own is a loop waiting to happen: it tells a port to forward immediately, so if somebody plugs a switch into it the loop is live before spanning tree can react. BPDU guard is the safety catch — the instant a BPDU arrives, the port is shut. Configure them as a pair, always.

>_ Configure — protection, per port and globally

```
! Per access port.
interface range GigabitEthernet0/5 - 20
  switchport mode access
  switchport access vlan 10
  spanning-tree portfast
  spanning-tree bpduguard enable
!
! Or globally, which applies to every access port automatically --
! far harder to forget on the next port somebody configures.
spanning-tree portfast default
spanning-tree portfast bpduguard default
!
! On a trunk facing a downstream switch that must never be root:
interface GigabitEthernet0/23
  spanning-tree guard root
```

```
!
! Loop guard, globally on all point-to-point links.
spanning-tree loopguard default
```

Q SW1# show interfaces status err-disabled

Port	Name	Status	Reason	Err-disabled Vlans
Gi0/7		err-disabled	bpduguard	

💡 Recovering an errdisabled port

Manually: shutdown then no shutdown on the interface — after removing whatever caused it. Automatically:

➤ Configure — automatic recovery from BPDU guard

```
errdisable recovery cause bpduguard
errdisable recovery interval 300
```

Automatic recovery is a convenience, not a fix. If a rogue switch is still plugged in the port will simply errdisable again every five minutes.

❗ Common Pitfalls

- **Leaving the root election to chance.** The oldest switch wins, and your traffic detours through the wiring closet nobody has visited in ten years.
- **PortFast without BPDU guard.** See above.
- **PortFast on a trunk to another switch.** It is for edge ports. There is a portfast trunk variant for specific cases, but the default answer is no.
- **Treating a blocking port as broken.** Altn BLK is spanning tree working.
- **Forgetting spanning tree is per VLAN.** A change to VLAN 10 does nothing for VLAN 20. Always name the VLANs.

🔧 Break & Fix — After a new switch was added, traffic takes a strange path.

An access switch was installed in a spare closet. Since then, throughput between two core switches has halved and show spanning-tree vlan 10 on the core reports that it is no longer the root.

Q CORE1# show spanning-tree vlan 10 | include Root ID|Priority|Address

Root ID	Priority	Address
	32778	0011.2233.4455
		Cost 19

Diagnosis. The root's priority is 32778 — the default of 32768 + 10 — and the MAC address is not the core's. The new access switch shipped with default priority and a lower MAC address than the core, so it won the election. Every VLAN's traffic now converges on a small access switch connected by a 100 Mbps link, which is where the cost of 19 comes from and why throughput halved.

Fix. Two steps, and do both. Set the intended root explicitly on the core: spanning-tree vlan 10,20,30 root primary, and the secondary on the other core switch. Then stop it happening again: apply spanning-tree guard root on the core ports facing access switches, so a switch advertising a superior BPDU is ignored rather than obeyed.

Lab 12.1 — Steer the tree, then protect it (*Packet Tracer*)

Topology. Three switches cabled in a triangle — deliberately looped — with PCs on two of them.

Task 1. Cable the triangle with no spanning tree changes. Identify the root from `show spanning-tree` and explain *why* that switch won.

Task 2. Record which port on which switch is blocking.

Task 3. Set SW1 as root `primary` and SW2 as root `secondary` for VLAN 1. Re-check and record what moved.

Task 4. Disconnect the link between SW1 and SW3. Time how long traffic takes to recover and note which port changed role.

Task 5. Enable PortFast and BPDU guard on an access port, then plug a switch into it. Record the log message and the port status, then recover the port.

Task 6. Enable `spanning-tree guard root` on SW1's port to SW3, then try to make SW3 root by lowering its priority. Record what happens instead.

Task 7. Extension. Time how long the network takes to recover from a link failure under `spanning-tree mode pvst`, then repeat under `rapid-pvst` and compare the two figures.

Checkpoint — Test Yourself

1. Three switches have default priorities and MAC addresses ending ...0100, ...0050 and ...0200. Which is root, and why?
2. A port shows `Altn BLK`. Is this a fault? Justify your answer.
3. Why must PortFast and BPDU guard be configured together?

Chapter Summary

- An Ethernet frame has no TTL, so a layer 2 loop never self-clears. A broadcast storm takes the network down in seconds.
- Lowest bridge ID becomes root; bridge ID is priority plus VLAN plus MAC. Set it deliberately or the oldest switch wins.
- Root port per switch, designated port per segment, everything else blocks.
- Rapid PVST+ runs one rapid instance per VLAN. Configure it, and remember every command is per VLAN.
- PortFast with BPDU guard on access ports; root guard facing downstream switches; loop guard on trunks. Errdisable is recovered by `shutdown/no shutdown` after removing the cause.

Review Questions

1. (MCQ) What is the effective priority of a switch in VLAN 20 with a configured priority of 8192? (a) 8192 (b) 8212 (c) 28692 (d) 32788
2. (MCQ) Which feature errdisables a port when a BPDU is received? (a) PortFast (b) BPDU guard (c) root guard (d) loop guard
3. (Short) Explain why a layer 2 loop is more damaging than a routing loop.
4. (Short) A port is in `root-inconsistent` state. Which feature put it there, and what was it protecting against?

5. **(Applied)** You inherit a three-switch network where the root is a random access switch and no protection features are configured. Write the commands to make the core the root, provide a backup root, and prevent recurrence — and say in what order you would apply them on a live network, with a reason.

CDP, LLDP and Documenting a Network

EXAM TOPICS **2.3**

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** what discovery protocols advertise and what they cannot tell you.
- **Configure** CDP and LLDP, including disabling them where they should not run.
- **Validate** a network diagram against the devices themselves, which is precisely what topic **2.3** asks.
- **Use** discovery output to find a mismatch — native VLAN, duplex or an undocumented device.

🔗 Before You Start

Chapter 10 for native VLANs, Chapter 2 for duplex, Chapter 7 for filtering output.

📖 Key Terms in This Chapter

CDP • LLDP • LLDP-MED • neighbour • holdtime • advertisement • capability code • platform • local interface • port ID

🎓 The verb here is *validate*

Topic **2.3** is “validate the accuracy of network documentation using CDP and LLDP”. It is not asking you to describe the protocols. It is asking you to take a diagram that claims SW1 Gi0/1 connects to R1 Gi0/0, and prove from the devices whether that is true. Expect to be shown a diagram and some `show cdp neighbors` output that contradicts it.

13.1 Two protocols, one job

	CDP	LLDP
Standard	Cisco proprietary	IEEE 802.1AB, open
Default state	On (most Cisco platforms)	Off (most Cisco platforms)
Timers	Advertise 60 s, hold 180 s	Advertise 30 s, hold 120 s
Sees	Cisco devices	Anything that speaks LLDP, including other vendors, printers and phones
Voice extension	—	LLDP-MED, used to tell phones their voice VLAN

Run both in a mixed estate: CDP for the Cisco gear, LLDP for everything else.

What a neighbour advertisement carries

Device identifier, the local and remote interface, the platform and capabilities, and depending on the protocol and platform: IP address, IOS version, native VLAN, duplex and power requirements.

Crucially, a neighbour is **directly connected at layer 2**. A device two switches away never appears. That is exactly what makes the output usable as evidence of physical cabling.

13.2 Reading the output

SW1# show cdp neighbors

Capability Codes: R - Router, T - Trans Bridge, B - Source Route Bridge
S - Switch, H - Host, I - IGMP, r - Repeater, P - Phone

Device ID	Local Intrfce	Holdtme	Capability	Platform	Port ID
R1	Gig 0/1	142	R S I	C1111	Gig 0/0/0
SW2	Gig 0/2	167	S I	WS-C2960	Gig 0/1
SW3	Gig 0/3	121	S I	WS-C2960	Gig 0/24
AP-EAST	Gig 0/10	155	H	AIR-AP2802	Gig 0

Read it as four columns that matter: **Device ID** (who), **Local Intrfce** (my port), **Port ID** (their port), **Capability** (what kind of device). Those four are the cabling diagram, taken from the devices rather than from a drawing somebody updated in 2019.

SW1# show cdp neighbors detail

```

-----
Device ID: R1
Entry address(es):
  IP address: 10.0.0.1
Platform: Cisco C1111-4P, Capabilities: Router Switch IGMP
Interface: GigabitEthernet0/1, Port ID (outgoing port): GigabitEthernet0/0/0
Holdtime : 142 sec

Version :
Cisco IOS XE Software, Version 17.09.04a

Duplex: full
Native VLAN: 999

```

detail adds the IP address, the software version, the duplex and the native VLAN — which is how a native VLAN mismatch gets caught before it causes a security incident.

Configure — running both protocols, and switching them off where they do not belong

```

! CDP is on by default; LLDP is not.
lldp run
!
! Turn CDP off on a port facing anything you do not control.
! Advertising your platform and IOS version to a guest network,
! an ISP handoff or an untrusted uplink is free reconnaissance.
interface GigabitEthernet0/24
  description ISP handoff - no discovery
  no cdp enable
  no lldp transmit
  no lldp receive
!
! Globally, if policy demands it:
! no cdp run

```


⚠ Do not disable it everywhere on principle

There is a reflex to switch CDP off across the estate “for security”. Weigh it properly: internally, CDP and LLDP are how you validate cabling, how phones learn their voice VLAN (Chapter 9), and how native VLAN and duplex mismatches are detected. Disable them on untrusted edges — ISP links, guest ports, DMZ segments — and leave them running inside.

13.3 Validating a diagram

📄 Worked Example: The diagram says one thing, the switch says another

The documentation claims:

- SW1 Gi0/2 → SW2 Gi0/1
- SW1 Gi0/3 → SW3 Gi0/1

The output above shows Gi0/2 to SW2 Gi0/1 — correct — but Gi0/3 to SW3 Gig 0/24, not Gi0/1.

What that means in practice. Somebody re-patched SW3 and did not update the diagram. It may be harmless. It may not: if SW3 Gi0/24 is configured as an access port while Gi0/1 was the trunk, the uplink is carrying one VLAN instead of all of them, and the fault will surface later as “the new VLAN does not work on the third floor”.

What to do. Correct the diagram, then check that the port actually in use carries the configuration the design intended. Discovery output tells you what is plugged in; it does not tell you whether the port is configured correctly. That is a second check.

⚠ Common Pitfalls

- **Expecting to see non-adjacent devices.** Neighbours are one hop. A device that does not appear may simply be further away.
- **Concluding a device is absent when CDP is off.** A non-Cisco switch, or one with CDP disabled, is invisible to CDP and may still be there. Cross-check with LLDP and the MAC address table.
- **Trusting stale entries.** Holdtime counts down from 180 s. An entry can persist for three minutes after a device is unplugged.
- **Forgetting lldp run.** Without it, `show lldp neighbors` is empty and you may wrongly conclude nothing is connected.
- **Reading the port columns backwards.** Local Intrfce is yours; Port ID is theirs.

🔧 Break & Fix — An undocumented switch appears in the discovery table.

An audit of SW1 shows a neighbour that is not on any diagram:

```
SW1# show cdp neighbors GigabitEthernet0/14 detail
```

```
-----
Device ID: Switch
Entry address(es):
Platform: cisco WS-C2960-8TC-L, Capabilities: Switch IGMP
Interface: GigabitEthernet0/14, Port ID (outgoing port): FastEthernet0/1
Holdtime : 131 sec

Version :
Cisco IOS Software, C2960 Software, Version 12.2(55)SE
```

Diagnosis. An eight-port switch with the default hostname `Switch`, no management address, and a decade-old IOS version, connected to an access port — somebody has plugged a desk

switch into a wall socket to get more ports. It is unmanaged, it participates in spanning tree, and it can influence the root election (Chapter 12).

Fix. Short term: confirm what is downstream of it before unplugging anything, because there are probably users behind it. Long term, this is exactly what `spanning-tree bpduguard` on access ports is for — with it configured, the port would have errdisabled the moment the switch was connected and the problem would have announced itself instead of being found in an audit months later.

Lab 13.1 — Draw the network from the devices (*Packet Tracer*)

Topology. Four switches and a router, cabled by your instructor or a partner without telling you how, plus a deliberately wrong diagram.

Task 1. Using only `show cdp neighbors` from each device, draw the physical topology. Do not look at the cables.

Task 2. Compare your drawing with the supplied diagram and list every discrepancy.

Task 3. Run `show cdp neighbors detail` on one link and record the IOS version, duplex and native VLAN of the neighbour.

Task 4. Create a native VLAN mismatch on one trunk and find the resulting log message.

Task 5. Enable `lldp run` everywhere and compare `show lldp neighbors` with the CDP view. Account for any difference.

Task 6. Disable CDP on one port and confirm the neighbour disappears — noting how long it takes.

Task 7. Extension. Write a short procedure a junior technician could follow to validate a patching change using discovery output alone.

Checkpoint — Test Yourself

1. `show cdp neighbors` lists a device you know is three switches away. What must be true?
2. You unplug a device and it still appears in the table. Why, and for how long?
3. Which single command would tell you a neighbour's IOS version and native VLAN?

Chapter Summary

- CDP is Cisco and on by default; LLDP is open and off by default. Run both in a mixed estate.
- Neighbours are directly connected only — which is what makes the output evidence of cabling.
- Local `Intf` is your port, Port ID is theirs. Those two columns plus the device name are the diagram.
- `detail` adds IP, version, duplex and native VLAN, and catches mismatches before they cause incidents.
- Disable discovery on untrusted edges, not everywhere. Holdtime means entries persist for minutes after a device is removed.

Review Questions

1. (MCQ) Which command enables LLDP globally? (a) `lldp enable` (b) `lldp run` (c) `lldp transmit` (d) `no lldp disable`
2. (MCQ) In `show cdp neighbors`, which column names the port on the *remote* device? (a) Local `Intf` (b) Port ID (c) Platform (d) Capability

3. **(Short)** Give two reasons to leave CDP running inside a network and one reason to disable it at an edge.
4. **(Short)** A diagram says two switches are connected. Neither appears in the other's CDP table. Give three explanations.
5. **(Applied)** You are handed a network with no documentation. Describe a repeatable procedure, naming the commands, to produce an accurate physical topology diagram and state two things your procedure cannot establish.

Troubleshooting Layer 2

EXAM TOPICS **2.4**

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Apply** a repeatable order of checks to a layer 2 fault instead of guessing.
- **Select** the right `show` command for a symptom, and read what it returns.
- **Use** `show logging`, ping, extended ping and capture output together to localise a fault.
- **Diagnose** the six faults that account for most switched-network problems.

🔗 Before You Start

All of Part II. This chapter assembles Chapter 8 to Chapter 13 into a method.

📖 Key Terms in This Chapter

symptom • hypothesis • scope • errdisable • show logging • severity • divide and conquer • bottom-up • baseline

🎓 This chapter is the exam's heaviest verb

Topic **2.4** — “troubleshoot basic Layer 2/Layer 3 connectivity and device operations using `show` commands (including `show logs`), ping, extended ping, trace route, and packet capture output” — is one of eight *troubleshoot* topics. Together with the eight *configure* topics they are 55% of the paper. The method below is worth more marks than any single technology in this book.

14.1 Scope first, then layers

💡 Ask “how many?” before you ask “why?”

The single most useful question is how much is broken, because it tells you where to look before you have run a command.

- **One device** → that port, that cable, that host's configuration.
- **One VLAN, everywhere** → the VLAN itself: does it exist, is it shut, is it allowed on the trunks, does its SVI have an address?
- **Everything on one switch** → that switch's uplink, or spanning tree.
- **Everything, everywhere** → a loop, the root bridge, or a routing or gateway failure.
- **Intermittent, under load** → physical layer, duplex, or a saturated link. Almost never a VLAN.

💡 Then work bottom-up

Once you know the scope, walk the layers from the bottom. Each step is one command and rules out a whole class of cause.

1. **Is the port up?** `show interfaces status`
2. **Is it clean?** `show interfaces` — errors, duplex, speed

3. Is it in the right VLAN? `show interfaces switchport`
4. Does the switch see the device? `show mac address-table`
5. Does the VLAN reach the next switch? `show interfaces trunk`
6. Is spanning tree forwarding? `show spanning-tree vlan`
7. Is there a gateway? `show ip interface brief`

Resist starting at step 7 because the user said “I cannot reach the internet”.

14.2 The first three commands

Q SW1# `show interfaces status`

Port	Name	Status	Vlan	Duplex	Speed	Type
Gi0/1	Uplink to SW2	connected	trunk	a-full	a-1000	10/100/1000
Gi0/5	Desk 5	connected	10	a-full	a-100	10/100/1000
Gi0/6	Desk 6	notconnect	10	auto	auto	10/100/1000
Gi0/7	Desk 7	err-disabled	10	auto	auto	10/100/1000
Gi0/8	Desk 8	connected	15	a-full	a-1000	10/100/1000

One screen, four different faults visible. Gi0/6 `notconnect` — nothing plugged in, or a dead cable. Gi0/7 `err-disabled` — a protection feature fired; find out which. Gi0/8 is in VLAN 15, which may or may not be intended — compare against the design.

Q SW1# `show logging | include LINK|LINEPROTO|ERR|SPANTREE`

```
%LINK-3-UPDOWN: Interface GigabitEthernet0/6, changed state to down
%SPANTREE-2-BLOCK_BPDUGUARD: Received BPDU on port Gi0/7 with BPDU Guard
enabled. Disabling port.
%PM-4-ERR_DISABLE: bpduguard error detected on Gi0/7, putting Gi0/7 in
err-disabled state
%CDP-4-NATIVE_VLAN_MISMATCH: Native VLAN mismatch discovered on
GigabitEthernet0/1 (999), with SW2 GigabitEthernet0/1 (1).
```

Read the log before you read anything else

The log frequently contains the answer in plain English, timestamped. Above, it names the errdisable cause and a native VLAN mismatch nobody had reported — a fault that is silently bridging VLAN 999 and VLAN 1 between two switches.

Filter it. `show logging | include` with the keywords above turns hundreds of lines into the handful that matter.

14.3 The six faults, and what each looks like

Fault	Symptom	Command that proves it
Port in the wrong VLAN	One user isolated; can reach some hosts, not others	<code>show interfaces switchport</code> — check Access Mode VLAN
VLAN missing from a trunk	One VLAN fails between switches; others fine	<code>show interfaces trunk</code> — compare allowed lists at both ends
Native VLAN mismatch	Nothing obviously down; VLANs leak	<code>show cdp neighbors detail</code> , or the log message above
Errdisabled port	One port dead after somebody plugged something in	<code>show interfaces status</code> <code>err-disabled</code>

Fault	Symptom	Command that proves it
Spanning tree blocking	A path that should work does not; redundant link idle	<code>show spanning-tree vlan N</code> — look for Altn BLK
Duplex or speed mismatch	Slow, not down; errors climbing under load	<code>show interfaces</code> — collisions, late collisions, CRC

Worked Example: Working a fault end to end

Symptom. “Nobody in accounts can reach the file server since this morning. Sales are fine.”

Scope. One VLAN, multiple users, more than one switch. That rules out a single cable and points at the VLAN or its path.

1. `show interfaces status` on the access switch — accounts ports are connected in VLAN 30. Not a port problem.
2. `show mac address-table vlan 30` — addresses are being learned locally. The devices are alive and the switch sees them.
3. `show interfaces trunk` on the uplink — VLAN 30 appears in “allowed” but *not* in “forwarding and not pruned”.
4. `show spanning-tree vlan 30` — the uplink is Altn BLK.
5. `show logging | include SPANTREE` — a topology change overnight; a second switch was added and won the root election for VLAN 30.

Cause. Not a VLAN fault at all. The new switch became root for VLAN 30 only, which moved the tree and put the accounts uplink into blocking. Sales, in VLAN 20, still had the old root and were unaffected — which is exactly why only one VLAN broke.

Fix. Set the intended root explicitly for all VLANs, add root guard on the ports facing access switches, and confirm the uplink returns to Desg FWD.

Common Pitfalls

- **Changing something before you have measured.** If you reconfigure before capturing the current state you can no longer tell whether you fixed it or moved it.
- **Changing more than one thing at once.** Then you do not know which change worked, and you cannot write it up.
- **Believing the reporter’s diagnosis.** “The switch is broken” usually means “my thing does not work”. Take the symptom, discard the theory.
- **Ignoring what still works.** The working half of the network is evidence. Sales working and accounts failing localised the fault above faster than any command did.
- **Skipping the log.** It is timestamped and often explicit.

Break & Fix — Two symptoms, one cause, and a misleading first look.

Users report intermittent slowness. One user cannot connect at all. Both started after a contractor visited the comms room.

 SW1# `show interfaces status | exclude connected`

Port	Name	Status	Vlan	Duplex	Speed	Type
Gi0/7	Desk 7	err-disabled	10	auto	auto	10/100/1000

```
Q SW1# show logging | include ERR_DISABLE|MACFLAP
```

```
%PM-4-ERR_DISABLE: bpduguard error detected on Gi0/7, putting Gi0/7 in
err-disable state
%SW_MATM-4-MACFLAP_NOTIF: Host 0011.2233.4455 in vlan 10 is flapping
between port Gi0/7 and port Gi0/12
```

Diagnosis. Both symptoms are one event. The contractor plugged an unmanaged switch into Gi0/7 and, to “make sure it worked”, also patched it into Gi0/12. That created a loop: the MAC flap is the loop being seen from two ports, and the intermittent slowness is the resulting flooding. BPDU guard then errdisabled Gi0/7, which is the one port that is definitively dead — and which, by shutting, largely saved the network.

Fix. Remove the second patch lead. Confirm the flapping stops. Then recover the port with `shutdown / no shutdown` — *after* the cause is gone, not before, or it will simply errdisable again. Finally, note that BPDU guard did its job: without it this would have been a full broadcast storm rather than one dead port.

Lab 14.1 — Fix five faults against the clock (*Packet Tracer*)

Topology. Three switches, one router, six PCs across three VLANs — built, working, and then broken by a partner.

Task 1. Build the network and confirm it works fully. **Capture a baseline:** `show interfaces status`, `show interfaces trunk`, `show spanning-tree vlan` and `show ip interface brief` from every device. You cannot troubleshoot without knowing what right looks like.

Task 2. Hand the network to a partner, who introduces **five** faults from the table above without telling you which.

Task 3. Find and fix them, writing down for each: the symptom, the scope, the command that proved it, and the fix. Give yourself twenty minutes.

Task 4. Swap roles.

Task 5. Compare notes: which fault took longest, and would starting with scope rather than commands have found it faster?

Task 6. Extension. Introduce a fault that produces *no* log message — a wrong access VLAN is a good one — and discuss why the log is a first stop rather than a complete answer.

Checkpoint — Test Yourself

1. Users on one VLAN across three switches lose connectivity; other VLANs are fine. What are the two most likely causes, and which command separates them?
2. A port is errdisabled. You issue `no shutdown` and it errdisables again within seconds. What have you forgotten?
3. Why does knowing what *still works* narrow a fault faster than knowing what is broken?

Chapter Summary

- Establish scope first — one device, one VLAN, one switch, or everything. Scope points at the layer before you type anything.
- Then work bottom-up: port up, port clean, right VLAN, MAC learned, VLAN on the trunk, spanning tree forwarding, gateway present.
- Read `show logging` early. It is timestamped and often explicit.

- Six faults cover most cases: wrong VLAN, VLAN missing from a trunk, native VLAN mismatch, errdisable, spanning tree blocking, duplex mismatch.
- Capture a baseline while the network works. Change one thing at a time, and remove the cause before recovering an errdisabled port.

Review Questions

1. **(MCQ)** A single VLAN fails between two switches while others work. Which command should you run first? (a) `show ip route` (b) `show interfaces trunk` (c) `show version` (d) `show mac address-table`
2. **(MCQ)** Which status means a protection feature has shut a port? (a) notconnect (b) disabled (c) err-disabled (d) monitoring
3. **(Short)** Explain why establishing scope before running commands makes troubleshooting faster, with an example.
4. **(Short)** Give one fault from this chapter that produces a clear log message and one that produces none, and say how you would find the second.
5. **(Applied)** Write a troubleshooting procedure of no more than eight steps that a first-line technician could follow for “a user cannot reach the network”, naming the command at each step and the decision it resolves.

PART III

IP Routing

A switch moves frames within a VLAN. Everything that has to cross between VLANs, between buildings or onto the internet is the router's problem, and this part is how a router decides.

Domain 3 is 20% of the exam across four topics, and the verbs are worth noting: two ask you to *interpret* — read a routing table, read the status of a redundancy protocol — one asks you to *troubleshoot* static routing, and only one asks you to *configure*, namely OSPF. So this part is as much about reading output correctly as about typing commands.

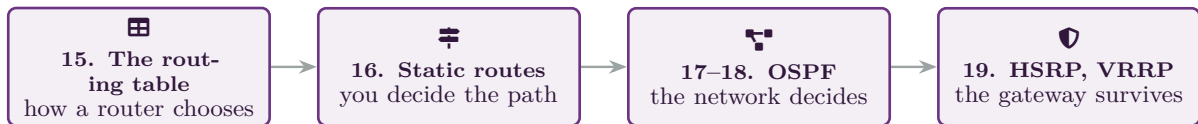


Figure P3.1: Part III. Static routing and OSPF are two answers to the same question; first Chapter 15 establishes how the router picks between whatever answers it has.

One idea underpins the whole part

A router builds *one* routing table from every source available to it — connected interfaces, static routes you typed, and routes learned from OSPF — and then forwards on the single best entry for each destination. Administrative distance decides which *source* wins; the longest prefix match decides which *entry* wins. Get those two rules straight in Chapter 15 and the rest of the part is configuration detail.

The Routing Table and How a Router Decides

EXAM TOPICS **3.1**

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Read** every column and code in `show ip route`.
- **Apply** the longest prefix match rule to determine which entry a given packet uses.
- **Explain** administrative distance and predict which source wins when two protocols offer the same prefix.
- **Distinguish** the three reasons a packet is dropped: no route, a route to null, and an unreachable next hop.

🔗 Before You Start

Chapter 4 for prefixes and masks — this chapter is unreadable without fluent subnetting.
Chapter 10 for SVIs as gateways.

📖 Key Terms in This Chapter

routing table • prefix • next hop • exit interface • administrative distance • metric • longest prefix match • directly connected • local route • default route • gateway of last resort • null0

15.1 What a router is actually doing

A switch asks “which *port* is this MAC address on?”. A router asks a different and slightly harder question, once for every packet:

Of everything I know how to reach, which entry best matches this destination, and where do I send the packet next?

📋 Routing table

The list of destinations a router knows how to reach, and for each one, the **next hop** — the neighbouring device to hand the packet to — and the interface to send it out of.

A router does **not** know the whole path. It knows one step. Every router along the way makes the same one-step decision independently, and the packet arrives because each of them was right about its own next hop. That is worth sitting with, because it explains why a fault three hops away looks like a working router with traffic disappearing into it.

💡 One line, before twenty

Every entry in the table has the same shape. Read this one slowly and the full dump below stops being a wall of text:

```
0    10.1.0.0/16 [110/20] via 10.255.0.2, 00:14:22, GigabitEthernet0/1
```

- **0** — **how** the router learned it. **0** is OSPF, **S** static, **C** connected, **L** local. The legend at the top of the output lists them all.

- 10.1.0.0/16 — **what** it reaches: the destination network and its prefix length.
- [110/20] — **administrative distance** and **metric**. 110 is how much the router trusts the source; 20 is how good this particular path is. Section 15.4 explains why those are two separate numbers.
- via 10.255.0.2 — the **next hop**.
- 00:14:22 — how long the route has been known.
- GigabitEthernet0/1 — the **exit interface**.

Connected and local routes have no distance, metric or next hop, because there is nothing between the router and them.

15.2 Reading the table

 R1# show ip route

```
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       O - OSPF, IA - OSPF inter area, N1 - OSPF NSSA external type 1
       E1 - OSPF external type 1, E2 - OSPF external type 2
       * - candidate default, U - per-user static route

Gateway of last resort is 203.0.113.1 to network 0.0.0.0

S*    0.0.0.0/0 [1/0] via 203.0.113.1
      10.0.0.0/8 is variably subnetted, 4 subnets, 2 masks
O      10.1.0.0/16 [110/20] via 10.255.0.2, 00:14:22, GigabitEthernet0/1
C      10.255.0.0/30 is directly connected, GigabitEthernet0/1
L      10.255.0.1/32 is directly connected, GigabitEthernet0/1
S      10.9.9.0/24 [1/0] via 10.255.0.2
      192.168.10.0/24 is directly connected, Vlan10
C      192.168.10.0/24 is directly connected, Vlan10
L      192.168.10.1/32 is directly connected, Vlan10
```

Element	What it tells you
Code letter	Where the route came from. C connected, L local, S static, O OSPF
Prefix and mask [110/20]	The destination network this entry covers Administrative distance / metric . 110 is OSPF's distance; 20 is OSPF's cost for this path
via 10.255.0.2	The next hop — the neighbouring router to hand the packet to
Interface	The exit interface
00:14:22	How long the route has been in the table. A route that keeps resetting is flapping

C and L are not the same thing

C is the *subnet* on a directly connected interface — the range the router can reach by ARP. L is a /32 host route for the router's *own* address on that interface, so that traffic addressed to the router itself is delivered locally rather than forwarded.

You do not configure L routes; the router creates one for every address you give an interface. They appear in every table and are not a fault.

15.3 Rule one: longest prefix match

💡 The most specific route wins, regardless of anything else

When several entries could carry a packet, the router uses the one with the **longest prefix** — the most 1 bits in the mask. Administrative distance and metric never override this; they only choose between entries with the *same* prefix length.

📊 Worked Example: Which entry carries the packet?

Using the table above, a packet for 10.9.9.20:

1. 0.0.0.0/0 — matches everything. Prefix length 0.
2. 10.1.0.0/16 — does 10.9.9.20 fall inside? The /16 covers 10.1.0.0–10.1.255.255. No.
3. 10.9.9.0/24 — covers 10.9.9.0–10.9.9.255. Yes. Prefix length 24.

Two entries match: the default and the /24. The /24 is longer, so the packet goes via 10.255.0.2. Note that the default route has a *better* administrative distance (1 versus 1 — equal here) and it still loses, because prefix length is decided first.

Now a packet for 172.20.5.1: only 0.0.0.0/0 matches, so it takes the gateway of last resort.

15.4 Rule two: administrative distance

📖 Administrative distance

Administrative distance ranks the trustworthiness of a route's *source*. When two sources offer the same prefix at the same length, the lower distance is installed and the other is discarded — it does not even appear in the table.

Source	Distance	Note
Connected	0	Nothing beats seeing it yourself
Static	1	You said so, so the router believes you
eBGP	20	
OSPF	110	The only dynamic protocol in this exam
RIP	120	Legacy
Unreachable / unusable	255	Never installed

💡 Why a static route beats OSPF, and when that bites

A static route at distance 1 will always displace an OSPF route at 110 for the same prefix. That is usually what you want — until the static route's path fails. OSPF would have re-routed; the static route sits there pointing at a dead path, because a static route has no way of knowing anything is wrong beyond the directly connected interface.

The deliberate use of this is the **floating static route**: a backup static configured with a distance *higher* than the dynamic protocol, so it stays out of the table until the dynamic route disappears. That is Chapter 16.

15.5 Three ways a packet dies

Cause	What the router does	How you see it
No matching route	Drops the packet and sends ICMP unreachable	U in ping output; <code>show ip route</code> has no entry and no default
Route points to Null0	Silently discards	An entry via Null0. Often deliberate, for summarisation or blackholing
Next hop unreachable	Route exists but cannot be resolved to a next hop	Route is in the table; pings fail; <code>show ip route</code> for the next hop shows nothing

⚠ A route in the table is not proof of reachability

The presence of an entry means the router knows where it *would* send the packet. It says nothing about whether the next hop is alive, whether the far end has a route back, or whether an access list drops it en route.

Confirm with an extended ping sourced from the relevant interface (Chapter 6) — that tests the return path too, which a route entry never does.

Q R1# show ip route 10.9.9.20

```
Routing entry for 10.9.9.0/24
  Known via "static", distance 1, metric 0
  Routing Descriptor Blocks:
    * 10.255.0.2
      Route metric is 0, traffic share count is 1
```

Querying a specific address rather than reading the whole table is faster and is what the exam expects when it asks which entry would be used.

! Common Pitfalls

- **Comparing metrics across protocols.** An OSPF cost of 20 and a RIP hop count of 2 are not comparable. Administrative distance decides first, and only then does metric compare *within* one protocol.
- **Thinking a better distance beats a longer prefix.** It does not. Prefix length is decided first, always.
- **Missing the gateway of last resort line.** If it says not set, there is no default and anything unmatched is dropped.
- **Reading L entries as duplicates.** They are the router's own addresses and belong there.
- **Assuming symmetry.** Traffic reaching a destination does not mean the reply can return. Most one-way faults are a missing reverse route.

🔧 Break & Fix — The route is there and the ping still fails.

Users on 192.168.10.0/24 cannot reach 10.9.9.0/24. R1 has the route:

Q R1# show ip route 10.9.9.0

```
Routing entry for 10.9.9.0/24
  Known via "static", distance 1, metric 0
  Routing Descriptor Blocks:
    * 10.255.0.2
```

A ping from R1 to 10.9.9.20 succeeds. A ping from a user's PC fails.

Diagnosis. The difference between the two pings is the source address. R1's own ping is sourced from 10.255.0.1, which the far end knows about because it is the link between them. The PC's traffic is sourced from 192.168.10.x, and the far router has no route back to that subnet. The forward path is fine; the return path does not exist.

Confirm it without leaving R1, using an extended ping that borrows the user's source address:

```
R1# ping 10.9.9.20 source Vlan10

Sending 5, 100-byte ICMP Echos to 10.9.9.20, timeout is 2 seconds:
Packet sent with a source address of 192.168.10.1
.....
Success rate is 0 percent (0/5)
```

Same destination, different source, opposite result — which is proof, not inference.

Fix. Add a route on the far router back to 192.168.10.0/24. Then re-run the same extended ping and expect five exclamation marks.

Lab 15.1 — Read the table, then predict the path (*Packet Tracer*)

Topology. Three routers in a line, a LAN on each end, and a second path between the outer two.

Task 1. Address everything and confirm the connected routes appear. Identify every C and L entry and explain each L.

Task 2. Add static routes so the two LANs can reach each other. Record the distance and metric shown.

Task 3. Add a default route on one router. Note the S* and the *gateway of last resort* line.

Task 4. Write down, before testing, which entry a packet to each of four addresses would use. Verify each with `show ip route <address>`.

Task 5. Add a second, more specific route for one destination and confirm it wins over the less specific one, regardless of distance.

Task 6. Remove the reverse route on the far router and reproduce the one-way fault above using `ping ... source`.

Task 7. Extension. Add a route pointing to Null0 for an unused prefix and observe how traffic to it fails compared with having no route at all.

Checkpoint — Test Yourself

1. A table contains 10.0.0.0/8 via OSPF and 10.5.0.0/16 via a static route. Which carries a packet to 10.5.1.1, and why?
2. Both OSPF and a static route offer 192.168.50.0/24. Which is installed, and what happens to the other?
3. A route exists but pings fail. Name three causes and the single command that distinguishes the most common one.

Chapter Summary

- One table, many sources. C connected, L the router's own address, S static, O OSPF.
- [distance/metric]. Distance ranks the source; metric ranks paths within one protocol and is never compared across protocols.
- Longest prefix match decides first. Distance only breaks ties between equal-length prefixes.

- Connected 0, static 1, OSPF 110. A static route displaces OSPF and will not fail over unless you make it float.
- A route in the table proves intent, not reachability. Test the return path with an extended ping.

Review Questions

1. **(MCQ)** A router has 0.0.0.0/0, 172.16.0.0/16 and 172.16.8.0/24. Which carries a packet to 172.16.8.50? (a) the default (b) the /16 (c) the /24 (d) it is dropped
2. **(MCQ)** What is the administrative distance of a static route? (a) 0 (b) 1 (c) 110 (d) 120
3. **(Short)** Explain the difference between a C and an L entry and why both exist.
4. **(Short)** Why can an OSPF cost never be compared with a RIP hop count?
5. **(Applied)** A router shows a static route to a remote subnet, and pings from the router succeed while pings from its LAN fail. Explain the likely cause, the exact command that would prove it, and the fix — including which device you would change.

Static Routing, IPv4 and IPv6

EXAM TOPICS **3.2**

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Configure** the four kinds of static route the blueprint names: default, network, host and floating.
- **Choose** correctly between a next-hop address, an exit interface, and both — a choice that matters more than it looks.
- **Configure** static routing for IPv6 alongside IPv4.
- **Troubleshoot** a static route that is absent from the table, or present and not working.

🔗 Before You Start

Chapter 15 — longest prefix match and administrative distance are assumed throughout. Chapter 5 for the v6 half.

📖 Key Terms in This Chapter

static route • default route • network route • host route • floating static • next hop • exit interface • fully specified route • recursive lookup • gateway of last resort

🎓 The verb is *troubleshoot*, and the sub-topics are a checklist

Topic 3.2 is “troubleshoot IPv4 and IPv6 static routing” and then lists exactly four kinds: default route, network route, host route, floating static. Expect to be shown a table and a symptom rather than asked to recite syntax — but you cannot diagnose what you cannot write, so learn the syntax first.

16.1 The four kinds

Kind	Prefix	What it is for
Network	A real subnet, e.g. /24	The ordinary case: reach that network via this neighbour
Default	0.0.0.0/0	“Everything I do not otherwise know.” Points at the internet or the core
Host	/32 (or /128)	One specific address. Used to steer a single server down a particular path
Floating	Any of the above, with a raised distance	A backup that stays out of the table until the preferred route disappears

➤ Configure — all four, on IPv4

```
! Network route -- reach 10.9.9.0/24 through the neighbour at 10.255.0.2.
ip route 10.9.9.0 255.255.255.0 10.255.0.2
!
! Default route -- the gateway of last resort.
```

```
ip route 0.0.0.0 0.0.0.0 203.0.113.1
!
! Host route -- this one server goes a different way.
ip route 10.9.9.50 255.255.255.255 10.255.1.2
!
! Floating static -- distance 200 keeps it out of the table while
! OSPF (distance 110) has a route. It appears only if OSPF loses one.
ip route 10.1.0.0 255.255.0.0 10.255.2.2 200
```

💡 Why the host route wins

All four routes above can match 10.9.9.50. The /32 host route is the longest prefix, so it is used — ahead of the /24, ahead of the default. Longest prefix match decides before distance is even considered (Chapter 15).

16.2 Next hop, exit interface, or both

This is the choice that produces the most subtle static-routing faults.

Form	Example	Behaviour
Next hop only	ip route 10.9.9.0 255.255.255.0 10.255.0.2	The router must look up how to reach 10.255.0.2 — a recursive lookup . Correct on any link
Exit interface only	ip route 10.9.9.0 255.255.255.0 Serial0/0/0	No recursion. Fine on point-to-point links; dangerous on Ethernet
Fully specified	ip route 10.9.9.0 255.255.255.0 Gi0/1 10.255.0.2	Both. No recursion and no ambiguity. The safest form on Ethernet

⚠️ Exit-interface-only routes on Ethernet are a trap

Ethernet is multi-access: many devices share the segment. A route that names only an exit interface tells the router “send it out here” without saying to whom, so the router must ARP for *every destination address* it forwards, hoping something answers. On a link where proxy ARP is enabled it appears to work; where it is not, it silently fails; and either way the ARP table fills with entries that should not exist.

On a serial point-to-point link there is only one possible recipient, so the form is harmless. On Ethernet, always give the next hop — alone or fully specified.

🔍 R1# show ip route static

```
10.0.0.0/8 is variably subnetted, 5 subnets, 3 masks
S    10.1.0.0/16 [200/0] via 10.255.2.2
S    10.9.9.0/24 [1/0] via 10.255.0.2
S    10.9.9.50/32 [1/0] via 10.255.1.2
S*   0.0.0.0/0 [1/0] via 203.0.113.1
```

Note the floating static at [200/0] is shown here — meaning OSPF currently has no route for 10.1.0.0/16, so the backup has taken over. When OSPF is healthy that line is absent from the table entirely.

16.3 Static routing for IPv6

The syntax mirrors IPv4, with one wrinkle worth learning.

>_ Configure — IPv6 static routes

```

ipv6 unicast-routing
!
! Network route via a global next-hop address.
ipv6 route 2001:db8:acad:20::/64 2001:db8:acad:99::2
!
! Default route.
ipv6 route ::/0 2001:db8:acad:99::2
!
! Host route.
ipv6 route 2001:db8:acad:20::50/128 2001:db8:acad:99::2
!
! Floating static.
ipv6 route 2001:db8:acad:30::/64 2001:db8:acad:98::2 200
!
! Via a LINK-LOCAL next hop -- the exit interface is mandatory,
! because fe80:: addresses are only unique per link and the router
! has no way to know which interface you mean.
ipv6 route 2001:db8:acad:40::/64 GigabitEthernet0/1 fe80::2

```

💡 Why link-local next hops need the interface

A link-local address is valid on one link only, and the same `fe80::2` may legitimately exist on three different interfaces of the same router. Naming the interface removes the ambiguity. IOS rejects a link-local next hop without one.

This matters because routing protocols, including OSPFv3, use link-local addresses as their next hop — so link-local next hops are the normal case in IPv6, not an oddity.

🔍 R1# show ipv6 route static

```

IPv6 Routing Table - default - 7 entries
Codes: C - Connected, L - Local, S - Static, U - Per-user Static route
S    ::/0 [1/0]
      via 2001:DB8:ACAD:99::2
S    2001:DB8:ACAD:20::/64 [1/0]
      via 2001:DB8:ACAD:99::2
S    2001:DB8:ACAD:40::/64 [1/0]
      via FE80::2, GigabitEthernet0/1

```

16.4 Floating statics, properly**📋 Worked Example: A backup path that only activates on failure**

R1 reaches 10.1.0.0/16 through OSPF over its primary link, and has a slow secondary link to the same destination.

1. OSPF's route has distance 110.
2. Configure the backup with a distance *above* 110: `ip route 10.1.0.0 255.255.0.0 10.255.2.2 200`.
3. While OSPF is up, only the OSPF route is installed. `show ip route` shows `0 10.1.0.0/16 [110/20]` and no static entry at all.
4. The primary link fails. OSPF withdraws its route. The static route, now the only candidate, is installed as `S 10.1.0.0/16 [200/0]`.
5. The primary returns. OSPF's route at 110 beats 200 and displaces the static again.

The number 200 is not magic — it just has to exceed the distance of whatever it is backing up. Back up a *static* route (distance 1) and any value from 2 upwards will do.

! Common Pitfalls

- **A floating static with too low a distance.** Set it below the dynamic protocol's and it takes over permanently, which is the opposite of a backup.
- **Exit-interface-only routes on Ethernet.** See the warning above.
- **Forgetting the reverse route.** A static route is one-directional. Both ends need one, and forgetting this is the commonest static-routing fault of all.
- **A next hop that is not directly reachable.** The route will not be installed at all; the router silently discards it. `show ip route` simply has no entry, which looks like you never typed it.
- **Omitting the interface on an IPv6 link-local next hop.** Rejected outright.

🔧 Break & Fix — The static route was typed and it is not in the table.

An engineer configures `ip route 10.9.9.0 255.255.255.0 10.255.0.2` and `show ip route static` shows nothing. No error was displayed when the command was entered.

```
R1# show ip route 10.255.0.2
```

```
% Network not in table
```

Diagnosis. A static route is only installed if the router can resolve its next hop. 10.255.0.2 is not reachable — there is no connected subnet covering it — so the route is accepted into the configuration and never enters the routing table. This is why the command produced no error: the syntax was perfect and the topology was not.

Check the interface that should reach it:

```
R1# show ip interface brief | include Gigabit
```

```
GigabitEthernet0/1    10.255.0.1    YES manual administratively down down
```

Fix. The interface is shut. `no shutdown` on Gi0/1, and the connected route for 10.255.0.0/30 appears, the next hop resolves, and the static route installs itself immediately. Confirm with `show ip route static` — and remember the general rule: **a static route missing from the table is nearly always an unreachable next hop**, not a typing error.

🏠 Lab 16.1 — Four routes, one failure (Packet Tracer)

Topology. Three routers — R1, R2, R3 — with R1 connected to R2 by a fast link and to R3 by a slow one, and R2 and R3 both connected to a LAN at the far side. A LAN on R1.

Task 1. Address everything. Confirm connected routes on all three and that neighbours ping.

Task 2. On R1, add a network route to the far LAN via R2. Test from the R1 LAN — it will fail. Diagnose why, then add the reverse route and re-test.

Task 3. Add a default route on R2 and R3 pointing back at R1. Note the S* and the gateway of last resort line.

Task 4. Add a host route on R1 for one server in the far LAN, via R3. Prove with `show ip route <address>` that it is preferred over the network route.

Task 5. Add a floating static via R3 for the far LAN with distance 5. Confirm it is *not* in the table.

Task 6. Shut R1's link to R2. Confirm the floating static appears and traffic still flows. Restore the link and watch it withdraw.

Task 7. Extension. Repeat steps 2 and 5 for IPv6, including one route via a link-local next hop.

? Checkpoint — Test Yourself

1. You configure a static route and it never appears in the table, with no error message. What is the most likely cause and which command confirms it?
2. A floating static backing up an OSPF route is configured with distance 100. What will happen, and why is it wrong?
3. Why must an IPv6 static route using a link-local next hop also name an exit interface?

📋 Chapter Summary

- Four kinds: network, default (0.0.0.0/0), host (/32) and floating. The host route wins on longest prefix match.
- Give a next hop on Ethernet — alone or fully specified. Exit-interface only belongs on point-to-point links.
- IPv6 mirrors IPv4, but a link-local next hop *must* name the exit interface.
- A floating static needs a distance above whatever it backs up: over 110 for OSPF, over 1 for another static.
- A static route absent from the table almost always means an unresolvable next hop. Routes are one-way; configure both ends.

🔧 Review Questions

1. (MCQ) Which command creates a default route via 203.0.113.1?
(a) `ip route 0.0.0.0 255.255.255.255 203.0.113.1`
(b) `ip route 0.0.0.0 0.0.0.0 203.0.113.1`
(c) `ip default-gateway 203.0.113.1`
(d) `ip route default 203.0.113.1`
2. (MCQ) A floating static backs up an OSPF route. Which distance is suitable? (a) 0 (b) 1 (c) 100 (d) 200
3. (Short) Explain why an exit-interface-only static route is acceptable on a serial link but risky on Ethernet.
4. (Short) A host route and a network route both match a destination. Which is used, and does administrative distance play any part?
5. (Applied) R1 must reach 192.168.80.0/24 via R2 normally and via R3 if that path fails, and must send everything else to the internet via 203.0.113.1. Write the complete configuration and state which `show` output would prove the backup is correctly dormant.

OSPFv2, Single Area

EXAM TOPICS 3.3

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** how OSPF forms adjacencies and why they sometimes do not.
- **Configure** single-area OSPFv2 with an explicit router ID, on both broadcast and point-to-point links.
- **Predict** and **influence** DR and BDR election.
- **Troubleshoot** the five mismatches that prevent an adjacency, using `show ip ospf neighbor` and `show ip ospf interface`.

🔗 Before You Start

Chapter 15 for administrative distance and metrics; Chapter 16 for the contrast with a route you configure by hand.

📖 Key Terms in This Chapter

OSPF • link-state • adjacency • neighbour • hello • router ID • area 0 • DR • BDR • DROTHER • cost • reference bandwidth • wildcard mask • passive interface • FULL state • 2-WAY state

🎓 What the blueprint asks for, precisely

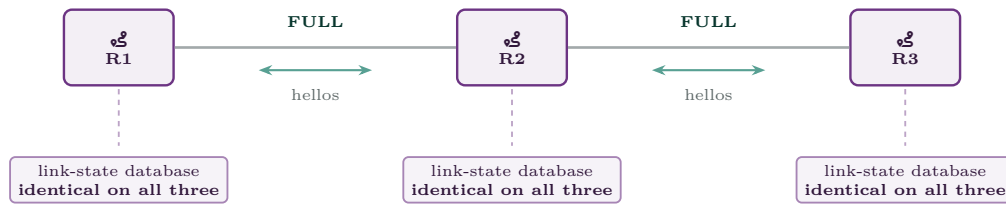
Topic 3.3 is “configure single area OSPFv2 for IPv4 and OSPFv3 for IPv6” with four sub-points: neighbour adjacencies *excluding authentication*, point-to-point, broadcast with DR/BDR selection, and router ID. Multi-area OSPF, authentication, virtual links and route summarisation are all out of scope. Learn these four properly rather than the whole protocol thinly.

17.1 What OSPF does that a static route cannot

💡 Link state, in three sentences

Every OSPF router floods a description of its own links to every other router in the area. Each router therefore ends up with an identical map of the whole area, which it stores in a **link-state database**. Each then runs the shortest-path algorithm against that map to work out its own best route to everywhere.

Because every router has the whole map, a failure anywhere is known everywhere within seconds, and every router recalculates independently and consistently. That is the property a static route can never have.



Each router floods a description of *its own* links. Every router therefore holds the same map — and then computes its *own* best path across it. Nobody is told what their routes are.

Figure 17.1: Link state in one picture. The database is shared and identical; the routing table each router builds from it is its own.

17.2 Adjacencies, and the five things that must match

Two routers become neighbours by exchanging hello packets on a shared link. The hello carries several parameters, and **if any of them disagree, no adjacency forms** — silently.

Must match	What goes wrong if it does not
Area ID	Different areas on the same link: no adjacency. In single-area OSPF everything is area 0
Hello and dead intervals	Defaults are 10 s and 40 s on broadcast links. Change one end only and the adjacency never forms
Subnet and mask	The two interfaces must be in the same subnet. A mask typo is a common cause
Network type	Broadcast on one end and point-to-point on the other prevents adjacency or leaves it stuck
Authentication	Out of scope for this exam, but real: mismatched keys prevent adjacency

Two further conditions are worth adding because they cause the same symptom: the **router IDs must be unique**, and the interface must not be **passive**.

Router ID, and how it is chosen

The **router ID** is a 32-bit value written like an IP address that uniquely identifies the router in OSPF. It is selected in this order:

1. The value configured with `router-id` — always use this.
2. Otherwise, the highest IP address on any *loopback* interface.
3. Otherwise, the highest IP address on any active physical interface.

Left to itself the ID can change when an interface goes down, which forces the adjacency to rebuild. Configure it explicitly and the problem disappears.

17.3 Configuring single-area OSPFv2

Configure — OSPF on R1, area 0

```

! A loopback gives a stable ID and a pingable address that never
! goes down with a physical interface.
interface Loopback0
  ip address 1.1.1.1 255.255.255.255
!
router ospf 10
  
```

```

! The process number is LOCAL. R2 may use a different one and
! the two will still peer -- unlike EIGRP or BGP.
router-id 1.1.1.1
!
! The network statement uses a WILDCARD mask -- the inverse of a
! subnet mask. It means "enable OSPF on any interface whose
! address falls in this range, and advertise that subnet".
network 10.255.0.0 0.0.0.3 area 0
network 192.168.10.0 0.0.0.255 area 0
!
! Do not send hellos out of a LAN with no other routers on it.
! It saves bandwidth and removes an attack surface, while the
! subnet is still advertised.
passive-interface Vlan10
!
! Make cost reflect modern link speeds -- see the note below.
auto-cost reference-bandwidth 10000

```

💡 Wildcard masks, quickly

A wildcard mask is the subnet mask with every bit inverted: 0 means “this bit must match”, 255 means “do not care”.

- /24 → mask 255.255.255.0 → wildcard 0.0.0.255
- /30 → mask 255.255.255.252 → wildcard 0.0.0.3
- A single interface → wildcard 0.0.0.0

The arithmetic is 255 minus each octet of the subnet mask. Wildcards return in Chapter 24.

⚠️ Reference bandwidth: change it everywhere or nowhere

OSPF cost is $\text{reference bandwidth} \div \text{interface bandwidth}$, and the default reference is 100 Mbps. That means a 1 Gbps link and a 10 Gbps link both come out at cost 1 — OSPF cannot tell them apart, and will happily choose the slower one.

Raising the reference to 10000 (10 Gbps) restores the distinction. But cost is compared across the area, so **every router must use the same reference bandwidth** or the path calculations disagree and traffic takes strange routes. Change it on all of them, or on none.

17.4 DR and BDR on broadcast links

💡 Why a designated router exists

On a broadcast segment with n routers, full adjacencies between every pair would be $n(n-1)/2$ relationships — ten routers would need forty-five. Instead one router is elected **DR** and one **BDR**, and everyone else becomes a **DROTHER** adjacent only to those two. Flooding goes through the DR.

The election: **highest OSPF interface priority** wins; if priorities tie, **highest router ID** wins. Default priority is 1. Priority 0 means “never be DR”.

Two things surprise people. The election is **per segment**, not per router — a router can be DR on one link and DROTHER on another. And it is **non-preemptive**: a new router with a better priority does not take over until the current DR fails. To force it, clear the process or bounce the interface.

➤ Configure — influencing the election, and avoiding it

```
! Make this router the DR on the segment.
interface GigabitEthernet0/0
  ip ospf priority 255
!
! Never allow this one to be DR or BDR.
interface GigabitEthernet0/1
  ip ospf priority 0
!
! On a link with exactly two routers, skip the election entirely.
! Point-to-point forms an adjacency faster and elects nobody.
interface GigabitEthernet0/2
  ip ospf network point-to-point
```

17.5 Verifying

Q R1# show ip ospf neighbor

Neighbor ID	Pri	State	Dead Time	Address	Interface
2.2.2.2	1	FULL/DR	00:00:33	10.255.0.2	GigabitEthernet0/1
3.3.3.3	1	FULL/BDR	00:00:31	10.255.0.3	GigabitEthernet0/1
4.4.4.4	0	FULL/ -	00:00:38	10.255.1.2	GigabitEthernet0/2

State	Meaning
FULL	Databases synchronised. This is the healthy state
2-WAY	Normal <i>between two DROTHERs</i> on a broadcast segment — they see each other but do not need full adjacency. Not a fault
EXSTART, EXCHANGE	Mid-negotiation. Stuck here usually means an MTU mismatch
INIT	Hellos received but this router is not listed in them — often one-way communication or an access list
FULL/ -	Full, on a point-to-point link where there is no DR

Q R1# show ip ospf interface GigabitEthernet0/1

```
GigabitEthernet0/1 is up, line protocol is up
Internet Address 10.255.0.1/30, Area 0, Attached via Network Statement
Process ID 10, Router ID 1.1.1.1, Network Type BROADCAST, Cost: 10
Designated Router (ID) 2.2.2.2, Interface address 10.255.0.2
Backup Designated router (ID) 3.3.3.3, Interface address 10.255.0.3
Timer intervals configured, Hello 10, Dead 40, Wait 40, Retransmit 5
Neighbor Count is 2, Adjacent neighbor count is 2
```

This one output carries four of the five things that must match: area, network type, hello and dead intervals, and the subnet. When an adjacency will not form, run it on *both* routers and compare line by line.

! Common Pitfalls

- **Assuming the process number must match.** It is locally significant. router ospf 10 peers happily with router ospf 65.
- **Using a subnet mask in the network statement.** It takes a wildcard. network 10.255.0.0 255.255.255.252 does not do what you expect.
- **Leaving the router ID to chance.** It changes when interfaces change, and every adjacency rebuilds.
- **Panicking at 2-WAY.** Between two DROTHERs it is correct.

- **Changing reference bandwidth on one router.** See the warning above.
- **Forgetting passive-interface on user VLANs.** Hellos on a LAN full of PCs are wasted and expose the protocol.

🔧 Break & Fix — Adjacency stuck in EXSTART.

Two routers see each other but never reach FULL:

```
R1# show ip ospf neighbor
```

Neighbor ID	Pri	State	Dead Time	Address	Interface
2.2.2.2	1	EXSTART/DR	00:00:35	10.255.0.2	GigabitEthernet0/1

Hellos are being exchanged — so area, timers, subnet and network type all match, or the neighbour would not be listed at all. The failure is later, during database exchange.

```
R1# show ip ospf interface GigabitEthernet0/1 | include MTU|Internet
```

```
Internet Address 10.255.0.1/30, Area 0, Attached via Network Statement
MTU is 1500 bytes
```

```
R2# show ip ospf interface GigabitEthernet0/1 | include MTU|Internet
```

```
Internet Address 10.255.0.2/30, Area 0, Attached via Network Statement
MTU is 1400 bytes
```

Diagnosis. An MTU mismatch. OSPF checks MTU during database description exchange and refuses to proceed if the two ends disagree — which is exactly why the adjacency reaches EXSTART and stops there. Being stuck in EXSTART or EXCHANGE is close to diagnostic of this.

Fix. Make the MTUs match: `ip mtu 1500` on R2's interface, or 1400 on R1's if the smaller value was deliberate. The adjacency will move to FULL within a dead interval. Do not use `ip ospf mtu-ignore` except as a temporary measure — it suppresses the symptom and leaves a real MTU mismatch to fragment traffic later.

🏢 Lab 17.1 — Three routers, one area (*Packet Tracer*)

Topology. R1, R2, R3 on a shared Ethernet segment; each with a LAN; R1 also point-to-point to R3.

Task 1. Configure loopbacks and explicit router IDs 1.1.1.1, 2.2.2.2, 3.3.3.3. Enable OSPF area 0 on all links and LANs, with LANs passive.

Task 2. Confirm every neighbour reaches FULL. Record which router became DR and which BDR, and explain the election from the router IDs.

Task 3. Set `ip ospf priority 255` on R3's segment interface. Note that nothing changes, then bounce the interfaces and note that it does. Explain why.

Task 4. On the R1–R3 link, set `ip ospf network point-to-point` at both ends. Confirm the state becomes FULL/ - with no DR.

Task 5. Change the hello interval on R1 only. Watch the adjacency drop, and find the mismatch with `show ip ospf interface`.

Task 6. Set `ip mtu 1400` on one end and reproduce the EXSTART fault above.

Task 7. Extension. Raise the reference bandwidth on one router only and observe the asymmetric routing that results.

Checkpoint — Test Yourself

1. Two routers on a broadcast segment show 2-WAY and never go FULL. Under what circumstance is this correct rather than a fault?
2. Which wildcard mask enables OSPF on 172.16.8.0/22?
3. A router's OSPF adjacencies all rebuild every time one interface flaps. What is the cause, and what is the one-line fix?

Chapter Summary

- OSPF floods a description of every link, so all routers in the area share one map and converge consistently.
- Area, hello and dead timers, subnet and mask, and network type must all match, or no adjacency forms. Router IDs must be unique.
- Configure the router ID explicitly; otherwise it moves and adjacencies rebuild.
- Network statements take wildcard masks. The process number is local.
- DR and BDR are elected per segment by priority then router ID, and the election is non-preemptive. `ip ospf network point-to-point` avoids it entirely on two-router links.
- FULL is healthy; 2-WAY between DROTHERs is healthy; stuck in EXSTART means MTU.

Review Questions

1. (MCQ) Which is chosen as router ID if no `router-id` is configured and a loopback exists?
(a) lowest physical interface address (b) highest loopback address (c) highest physical address
(d) the process number
2. (MCQ) Two routers are stuck in EXSTART. The most likely cause is (a) area mismatch
(b) MTU mismatch (c) timer mismatch (d) duplicate router ID
3. (Short) Explain why the OSPF process number does not need to match between neighbours, and name a protocol where the equivalent number does.
4. (Short) Give the reason for setting `ip ospf priority 0` on an interface, and name a situation where you would.
5. (Applied) Write a complete single-area OSPFv2 configuration for a router with a loopback, one point-to-point link to a neighbour and two user VLANs, and list the two `show` commands you would use to prove it is correct.

OSPFv3 for IPv6

EXAM TOPICS **3.3** Learning Outcomes

By the end of this chapter you will be able to:

- **State** what OSPFv3 keeps from OSPFv2 and what it changes.
- **Configure** single-area OSPFv3 on interfaces rather than with network statements.
- **Explain** why an IPv6-only router still needs an IPv4-format router ID, and configure one.
- **Troubleshoot** an OSPFv3 adjacency, including the failure unique to it: a missing router ID.

 Before You Start

Chapter 17 — this chapter assumes it and covers only the differences. Chapter 5 for link-local addressing.

 Key Terms in This Chapter

OSPFv3 • router ID • link-local next hop • per-interface configuration • address family • ff02::5 • ff02::6

 One topic, two protocols

Topic 3.3 covers OSPFv2 and OSPFv3 together, and OSPFv3 is new in v2.0 — it was not in the v1.1 blueprint at all. The exam will not ask you to re-learn OSPF; it will ask whether you can configure the v6 version, which means knowing the handful of things that differ.

18.1 What stays the same

Almost everything conceptual. The link-state mechanism, the database, the shortest-path calculation, areas, the DR and BDR election with its priority-then-router-ID tie-break, the FULL and 2-WAY states, cost as the metric, hello and dead intervals that must match, and the administrative distance of 110 are all identical to Chapter 17.

So is the troubleshooting. If an OSPFv3 adjacency will not form, the causes are the same list: area, timers, network type, MTU, duplicate router ID.

18.2 What changes

	OSPFv2	OSPFv3
Enabled by	network statements under the process	Per interface , with <code>ipv6 ospf 10 area 0</code>
Advertises	The subnets matched by network statements	Every prefix on an enabled interface, automatically
Neighbour address	The interface's IPv4 address	The neighbour's link-local address
Multicast groups	224.0.0.5, 224.0.0.6	ff02::5, ff02::6

	OSPFv2	OSPFv3
Router ID	32-bit, may be taken from an interface	32-bit, in IPv4 format, and must be configured if no IPv4 address exists anywhere
Authentication	Built into the protocol	Delegated to IPsec

⚠ The failure unique to OSPFv3

The router ID is still a 32-bit value written like an IPv4 address, even in a network with no IPv4 at all. If the router has *no* IPv4 address on any interface and you have not configured `router-id`, the process will not start. IOS says so, but the message is easily missed in a long paste:

```
% OSPFv3-1-IPV6 Router process 10 could not pick a router-id, please configure manually
```

Always configure it explicitly. It costs one line and removes the whole class of problem.

18.3 Configuring it

➤ Configure — single-area OSPFv3 on R1

```
ipv6 unicast-routing
!
! The process exists mainly to hold the router ID; interfaces do
! the rest of the work.
ipv6 router ospf 10
  router-id 1.1.1.1
  passive-interface Vlan10
!
! Enable per interface. Every IPv6 prefix on the interface is
! advertised -- there is no network statement and no wildcard mask.
interface GigabitEthernet0/1
  description Link to R2
  ipv6 address 2001:db8:acad:99::1/64
  ipv6 ospf 10 area 0
!
interface Vlan10
  ipv6 address 2001:db8:acad:10::1/64
  ipv6 ospf 10 area 0
!
! Point-to-point on a two-router link, exactly as in OSPFv2.
interface GigabitEthernet0/2
  ipv6 ospf network point-to-point
```

💡 No network statements is a simplification, not a trap

In OSPFv2 you had to compute a wildcard mask and remember that the statement both enables the interface and advertises the subnet. OSPFv3 drops all of that: enable the interface and every prefix on it is advertised, including additional addresses added later. The practical consequence is that `passive-interface` becomes more important, not less — it is now the only way to advertise a LAN's prefix without sending hellos into it.

18.4 Verifying

Q R1# show ipv6 ospf neighbor

OSPFv3 Router with ID (1.1.1.1) (Process ID 10)

Neighbor ID	Pri	State	Dead Time	Interface ID	Interface
2.2.2.2	1	FULL/BDR	00:00:31	4	GigabitEthernet0/1
3.3.3.3	1	FULL/DR	00:00:35	4	GigabitEthernet0/1

The neighbour IDs are still IPv4-format. The column that replaces the IPv4 address is **Interface ID** — the neighbour's own index for its interface, which matters only when reading the database.

Q R1# show ipv6 route ospf

```
IPv6 Routing Table - default - 9 entries
Codes: C - Connected, L - Local, S - Static, O - OSPF intra, OI - OSPF inter
O   2001:DB8:ACAD:20::/64 [110/2]
    via FE80::2, GigabitEthernet0/1
O   2001:DB8:ACAD:30::/64 [110/3]
    via FE80::3, GigabitEthernet0/1
```

⚠ The next hop is a link-local address, and that is correct

Every OSPFv3 route resolves to a `fe80::` next hop plus an interface. This looks wrong to anyone used to IPv4 and is not: link-local addresses are guaranteed to exist on every IPv6 interface and cannot be renumbered, so they are the stable choice. This is also why the interface must always be named alongside them (Chapter 16).

Q R1# show ipv6 ospf interface brief

Interface	PID	Area	Intf ID	Cost	State	Nbrs	F/C
Gi0/1	10	0	4	1	BDR	2/2	
Gi0/2	10	0	5	1	P2P	1/1	
Vl10	10	0	8	1	DR	0/0	

! Common Pitfalls

- **No router ID on an IPv6-only router.** The process will not start. This is the single most common OSPFv3 fault.
- **Looking for network statements.** There are none; enable the interface.
- **Forgetting ipv6 unicast-routing.** As ever. Interfaces come up, OSPFv3 forms adjacencies, and nothing is forwarded.
- **Expecting a global next hop in the routing table.** Link-local is correct.
- **Assuming OSPFv2 and OSPFv3 share anything at runtime.** They are separate processes with separate databases and separate neighbours. A working v2 adjacency tells you nothing about v3 on the same link.

🔧 Break & Fix — OSPFv3 configured on both routers; no neighbours at all.

R1 and R2 both have IPv6 addresses on the link, ping each other's link-local addresses successfully, and have identical-looking OSPFv3 configuration. `show ipv6 ospf neighbor` is empty on both.

Q R1# show ipv6 ospf

```
%OSPFv3: Router process 10 is not running, please configure a router-id
```

Diagnosis. Neither router has any IPv4 address anywhere — this is an IPv6-only lab — so neither could derive a 32-bit router ID, and the process never started. The interface

commands were accepted and are sitting in the configuration doing nothing, which is why the configuration looks right.

The tell is that *both* ends are empty. A mismatch of area, timers or MTU usually leaves at least one side reporting something.

Fix. Configure a router ID on each router explicitly:

➤ Configure — starting the process

```
ipv6 router ospf 10
 router-id 1.1.1.1
```

The process starts immediately and adjacencies form within a hello interval. Configure a distinct ID on R2 — duplicate router IDs are the next fault along, and they produce a flapping adjacency rather than none.

🔧 Lab 18.1 — Dual-stack routing, both protocols side by side (*Packet Tracer*)

Topology. The three-router topology from Chapter 17, now addressed for IPv4 and IPv6 simultaneously.

Task 1. Keep the working OSPFv2 configuration. Add IPv6 addresses to every link and LAN, and `ipv6 unicast-routing` everywhere.

Task 2. Configure OSPFv3 with explicit router IDs matching the v2 ones. Enable it per interface, LANs passive.

Task 3. Confirm `show ipv6 ospf neighbor` reaches FULL on every link, and that the DR and BDR are the same routers as for OSPFv2. Explain why they need not have been.

Task 4. Compare `show ip route ospf` with `show ipv6 route ospf`. Note that v4 next hops are global and v6 next hops are link-local.

Task 5. Remove the router ID from one router and clear the process. Reproduce the “process not running” fault, then fix it.

Task 6. Set one link to `ipv6 ospf network point-to-point` at one end only and observe the result.

Task 7. Extension. Shut a link and time how long each protocol takes to reconverge. They should be comparable — the algorithm is the same.

❓ Checkpoint — Test Yourself

1. Why does an IPv6-only router still need a router ID in IPv4 format, and what happens if it cannot get one?
2. OSPFv2 works on a link and OSPFv3 does not. What does the working v2 adjacency prove about the v3 problem?
3. Where is OSPFv3 enabled, and what is advertised as a result?

📋 Chapter Summary

- OSPFv3 is OSPF. The database, the algorithm, the states, the DR election and the troubleshooting are unchanged.
- It is enabled per interface with `ipv6 ospf <pid> area <n>`, and every prefix on that interface is advertised. No network statements, no wildcards.
- Neighbours are reached by link-local address, so routes show a `fe80::` next hop plus an interface. That is correct.

- The router ID is still 32-bit IPv4 format and **must** be configured on an IPv6-only router, or the process will not start.
- v2 and v3 are independent processes. One working says nothing about the other.

Review Questions

1. **(MCQ)** How is OSPFv3 enabled on an interface? (a) `network` statement under the process (b) `ipv6 ospf 10 area 0` on the interface (c) automatically, on all IPv6 interfaces (d) `ipv6 router ospf 10` on the interface
2. **(MCQ)** Which multicast group do OSPFv3 routers use for hellos? (a) 224.0.0.5 (b) ff02::1 (c) ff02::5 (d) ff02::a
3. **(Short)** Explain why an OSPFv3 route's next hop is a link-local address rather than a global one.
4. **(Short)** A router running only IPv6 reports that the OSPFv3 process is not running. State the cause and the fix.
5. **(Applied)** Write the OSPFv3 configuration for a dual-stack router with one point-to-point uplink and two user VLANs, and explain which two commands have no OSPFv2 equivalent and why.

First Hop Redundancy: HSRP and VRRP

EXAM TOPICS **3.4**

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** the problem a first hop redundancy protocol solves, and why routing protocols do not solve it.
- **Interpret** the operational status of HSRP and VRRP — which the blueprint asks for by name.
- **Configure** basic HSRP and VRRP with priority and preemption.
- **Diagnose** the two classic faults: both routers active, and a gateway that fails over but does not fail back.

🔗 Before You Start

Chapter 15 for gateways; Chapter 10 for SVIs, which is where these protocols usually live.

📖 Key Terms in This Chapter

first hop redundancy • virtual IP • virtual MAC • HSRP • VRRP • active • standby • master • backup • priority • preemption • group number • tracking

🎓 The verb is *interpret*, not *configure*

Topic 3.4 is “interpret the operational status of First Hop Redundancy Protocols (HSRP and VRRP)”. You are far more likely to be shown `show standby brief` and asked which router is forwarding, or why a failover did not happen, than to be asked to configure it from scratch. Configuration is here because you cannot interpret what you have never built — but read the verification sections hardest.

19.1 The problem

Every host is configured with one default gateway address. If the router holding that address fails, every host on the subnet is cut off — and no routing protocol helps, because the hosts are not running one. They have a single static address and no way to learn a new one quickly.

💡 The solution: a gateway address that is not tied to a router

Two routers share a **virtual IP address** and a **virtual MAC address**. Hosts are configured with the virtual IP as their gateway and never know that two routers exist. One router is elected to answer for it; the other waits.

When the active router fails, the standby takes over *both* the virtual IP and the virtual MAC. Because the MAC does not change, the hosts’ ARP caches stay valid and no host has to do anything at all. That is the whole trick, and it is why the virtual MAC matters as much as the virtual IP.

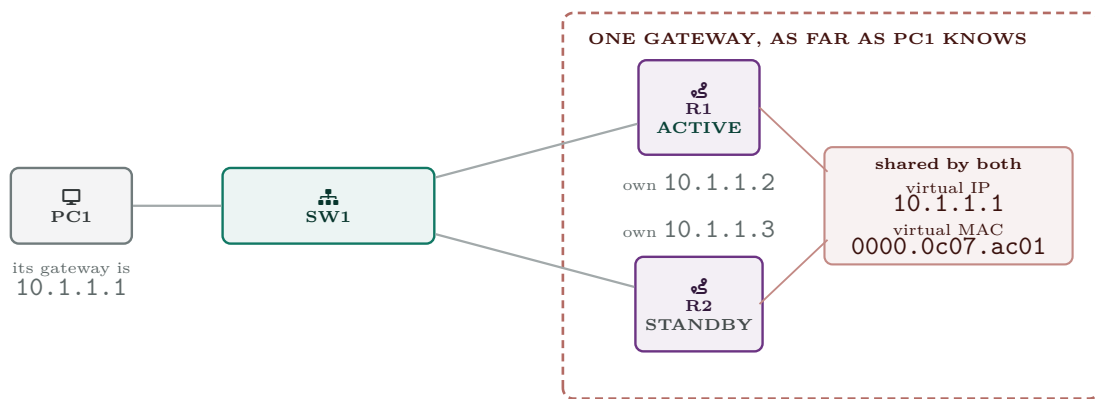


Figure 19.1: The host is configured once and never reconfigured. Failover moves the virtual IP *and* the virtual MAC, which is why nothing on the LAN has to re-learn anything.

	HSRP	VRRP
Standard	Cisco proprietary	IETF, RFC 5798
Role names	Active and standby	Master and backup
Default priority	100	100
Preemption	Off by default	On by default
Virtual MAC	0000.0c07.acXX (XX = group)	0000.5e00.01XX
Virtual IP	Must be an address no router owns	May be an address one router owns; that router then always wins

⚠ The preemption default is the difference that catches people

HSRP does **not** preempt by default. Configure a router with priority 150, reload it, and when it comes back the other router — priority 100 — keeps the active role. Your carefully planned primary is now the standby, and everything still works, so nobody notices until the second failure.

VRRP preempts by default, so it behaves the way most people expect.

If you want a specific router to be primary, configure **standby 10 preempt** explicitly. This is the single most commonly missed HSRP command.

19.2 Configuring HSRP

>_ Configure — HSRP on two multilayer switches, VLAN 10

```
! ----- CORE1: the intended primary -----
interface Vlan10
ip address 192.168.10.2 255.255.255.0
! The virtual IP -- this is what hosts use as their gateway.
standby 10 ip 192.168.10.1
! Higher priority wins the election.
standby 10 priority 150
! Without this, CORE1 will NOT take the role back after a reload.
standby 10 preempt
! Give up the role if the uplink dies -- otherwise CORE1 stays
! active while having no path to anywhere.
```

```
standby 10 track GigabitEthernet0/24 60
standby 10 name VLAN10-GW
!
! ----- CORE2: the standby -----
interface Vlan10
ip address 192.168.10.3 255.255.255.0
standby 10 ip 192.168.10.1
standby 10 priority 100
standby 10 preempt
```

💡 Tracking, and why it is not optional

Without tracking, the active router keeps the role as long as its own VLAN interface is up. If its *uplink* fails, it is still active, still answering for the gateway, and has nowhere to send the traffic — a black hole that the standby router cannot detect, because from its point of view the active router is perfectly healthy.

`standby 10 track Gi0/24 60` decrements the priority by 60 when that interface goes down: 150 becomes 90, which is below CORE2's 100, so CORE2 preempts and takes over. The decrement must be large enough to cross the gap between the two priorities, or nothing happens.

19.3 Interpreting the status

Q CORE1# show standby brief

P indicates configured to preempt.
|

Interface	Grp	Pri	P	State	Active	Standby	Virtual IP
Vl10	10	150	P	Active	local	192.168.10.3	192.168.10.1
Vl20	20	100	P	Standby	192.168.10.3	local	192.168.20.1

Read it column by column. Pri is this router's priority. P means preemption is configured — its absence is the fault described above. State is the role. Active and Standby name who holds each role, where local means this router.

Note that VLAN 10 and VLAN 20 have opposite roles here. That is deliberate load sharing: half the VLANs are active on each core switch, so both boxes carry traffic instead of one sitting idle.

Q CORE1# show standby Vlan10

```
Vlan10 - Group 10
State is Active
  2 state changes, last state change 04:12:19
Virtual IP address is 192.168.10.1
Active virtual MAC address is 0000.0c07.ac0a
  Local virtual MAC address is 0000.0c07.ac0a (v1 default)
Hello time 3 sec, hold time 10 sec
Preemption enabled
Active router is local
Standby router is 192.168.10.3, priority 100 (expires in 8.048 sec)
Priority 150 (configured 150)
  Track interface GigabitEthernet0/24 state Up decrement 60
```

Two lines to check first: Priority 150 (configured 150) — if the two numbers differ, a tracked interface is down and has decremented it — and Preemption enabled.

Q CORE1# show vrrp brief

Interface	Grp	Pri	Time	Own	Pre	State	Master addr	Group addr
Vl30	30	110	3570	Y	Y	Master	192.168.30.2	192.168.30.1

! Common Pitfalls

- **Forgetting preempt on HSRP.** The intended primary never takes the role back.
- **Mismatched group numbers.** Two routers with different group numbers on the same VLAN both become active, each for its own group. The symptom is intermittent connectivity as hosts ARP for a virtual IP that two devices answer.
- **Mismatched virtual IPs.** Same result.
- **A track decrement too small to matter.** 150 with a decrement of 10 still beats 100. Size the decrement against the gap.
- **Configuring the virtual IP as a router's real address in HSRP.** Not allowed. VRRP permits it; HSRP does not.
- **Assuming the gateway works because the routers are up.** Test by pinging the *virtual* address from a host, then failing the active router and pinging again.

🔧 Break & Fix — Both routers say they are active.

Users on VLAN 10 report intermittent loss. Both core switches report the active role for group 10.

Q CORE1# show standby brief

Interface	Grp	Pri	P	State	Active	Standby	Virtual IP
Vl10	10	150	P	Active	local	unknown	192.168.10.1

Q CORE2# show standby brief

Interface	Grp	Pri	P	State	Active	Standby	Virtual IP
Vl10	11	100	P	Active	local	unknown	192.168.10.1

Diagnosis. Look at the Grp column: CORE1 is in group 10, CORE2 in group 11. They are not peers at all — they are two independent HSRP groups that happen to share a virtual IP address. Each sees no partner, so each elects itself active, and each answers ARP for 192.168.10.1 with a *different* virtual MAC (0000.0c07.ac0a against 0000.0c07.ac0b). Hosts get whichever reply arrives first, and traffic splits unpredictably between two routers.

The giveaway is **Standby unknown** on both sides. A healthy pair always names its partner.

Fix. Correct the group number on CORE2 to 10. Within a hello interval the two will see each other, the priorities decide, and one becomes standby. Confirm that each side names the other in the **Standby** column, and that only one virtual MAC is now in use.

🔧 Lab 19.1 — Fail a gateway and watch it survive (Packet Tracer)

Topology. Two multilayer switches as redundant gateways for VLANs 10 and 20; an access switch; two PCs.

Task 1. Address both cores in VLAN 10 and configure HSRP group 10 with virtual IP 192.168.10.1, CORE1 at priority 150 with preempt.

Task 2. Point the PCs at the virtual IP and confirm they can reach the rest of the network. Record the ARP entry a PC holds for its gateway — note the virtual MAC.

Task 3. Run `show standby brief` on both. Identify active, standby and the preempt flag.

Task 4. Start a continuous ping from a PC, then shut CORE1's VLAN 10 interface. Count how many pings are lost and confirm the PC's ARP entry has *not* changed.

Task 5. Bring CORE1 back and confirm it resumes the active role. Then remove **preempt**, repeat, and observe that it does not.

Task 6. Configure tracking on CORE1's uplink with a decrement of 60. Shut the uplink and confirm failover; use a decrement of 10 instead and confirm it does not.

Task 7. Configure VLAN 20 with CORE2 as primary, so the two share the load.

Task 8. Extension. Repeat the first four steps with VRRP on a third VLAN and note the difference in default preemption behaviour.

? Checkpoint — Test Yourself

1. Why does a host not need to update its ARP cache when the active router fails?
2. `show standby brief` shows `Standby unknown` on both routers. What does that indicate?
3. A router is configured with priority 150 and a tracked interface with decrement 10. Its peer is at 100. The tracked interface fails. What happens, and is it what was intended?

✓ Chapter Summary

- Hosts have one static gateway address, so redundancy has to be provided below them: two routers share a virtual IP *and* a virtual MAC.
- HSRP is Cisco, uses active and standby, and **does not preempt by default**. VRRP is standard, uses master and backup, and does.
- Priority decides; preemption decides whether the better router takes the role back.
- Track the uplink, with a decrement large enough to cross the priority gap, or the active router can become a black hole.
- Interpret `show standby brief` by columns: priority, the P flag, state, and who is named as partner. `Standby unknown` means the two are not peering.

🔧 Review Questions

1. (MCQ) Which protocol preempts by default? (a) HSRP (b) VRRP (c) both (d) neither
2. (MCQ) In `show standby brief`, what does the P column indicate? (a) priority (b) preemption configured (c) primary (d) point-to-point
3. (Short) Explain why the virtual MAC address matters as much as the virtual IP address during a failover.
4. (Short) An HSRP router is active but has lost its uplink. What feature prevents this, and how must it be sized?
5. (Applied) Two core switches serve VLANs 10, 20, 30 and 40. Design an HSRP arrangement that shares the load evenly and survives the loss of either switch, and write the configuration for one VLAN on each switch.

PART IV

Network Services and Security

Parts I to III built a network that forwards packets. Nobody uses a network like that. Users expect an address to appear without asking for one, a name to resolve to that address, a private network to reach the internet through one public address, and somebody to have decided which of them is allowed where.

That is this part. It is the largest single block of the book because it carries two blueprint domains at once: all seven topics of domain 4, plus topic 1.7 from domain 1, which sits here rather than in Part I because troubleshooting DHCP is impossible before you know what a relay is doing.

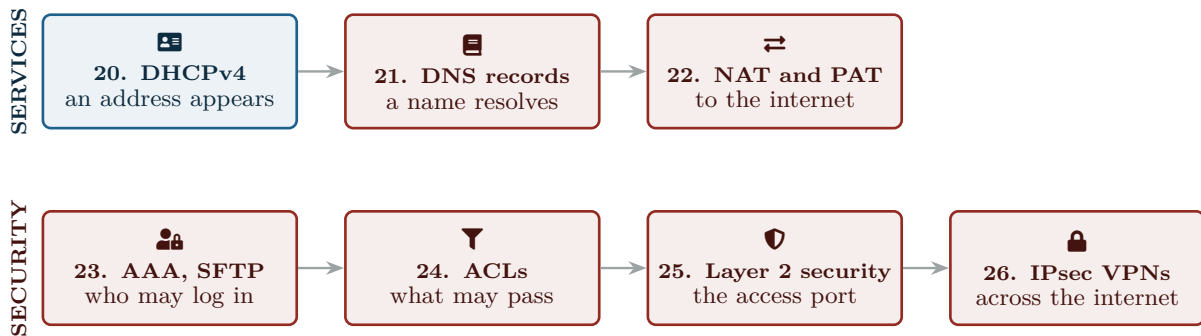


Figure P4.1: Part IV. Three chapters make the network usable; four decide who is allowed to use it. Chapter 20 belongs to domain 1 and the rest to domain 4.

Six verbs, and only one of them is *describe*

Topic 1.7 says *troubleshoot*. Topics 4.1, 4.3, 4.6 and 4.7 say *configure*. Topic 4.2 says *manage* and 4.4 says *diagnose*. Only 4.5, the VPN topic, says *describe* — which is why Chapter 26 is the shortest chapter in the part and has no lab.

Read that distribution as an instruction. Four of these seven chapters will be examined by putting a configuration in front of you and asking what it does or what is wrong with it. Reading an ACL correctly is worth more marks here than being able to recite what a wildcard mask is.

Two platforms from here on

Parts I–III were written for Packet Tracer, which models switching and routing well enough. From this part onward the labs move to **CML** or **GNS3** where the platform matters: topic 4.3 names **IOS XE** explicitly, and SFTP, SCP and AAA against a real RADIUS server need a control plane that Packet Tracer only pretends to have.

Every lab header names its platform. Where a lab can still be done usefully in Packet Tracer with a caveat, the caveat is stated rather than hidden.

DHCP: Client, Server and Relay

EXAM TOPICS **1.7**

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Describe** the four DHCPv4 messages and say which of them are broadcast and why that matters.
- **Configure** an IOS router in each of the three roles: client, server and relay agent.
- **Interpret** `show ip dhcp binding`, `show ip dhcp pool` and `show ip dhcp conflict`.
- **Troubleshoot** a host that has no address, has the wrong address, or has an address but no connectivity — and tell those three apart before touching anything.

🔗 Before You Start

Chapter 4 for addressing and subnet boundaries. Chapter 8 and Chapter 10 for SVIs — a DHCP fault is very often a VLAN fault wearing a disguise. Chapter 6 for UDP and for broadcast versus unicast.

📖 Key Terms in This Chapter

DORA • DHCPDISCOVER • lease • binding • relay agent • ip helper-address • giaddr • excluded address • APIPA • conflict

🎓 The verb is *troubleshoot*, not *configure*

Topic 1.7 reads *troubleshoot DHCPv4 client, server, and relay on IOS devices*. It is worth reading twice. The exam is not primarily asking you to build a DHCP pool from nothing — it is asking you to be shown a pool, a relay and a client that between them are not working, and to say which of the three is at fault.

Note also **DHCPv4**. There is no DHCPv6 in this topic. IPv6 hosts in this course get their addresses by SLAAC (Chapter 5).

That is why this chapter spends more space on *how to tell the three failure classes apart* than on the pool syntax, which is four lines long.

20.1 What DHCP is solving

A host that has just booted knows its own MAC address and nothing else. It has no IP address, so it cannot be the destination of a routed packet; it does not know the address of a server, so it cannot ask one directly; and it does not know its own subnet, so it cannot even work out what a sensible address would be.

Everything odd about DHCP follows from that opening position. The first message has to be a broadcast, because there is no unicast address to use in either direction. And broadcasts do not cross routers, which is the whole reason relay agents exist.

📖 DHCPv4

A client-server protocol that leases an IPv4 address and a set of configuration options — default gateway, DNS servers, domain name, lease time — to a host for a bounded period. The client runs on UDP port 68, the server on UDP port 67.

20.1.1 DORA

Four messages, in a fixed order, remembered as **DORA**.

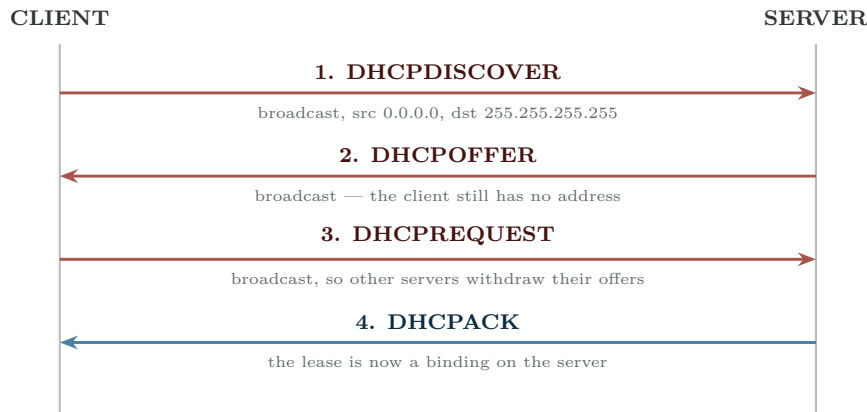


Figure 20.1: DORA. Three of the four messages are broadcasts, which is why a router between client and server stops the process dead unless it is configured to relay.

Two details in that figure earn their place on the exam.

- **The REQUEST is broadcast, not unicast.** If two servers both offered, the request names the one the client accepted, and the other server sees it and returns its offered address to the pool. A unicast request would leave the second address reserved for nothing.
- **The client is not finished at the OFFER.** A host that receives an offer and then loses the server still has no address. “It got an offer” is not the same as “it is working”.

💡 Renewal is not DORA

At half the lease time (T1), a client that already has an address sends a *unicast* DHCPREQUEST straight to its server and expects an ACK. There is no discover and no offer. This matters for troubleshooting: a fault that only breaks broadcast forwarding — a dead relay, a wrong VLAN — can leave existing hosts working perfectly for hours while every newly booted host fails. “It works for everyone except the machines that rebooted this morning” is a relay fault until proved otherwise.

20.2 Role 1: IOS as a client

A router interface can take its address from DHCP. This is normal on the WAN interface of a small branch router.

>_ Configure — a router that leases its own WAN address

```
interface GigabitEthernet0/0/0
  description WAN to ISP
  ip address dhcp
  no shutdown
```

🔍 R1# show ip interface brief

Interface	IP-Address	OK?	Method	Status	Protocol
GigabitEthernet0/0/0	203.0.113.24	YES	DHCP	up	up
GigabitEthernet0/0/1	192.168.10.1	YES	manual	up	up

The **Method** column is the whole point of that output. DHCP means the address was leased; **manual** means somebody typed it. A WAN interface showing **unassigned** with the line protocol up is a router that asked and got no answer.

On the exam

A router that is a DHCP client normally also wants the default route the lease carries. It installs one automatically as a static default with an administrative distance of 254, so anything you configure yourself will win. Check with **show ip route static** if a branch router is reaching the internet by a path you did not expect.

20.3 Role 2: IOS as a server

Small sites often use the router itself rather than a dedicated server. Four things must be right, and they must be right *together*.

>_ Configure — a DHCP pool for VLAN 10

```
! 1. Reserve the addresses that are already spoken for -- the gateway,
!   the switch management SVIs, the printers, the servers.
! Do this FIRST. A pool created before the exclusions can hand out
! the gateway address in the seconds between the two commands.
ip dhcp excluded-address 192.168.10.1 192.168.10.20
ip dhcp excluded-address 192.168.10.250 192.168.10.254
!
ip dhcp pool VLAN10-STAFF
  network 192.168.10.0 255.255.255.0
  default-router 192.168.10.1
  dns-server 192.168.20.10 192.168.20.11
  domain-name campus.example.edu
  lease 0 8 0
!
! lease is days hours minutes. "lease 0 8 0" is eight hours, which
! suits a teaching lab; "lease infinite" suits nothing.
```

The network statement must match the interface's subnet

The router decides which pool to use by looking at the address of the interface the request arrived on — or, for a relayed request, at the **giaddr** field. If VLAN 10's SVI is 192.168.10.1/24 and the pool says **network 192.168.10.0 255.255.255.0**, they match and the pool is used. If somebody later rennumbers the SVI to /25 and leaves the pool at /24, the pool no longer matches, the router silently declines to serve, and the symptom is a host with no address at all — with a perfectly valid-looking pool sitting in the configuration.

R1# show ip dhcp binding

```
Bindings from all pools not associated with VRF:
IP address      Client-ID/      Lease expiration   Type   State
                Hardware address
192.168.10.21   0100.5079.6668.01   Aug 28 2026 06:14 AM   Automatic Active
192.168.10.22   0100.5079.6668.22   Aug 28 2026 06:19 AM   Automatic Active
```

R1# show ip dhcp pool

```
Pool VLAN10-STAFF :
Utilization mark (high/low) : 100 / 0
Subnet size (first/next)     : 0 / 0
Total addresses               : 254
Leased addresses              : 2
Excluded addresses            : 25
Pending event                 : none
```

```

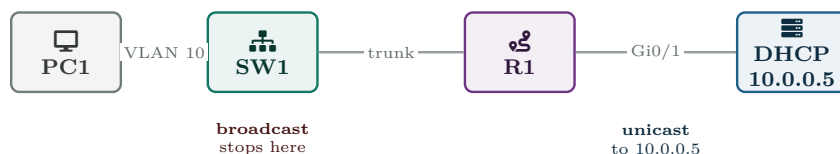
1 subnet is currently in the pool:
Current index  IP address range      Leased/Excluded/Total
192.168.10.23  192.168.10.1    - 192.168.10.254    2 / 25 / 254

```

Leased addresses against Total addresses is the number to read when somebody reports that “DHCP has stopped working” at nine in the morning. A pool that is full does not log an error to anybody who is watching; it simply stops answering.

20.4 Role 3: IOS as a relay agent

The DISCOVER is a broadcast, and a router does not forward broadcasts. So when the server is on a different subnet from the client — which is the normal case in any site with more than one VLAN — the router on the client’s subnet has to pick the broadcast up and forward it as a unicast.



➤ Configure — relaying VLAN 10’s requests to a central server

```

interface Vlan10
description Staff -- clients live here
ip address 192.168.10.1 255.255.255.0
ip helper-address 10.0.0.5

```

⚠ ip helper-address goes on the interface facing the *clients*

This is the single most common relay mistake and it is worth stating on its own. The command belongs on the SVI or interface in the subnet where the broadcasts are — the client side. Putting it on the interface facing the server does nothing at all, because no DISCOVER ever arrives there.

If a site has six client VLANs, the command goes on six SVIs.

💡 What giaddr does, and why the server needs it

When the relay forwards the DISCOVER it fills in the **gateway IP address** field, **giaddr**, with its own address on the interface that received the broadcast — 192.168.10.1 in the example above.

That single field does two jobs. It tells the server which subnet the client is on, so the server knows which pool to draw from; and it gives the server a unicast address to send the OFFER back to, so the relay can broadcast it onward to the client.

The consequence for troubleshooting: if the central server has no pool for 192.168.10.0/24, the relay is working perfectly and the client still gets nothing. Do not conclude “the relay is broken” from “the client got no address”.

🎓 ip helper-address forwards more than DHCP

By default it relays eight UDP services, not one — among them TFTP (69), DNS (53), NetBIOS name service (137) and TACACS (49) as well as DHCP (67/68). That is occasionally useful and occasionally a surprise, and it is a fair exam question. Trim it with `ip forward-protocol udp <port>` if a site wants only DHCP relayed.

20.5 Troubleshooting: three symptoms, three places to look

This is the part the blueprint actually asks for. Almost every DHCP fault presents as one of three symptoms, and each points at a different part of the path. Establish which one you have *before* you start changing configuration.

Symptom on the host	What it proves	Where to look
169.254.x.x, or “no address”	The host asked and heard nothing back. The failure is before the server ever formed an opinion.	Client VLAN and access port first (Chapter 14); then the relay — is <code>ip helper-address</code> on the client-side SVI? Then whether the SVI is up.
An address, but from the wrong subnet	Something answered, but from the wrong pool — or a rogue server answered first.	<code>giaddr</code> : the client is in a different VLAN from the one you think. Or an unauthorised server on the segment — see Chapter 25.
The right address, but nothing works	DHCP succeeded. This is not a DHCP fault at all.	The <i>options</i> , not the address: <code>default-router</code> and <code>dns-server</code> . A host with a good address and a wrong gateway reaches its own VLAN and nothing else.

⚠ The third row is the one people waste an afternoon on

“DHCP is broken” is reported when a user cannot reach the internet. Run `ipconfig /all` on the host. If it has an address in the right subnet, DHCP has done its job perfectly and the problem is elsewhere — routing, NAT, DNS or an ACL. Check the gateway and DNS values the lease carried, then leave DHCP alone.

20.5.1 Conflicts

Q R1# show ip dhcp conflict

IP address	Detection method	Detection time
192.168.10.24	Ping	Aug 27 2026 09:12 AM
192.168.10.31	Gratuitous ARP	Aug 27 2026 09:41 AM

A conflict means the router offered an address and then found somebody already using it. `Ping` means the server tested the address before offering and got a reply; `Gratuitous ARP` means a client took the address and immediately heard another host claim it.

Either way the cause is the same: a statically addressed device inside the pool range that nobody excluded. Conflicted addresses are removed from the pool and stay out, so a site that never clears them slowly runs out of addresses. Fix the exclusions first, then `clear ip dhcp conflict *`.

⚠ Common Pitfalls

- **Pool created before the exclusions.** There is a real window in which the gateway address can be leased to a laptop. Exclude first.
- **`ip helper-address` on the server-side interface.** Does nothing. It goes on the client side.
- **Pool network not matching the SVI’s subnet or mask.** Silent refusal to serve.
- **Assuming a working host proves DHCP works.** Existing leases renew by unicast and survive faults that stop every new host dead.

- **Forgetting the SVI must be up.** An SVI with no access port in its VLAN in the up state is down, so the pool it serves is unreachable — a layer 2 fault presenting as a DHCP fault.
- **Reading 169.254.x.x as a server problem.** It means nothing answered. Nine times in ten the fault is on the client's own switch port.

 **Break & Fix** — Every host in VLAN 30 gets 169.254 addresses. VLAN 10 and VLAN 20 are fine.

The DHCP server is a central router, R1, reached through a distribution switch. VLANs 10, 20 and 30 all have SVIs on DSW1, all three have pools on R1, and only VLAN 30 fails. Static addressing inside VLAN 30 works, and a statically addressed host in VLAN 30 can ping R1.

 DSW1# show run interface vlan 30

```
interface Vlan30
description Lab -- clients
ip address 192.168.30.1 255.255.255.0
!
```

Diagnosis. There is no `ip helper-address`. VLAN 10 and VLAN 20 have one; VLAN 30 was added later and the line was missed. Everything else about VLAN 30 is correct, which is exactly why static addressing works and why the subnet is reachable — routing is fine, layer 2 is fine, and only the broadcast relay is absent.

The tell is that the fault is confined to *one* VLAN. A server fault or a routing fault would take out all three; a switch-port fault would take out one host. One whole VLAN failing while its neighbours work points at something configured per-SVI, and on this path there is only one such thing.

Fix.

 **Configure** — restoring the relay

```
interface Vlan30
ip helper-address 10.0.0.5
```

Then check the second half. Confirm R1 actually has a pool whose `network` statement matches 192.168.30.0/24. A relay pointing at a server with no matching pool produces the identical symptom, and fixing one half of a two-part fault and declaring victory is how faults come back the next morning.

 **Lab 20.1** — DHCP in all three roles, then broken deliberately (*Packet Tracer*)

Topology. SW1 carrying VLANs 10, 20 and 30; DSW1 holding the three SVIs; R1 as the central server on 10.0.0.0/24; two PCs per VLAN.

- Task 1.** Address the SVIs and confirm inter-VLAN routing works with static host addresses before introducing DHCP at all. Do not skip this — it separates the layer 3 work from the DHCP work.
- Task 2.** On R1, exclude the first twenty addresses of each subnet, then build three pools with gateway, DNS and an eight-hour lease.
- Task 3.** Add `ip helper-address` to VLAN 10's SVI only. Boot one PC in VLAN 10 and one in VLAN 20. Record what each receives, and explain the difference in terms of `giaddr` before adding the second helper.
- Task 4.** Add the remaining helpers. Confirm all six PCs lease correctly and read `show ip dhcp binding` and `show ip dhcp pool` on R1.

Task 5. Break 1. Change VLAN 20's SVI mask to /25, leaving the pool at /24. Release and renew a VLAN 20 host. Explain the result.

Task 6. Break 2. Give a PC a static address inside a pool range, then force new leases until a conflict is detected. Read `show ip dhcp conflict`, fix the exclusion, and clear it.

Task 7. Break 3. Remove `default-router` from VLAN 30's pool. Renew a host. It will lease successfully and reach its own VLAN only. Write down the exact test that distinguishes this from a routing failure.

Task 8. Extension. Configure R1's uplink as `ip address dhcp` against a second router acting as an ISP, and observe the default route that appears with distance 254.

? Checkpoint — Test Yourself

1. Which of the four DORA messages are broadcast, and what would break if the REQUEST were unicast?
2. A host has 169.254.8.31. What single fact does that establish, and what does it *not* establish?
3. On which interface does `ip helper-address` belong, and what does the relay write into `giaddr`?
4. Existing hosts work; every rebooted host fails. What class of fault does that pattern point to, and why?
5. A host has the correct address and gateway but cannot browse the web. Is this a DHCP fault? Justify the answer.

✓ Chapter Summary

- DORA: Discover, Offer, Request, Ack. The first three are broadcast, which is why routers block the process by default.
- Renewal at T1 is a *unicast* request with no discover, so existing hosts survive faults that kill every new one.
- As a client: `ip address dhcp`, and the `Method` column of `show ip interface brief` proves it.
- As a server: exclude first, then a pool whose `network` statement matches the serving interface's subnet exactly.
- As a relay: `ip helper-address` on the **client-side** interface. It writes `giaddr`, which selects the pool and carries the reply back.
- Three symptoms, three places: no address means nothing answered; wrong subnet means the wrong pool or a rogue server; right address but no connectivity is not a DHCP fault.
- Conflicts come from static addresses inside the pool range. Fix the exclusion, then `clear ip dhcp conflict *`.

🔧 Review Questions

1. (MCQ) A client receives 169.254.14.2. What has happened?
 - (a) The server offered an address from the wrong pool
 - (b) The client received no DHCP reply and self-assigned
 - (c) The lease expired and was not renewed
 - (d) The relay agent rewrote `giaddr` incorrectly
2. (MCQ) Where must `ip helper-address` be configured?

- (a) On the interface facing the DHCP server
 - (b) On the SVI in the clients' own subnet
 - (c) On every interface of the router
 - (d) Under the `ip dhcp pool` for that subnet
3. **(Short)** Explain why the DHCPREQUEST is broadcast rather than sent directly to the server that made the offer.
4. **(Short)** A pool is configured, the SVI is up, the helper address is correct, and clients still receive nothing. Name two configuration errors that would produce exactly this and say how you would distinguish them.
5. **(Applied)** Write the complete configuration for a router serving two VLANs, 10.1.10.0/24 and 10.1.20.0/24, reserving the first ten addresses of each for infrastructure, with a four-hour lease and two DNS servers. Then state which single command you would add if the server were moved to a third subnet.
6. **(Applied)** You are handed `show ip dhcp pool` output showing `Leased addresses: 251` of `Total addresses: 254`. Users report intermittent failures. Give the immediate mitigation and the permanent fix, and say which you would do first and why.

DNS, and Diagnosing Records

EXAM TOPICS **4.4**

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Trace** a name resolution from a stub resolver to an authoritative server, and say which step a given failure is stuck at.
- **Identify** the six record types the blueprint names — A, AAAA, CNAME, MX, NS and PTR — and state what each is for.
- **Use** nslookup and dig to query a specific record type against a specific server.
- **Diagnose** the three failure classes the topic names: a host unreachable by name, a web application that half works, and mail that is rejected.

🔗 Before You Start

Chapter 6 for UDP, TCP and ports. Chapter 20 — the DNS server address usually arrives in a lease, so a DNS fault is sometimes a DHCP option fault. Chapter 5 for what an AAAA record contains.

📖 Key Terms in This Chapter

resolver • recursion • authoritative • TTL • zone • A • AAAA • CNAME • MX • NS • PTR • NXDOMAIN • SERVFAIL

📖 Read the topic wording carefully

Topic **4.4** is *diagnose issues with DNS records (A, AAAA, CNAME, MX, NS, and PTR) to support host, web application, and mail server access by name*.

Three things follow. The verb is *diagnose*, so you will be given a resolution that is wrong and asked why. The record types are enumerated, so those six and no others are examinable — there is no SOA, SRV or TXT question waiting for you. And the three purposes are enumerated too, which is a strong hint that questions will be framed as “the website works but mail bounces” rather than as “define a CNAME”.

You are not asked to configure a DNS server. You are asked to read one.

21.1 How a name becomes an address

📖 DNS

A distributed, hierarchical database mapping names to records. Queries normally travel over UDP port 53; responses too large for a datagram, and zone transfers between servers, use TCP port 53.

💡 The stub resolver explains most “it works for me” arguments

Your host is not doing the work in that figure. It sends one query to whichever resolver DHCP gave it and waits for a complete answer. Everything downstream — which root it used, whether the TLD was slow, whether a cached record is stale — is invisible to the host and different for every resolver.

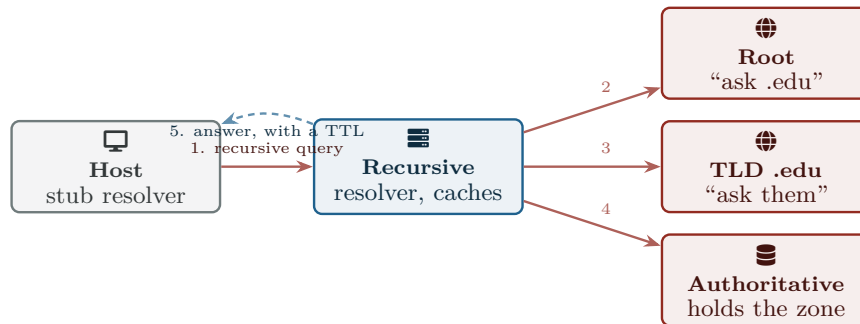


Figure 21.1: Only step 1 is recursive. The host asks one question and waits; the resolver does the walking and caches what it learns for the TTL.

So when one user cannot reach a site and another can, the first question is not “what is wrong with the site” but **“are they using the same resolver?”**. `ipconfig /all` or `resolvectl status` answers it in one line, and the answer resolves the argument more often than not.

21.1.1 TTL, and why a fix appears not to work

Every record carries a **time to live** in seconds. A resolver that answers from cache serves the record until the TTL expires, and it does not re-check early because somebody in another organisation changed something.

This is the source of the single most common DNS complaint: “we fixed the record an hour ago and it is still wrong.” If the old record had a 24-hour TTL, it is still wrong for up to 24 more hours, for anyone whose resolver cached it. There is no remote command to flush somebody else’s cache.

⚠ The practical consequence

Lower the TTL *before* a planned change, not after. Drop it to 300 seconds a day ahead, make the change, verify, then raise it again. Doing this in the right order is the difference between a five-minute cutover and a day of intermittent failure — and “why did the change take so long to appear” is a fair short-answer exam question.

21.2 The six record types

Type	Maps	Example	Failure it causes when wrong
A	Name → IPv4 address	<code>www</code> → 203.0.113.10	Name does not resolve at all, or resolves to a dead host
AAAA	Name → IPv6 address	<code>www</code> → 2001:db8::10	Slow or failing access from dual-stack clients only
CNAME	Name → another <i>name</i>	<code>shop</code> → <code>www</code>	Loops, or a name that cannot also hold other records
MX	Domain → mail exchanger, with a preference number	<code>example.edu</code> → 10 mail1	Inbound mail is not delivered or goes to the wrong host

Type	Maps	Example	Failure it causes when wrong
NS	Zone → the servers authoritative for it	example.edu → ns1	The whole zone becomes unresolvable — the widest fault here
PTR	IP address → name (reverse)	10.113.0.203. in-addr.arpa → mail1	Outbound mail rejected or scored as spam; logs show IPs

💡 Reverse lookups are a separate tree, not a mirror

A PTR record does not live in your forward zone. It lives in `in-addr.arpa` for IPv4 (or `ip6.arpa` for IPv6), with the octets reversed, and it is delegated to *whoever owns the address block* — normally your ISP or cloud provider, not you.

That is why forward and reverse so often disagree. Adding an A record for a new mail server takes you thirty seconds; adding the matching PTR may require a support ticket to somebody else. The exam likes this asymmetry because it explains a real failure: mail that leaves cleanly and is rejected at the far end.

21.3 Querying: the two tools

>_ Windows — `nslookup -type=MX example.edu 8.8.8.8`

```
Server:  dns.google
Address:  8.8.8.8

Non-authoritative answer:
example.edu      MX preference = 10, mail exchanger = mail1.example.edu
example.edu      MX preference = 20, mail exchanger = mail2.example.edu

mail1.example.edu      internet address = 203.0.113.25
mail2.example.edu      internet address = 203.0.113.26
```

Three things in that command line, and all three matter when diagnosing.

- `-type=MX` asks for one record type. Without it you get A records and would wrongly conclude the MX records are missing.
- `8.8.8.8` at the end queries *that* server, bypassing the host's configured resolver. This is how you prove whether a fault is local caching or genuinely upstream.
- `Non-authoritative answer` means it came from a cache. An authoritative answer comes only from a server that holds the zone.

>_ Linux or macOS — `dig +noall +answer www.example.edu A`

```
www.example.edu.      3600    IN      CNAME   web-lb.example.edu.
web-lb.example.edu.   300     IN      A       203.0.113.10
```

That two-line answer is worth studying. It shows a CNAME being followed: the name asked for is an alias, the alias points at another name, and the resolver returned both the alias and the address it eventually reached. The numbers 3600 and 300 are the two TTLs, and they are independent — the alias is cached for an hour, the address behind it for five minutes.

📖 Read the status line first

When `dig` is run without `+noall +answer`, the header carries a status, and it narrows the fault immediately:

- NOERROR with an empty answer section — the name exists, but not with that record type. Very often the real answer in an exam question.
 - NXDOMAIN — the name does not exist. A typo, or a record never created.
 - SERVFAIL — the resolver could not complete. Broken delegation, an unreachable authoritative server, or a validation failure.
 - REFUSED — the server will not answer you. Usually an access control, not a record problem.
- NOERROR-with-no-answer and NXDOMAIN look identical to a user and mean completely different things to you.

21.4 DNS on IOS itself

A router resolving names is a convenience for you, not a service for users, but it appears in labs and it has one famous side effect.

>_ Configure — name resolution on a router

```
ip domain lookup
ip name-server 192.168.20.10 192.168.20.11
ip domain name campus.example.edu
!
! A local override, no server needed. Useful in a lab and useful when
! you want a name to resolve even if DNS is the thing that is broken.
ip host CORE1 10.0.0.1
```

⚠ The mistyped-command hang

Type a command wrong in EXEC mode — `shwo ip route` — and IOS assumes you meant to telnet to a host called `shwo`. With `ip domain lookup` enabled and no reachable server, the CLI freezes for tens of seconds on every typo.

On a lab device the fix is `no ip domain lookup`. On a production device, where you may want resolution, use `line con 0` then `transport preferred none`, which stops the CLI from treating unknown words as hostnames without disabling DNS.

21.5 Diagnosing the three named failures

The blueprint names three purposes. Take them one at a time, because each has a characteristic record behind it.

21.5.1 Host access by name

- `ping <name>` fails, `ping <address>` works → it is DNS, not the network. This one test separates the two and should always be first.
- Both fail → it is not DNS. Stop here and go to Chapter 28.
- Name resolves to an address, but the *wrong* one → a stale cache or a stale A record. Query an authoritative server directly and compare.

21.5.2 Web application access

The characteristic web fault is *partial*: the site loads for some people, or loads slowly, or works on one machine and not another. Two causes account for most of it.

- **An AAAA record pointing somewhere that does not answer.** A dual-stack client prefers IPv6, tries the AAAA address, waits for a timeout, and only then falls back to IPv4. The site works — after a painful delay — for v6-capable clients and instantly for everyone else. A missing AAAA is harmless; a *wrong* AAAA is much worse than none.

- **A CNAME where there should not be one.** A name that is a CNAME cannot hold any other record. Putting a CNAME at a zone apex therefore conflicts with the zone's own NS records, and resolvers behave inconsistently. `www` may safely be a CNAME; the bare domain may not.

21.5.3 Mail server access

Mail exercises three of the six record types at once, and the exam knows it.

- **Inbound fails** → look at MX. Does it exist? Does the name it gives have an A record? An MX pointing at a name with no address is a common and completely silent fault.
- **Outbound is rejected by the recipient** → look at PTR. Receiving servers routinely check that the connecting IP reverse-resolves to a name, and that the name forward-resolves back to the same IP. If either half is missing, mail is rejected or scored as spam.
- **Nothing about the domain resolves** → look at NS. A delegation problem takes out MX, A and everything else simultaneously, which is how you tell it apart from a record-level fault.

⚠ MX must point at a name with an A record, never at a CNAME

Two rules, both examinable. The value of an MX record must be a hostname, not an IP address. And that hostname must resolve by A or AAAA directly — pointing an MX at a CNAME is prohibited by the standards and rejected by some mail servers and tolerated by others, which produces the worst kind of fault: mail from *some* senders is delivered and from others is not.

! Common Pitfalls

- **Querying without `-type=`.** You get A records, see them, and wrongly declare the MX or NS records missing.
- **Not naming a server in the query.** You test your own cache instead of the truth. Always query the authoritative server too.
- **Treating NXDOMAIN and empty-NOERROR as the same.** One means the name is absent; the other means the *record type* is.
- **Forgetting TTL.** A correct fix looks like a failed fix for as long as the old TTL.
- **Assuming reverse DNS is yours to fix.** PTR usually lives with whoever owns the address block.
- **Blaming DNS for a routing fault.** If ping by address also fails, DNS is not involved.

🔧 Break & Fix — The website works. Mail to the domain bounces with “no route to host”.

`example.edu` loads correctly in a browser from everywhere. Mail sent to `staff@example.edu` from outside is returned by the sender's server. Mail *between* internal users is fine. The mail server itself is up, is listening on port 25, and can be reached by IP address from outside.

```
>_ Linux — dig +noall +answer example.edu MX
example.edu.      3600    IN      MX      10 mail.example.edu.
```

```
>_ Linux — dig +noall +answer mail.example.edu A
```

Diagnosis. The MX record exists and is syntactically fine. The name it points at, `mail.example.edu`, has no A record — the answer section is empty. A sending server looks up the MX, gets a name, tries to resolve that name to an address, gets nothing, and has nowhere to connect to.

Everything else being healthy is what makes this hard to spot. The web works because `www` has its own A record. Internal mail works because internal clients are configured with the server's address directly and never consult the MX. Only *external inbound* mail follows the full MX-then-A path, so only external inbound mail fails.

The tell is the shape of the failure: one direction, one protocol, one class of sender. Faults that broad in effect but that narrow in scope are almost always in a record that only one code path reads.

Fix. Add the missing A record for `mail.example.edu` pointing at the server's public address, then re-query to confirm. Two further checks before closing it:

- Confirm the PTR for that address resolves back to `mail.example.edu`, or outbound mail will be the next ticket.
- Confirm the MX value is not itself a CNAME.

And expect a delay. The negative answer has been cached by every resolver that tried, for the zone's negative-caching TTL. Mail will resume as those expire, not the moment you save the record.

Lab 21.1 — Diagnosing a zone you did not build (CML or GNS3, with a BIND or Windows DNS server)

Platform note. Packet Tracer's DNS server holds only A and CNAME records and cannot express MX, NS or PTR, so it cannot support this lab. Use a real resolver in CML or GNS3, or run BIND in a container alongside a virtual network.

Topology. One router with `ip name-server` configured, one DNS server holding `example.edu`, and two clients — one dual-stack, one IPv4 only.

Task 1. Load the zone supplied with this lab. Do not read the zone file yet; diagnose from the client side first, as you would in practice.

Task 2. For each of the six record types, run a query that returns it, using `nslookup` on one client and `dig` on the other. Record the exact command in each case.

Task 3. Find every fault in the zone using queries alone. There are four. Write a one-line diagnosis for each *before* opening the zone file.

Task 4. Open the zone file and check your diagnoses. Note any fault you missed and what query would have found it.

Task 5. Break 1. Point `www` at a CNAME chain three deep, with the last entry pointing back at the first. Query it and describe the result.

Task 6. Break 2. Add an AAAA record for `www` pointing at an unreachable IPv6 address. Browse from the dual-stack client and from the IPv4-only client. Time both. Explain the difference to somebody who has not read this chapter.

Task 7. Break 3. Set the A record TTL to 3600, change the address, and observe how long the old value survives on a client that had already queried it. Then repeat with a TTL of 60.

Task 8. Extension. Disable `ip domain lookup` on the router, mistype a command, and compare the behaviour with it enabled and no reachable server.

Checkpoint — Test Yourself

1. `ping www.example.edu` fails and `ping 203.0.113.10` succeeds. What has this proved, and what is the next command you run?
2. What is the difference between `NXDOMAIN` and `NOERROR` with an empty answer section?
3. Why can a name that is a CNAME not also hold an MX record?

4. A site is slow for some users and instant for others. Which record type would you check first, and why does it affect only some clients?
5. Whose responsibility is a PTR record for your public address, and what breaks when it is missing?

Chapter Summary

- A host runs a stub resolver: one query, one answer. All the recursion happens at the resolver, which caches for the TTL.
- Six examinable types: A and AAAA give addresses, CNAME gives another name, MX gives a mail exchanger with a preference, NS delegates a zone, PTR reverses an address into a name.
- Query a *type* and a *server* explicitly, or you are testing neither the record you meant nor the source you meant.
- NXDOMAIN is a missing name; empty NOERROR is a missing record type; SERVFAIL is usually a broken delegation.
- Host fault: does it ping by address? Web fault: suspect AAAA or an apex CNAME. Mail fault: MX inbound, PTR outbound, NS if everything is gone.
- MX must name a host that has an A record, and must not name a CNAME.
- TTL means a correct fix takes time to become visible. Lower it before a change, not after.

Review Questions

1. **(MCQ)** Outbound mail from your server is being rejected as spam by several recipients. Which record is the first to check?
(a) MX (b) A (c) PTR (d) NS
2. **(MCQ)** dig returns status NOERROR with an empty answer section. What does this mean?
(a) The name does not exist
(b) The name exists but has no record of the type requested
(c) The resolver could not reach an authoritative server
(d) The server refused the query
3. **(Short)** Explain why an MX record must not point at a CNAME, and describe the symptom when one does.
4. **(Short)** A record was corrected an hour ago and users still reach the old address. Explain why, and state what should have been done the day before.
5. **(Applied)** A domain's website, mail and every subdomain all stop resolving at once, while the servers themselves are up and reachable by address. Name the record type at fault, justify the choice from the scope of the outage, and give the query that would confirm it.
6. **(Applied)** Write the nslookup and the dig command that each ask an authoritative server at 192.0.2.53 for the NS records of example.edu, and explain what specifying the server proves that omitting it would not.

NAT and PAT on IOS XE

EXAM TOPICS **4.3**

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** the four NAT address terms and identify each one in a translation table.
- **Configure** static NAT, dynamic NAT and PAT on an IOS XE router, including port forwarding.
- **Interpret** `show ip nat translations` and `show ip nat statistics`.
- **Troubleshoot** the four faults that account for almost every broken NAT: missing interface markers, a wrong ACL, an exhausted pool, and an order-of-operations misunderstanding.

🔗 Before You Start

Chapter 4 for private address ranges. Chapter 15 — NAT does not replace routing, and a router that cannot route to the internet will not be fixed by NAT. Chapter 6 for port numbers, which is what PAT actually manipulates.

📖 Key Terms in This Chapter

inside local • inside global • outside global • outside local • static NAT • dynamic NAT • PAT • overload • translation table • port forwarding

🎓 IOS XE is named in the topic

Topic 4.3 is *configure NAT/PAT on IOS XE routers*. The NAT configuration syntax is the same on IOS and IOS XE, so nothing here is wasted on either platform — but the interface names differ (`GigabitEthernet0/0/0` on XE against `GigabitEthernet0/0` on classic IOS), and this chapter uses XE names throughout so that they look normal to you in the exam.

22.1 The problem NAT solves, and the one it does not

There are about 3.7 billion usable public IPv4 addresses and far more devices than that. RFC 1918 set aside three ranges — 10.0.0.0/8, 172.16.0.0/12 and 192.168.0.0/16 — that anyone may use internally and that no router on the internet will forward. NAT is what lets a network built from those addresses reach the internet anyway.

⚠️ NAT is not a security feature

It is frequently described as one, and it is not. It happens to make unsolicited inbound connections difficult, because there is no translation entry to match them against — but that is a side effect of an address-conservation mechanism, not a policy. Anything you actually want blocked should be blocked by an ACL (Chapter 24) or a firewall, which can be audited and which says what it means.

The exam has been known to ask this directly. NAT conserves addresses.

22.2 The four terms

This is the part students get wrong, and it is worth slowing down for. There are two axes, not one. **Inside** and **outside** say which *host* you mean. **Local** and **global** say which *side of the router* you are standing on when you look at the address.

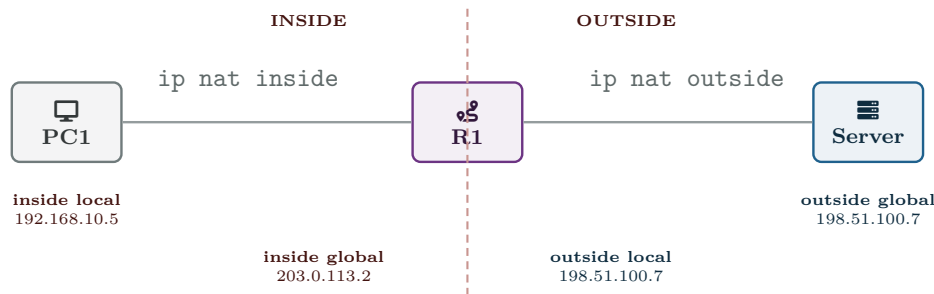


Figure 22.1: The same two hosts, four names. *Inside* and *outside* name the host; *local* and *global* name the viewpoint.

Term	What it is	Who sees it
Inside local	The private address really configured on the internal host	The internal host and everything inside
Inside global	The public address the internal host is translated to	Everything on the internet
Outside global	The real public address of the external host	Everyone, inside and out
Outside local	How the external host appears to internal hosts	Internal hosts. Usually identical to outside global

💡 Why outside local usually equals outside global

Because plain NAT only rewrites the *inside* host's address. The external server's address is untouched in both directions, so both names describe the same number. The two differ only when you translate the outside side as well (`ip nat outside source`), which is rare and outside this exam.

That is good news: for CCNA purposes there are effectively *three* addresses to track, and the one that changes is the inside host's.

22.3 Three kinds of NAT

22.3.1 Static NAT — one to one, permanent

Used when an internal server must be reachable from outside at a fixed public address.

>_ Configure — static NAT for an internal web server

```
! The mapping itself: inside local first, inside global second.
ip nat inside source static 192.168.10.50 203.0.113.50
!
! Without these two lines nothing is translated. This is the most
! commonly forgotten pair of commands in the whole topic.
interface GigabitEthernet0/0/1
  description LAN
  ip address 192.168.10.1 255.255.255.0
  ip nat inside
!
```



```
interface GigabitEthernet0/0/0
description WAN to ISP
ip address 203.0.113.2 255.255.255.252
ip nat outside
```

22.3.2 Dynamic NAT — many to a few, from a pool

Internal hosts borrow a public address from a pool for the life of a conversation. When the pool is empty, further hosts get nothing.

>_ Configure — dynamic NAT from a small pool

```
! 1. Which internal addresses are allowed to be translated.
access-list 1 permit 192.168.10.0 0.0.0.255
!
! 2. The public addresses available to lend.
ip nat pool BRANCH-POOL 203.0.113.10 203.0.113.20 netmask 255.255.255.0
!
! 3. Tie them together.
ip nat inside source list 1 pool BRANCH-POOL
```

⚠ Dynamic NAT fails silently when the pool runs out

Eleven addresses in that pool means the twelfth simultaneous host gets no translation and its packets are dropped. There is no message to the user and no log entry unless you asked for one. The symptom is “the internet works in the morning and stops after lunch”, and the evidence is one line of `show ip nat statistics`.

This is exactly why PAT exists, and why almost nobody deploys plain dynamic NAT.

22.3.3 PAT — many to one, the normal case

PAT (also called NAT overload) translates the port as well as the address, so a single public address serves thousands of internal hosts. Add one keyword.

>_ Configure — PAT onto the WAN interface address

```
access-list 1 permit 192.168.10.0 0.0.0.255
!
! "interface" instead of a pool: use whatever address the WAN
! interface currently has. This survives a DHCP-assigned WAN address
! changing, which a pool would not.
ip nat inside source list 1 interface GigabitEthernet0/0/0 overload
!
interface GigabitEthernet0/0/1
ip nat inside
!
interface GigabitEthernet0/0/0
ip nat outside
```

🎓 The word overload is the whole difference

`ip nat inside source list 1 pool BRANCH-POOL` is dynamic NAT. `ip nat inside source list 1 pool BRANCH-POOL overload` is PAT using a pool. `ip nat inside source list 1 interface Gi0/0/0 overload` is PAT using the interface address.

One keyword changes the behaviour from “runs out” to “does not run out”. Exam questions have been built on exactly that keyword being present or absent.

22.3.4 Port forwarding — one port in, one host

A special case of static NAT that maps a single port rather than the whole address, so one public address can host several internal services.

➤ Configure — publishing two internal services on one public address

```
ip nat inside source static tcp 192.168.10.50 80 203.0.113.2 80
ip nat inside source static tcp 192.168.10.60 22 203.0.113.2 2222
!
```

! The second line also changes the port: the outside world connects to 2222 and the internal host still listens on 22.

22.4 Verifying

🔍 R1# show ip nat translations

Pro	Inside global	Inside local	Outside local	Outside global
tcp	203.0.113.2:1055	192.168.10.5:1055	198.51.100.7:80	198.51.100.7:80
tcp	203.0.113.2:1056	192.168.10.6:1055	198.51.100.7:80	198.51.100.7:80
tcp	203.0.113.2:80	192.168.10.50:80	---	---

Read that output carefully, because it contains the whole idea of PAT in three lines.

- Rows one and two are two different internal hosts that both happened to choose source port 1055. The router gave the second one 1056 on the outside. That port rewrite is the reason one public address can serve a campus.
- The Pro column shows tcp, so these are PAT entries. A plain static or dynamic NAT entry has -- in that column and no ports.
- Row three has dashes in both outside columns. That is a static mapping sitting ready with no conversation currently using it.

🔍 R1# show ip nat statistics

```
Total active translations: 3 (1 static, 2 dynamic; 2 extended)
Peak translations: 412, occurred 00:31:09 ago
Outside interfaces:
  GigabitEthernet0/0/0
Inside interfaces:
  GigabitEthernet0/0/1
Hits: 20481 Misses: 0
Expired translations: 1173
Dynamic mappings:
-- Inside Source
[Id: 1] access-list 1 interface GigabitEthernet0/0/0 refcount 2
```

💡 The three lines to read first when NAT is broken

- **Inside interfaces / Outside interfaces.** If either list is empty, stop. Nothing else matters until both are populated.
- **Misses.** A non-zero and rising miss count on a dynamic mapping means packets arrived wanting translation and did not get one — an exhausted pool, or an ACL that did not match.
- **Dynamic mappings.** Confirms which ACL and which pool or interface are actually in force, which is not always the one you think you configured.

22.5 Order of operations, and why it catches people

The order in which a router applies routing, NAT and ACLs is not symmetric, and one exam-favourite consequence falls out of it.

Inside to outside	Outside to inside
1. Inbound ACL on the inside interface	1. Inbound ACL on the outside interface
2. Routing decision	2. Un-translate (NAT outside to inside)
3. Translate (NAT inside to outside)	3. Routing decision
4. Outbound ACL on the outside interface	4. Outbound ACL on the inside interface

⚠ The consequence you will be asked about

An inbound ACL on the *outside* interface is evaluated **before** the packet is un-translated. So it sees the *inside global* address — 203.0.113.50 — and not the private one. An ACL written against 192.168.10.50 on that interface will never match anything.

An inbound ACL on the *inside* interface is evaluated before translation too, so it sees the private address. Same rule, opposite result: **ACLs always see the address that is on the wire at the interface they are attached to.** If you can hold on to that sentence you never need to memorise the table.

! Common Pitfalls

- **No ip nat inside / ip nat outside.** The single most common NAT fault. The translation rule is present, the interfaces are not marked, and nothing happens.
- **Marking the wrong interface as inside.** On a router with several LANs, only the marked ones are translated.
- **A permit ip any any ACL used to select NAT traffic.** It will translate traffic destined for other internal subnets too, breaking internal routing in ways that look like a routing fault.
- **Forgetting overload.** Works for the first few users and then does not.
- **Expecting to ping the inside global address from inside.** It usually does not work, and it does not mean NAT is broken. Test from outside.
- **Editing a rule without clearing translations.** Existing entries persist. `clear ip nat translation *` after every change, or you are testing the old configuration.
- **Using an ACL number twice.** The NAT selection ACL and a filtering ACL with the same number are the same ACL.

🔧 Break & Fix — NAT is configured, the translation table is empty, and internal hosts cannot reach the internet.

R1 has a default route, can ping its ISP next hop, and can ping 8.8.8.8 from its own CLI. Internal hosts can ping R1's LAN interface but nothing beyond it. `show ip nat translations` returns nothing at all.

Q R1# show run | include ip nat

```
ip nat inside source list 1 interface GigabitEthernet0/0/0 overload
ip nat inside
```

Q R1# show ip nat statistics

```
Total active translations: 0 (0 static, 0 dynamic; 0 extended)
Outside interfaces:
Inside interfaces:
  GigabitEthernet0/0/1
Hits: 0 Misses: 0
```

Diagnosis. Outside interfaces: is empty. The WAN interface was never marked `ip nat outside`. NAT needs a boundary, and a boundary needs two sides — with only one side declared, the router has no rule to apply and does not attempt translation at all.

Hits: 0 and Misses: 0 together are the confirming detail, and they are worth separating. Misses would mean packets arrived wanting translation and could not get one — a pool or ACL problem. Both counters at zero means no packet was ever *considered* for translation. That distinction points at the interface markers rather than at the rule.

The router itself reaching 8.8.8.8 is the red herring. Traffic the router originates is sourced from its own public interface and needs no translation, so it proves the WAN link and the default route are fine and says nothing about NAT.

Fix.

>_ Configure — declaring the outside boundary

```
interface GigabitEthernet0/0/0
ip nat outside
```

Then clear `ip nat translation *` and re-test from a host. Confirm entries appear and that Hits is climbing.

Lab 22.1 — PAT for a branch, then a published server (CML or GNS3 with IOS XE)

Platform note. Packet Tracer supports this lab, but the topic names IOS XE and the interface naming differs. If you use Packet Tracer, translate the interface names as you go and note where they differ.

Topology. R1 as the branch router with 192.168.10.0/24 inside and 203.0.113.0/30 to ISP; ISP with a loopback of 198.51.100.7 standing in for the internet; three internal hosts and one internal server at 192.168.10.50.

- Task 1.** Build and verify routing first, with no NAT. Confirm R1 can ping 198.51.100.7 and that internal hosts cannot. Explain precisely why the hosts fail, in terms of the ISP's routing table.
- Task 2.** Configure PAT using the WAN interface address. Verify from all three hosts and read `show ip nat translations`.
- Task 3.** Generate connections from two hosts using the same source port (open the same service twice). Find the row where the router had to change the port and record both values.
- Task 4.** Add a static NAT mapping for the server and confirm from ISP that it is reachable inbound.
- Task 5.** Change the static mapping to port forwarding on port 8080, and confirm the server still listens on 80.
- Task 6. Break 1.** Remove `ip nat outside`. Predict what `show ip nat statistics` will show *before* you look, then look.
- Task 7. Break 2.** Replace PAT with dynamic NAT from a two-address pool, no overload. Bring up three simultaneous sessions and observe the third fail. Find the miss counter.
- Task 8. Break 3.** Apply an inbound ACL on the outside interface denying 192.168.10.50. Confirm it has no effect on inbound traffic to the server, and explain why using the order-of-operations table.
- Task 9. Extension.** Change the NAT selection ACL to permit `ip any any`, add a second internal LAN, and observe what breaks between the two internal subnets.

🔍 Checkpoint — Test Yourself

1. Give the four NAT terms for a host 10.1.1.9 translated to 198.51.100.4 contacting a server at 203.0.113.80.
2. What single keyword turns dynamic NAT into PAT, and what failure does it prevent?
3. `show ip nat statistics` shows Hits: 0 and Misses: 0. What does that rule out, and what does it point to?
4. An inbound ACL on the outside interface must match a translated inside host. Which address does it have to name, and why?
5. Why is NAT not a security control?

📋 Chapter Summary

- Inside/outside names the host; local/global names the viewpoint. Outside local and outside global are normally the same address.
- Static NAT is a permanent one-to-one map. Dynamic NAT lends from a pool and fails silently when empty. PAT adds `overload` and translates ports too, so it does not run out.
- `ip nat inside` and `ip nat outside` on the interfaces are not optional. Without both, no translation is attempted.
- `show ip nat translations` for what is happening now; `show ip nat statistics` for whether the rule is even being reached.
- ACLs see the address on the wire at the interface they are attached to — translated on the outside, untranslated on the inside.
- `clear ip nat translation *` after every change, or you are testing stale entries.
- NAT conserves addresses. It is not a firewall.

🔧 Review Questions

1. (MCQ) Which command configures PAT using the WAN interface's own address?
 - (a) `ip nat inside source list 1 interface Gi0/0/0`
 - (b) `ip nat inside source list 1 interface Gi0/0/0 overload`
 - (c) `ip nat inside source static 192.168.1.0 Gi0/0/0`
 - (d) `ip nat outside source list 1 interface Gi0/0/0 overload`
2. (MCQ) In `show ip nat translations`, which column holds the private address of the internal host?
 - (a) Inside global
 - (b) Inside local
 - (c) Outside local
 - (d) Outside global
3. (Short) Explain why an ACL applied inbound on the outside interface cannot match an internal host's private address.
4. (Short) Users report that internet access works early in the day and fails later. Give the most likely NAT cause and the single show command that would confirm it.
5. (Applied) Write a complete configuration for a branch translating 172.16.4.0/24 to the address of GigabitEthernet0/0/0, while also publishing an internal mail server at 172.16.4.25 on port 25 of the same public address. State the order in which you would enter the commands and why the interface markers should go in first.
6. (Applied) A colleague reports that NAT “stopped working” right after they corrected the selection ACL. The configuration is now correct. Explain what they have most likely failed to do and give the command.

Device Access, AAA and Secure File Transfer

EXAM TOPICS **4.1** **4.2**

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Configure** local usernames, enable secrets and SSH-only access to the console and VTY lines.
- **Configure** a router or switch as an AAA client against TACACS+ and against RADIUS, with a local fallback that cannot lock you out.
- **Compare** TACACS+ and RADIUS and choose correctly between them.
- **Manage** configuration and image files over SCP and SFTP, and say why v2.0 replaced TFTP with them.

🔗 Before You Start

Chapter 7 for line configuration and privilege levels. Chapter 21 for `ip domain name`, which SSH key generation requires. Chapter 6 for the difference between TCP and UDP, which is one of the real distinctions between TACACS+ and RADIUS.

📖 Key Terms in This Chapter

AAA • authentication • authorization • accounting • TACACS+ • RADIUS • method list • fallback • `enable secret` • SCP • SFTP • privilege level

🎓 Two topics, one chapter, and a v2.0 change worth noticing

Topic **4.1** is *configure network devices with local usernames and as an AAA client (TACACS+ and RADIUS) for management*. Topic **4.2** is *manage device configuration and software files using secure file transfer operations with SFTP/SCP*.

They sit together because they are the same job: getting onto a device safely, and moving files onto it safely. And **4.2** is a genuine v2.0 change — v1.1 examined TFTP. TFTP is UDP, has no authentication of any kind, and sends everything in clear text. It has been removed from the blueprint and replaced by two protocols that fix all three problems. If a practice question offers TFTP as the secure answer, the question predates your exam.

23.1 Local access done properly

Before any server is involved, get the device itself right. Everything in this section should be on every device you build, including the ones that later use AAA — because AAA servers fail, and this is what you fall back to.

➤ Configure — baseline secure access

```
hostname CAMPUS-SW1
!
! A domain name is required before RSA keys can be generated: the key
! is named host.domain.
ip domain name campus.example.edu
!
! script is type 9. Use it where the platform offers it. Never use
```

```

! "username ... password", which stores type 0 or type 7.
username netadmin privilege 15 algorithm-type scrypt secret S0me-Long-Passphrase
username helpdesk privilege 1 algorithm-type scrypt secret AnOther-Passphrase
!
enable algorithm-type scrypt secret Enable-Passphrase-Here
!
! 2048 is the practical minimum; below 768 the device will refuse to
! run SSH version 2 at all.
crypto key generate rsa modulus 2048
ip ssh version 2
ip ssh time-out 60
ip ssh authentication-retries 3
!
line console 0
 login local
 exec-timeout 10 0
 logging synchronous
!
line vty 0 15
 login local
 transport input ssh
 exec-timeout 10 0

```

⚠ transport input ssh is the line that matters

Generating keys and enabling SSH does not disable Telnet. Until you say `transport input ssh`, the device still accepts plain-text logins on port 23, and an auditor will find it. The default on many platforms is `transport input all`.

The reverse mistake is worse: typing `transport input ssh` *before* the keys exist locks you out of a remote device completely. Generate keys first, prove SSH works from a second window, then restrict the transport.

💡 Password types, and why service password-encryption is not enough

- **Type 0** — plain text. Visible in the configuration.
- **Type 7** — Cisco's reversible obfuscation, applied by `service password-encryption`. It is trivially reversed by tools that have existed for decades. It stops shoulder-surfing and nothing else.
- **Type 5** — salted MD5. A real hash, but MD5 is old.
- **Type 8 (PBKDF2) and type 9 (scrypt)** — current. Use `algorithm-type sha256` or `algorithm-type scrypt`.

So `service password-encryption` is still worth enabling — it covers things that can only be type 7, such as line passwords and TACACS+ keys — but it is not a substitute for storing user passwords as type 8 or 9.

`enable secret` is hashed; `enable password` is not. If both exist, the secret wins and the password is dead configuration that leaks a credential. Delete it.

23.2 AAA: three separate questions

☰ AAA

Authentication — who are you? **Authorization** — what are you allowed to do? **Accounting** — what did you actually do? Three distinct questions, answered by method lists that a device consults in order.

23.2.1 Method lists

A method list is an ordered list of ways to answer one of those questions. The device tries them left to right and moves on only when a method is *unreachable*, not when it returns a refusal.

⚠ The distinction that saves you and the one that locks you out

If the TACACS+ server answers “no, wrong password”, that is an answer. The device stops there and rejects you — it does not fall through to the local database. Fallback happens only when the server does not respond at all.

This is correct behaviour and it surprises people. A fallback method is protection against a *dead server*, not against a forgotten password.

➤ Configure — AAA against TACACS+, with a local fallback

```
aaa new-model
!
! The server. This is the IOS 15 / IOS XE syntax; older configurations
! used a single "tacacs-server host" line.
tacacs server TAC1
  address ipv4 10.0.0.20
  key SharedSecretHere
!
tacacs server TAC2
  address ipv4 10.0.0.21
  key SharedSecretHere
!
aaa group server tacacs+ TACGROUP
  server name TAC1
  server name TAC2
!
! Authentication: try the group, then the local database if no server
! answers. The trailing "local" is not optional in any network you
! intend to keep working.
aaa authentication login default group TACGROUP local
aaa authentication enable default group TACGROUP enable
!
! Authorization: what privilege level does this user get on login?
aaa authorization exec default group TACGROUP local if-authenticated
!
! Accounting: a record of every session and every command at level 15.
aaa accounting exec default start-stop group TACGROUP
aaa accounting commands 15 default start-stop group TACGROUP
!
! Source the packets from a stable address so the server's client
! entry always matches, whichever interface the traffic leaves by.
ip tacacs source-interface Loopback0
```

➤ Configure — the same device as a RADIUS client

```
radius server RAD1
  address ipv4 10.0.0.30 auth-port 1812 acct-port 1813
  key AnotherSharedSecret
!
aaa group server radius RADGROUP
  server name RAD1
!
aaa authentication login default group RADGROUP local
aaa authorization exec default group RADGROUP local
ip radius source-interface Loopback0
```



```
Q CAMPUS-SW1# show aaa servers | include host|state
```

```
RADIUS: id 1, priority 1, host 10.0.0.30, auth-port 1812, acct-port 1813
State: current UP, duration 43112s, previous duration 0s
```

23.3 TACACS+ against RADIUS

Both are AAA protocols and they are not interchangeable. This table is examinable more or less as it stands.

	TACACS+	RADIUS
Transport and port	TCP 49	UDP 1812 authentication, 1813 accounting
What is encrypted	The entire packet body	The password field only. Everything else, usernames included, is in clear
AAA separation	Fully separate. Authorization can be asked per command	Authentication and authorization are combined in one exchange
Origin	Cisco-developed, later published	IETF standard, universally implemented
Normal use	Device administration — controlling who may log in to routers and switches and what they may type	Network access — 802.1X, wireless clients, VPN users
Per-command control	Yes, and this is its main advantage	No, beyond a privilege level granted at login

The one-sentence version

TACACS+ manages administrators; RADIUS manages users. If a question is about who may configure a switch, the answer is TACACS+. If it is about a laptop authenticating onto a wireless network, the answer is RADIUS.

Two supporting facts worth memorising because they are asked directly: TACACS+ is TCP 49 and encrypts the whole payload; RADIUS is UDP and encrypts only the password.

23.4 Moving files securely

23.4.1 Why not TFTP

TFTP has no user authentication, no encryption, and runs over UDP with no integrity protection beyond a checksum. Anyone who can reach the server can read any file whose name they can guess, and a configuration file in flight is readable by anyone on the path — including the shared secrets in it. That is the case for its removal from the blueprint.

23.4.2 SCP — the device as a server

SCP runs over SSH, so it inherits SSH's authentication and encryption. The device can act as an SCP *server*, letting you pull and push files from a workstation.

Configure — enabling the SCP server

```
! SCP will not start without all three of these. The authorization
! line in particular is easy to miss, and its absence produces a
! connection that authenticates and then refuses every transfer.
```

```

aaa new-model
aaa authentication login default local
aaa authorization exec default local
!
ip scp server enable

```

With that in place, from a workstation:

```

>_ Linux or macOS — scp netadmin@10.0.0.1:running-config ./sw1-backup.cfg

netadmin@10.0.0.1's password:
running-config                100% 8241      1.2MB/s   00:00

```

23.4.3 SFTP — the device as a client

IOS is an SFTP *client* but not an SFTP server. So the direction of travel is the device fetching from, or sending to, a server you run.

```

Q CAMPUS-SW1# copy sftp: flash:

```

```

Address or name of remote host []? 10.0.0.40
Source username [netadmin]? images
Source filename []? cat9k_iosxe.17.12.04.SPA.bin
Destination filename [cat9k_iosxe.17.12.04.SPA.bin]?
Accessing sftp://images@10.0.0.40/cat9k_iosxe.17.12.04.SPA.bin...
Loading file ! [OK - 1103237120 bytes]
1103237120 bytes copied in 412.338 secs (2675281 bytes/sec)

```

>_ Configure — backing the configuration up on a schedule

```

! One-off, non-interactive form. The password appears on the command
! line, so prefer key-based SCP where the platform supports it.
! do copy running-config scp://netadmin@10.0.0.40/CAMPUS-SW1.cfg
!
! Automatic archive: a copy every time someone saves, kept off-box.
archive
 path scp://netadmin@10.0.0.40/archive/$h-
 write-memory
 maximum 14

```

💡 Verify the image before you reload onto it

A transfer that reports [OK] has moved the bytes it was given. It has not told you the file is the image Cisco shipped. Before reloading:

- `verify /md5 flash:cat9k_iosxe.17.12.04.SPA.bin` and compare against the hash published with the download.
- `show flash:` to confirm the size matches.
- Confirm free space *before* the copy, not after — a partial copy that filled flash can leave a device unable to boot.

❗ Common Pitfalls

- **aaa authentication login default group TACGROUP with no local.** When the server is unreachable, nobody can log in — not even at the console. This is the classic lock-out.
- **Testing AAA by closing your session.** Always open a second session and prove it works before closing the first.
- **Mismatched shared key.** Authentication fails with no useful message on the client. Check the server's logs, not the switch's.

- **No ip tacacs source-interface.** Packets leave by whichever interface routing picks, the server does not recognise the source, and the failure is intermittent by design.
- **transport input ssh before the keys exist.** Instant remote lock-out.
- **Enabling SCP without aaa authorization exec.** Logs in, transfers nothing.
- **Believing service password-encryption protects anything.** Type 7 is reversible.
- **Reaching for TFTP out of habit.** It is off the blueprint and it was off the good-practice list long before that.

🔧 Break & Fix — Nobody can log in to any switch. The AAA server is up and other services work.

A campus of forty switches, all authenticating to TACACS+. Overnight, logins begin failing everywhere. The TACACS+ server is running, responds to ping from a workstation, and is authenticating VPN users normally. Console access fails as well as SSH.

```
Q CAMPUS-SW1# show run | include aaa authentication
```

```
aaa authentication login default group TACGROUP local
aaa authentication enable default group TACGROUP enable
```

```
Q CAMPUS-SW1# show aaa servers | include host|state
```

```
TACACS+: id 1, priority 1, host 10.0.0.20, auth-port 49
State: current UP, duration 61204s
```

Diagnosis. The configuration has a correct fallback and the server is reachable and marked UP. That combination rules out both of the usual suspects at once — and it is the clue.

Because the server is reachable, the device sends the request and gets an *answer*. The answer is a rejection. Fallback to *local* never fires, because fallback happens on silence, not on refusal. The fault is therefore on the server: overnight, the group that grants network-device administration was changed, and the server is now correctly refusing everyone.

The tell is that console access fails too. A network-side or transport-side fault would leave the console working, because the console does not depend on the network. A fault that follows you to the console is in the authentication decision itself.

Immediate recovery. You need an account the switch can validate without the server. If a local account exists, use it — but the method list consults TACACS+ first and TACACS+ is answering, so it will not be reached. Break the server's reachability from the switch deliberately, or use the password-recovery procedure on one switch, restore local access, and work from there.

The lasting fix is on the server, not the switches. Restore the administration group.

And the lesson to write down. A local fallback protects you from an unreachable server. It does not protect you from a reachable server that says no. Sites that care about this keep a break-glass local account and test it on a schedule — and keep at least one device out of AAA entirely, reachable out-of-band, as the way back in.

🏢 Lab 23.1 — Secure access, AAA, and a backup you can restore (CML or GNS3, with a TACACS+ or RADIUS server)

Platform note. Packet Tracer models a simplified AAA server and will carry tasks 1–4. Tasks 5–8 need a real server (tac_plus or FreeRADIUS) and a real SSH stack, so use CML or GNS3.

Topology. Two switches and a router, a management VLAN, a Linux host running the AAA server and an SFTP server, and your own workstation.

- Task 1.** Build baseline access on all three devices: hostname, domain name, two local users at different privilege levels, enable secret, RSA keys, SSH version 2, `transport input ssh`. Verify by SSH and confirm Telnet is refused.
- Task 2.** Prove the password types. Run `show run | include username` and identify the type number on each. Add one user with `password` instead of `secret` and compare, then remove it.
- Task 3.** Configure AAA against the server with a local fallback. **Open a second SSH session before you test**, and keep it open throughout.
- Task 4.** Verify with `show aaa servers` and by logging in as a server-defined user. Confirm the privilege level you land in.
- Task 5. Break 1.** Stop the AAA server. Log in again and confirm the local fallback works. Record how long the device waits before falling back.
- Task 6. Break 2.** Restart the server and disable your test account on it. Attempt to log in. Explain why the local fallback does *not* rescue you, and relate this to the Break & Fix above.
- Task 7. Break 3.** Remove `local` from the method list, stop the server, and log out of every session. Recover the device. Write down how long it took and what you would have done differently.
- Task 8.** Enable the SCP server and copy the running configuration to your workstation. Then omit `aaa authorization exec` and observe the different failure.
- Task 9.** Configure archive to write to SFTP on every `write memory`. Make a change, save, and confirm the archived file appears.
- Task 10. Extension.** Copy an image by SFTP, verify its MD5 against the published hash, and deliberately truncate a copy to see what `verify` reports.

🔍 Checkpoint — Test Yourself

1. In `aaa authentication login default group TACGROUP local`, when is `local` used and when is it not?
2. Give the transport, port and encryption scope of TACACS+ and of RADIUS, and say which you would choose for controlling switch administrators.
3. Why must `ip domain name` be set before generating RSA keys?
4. What three commands does the IOS SCP server require, and what is the symptom when the authorization line is missing?
5. Why did v2.0 replace TFTP with SFTP and SCP? Give three reasons.

📋 Chapter Summary

- Baseline first: hostname, domain name, local users with `algorithm-type script secret`, `enable secret`, RSA keys, `ip ssh version 2`, `login local`, `transport input ssh`.
- Type 7 is obfuscation, not encryption. Type 8 and 9 are the current choices. `enable password` should not exist.
- AAA answers three separate questions through ordered method lists. A method is skipped only when it does not respond — a rejection ends the list.
- Always end an authentication method list with `local`, and always test from a second session.
- TACACS+: TCP 49, whole packet encrypted, full AAA separation, per-command authorization, device administration. RADIUS: UDP 1812/1813, password only encrypted, combined authentication and authorization, network access.

- SCP puts file transfer inside SSH; IOS can be an SCP server and an SFTP client. Both replace TFTP, which authenticates nobody and encrypts nothing.
- Verify an image's hash before reloading onto it.

Review Questions

1. **(MCQ)** Which protocol encrypts the entire packet body and uses TCP port 49?
(a) RADIUS (b) TACACS+ (c) SNMPv3 (d) SCP
2. **(MCQ)** A device is configured `aaa authentication login default group SRV local`. The server is reachable and rejects the credentials supplied. What happens?
(a) The local database is consulted and the user may still log in
(b) The login is refused; the local database is not consulted
(c) The device retries the server three times, then falls back
(d) The user is granted privilege level 1
3. **(Short)** Explain the difference between `enable password` and `enable secret`, and say what to do if a configuration contains both.
4. **(Short)** Give three concrete reasons SFTP or SCP should be used in place of TFTP for moving a configuration file.
5. **(Applied)** Write the configuration that makes a switch an AAA client of two TACACS+ servers with a local fallback, sourcing its packets from `Loopback0`, and accounting for every level-15 command. Then state the two verification steps you would take before ending your session.
6. **(Applied)** A colleague has enabled the SCP server. Logins succeed, transfers fail immediately. Give the missing command and explain why authentication alone is not sufficient.

Access Control Lists

EXAM TOPICS **4.6**

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Calculate** a wildcard mask for any subnet, and recognise the common ones on sight.
- **Configure** standard, extended, numbered and named IPv4 ACLs.
- **Decide** where to place an ACL and in which direction, and justify the choice.
- **Read** an existing ACL and predict exactly which traffic it permits — which is what the exam asks far more often than it asks you to write one.

🔗 Before You Start

Chapter 4 for subnet masks — a wildcard mask is meaningless without them. Chapter 6 for protocol numbers and well-known ports. Chapter 22 for the order of operations, which decides which address an ACL actually sees.

📖 Key Terms in This Chapter

access control list • wildcard mask • implicit deny • standard ACL • extended ACL • named ACL • sequence number • ip access-group • access-class • established

🎓 IPv4 only, and all four kinds

Topic **4.6** reads *configure IPv4 access control lists (standard, extended, numbered, and named ACLs)*. IPv6 ACLs are not on this blueprint.

All four named kinds are examinable, but note they are two axes, not four categories: an ACL is standard *or* extended by what it can match, and numbered *or* named by how you refer to it. All four combinations exist.

24.1 What an ACL does

📋 Access control list

An ordered list of **permit** and **deny** statements. Each packet is tested against the entries from the top down; the **first match wins** and testing stops. A packet matching nothing hits an invisible **deny** any at the end and is dropped.

Three consequences follow from that definition, and between them they cause most ACL faults.

- **Order is behaviour.** Moving one line changes what the ACL does. A specific rule placed after a general one that already matched is dead configuration.
- **The implicit deny is always there.** An ACL that permits one thing denies everything else. There is no way to switch it off; if you want everything else through, end the list with **permit ip any any**.
- **An unapplied ACL does nothing.** Writing the list and applying it to an interface are two separate acts, and forgetting the second is common.

24.2 Wildcard masks

A wildcard mask says which bits of an address must match. A **0** bit means “must match”; a **1** bit means “ignore”. That is the inverse of a subnet mask, and the arithmetic is: subtract each octet of the subnet mask from 255.

Subnet mask	Wildcard	Matches
255.255.255.255 (/32)	0.0.0.0	Exactly one host. Shorthand: <code>host 10.1.1.5</code>
255.255.255.192 (/26)	0.0.0.63	One 64-address subnet
255.255.255.0 (/24)	0.0.0.255	One 256-address subnet
255.255.252.0 (/22)	0.0.3.255	Four consecutive /24s
255.255.0.0 (/16)	0.0.255.255	An entire class-B-sized range
—	255.255.255.255	Anything. Shorthand: <code>any</code>

Worked Example: Wildcard for 172.16.8.0/21

The mask for a /21 is 255.255.248.0.

Subtract each octet from 255: $255 - 255 = 0$, $255 - 255 = 0$, $255 - 248 = 7$, $255 - 0 = 255$. The wildcard is 0.0.7.255, and the ACL entry is `permit ip 172.16.8.0 0.0.7.255 any`. It matches 172.16.8.0 through 172.16.15.255 — eight consecutive /24s, which is what a /21 is.

The check that catches arithmetic slips

Add the wildcard to the network address. The result should be the last address in the range you meant. $172.16.8.0 + 0.0.7.255 = 172.16.15.255$. If that is not the broadcast address of the block you intended, the wildcard is wrong.

Do this every time. It takes five seconds and it catches the off-by-one that otherwise costs you a lab.

24.3 Standard ACLs

A standard ACL matches the **source address only**. Numbered 1–99 and 1300–1999.

Configure — a numbered standard ACL

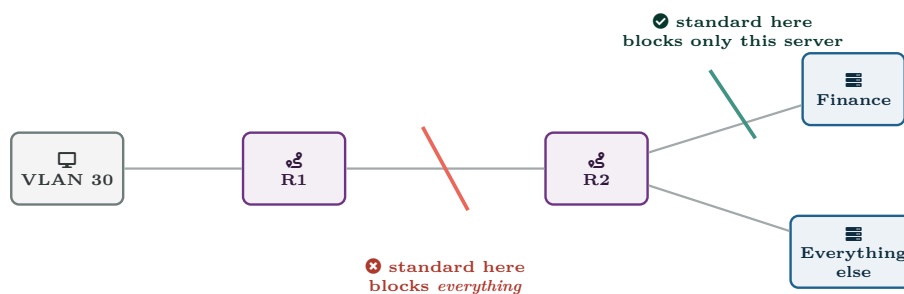
```
! Order matters: the specific deny must precede the general permit,
! or the permit matches first and the deny is never reached.
access-list 10 remark Block the lab subnet, allow the rest of campus
access-list 10 deny 192.168.30.0 0.0.0.255
access-list 10 permit 192.168.0.0 0.0.255.255
!
interface GigabitEthernet0/0/1
 ip access-group 10 out
```

Place a standard ACL close to the destination

Because it can only see the source address, a standard ACL cannot distinguish “traffic from this subnet to the finance server” from “traffic from this subnet to anywhere”. Applied near the source, it blocks the host from the entire network, not just from the thing you wanted protected.

So the rule is: standard ACLs go as close to the destination as possible; extended ACLs go as close to the source as possible, because they can be precise enough to be safe there and it saves carrying doomed traffic across the network.

VLAN 30 must not reach the finance server — but must still reach everything else



A standard ACL sees only *who sent it*, so near the source it cannot tell the two servers apart. An **extended ACL** names the destination too, so it is safe at the source — and saves carrying doomed traffic across the network.

Figure 24.1: Why placement is part of the design. The same ACL is correct in one place and an outage in the other.

24.4 Extended ACLs

An extended ACL matches source, destination, protocol and port. Numbered 100–199 and 2000–2699.

The argument order is fixed and worth reciting until it is automatic:

```
access-list <n> {permit|deny} <protocol> <source> <src-port> <dest> <dest-port>
```

>_ Configure — an extended ACL letting a subnet reach only what it needs

```
access-list 110 remark Guest VLAN -- web, DNS and nothing else
!
! DNS to the campus resolvers, both transports.
access-list 110 permit udp 192.168.40.0 0.0.0.255 host 192.168.20.10 eq 53
access-list 110 permit tcp 192.168.40.0 0.0.0.255 host 192.168.20.10 eq 53
!
! Web out to anywhere.
access-list 110 permit tcp 192.168.40.0 0.0.0.255 any eq 80
access-list 110 permit tcp 192.168.40.0 0.0.0.255 any eq 443
!
! Let guests get an address. Without this the implicit deny stops
! DHCP and the VLAN never works at all.
access-list 110 permit udp any host 255.255.255.255 eq 67
!
! Ping to the gateway only, so users can prove their own link.
access-list 110 permit icmp 192.168.40.0 0.0.0.255 host 192.168.40.1
!
! Everything else is dropped by the implicit deny, logged so that
! surprises are visible.
access-list 110 deny ip any any log
!
interface GigabitEthernet0/0/2
 description Guest VLAN
 ip access-group 110 in
```

💡 eq 53 on the destination, not the source

In that list the port qualifier follows the *destination* address, because the guest is connecting to port 53 on the resolver. Its own source port is ephemeral and unpredictable, so it is left unspecified.

Getting this backwards — `permit udp 192.168.40.0 0.0.0.255 eq 53 any` — is a classic and produces an ACL that matches almost nothing. The position of the port in the line is the whole meaning.

established

`permit tcp any 192.168.10.0 0.0.0.255 established` matches only TCP segments with the ACK or RST flag set — that is, packets belonging to a conversation that somebody inside already started. A pure connection attempt from outside has only SYN set and is not matched.

It is a cheap way to say “replies to our traffic may return, new connections may not.” It is not stateful — it inspects flags, not connections — and the exam likes that distinction.

24.5 Named ACLs, and why they are what you should write

>_ Configure — the same policy, named and editable

```
ip access-list extended GUEST-OUT
remark Guest VLAN -- web, DNS and nothing else
10 permit udp 192.168.40.0 0.0.0.255 host 192.168.20.10 eq 53
20 permit tcp 192.168.40.0 0.0.0.255 host 192.168.20.10 eq 53
30 permit tcp 192.168.40.0 0.0.0.255 any eq 80
40 permit tcp 192.168.40.0 0.0.0.255 any eq 443
50 deny ip any any log
!
interface GigabitEthernet0/0/2
 ip access-group GUEST-OUT in
```

This is the practical advantage, and it is a large one

With sequence numbers you can insert and delete single lines:

- `ip access-list extended GUEST-OUT then 35 permit tcp ... eq 8080` inserts a rule between 30 and 40.
- `no 40` removes one entry and leaves the rest standing.

With a numbered ACL on older IOS, `no access-list 110` deletes the *entire list* — and if it is still applied to an interface, the interface now has an empty ACL, which denies everything. That is how a routine edit becomes an outage. Number your entries in tens so there is room to insert later, and prefer named ACLs.

24.6 Applying and verifying

Command	Where and what
<code>ip access-group <acl> in</code>	On an interface. Filters packets <i>arriving</i> . Evaluated before routing, so it sees traffic the router might have dropped anyway — cheaper.
<code>ip access-group <acl> out</code>	On an interface. Filters packets <i>leaving</i> after a routing decision. Does not filter traffic the router itself originates.
<code>access-class <acl> in</code>	On line vty. Controls who may open a management session. Use a standard ACL naming the permitted management subnets.

>_ Configure — restricting management access

```
ip access-list standard MGMT-HOSTS
 10 permit 10.0.0.0 0.0.0.255
 20 permit host 192.168.99.14
!
line vty 0 15
 access-class MGMT-HOSTS in
 transport input ssh
```

Q R1# show access-lists

```
Standard IP access list MGMT-HOSTS
 10 permit 10.0.0.0, wildcard bits 0.0.0.255 (214 matches)
 20 permit 192.168.99.14 (3 matches)
Extended IP access list GUEST-OUT
 10 permit udp 192.168.40.0 0.0.0.255 host 192.168.20.10 eq domain (8841 matches)
 20 permit tcp 192.168.40.0 0.0.0.255 host 192.168.20.10 eq domain (12 matches)
 30 permit tcp 192.168.40.0 0.0.0.255 any eq www (40218 matches)
 40 permit tcp 192.168.40.0 0.0.0.255 any eq 443 (338104 matches)
 50 deny ip any any log (2277 matches)
```

💡 The match counters are the diagnostic

That output tells you more than the configuration does.

- **A rule with zero matches** is either unnecessary or unreachable — shadowed by an earlier line. Both are worth investigating.
- **A rising counter on the final deny** tells you the ACL is dropping things, and log tells you what. This is the first place to look when an application “sometimes” fails.
- **Counters that do not move at all** mean the ACL is not in the path. Check it is actually applied, and in the direction you think.

clear access-list counters before a test gives you a clean baseline.

Q R1# show ip interface GigabitEthernet0/0/2 | include access list

```
Outgoing access list is not set
Inbound access list is GUEST-OUT
```

❗ Common Pitfalls

- **Forgetting the implicit deny.** Permit what you need and everything else stops. This is the single biggest cause of ACL outages.
- **A general permit above a specific deny.** First match wins, so the deny never runs.
- **A standard ACL applied near the source.** Blocks the host from everything, not from the one destination you meant.
- **Port qualifier on the wrong side.** eq 80 belongs with the destination for a client reaching a web server.
- **Writing the ACL and not applying it.** No error, no effect.
- **Applying it in the wrong direction.** in and out are relative to the interface, not to the building.
- **no access-list 110 on a live numbered ACL.** Deletes the whole list; the interface is left denying everything.
- **Blocking your own management session.** Think about vty before you press enter on a remote device.
- **Forgetting supporting protocols.** DHCP, DNS and ICMP all need explicit permits in a restrictive list, or the VLAN looks completely dead.

 **Break & Fix** — An ACL meant to stop one subnet reaching the finance server has cut that subnet off from everything.

The requirement was: hosts in 192.168.30.0/24 must not reach the finance server at 192.168.20.15. Everything else should be unaffected. After the change, VLAN 30 cannot reach the internet, the file server, or DNS. It can still reach hosts within its own VLAN.

 R1# show access-lists 20

```
Standard IP access list 20
 10 deny 192.168.30.0, wildcard bits 0.0.0.255 (18442 matches)
 20 permit any (0 matches)
```

 R1# show ip interface Vlan30 | include access list

```
Inbound access list is 20
Outgoing access list is not set
```

Diagnosis. Two things are wrong and they compound.

First, this is a *standard* ACL, so it can only match the source. It has no way to express “to the finance server” — the destination is invisible to it.

Second, it is applied **inbound on VLAN 30’s own SVI**, which is as close to the source as it is possible to get. Every packet leaving VLAN 30 for anywhere hits line 10 and is dropped. Traffic *within* the VLAN still works because it is switched and never reaches the SVI at all — which is exactly why the symptom looks like a routing fault rather than an ACL.

The confirming detail is line 20: **permit any** has **zero matches**. Not one packet from anywhere got past line 10. An ACL whose last line never matches is dropping everything it sees.

Fix. The requirement names a destination, so it needs an extended ACL, and an extended ACL can safely sit near the source.

 **Configure** — an ACL that expresses the actual requirement

```
ip access-list extended VLAN30-POLICY
 10 remark Deny only the finance server, permit the rest
 20 deny ip 192.168.30.0 0.0.0.255 host 192.168.20.15
 30 permit ip any any
!
interface Vlan30
 no ip access-group 20 in
 ip access-group VLAN30-POLICY in
```

Note line 30. Without it the extended ACL would reproduce the original outage by a different route. Then **clear access-list counters** and re-test: line 20 should accumulate matches only when someone tries the finance server, and line 30 should carry everything else.

Lab 24.1 — Writing, reading and repairing ACLs (*Packet Tracer, or CML*)

Topology. R1 routing four VLANs — staff 10, servers 20, lab 30, guest 40 — with a server at 192.168.20.10 running DNS and HTTP, a finance server at 192.168.20.15, and an internet-facing link.

Task 1. Confirm full connectivity between all four VLANs before applying any ACL. Record the tests you used; you will repeat them after every change.

Task 2. Write and apply a numbered standard ACL restricting management access to the VTY lines to the staff VLAN only. Test from staff and from lab.

- Task 3.** Convert it to a named ACL. Insert an additional permitted host between two existing lines without retyping the list.
- Task 4.** Write an extended named ACL for the guest VLAN permitting only DNS to 192.168.20.10, HTTP and HTTPS anywhere, and DHCP. Apply it inbound. Verify each permitted service works and at least three others do not.
- Task 5.** Read the match counters after your tests. Identify any rule with zero matches and explain why.
- Task 6. Break 1.** Remove the DHCP permit and release a guest host's lease. Predict the result before testing.
- Task 7. Break 2.** Move the `permit tcp ... eq 443` line above the `deny` that follows it, then below it, and observe the counters in each case.
- Task 8. Break 3.** Reproduce the Break & Fix above deliberately: use a standard ACL near the source to try to protect one server. Confirm the symptom, then repair it with an extended ACL.
- Task 9. Break 4.** Apply an ACL that blocks your own SSH session on a device you can still reach by console. Recover it, then write down the habit that would have prevented it.
- Task 10. Extension.** Add `permit tcp any 192.168.20.0 0.0.0.255 established` to an internet-facing inbound ACL. Verify that internal hosts can browse out and that inbound connections are refused, and explain why this is not a stateful firewall.

🔍 Checkpoint — Test Yourself

1. Give the wildcard mask for 10.20.16.0/20 and the range it matches.
2. Why is a standard ACL placed near the destination and an extended ACL near the source?
3. An ACL's final `permit any` shows zero matches. What does that tell you?
4. What does `established` match, and why is it not stateful?
5. Name three supporting protocols that a restrictive inbound ACL must explicitly permit, and give the symptom for each if it is missed.

📋 Chapter Summary

- Top down, first match wins, implicit `deny any` at the end. Order is behaviour.
- Wildcard mask = 255 minus each octet of the subnet mask. Check it by adding it to the network address.
- Standard (1–99, 1300–1999) matches source only — place near the destination. Extended (100–199, 2000–2699) matches source, destination, protocol and port — place near the source.
- Named ACLs with sequence numbers can be edited a line at a time. Numbered ones on older IOS cannot, and deleting one that is still applied denies everything.
- Apply with `ip access-group in|out` on an interface, or `access-class in` on the VTY lines.
- `show access-lists` match counters are the real diagnostic: a zero on the last line means everything is being dropped; a zero in the middle means a shadowed rule.
- Remember DHCP, DNS and ICMP, or a restrictive ACL makes a VLAN look dead.

🔧 Review Questions

1. (MCQ) Which wildcard mask matches exactly the range 172.20.32.0 to 172.20.39.255?

- (a) 0.0.7.255 (b) 0.0.15.255 (c) 0.0.3.255 (d) 255.255.248.0

2. (MCQ) A standard ACL should be applied:

- (a) as close to the source as possible
- (b) as close to the destination as possible
- (c) on every interface of the router
- (d) inbound only

3. (Short) Explain what happens when a numbered ACL that is applied to a live interface is deleted with `no access-list 110`.

4. (Short) An ACL permitting HTTP and HTTPS is applied inbound on a user VLAN and the whole VLAN immediately loses connectivity, including the ability to obtain an address. Explain why, and give the two entries that were missing.

5. (Applied) Write a named extended ACL that allows 10.10.10.0/24 to reach only SSH on 10.20.20.5, DNS on 10.20.20.10, and HTTPS anywhere, denying and logging everything else. State where you would apply it and in which direction, and justify both choices.

6. (Applied) You are given an ACL and told users cannot print. Describe, in order, the four commands you would run to establish whether the ACL is responsible, and say what each one would prove.

Layer 2 Security

EXAM TOPICS **4.7**

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Configure** port security with an appropriate maximum, learning method and violation mode.
- **Configure** DHCP snooping, and explain which ports must be trusted and why.
- **Configure** dynamic ARP inspection, including the case it cannot handle without help.
- **Configure** storm control and IPv6 RA guard.
- **Predict** the outage each of these causes when deployed carelessly — which is how they are examined and how they fail in practice.

🔗 Before You Start

Chapter 3 for MAC learning. Chapter 8 for access ports. Chapter 20 — DHCP snooping is unintelligible without DORA. Chapter 5 for router advertisements, which RA guard exists to police.

🔑 Key Terms in This Chapter

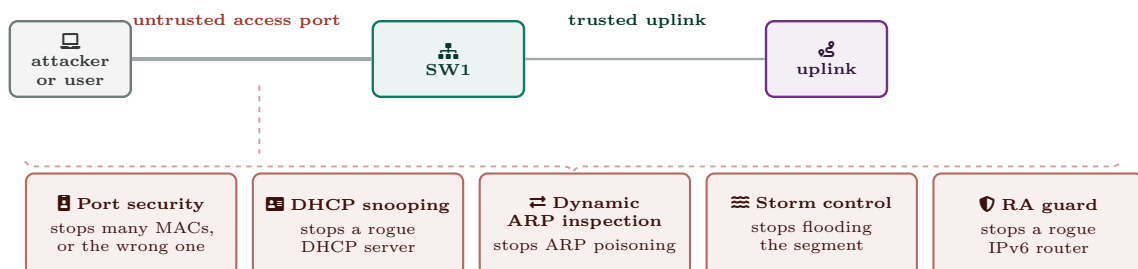
port security • sticky MAC • violation mode • err-disabled • DHCP snooping • trusted port • binding database • dynamic ARP inspection • ARP ACL • storm control • RA guard

🏠 Five named sub-topics, all *configure*

Topic **4.7** enumerates them: **4.7.a** DHCP snooping, **4.7.b** dynamic ARP inspection, **4.7.c** storm control, **4.7.d** RA guard, **4.7.e** port security.

Five features, one idea: **the access port is where the network meets people it does not control**, and each of these constrains one thing an attached device could otherwise do — claim many identities, answer as a DHCP server, lie in ARP, flood the segment, or claim to be an IPv6 router.

Learn them as that list of five abuses and the configuration follows.



All five live on the *access* port, and all five exist because the device plugged into it is not under your control.

Figure 25.1: Five features, five abuses, one port. The uplink is trusted; everything facing a user is not.

Deploy them one at a time

Each of these can black-hole a floor on its own, and two of them — DHCP snooping and dynamic ARP inspection — will do so if the uplink is left untrusted. Turn one on, on one switch, and watch it before doing the next.

25.1 Port security — limiting who may appear on a port

>_ Configure — a normal user access port

```
interface GigabitEthernet1/0/5
description Desk 5
switchport mode access
switchport access vlan 10
spanning-tree portfast
spanning-tree bpduguard enable
!
switchport port-security
switchport port-security maximum 2
switchport port-security mac-address sticky
switchport port-security violation restrict
switchport port-security aging time 60
switchport port-security aging type inactivity
```

switchport mode access must come first

Port security cannot be enabled on a port still in dynamic negotiation mode. If you type `switchport port-security` on a default port, IOS rejects it. Set the mode, then secure it — the same command-order rule as VLAN assignment in Chapter 8.

25.1.1 Choosing the maximum

The default is 1, which is almost never right.

- **One device on the port:** 1.
- **A PC behind an IP phone:** 3. The phone appears in the voice VLAN, the PC in the data VLAN, and the phone commonly registers a second address of its own. Setting 2 here produces violations that come and go with the phone's firmware — a genuinely miserable fault to chase.
- **A small unmanaged switch or a lab bench:** count the devices and add headroom, or do not use port security on that port at all.
- **A virtualisation host:** port security is usually the wrong tool. Every VM has its own MAC.

25.1.2 The three violation modes

Mode	Traffic	Notification	Use when
protect	Offending frames dropped, port stays up	None at all. No counter, no log	Almost never. Silent failure is worse than an outage
restrict	Offending frames dropped, port stays up	Counter increments, syslog and SNMP trap	User ports where you want evidence without an outage
shutdown	Port goes err-disabled — everything stops	Syslog, trap, and a very fast phone call	The default. High-security areas

💡 protect is a trap, and the exam knows it

It sounds like the gentle, sensible option. It is the one mode that gives you no evidence whatsoever: a user whose traffic is being silently discarded reports “the network is slow”, and nothing anywhere records why.

Prefer `restrict` when you do not want to drop the port, and `shutdown` when you do.

25.1.3 Recovering an err-disabled port

Manual recovery is `shutdown` then `no shutdown` on the interface — both, in that order; `no shutdown` alone does nothing to a port in `err-disabled`. To recover automatically:

>_ Configure — automatic recovery after five minutes

```
errdisable recovery cause psecure-violation
errdisable recovery cause bpduguard
errdisable recovery interval 300
```

🔍 SW1# show port-security interface GigabitEthernet1/0/5

```
Port Security          : Enabled
Port Status            : Secure-up
Violation Mode         : Restrict
Aging Time             : 60 mins
Aging Type             : Inactivity
Maximum MAC Addresses  : 2
Total MAC Addresses    : 2
Configured MAC Addresses : 0
Sticky MAC Addresses   : 2
Last Source Address:Vlan : 0050.7966.6800:10
Security Violation Count : 14
```

A non-zero Security Violation Count on a Secure-up port is the useful signal here: the port is working, and something is repeatedly being refused. That is either an attack or — far more often — a maximum set too low.

⚠️ Sticky addresses are learned, not saved

`mac-address sticky` converts dynamically learned addresses into running-configuration entries. They are *not* written to the startup configuration until you save. Reload without saving and the port relearns whatever is plugged in at that moment — which may be the wrong device, now permanently authorised.

If you use sticky, `write memory` is part of the procedure, not an afterthought.

25.2 DHCP snooping — stopping rogue servers

An attacker — or a helpful colleague with a home router — who answers DHCP first becomes the default gateway for everyone who believed them. DHCP snooping fixes this by deciding which ports are allowed to send *server* messages.

📖 Trusted and untrusted

On an untrusted port, DHCP *server* messages (OFFER, ACK, NAK) are dropped. Client messages pass. On a trusted port everything passes.

Every port is untrusted once snooping is enabled on a VLAN. You must explicitly trust the path toward the legitimate server.

>_ Configure — DHCP snooping on an access switch

```

ip dhcp snooping
ip dhcp snooping vlan 10,20,30
!
! Almost always required. The switch inserts option 82 into relayed
! requests; a router that is not configured to expect it discards the
! request, and every client fails. See the alert below.
no ip dhcp snooping information option
!
! The uplink carries the server's replies. Trust it.
interface GigabitEthernet1/0/48
  description Uplink to distribution
  ip dhcp snooping trust
!
! User ports stay untrusted, and are rate-limited so a single host
! cannot exhaust the pool by flooding requests.
interface range GigabitEthernet1/0/1-46
  ip dhcp snooping limit rate 10

```

⚠ The two faults that make snooping look broken

- **The uplink is not trusted.** The server's OFFER arrives on an untrusted port and is dropped. *Every* client on the switch fails, and the DHCP server logs nothing wrong because it answered.
- **Option 82.** By default the switch adds option 82 to requests it forwards, but with a giaddr of 0.0.0.0. Cisco routers discard such packets unless told otherwise. Either no ip dhcp snooping information option on the switch, or ip dhcp relay information trust-all on the router.

Both produce the same symptom — a whole switch full of clients with no addresses, immediately after a change that “should have been transparent”.

🔍 SW1# show ip dhcp snooping binding

MacAddress	IpAddress	Lease(sec)	Type	VLAN	Interface
00:50:79:66:68:00	192.168.10.21	27154	dhcp-snooping	10	GigabitEthernet1/0/5
00:50:79:66:68:22	192.168.10.22	27301	dhcp-snooping	10	GigabitEthernet1/0/7

💡 The binding table is the point

Stopping rogue servers is useful. Building this table is what makes the next two features possible: it is the switch's record of *which MAC legitimately holds which IP on which port*. Dynamic ARP inspection and IP source guard both consult it. So DHCP snooping is not an optional extra you turn on afterwards. It is the foundation, and it has to go first.

25.3 Dynamic ARP inspection — stopping ARP lies

ARP has no authentication. Any host can claim any address, and a host that claims to be the default gateway receives everyone's traffic. DAI checks every ARP message on an untrusted port against the snooping binding table and drops anything that disagrees.

>_ Configure — DAI on the same switch

```

! DHCP snooping must already be running on these VLANs.
ip arp inspection vlan 10,20,30
!

```

```

! Optional but recommended: also check that the Ethernet header
! addresses agree with the ARP payload, and that the IP is sane.
ip arp inspection validate src-mac dst-mac ip
!
! Trust the uplink, exactly as for snooping. Switch-to-switch links
! must be trusted or legitimate ARP is dropped between switches.
interface GigabitEthernet1/0/48
  ip arp inspection trust
!
! Rate-limit user ports. The default is 15 packets per second, and
! exceeding it err-disables the port.
interface range GigabitEthernet1/0/1-46
  ip arp inspection limit rate 15

```

⚠ The case DAI cannot handle on its own

A statically addressed device — a server, a printer, a legacy controller — never asks for a DHCP lease, so it has **no entry in the binding table**. When DAI is enabled, its ARP replies match nothing and are dropped, and the device disappears from the network. The fix is an ARP ACL naming the address and MAC explicitly:

>_ Configure — permitting statically addressed hosts

```

arp access-list STATIC-SERVERS
  permit ip host 192.168.20.15 mac host 0011.2233.4455
  permit ip host 192.168.20.16 mac host 0011.2233.4456
!
ip arp inspection filter STATIC-SERVERS vlan 20

```

Inventory your static devices *before* enabling DAI, not after the calls start.

Q SW1# show ip arp inspection statistics vlan 10

Vlan	Forwarded	Dropped	DHCP Drops	ACL Drops
----	-----	-----	-----	-----
10	88214	37	37	0

25.4 Storm control — limiting flooding

A broadcast storm — from a loop, a faulty NIC or a deliberate flood — can saturate a segment. Storm control caps how much of a port's bandwidth may be broadcast, multicast or unknown unicast.

>_ Configure — storm control on user ports

```

interface range GigabitEthernet1/0/1-46
  storm-control broadcast level 5.00 3.00
  storm-control multicast level 10.00
  storm-control action trap

```

Two numbers on the first line: the port suppresses broadcast traffic when it exceeds 5% of bandwidth and resumes when it falls below 3%. The gap prevents the port oscillating on and off at the threshold.

- **action trap** sends a notification and keeps suppressing — the usual choice.
- **action shutdown** err-disables the port. Decisive, and it turns a performance problem into an outage, so reserve it for ports where that is the right trade.
- Levels can also be given in pps or bps where a percentage is awkward.

Q SW1# show storm-control broadcast

Interface	Filter State	Upper	Lower	Current
Gi1/0/1	Forwarding	5.00%	3.00%	0.42%
Gi1/0/7	Blocking	5.00%	3.00%	6.11%

Blocking on Gi1/0/7 means it is suppressing right now. Storm control treats the symptom; a port that blocks repeatedly is telling you to go and find the loop (Chapter 12).

25.5 RA guard — the IPv6 equivalent

In IPv6 there is no DHCP server to impersonate for basic connectivity. Hosts learn their prefix and gateway from **router advertisements**, so a host that sends RAs — maliciously, or because someone enabled internet sharing on a laptop — becomes the default gateway for the segment. RA guard drops RAs arriving on ports that should not have any.

>_ Configure — RA guard on access ports

```

ipv6 nd raguard policy ACCESS-PORTS
  device-role host
!
ipv6 nd raguard policy UPLINKS
  device-role router
!
interface range GigabitEthernet1/0/1-46
  ipv6 nd raguard attach-policy ACCESS-PORTS
!
interface GigabitEthernet1/0/48
  ipv6 nd raguard attach-policy UPLINKS

```

🎓 Map each feature to the abuse it prevents

- **Port security** — one device claiming many MAC addresses, or an unauthorised device on a port.
- **DHCP snooping** — a rogue DHCP server handing out gateways.
- **Dynamic ARP inspection** — ARP poisoning, the on-path attack.
- **Storm control** — flooding, whether hostile or accidental.
- **RA guard** — a rogue IPv6 router. The v6 counterpart of DHCP snooping.

Matching questions on this list are close to free marks. Learn the five pairs.

❗ Common Pitfalls

- **Not trusting the uplink** for DHCP snooping or DAI. Takes out an entire switch.
- **Option 82 left on** with a Cisco relay that discards it. Same symptom, different cause.
- **Enabling DAI before inventorying static hosts**. Servers and printers vanish.
- **Port security maximum of 2 on a phone port**. Should be 3.
- **Using protect**. Silent drops and no evidence.
- **Sticky MACs never saved**. A reload re-learns whatever is plugged in.
- **no shutdown alone on an err-disabled port**. You must shutdown first.
- **Storm control with action shutdown on user ports**. Turns a transient into a truck roll.
- **Enabling everything at once on a Friday**. Each of these features can black-hole a floor. Deploy one at a time, on one switch, and watch.

 **Break & Fix** — After a security rollout, two servers and every printer stop responding. PCs are fine.

DHCP snooping and DAI were enabled across the access layer overnight. In the morning, all DHCP clients work normally — addresses lease, browsing works. But two application servers and every network printer are unreachable from everywhere, including from hosts in their own VLAN. The devices themselves are powered on, their link lights are up, and their switch ports show connected.

 SW1# show ip arp inspection statistics vlan 20

Vlan	Forwarded	Dropped	DHCP Drops	ACL Drops
20	4471	1902	1902	0

 SW1# show ip dhcp snooping binding | include 192.168.20

Diagnosis. The binding query returns nothing for the server subnet. The servers and printers are statically addressed, so they never leased anything and never appeared in the snooping binding table.

DAI validates every ARP message on an untrusted port against that table. With no entry, every ARP reply these devices send is dropped — so no other host can ever resolve their MAC address, and they become unreachable while remaining perfectly healthy. The DHCP Drops column confirms it: 1902 packets dropped specifically for having no binding, and zero ACL drops because no ARP ACL exists.

Two details make the diagnosis clean. First, DHCP clients are unaffected — they have bindings, so this is not a general layer 2 fault. Second, the devices are unreachable *within* their own VLAN, which rules out routing entirely: ARP is the only thing that operates there.

Fix. Give the static devices explicit permission.

 **Configure** — an ARP ACL for the static estate

```
arp access-list STATIC-HOSTS
 permit ip host 192.168.20.15 mac host 0011.2233.4455
 permit ip host 192.168.20.16 mac host 0011.2233.4456
 permit ip host 192.168.20.30 mac host 00aa.bbcc.dd01
 permit ip host 192.168.20.31 mac host 00aa.bbcc.dd02
!
ip arp inspection filter STATIC-HOSTS vlan 20
```

The process lesson. The rollout was not wrong, it was incomplete. DAI's dependency on the binding table is documented and predictable, and the work that was skipped was the inventory: every statically addressed device, its address and its MAC, before the feature is enabled. On a site of any size, the alternative is DHCP reservations for those devices instead, so that they build bindings like everyone else — which is the better long-term answer.

 **Lab 25.1** — Securing an access switch, one feature at a time (*Packet Tracer for tasks 1–6; CML for RA guard*)

Platform note. Packet Tracer supports port security, DHCP snooping, DAI and storm control. RA guard needs a real IOS image, so task 7 requires CML or GNS3.

Topology. SW1 with access ports in VLANs 10 and 20, an uplink to DSW1, a DHCP server beyond it, one statically addressed server in VLAN 20, a laptop you will use as the attacker, and an IP phone with a PC behind it.

- Task 1.** Baseline: confirm every host has connectivity and can obtain a lease. Record the MAC address of every device — you will need them.
- Task 2.** Configure port security on one access port with a maximum of 1 and `violation shutdown`. Move the cable to a second device and observe the `err-disable`. Recover it manually, then configure automatic recovery and repeat.
- Task 3.** Set the phone port to a maximum of 2 and confirm the violation. Raise it to 3 and confirm it clears. Explain the third address.
- Task 4.** Configure sticky learning, verify with `show run interface`, then reload *without* saving. Explain what you find.
- Task 5.** Attach the laptop running a DHCP server to an access port and confirm it can poison a client. Then enable DHCP snooping — **deliberately omitting** the trust on the uplink — and record the resulting symptom before fixing it.
- Task 6.** Enable DAI. Confirm the statically addressed server becomes unreachable, diagnose it from `show ip arp inspection statistics`, and repair it with an ARP ACL.
- Task 7.** Configure storm control at 1% with `action trap`, generate broadcast traffic, and watch the filter state change. Then change to `action shutdown` and compare the consequences.
- Task 8. RA guard (CML).** Enable IPv6 on the VLAN, confirm hosts get addresses by SLAAC, then advertise a rogue prefix from the laptop. Observe the hosts move to it. Apply RA guard and confirm the rogue RAs are dropped.
- Task 9. Extension.** Write the deployment runbook you would actually use for this rollout: the order, the pre-checks, what you would verify after each step, and the rollback for each.

? Checkpoint — Test Yourself

1. Name each of the five features and the specific abuse it prevents.
2. Why must DHCP snooping be configured before DAI, and what does DAI consult?
3. What is wrong with `violation protect`?
4. Why is 3 the right port-security maximum on a port with a phone and a PC?
5. A statically addressed printer stops responding after DAI is enabled. Explain the mechanism and give the fix.
6. Which two ports on an access switch almost always need to be trusted, and for which two features?

✓ Chapter Summary

- Five features, five abuses: port security (identity), DHCP snooping (rogue server), DAI (ARP poisoning), storm control (flooding), RA guard (rogue IPv6 router).
- Port security needs `switchport mode access` first. Maximum 3 for a phone plus PC. `restrict` for evidence without an outage, `shutdown` for enforcement, `protect` essentially never.
- Sticky addresses live in the running configuration only until you save.
- Recover an `err-disabled` port with `shutdown` then `no shutdown`, or configure `errdisable recovery`.
- DHCP snooping makes every port untrusted; trust the uplink, and expect to disable option 82 insertion.
- Its binding table is what DAI validates against — which is why statically addressed devices need an ARP ACL.

- Storm control has a rising and a falling threshold; **trap** suppresses and reports, **shutdown** err-disables.
- RA guard classifies ports as **host** or **router** and drops RAs from hosts.

Review Questions

1. **(MCQ)** Which violation mode drops offending traffic, keeps the port up, and generates no notification of any kind?
(a) **restrict** (b) **shutdown** (c) **protect** (d) **err-disable**
2. **(MCQ)** Dynamic ARP inspection validates ARP packets against:
(a) the MAC address table
(b) the DHCP snooping binding table
(c) the routing table
(d) the CAM table's sticky entries
3. **(Short)** Every client on one switch loses DHCP immediately after snooping is enabled. Give two distinct causes and the command that fixes each.
4. **(Short)** Explain why sticky MAC addresses can become a security problem after an unplanned reload.
5. **(Applied)** Write the configuration securing a 24-port access switch: user ports in VLAN 10 with phones, an uplink on port 24, DHCP snooping and DAI on VLAN 10, storm control at 3%, and RA guard. Mark clearly which commands are global and which are per-interface, and state the order you would deploy them in.
6. **(Applied)** You must enable DAI on a campus with an unknown number of statically addressed devices. Describe the procedure you would follow to avoid the outage in this chapter's Break & Fix, including what you would collect beforehand and how you would stage the change.

IPsec VPNs

EXAM TOPICS **4.5** Learning Outcomes

By the end of this chapter you will be able to:

- **Distinguish** site-to-site from remote-access VPNs and say which suits a given requirement.
- **Name** the protocols in the IPsec suite — IKE, ESP and AH — and state what each contributes.
- **Compare** tunnel mode and transport mode, and say which is used where.
- **Explain** why AH is effectively unusable through NAT, and what is used instead.

 Before You Start

Chapter 6 for ports and protocol numbers. Chapter 22 — the interaction between IPsec and NAT is most of what makes this topic interesting. Chapter 23 for the idea of authentication as distinct from encryption.

 Key Terms in This Chapter

IPsec • IKE • ISAKMP • ESP • AH • tunnel mode • transport mode • security association • NAT-T • site-to-site • remote access • GRE over IPsec

 The only *describe* topic in this part

Topic **4.5** is *describe IPsec remote access and site-to-site VPNs (protocols and transport modes)*. There is no configuration verb, and no crypto map or IKEv2 proposal syntax is required of you.

That is why this chapter is short and has no lab: writing one would teach you something the exam does not ask for, at the cost of time you need elsewhere in this part. What it *does* ask for is in the parenthesis — **the protocols and the transport modes** — so those two lists are what this chapter drills.

If you go on to CCNP Security, this chapter is the vocabulary you will build on. For now, know the names, the jobs and the trade-offs.

26.1 The problem

Two campus sites need to exchange traffic and the only path between them is the internet, which is shared with everyone and trusted by nobody. Anything sent in clear can be read; anything read can be altered; anything altered can be replayed.

A **virtual private network** makes that shared path behave like a private one. **IPsec** is the standard suite of protocols that does it at the network layer — which is the useful part, because it means applications need no changes and know nothing about it.

 The four services IPsec provides

- **Confidentiality** — encryption, so the payload cannot be read. Normally AES.
- **Integrity** — a keyed hash, so alteration is detectable. Normally SHA-2.

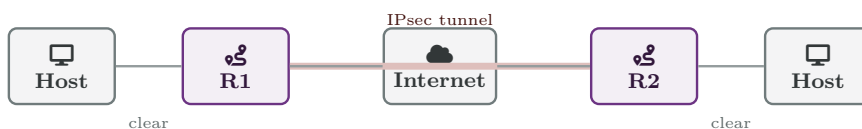
- **Authentication** — proof that the peer is who it claims to be. A pre-shared key or a digital certificate.
- **Anti-replay** — sequence numbers, so a captured packet cannot be re-sent later to useful effect.

Underpinning all four is **key management**: Diffie–Hellman, which lets two peers agree a shared secret over a channel an eavesdropper is watching.

26.2 Two deployment models

	Site-to-site	Remote access
Endpoints	Two gateways — routers or firewalls	One client device and a gateway
Who knows	Nobody. Hosts send ordinary packets; the gateways do the work	The user, who starts a client and authenticates
Lifetime	Permanent. Brought up and left up	On demand, per session
Typical use	Branch office to head office; campus to data centre	Staff working from home or travelling
Client software	None	Required — Cisco Secure Client, or a browser for a clientless TLS VPN
Authenticates	The <i>gateway</i> , by pre-shared key or certificate	The <i>user</i> , usually against RADIUS (Chapter 23)

Site-to-site



Remote access

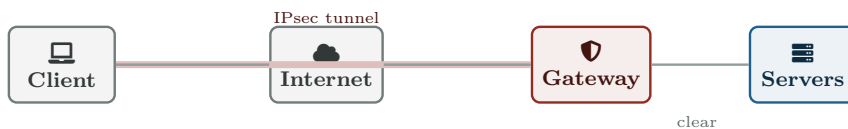


Figure 26.1: Where the tunnel starts decides the model. Site-to-site protects gateway to gateway; remote access protects all the way to the user's device.

💡 Traffic is only protected inside the tunnel

In the site-to-site diagram, traffic between the host and its own gateway is in clear. IPsec protects the internet crossing, not the LAN at either end. That is usually the right trade — but it means a VPN is not an answer to a threat already inside the building.

26.3 The protocols

Three names, and the exam expects you to know which does what.

Protocol	Identifier	Provides	Notes
IKE	UDP 500, and UDP 4500 with NAT-T	Peer authentication, key exchange, negotiation of everything else	The control plane. Nothing is encrypted until IKE has finished
ESP	IP protocol 50	Confidentiality, integrity, authentication, anti-replay	What is actually used. The only one of the three that encrypts
AH	IP protocol 51	Integrity, authentication, anti-replay — no encryption	Effectively obsolete. Breaks through NAT; see below

⚠ Two facts that are asked directly

ESP encrypts; AH does not. If a question asks which protocol provides confidentiality, the answer is ESP, every time.

ESP and AH are not TCP or UDP. They are IP protocols in their own right, numbered 50 and 51, sitting at the same layer as TCP (6) and UDP (17). This is why a firewall rule that permits “UDP 500” alone will let the tunnel negotiate and then carry no data — a memorable and very common real fault.

26.3.1 IKE, in two phases

- **Phase 1** builds a secure, authenticated channel between the two peers — the ISAKMP or IKE security association. The peers prove their identity (pre-shared key or certificate) and run Diffie–Hellman.
- **Phase 2** uses that channel to negotiate the security associations that will actually protect user traffic, including which traffic is to be protected at all.

IKEv2, the current version, does the same work in fewer exchanges, and adds built-in NAT traversal and liveness checking. New deployments use it.

📖 Security association

A one-directional agreement covering the algorithms, keys and lifetimes for one flow. Because it is one-directional, a working tunnel has *two* — one in each direction. Asymmetric failures, where traffic passes one way only, are usually a phase 2 mismatch.

26.4 Transport mode and tunnel mode

This is the second half of the topic’s parenthesis, and the distinction is entirely about **what gets encrypted and what IP header the packet travels under**.

	Transport mode	Tunnel mode
Encrypts	The payload only	The entire original packet
IP header	The original one is kept and is visible	A new header is added; the original is hidden
Overhead	Lower	Higher — an extra IP header

	Transport mode	Tunnel mode
Endpoints	The two hosts that are communicating	Two gateways, which are not the communicating hosts
Used for	Host-to-host on a trusted path; protecting a GRE tunnel that already carries the addressing	Site-to-site and remote-access VPNs — the normal case
Private addressing	Cannot hide it	Can carry it across the public internet

 If the question does not say otherwise, it is tunnel mode

Tunnel mode is the default for the VPNs this topic names. The single sentence to remember: **tunnel mode encrypts the original IP header and adds a new one; transport mode does not.** Every other difference in the table follows from that.

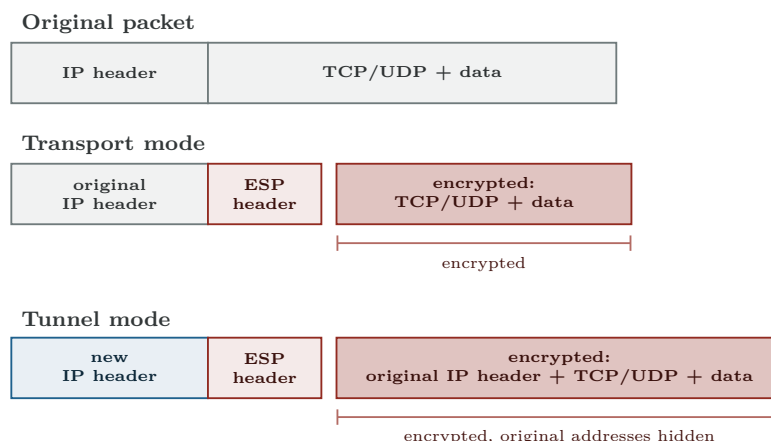


Figure 26.2: Transport mode keeps the original IP header. Tunnel mode encrypts it and adds a new one, which is what lets private addresses cross a public network.

26.5 IPsec and NAT

⚠ Why AH is dead

AH authenticates the *entire* packet, including the source and destination addresses in the IP header. NAT's whole job is to rewrite those addresses. So a NAT device on the path invalidates AH's integrity check by definition, and the far end discards the packet. There is no configuration that fixes this; the two mechanisms are logically incompatible. Since almost every internet path involves NAT somewhere, AH has been left behind, and ESP — which does not protect the outer header — is what everyone uses.

ESP has a milder version of the same problem: PAT needs port numbers to multiplex, and ESP is not a port-based protocol. **NAT traversal** solves it by detecting NAT on the path during IKE and wrapping ESP inside UDP port 4500, which PAT can then handle normally. It is automatic in IKEv2.

💡 GRE over IPsec, in one paragraph

IPsec tunnel mode carries unicast IP and nothing else — no multicast, and therefore no routing protocols. A site-to-site VPN that needs OSPF between the sites wraps the traffic in a GRE tunnel first, which *can* carry multicast, and then protects the GRE with IPsec. You will meet the phrase “GRE over IPsec”. For this exam, know why it exists: GRE brings the ability to run a routing protocol, IPsec brings the security, and neither does both.

26.6 When things do not work

Not examined as a configuration task, but these four explain most real failures and make the mechanics concrete.

- **Phase 1 never completes.** Mismatched pre-shared key, mismatched IKE proposal, or UDP 500 blocked. Nothing at all passes.
- **Phase 1 completes, phase 2 does not.** Mismatched transform set, or the two ends disagree about which traffic should be protected. The peers can talk; user traffic cannot.
- **Tunnel establishes, large packets fail.** The extra headers push packets past the path MTU. Small pings and SSH work, file transfers and web pages hang. A classic, and a good reminder that “it pings” is not “it works”.

- **Everything negotiates and no data flows.** A firewall permitting UDP 500 but not IP protocol 50.

! Common Pitfalls

- **Saying AH encrypts.** It does not. Only ESP does.
- **Calling ESP and AH “TCP” or “UDP”.** They are IP protocols 50 and 51.
- **Mixing up the modes.** Tunnel mode adds a new IP header; transport mode keeps the original.
- **Assuming a VPN protects the whole path.** It protects the tunnel.
- **Expecting a routing protocol to run over plain IPsec.** It cannot — multicast needs GRE.
- **Treating a successful ping as a working tunnel.** Test with something large enough to expose an MTU problem.

🔧 Break & Fix — The tunnel is up and shows both peers established. No user traffic crosses it.

A site-to-site VPN between R1 and R2. Both routers report an established IKE session with the other. Users at either site cannot reach the other site at all.

Q R1# show crypto isakmp sa

IPv4 Crypto ISAKMP SA	dst	src	state	conn-id	status
	198.51.100.9	203.0.113.2	QM_IDLE	1004	ACTIVE

Q R1# show crypto ipsec sa | include pkts|local ident|remote ident

```
local ident (addr/mask/prot/port): (192.168.10.0/255.255.255.0/0/0)
remote ident (addr/mask/prot/port): (192.168.20.0/255.255.255.0/0/0)
#pkts encaps: 0, #pkts encrypt: 0, #pkts digest: 0
#pkts decaps: 0, #pkts decrypt: 0, #pkts verify: 0
```

Diagnosis. Read the two outputs as a pair, because together they locate the fault precisely. QM_IDLE and ACTIVE mean **phase 1 succeeded**: the peers found each other, agreed on IKE parameters and authenticated. So the pre-shared key is right, UDP 500 is reaching both ends, and the peer addresses are correct. All of that is now ruled out.

But every counter is zero — in *both* directions. Nothing has been encrypted and nothing decrypted. Traffic is not being selected for the tunnel, or is being selected and then dropped before it is sent.

The identities are the place to look. R1 intends to protect 192.168.10.0/24 to 192.168.20.0/24. If R2's definition is not the exact mirror — a different mask, a wider or narrower range — phase 2 cannot agree on what the tunnel is for, and no traffic qualifies.

The class of fault, which is the point here. Phase 1 up with phase 2 idle is always a *policy* disagreement rather than a connectivity or credential one: the crypto ACL, the transform set, or the mode. Knowing which half of IKE succeeded halves the search before you have looked at anything.

The other candidate, if the identities do match, is a firewall permitting UDP 500 — which is why phase 1 worked — while dropping IP protocol 50. Symptoms are identical; the discriminator is whether **#pkts encaps** is zero (traffic never selected) or climbing while **#pkts decaps** stays at zero (sent, never arriving).

Checkpoint — Test Yourself

1. Which IPsec protocol provides confidentiality, and what is its IP protocol number?
2. State the difference between tunnel mode and transport mode in one sentence, and say which a site-to-site VPN uses.
3. Why can AH not survive a NAT device on the path?
4. What does phase 1 of IKE accomplish, and what does phase 2 accomplish?
5. Why is GRE needed if a site-to-site VPN must carry OSPF?
6. A tunnel is established and small pings succeed while file transfers hang. What is the likely cause?

Chapter Summary

- IPsec provides confidentiality, integrity, authentication and anti-replay, with Diffie–Hellman for key agreement.
- Site-to-site joins two gateways and is invisible to hosts; remote access runs from a user’s device to a gateway and authenticates the user.
- IKE (UDP 500, or 4500 with NAT-T) negotiates. ESP (IP protocol 50) protects data and **encrypts**. AH (IP protocol 51) authenticates but does **not** encrypt.
- Phase 1 builds the secure channel between peers; phase 2 builds the security associations that protect user traffic. SAs are one-directional, so a working tunnel has two.
- Tunnel mode encrypts the whole original packet and adds a new IP header — the normal case. Transport mode keeps the original header.
- AH is incompatible with NAT because it authenticates the header NAT rewrites. ESP with NAT-T (UDP 4500) is the working combination.
- Plain IPsec carries no multicast, so a VPN needing a routing protocol uses GRE over IPsec.

Review Questions

1. (MCQ) Which protocol provides encryption for IPsec?
(a) AH (b) ESP (c) IKE (d) GRE
2. (MCQ) In tunnel mode, what is encrypted?
(a) The payload only, with the original IP header retained
(b) The entire original packet, including its IP header
(c) The IP header only
(d) Nothing — tunnel mode provides integrity only
3. (Short) Explain why AH cannot be used across a NAT device, and name the mechanism that lets ESP work in the same situation.
4. (Short) A firewall administrator permits UDP 500 between two VPN gateways and reports that the tunnel “comes up but does not work”. Explain what is missing and why the tunnel appears healthy.
5. (Applied) A university needs branch campuses permanently connected to the data centre, with OSPF running between them, and separately needs staff to work from home. Recommend a VPN model for each requirement, name the protocols involved, and justify the use of GRE in one case and not the other.

6. **(Applied)** Given `show crypto isakmp sa` reporting QM_IDLE and `show crypto ipsec sa` reporting zero packets encapsulated, state which phase succeeded, what that rules out, and the two most likely causes.

PART V

Wireless, Virtualisation and Operations

This part looks like a miscellany and is not. It finishes two things at once: the three topics of domain 1 that had no natural home earlier — radio, clients and virtual infrastructure — and the whole of domain 5, which is about *operating* a network rather than building one.

Domain 5 is only 10% of the exam, and it is entirely new. Nothing in it existed under v1.1 in this form: agentic AI, prompt selection, and the five management approaches are all v2.0 additions, and SNMP and syslog have been re-cast around *interpreting* output rather than configuring it.

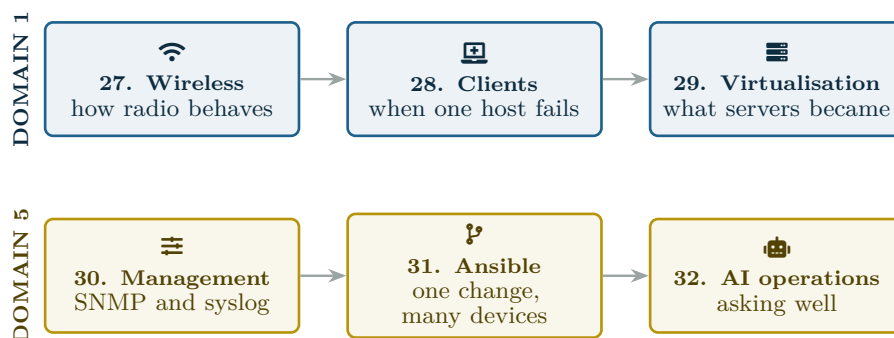


Figure P5.1: Part V. The top row completes domain 1; the bottom row is the whole of domain 5, and every topic in it is new to v2.0.

Ten per cent, and the cheapest marks in the exam

Domain 5 has six topics and almost no configuration. Four of them — [5.1](#), [5.2](#), [5.3](#) and [5.4](#) — are *describe* or *select* questions answerable from a page of well-organised facts. [5.6](#) asks you to *interpret* a syslog line, which is a skill you can acquire in an afternoon and keep for a career. Only [5.5](#), Ansible, asks you to *use* anything.

Candidates routinely under-prepare this domain because it is the smallest and because “AI” sounds unexaminable. It is 10% of the paper for perhaps 5% of the study effort of domain 2. Do not leave it until the last week.

What “agentic” is doing in a CCNA blueprint

Topic [5.1](#) names **agentic AI** specifically, and [5.2](#) asks you to choose a *prompt*. That is unusual for a Cisco exam and it is worth being straight about why it is there: network operations tooling now routinely includes systems that observe, suggest and — increasingly — act.

Chapter 32 treats this as an operational subject with real limits rather than a marketing one. The examinable content is small and specific: what distinguishes an agentic system from a generative one, and the four prompt components the blueprint names by hand.

Wireless Principles

EXAM TOPICS **1.5**

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Select** a band and a channel plan, and explain why 2.4 GHz has only three usable channels.
- **Describe** the RF characteristics that decide whether a link works: signal strength, noise, SNR, and what happens to radio waves on the way.
- **Compare** WEP, WPA, WPA2 and WPA3, and distinguish personal from enterprise authentication.
- **Identify** causes of interference, and tell co-channel interference apart from adjacent-channel interference.

🔗 Before You Start

Chapter 3 — wireless is a shared medium with collision *avoidance* rather than detection, and the contrast is the quickest way in. Chapter 9 for how access points attach to the wired network. Chapter 23 for RADIUS, which is what “enterprise” wireless authenticates against.

📖 Key Terms in This Chapter

band • channel • non-overlapping • channel bonding • dBm • RSSI • SNR • noise floor • attenuation • SSID • BSSID • WPA2 • WPA3 • SAE • 802.1X • co-channel interference

🎓 Four named sub-topics, and the verb is *describe*

Topic **1.5** is *describe wireless principles*, with four sub-topics named explicitly: **1.5.a** band and channel selection, **1.5.b** RF characteristics, **1.5.c** security protocols, **1.5.d** cause of interference.

There is **no wireless configuration on this blueprint**. v1.1 asked you to build a WLAN in a controller GUI; v2.0 does not. Do not spend time learning WLC menus for this exam — spend it on the four lists in this chapter, because that is exactly what is examined.

Access points do appear elsewhere: **2.2** covers connecting them to a switch port, which is Chapter 9’s material, and **1.6** covers the client end, which is Chapter 28’s.

27.1 Why wireless is different

A switch port is a private, full-duplex conversation between two devices. A radio channel is a room everybody shouts across. Three consequences shape everything else in this chapter.

- **It is half duplex and shared.** Only one device on a channel may transmit at a time. Ten clients on one access point share the airtime between them — the advertised data rate is not per-client bandwidth.
- **Collisions cannot be detected, only avoided.** A radio cannot listen while it transmits, so Ethernet’s CSMA/CD is impossible. Wireless uses CSMA/CA: listen, wait a random interval, transmit, and expect an acknowledgement for every frame.
- **The medium is unlicensed and shared with strangers.** Your neighbour’s network, a microwave oven and a Bluetooth headset all have the same right to the spectrum that you do.

Four names that get confused

- **SSID** — the network's name, as a human sees it. Several APs normally share one.
- **BSSID** — the MAC address of one radio on one AP. This is what a client is actually associated to.
- **BSS** — one AP's coverage area. **ESS** — several APs sharing an SSID so a client can roam between them.
- **Roaming** — a client deciding, on its own, to move from one BSSID to another within the same ESS. *The client decides*, not the network. This matters in Chapter 28.

27.2 Bands and channels

Band	Usable width	Non-overlapping 20 MHz channels	Character
2.4 GHz	About 83 MHz	Three: 1, 6 and 11	Long range, penetrates walls, hopelessly crowded
5 GHz	About 500 MHz	Around 25 , some requiring DFS	Shorter range, far more room, the sensible default
6 GHz	1200 MHz	59	Wi-Fi 6E and later only. Clean, but short range and newer clients only

Why 2.4 GHz has only three channels

The band is divided into channels numbered 1 to 11 in North America (1 to 13 in much of the rest of the world), spaced 5 MHz apart. But each channel is **20 MHz wide**. A 20 MHz signal spaced 5 MHz from its neighbour overlaps it heavily.

To avoid overlap you need five channel numbers of separation — which in a band of eleven gives you exactly three: **1, 6 and 11**. Any other plan puts two networks partly on top of each other, which is worse than putting them fully on top of each other. That last point is the one the exam tests, and it is explained below.

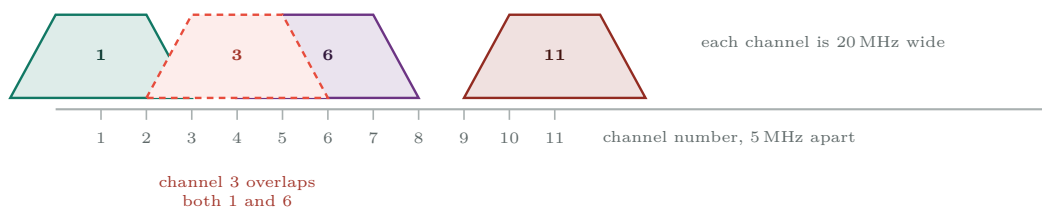


Figure 27.1: The 2.4 GHz band. Channels 1, 6 and 11 are the only trio that does not overlap; anything between them interferes with two networks instead of sharing one.

27.2.1 Channel bonding

Wider channels carry more data. 802.11n and later can bond adjacent 20 MHz channels into 40, 80 or 160 MHz.

⚠ The trade-off, which is examinable

Every doubling of channel width roughly doubles throughput — and **halves the number of non-overlapping channels available**, and raises the noise floor by about 3 dB because the receiver is listening to twice as much spectrum.

So: bonding is sensible at 5 and 6 GHz where there is room, and in a low-density deployment. It is **never** sensible at 2.4 GHz, where a 40 MHz channel consumes most of the band and leaves no clean plan for anyone, including you.

💡 DFS channels

Part of the 5 GHz band is shared with weather and military radar. Access points using those channels must run **dynamic frequency selection**: listen before transmitting, and vacate the channel within seconds if radar appears.

The practical consequence is a real fault you will meet: a DFS event moves an AP to a different channel and every client on it is briefly disconnected. If a site reports unexplained drops on 5 GHz at irregular intervals — particularly near an airport or a coast — DFS is the first thing to check.

27.3 RF characteristics

27.3.1 Measuring power in decibels

Radio power spans an enormous range, so it is measured logarithmically.

- **dBm** — power relative to 1 milliwatt. 0 dBm is 1 mW. Absolute.
- **dB** — a ratio, with no unit of its own. A gain or a loss.
- **dBi** — antenna gain relative to a theoretical isotropic radiator.

📊 Worked Example: The two rules that replace a calculator

The 3 dB rule: +3 dB doubles power, −3 dB halves it.

The 10 dB rule: +10 dB multiplies by ten, −10 dB divides by ten.

So 20 dBm is $10 \times 10 \times 1 \text{ mW} = 100 \text{ mW}$. And 23 dBm is double that again — 200 mW.

Going the other way: a client receiving −70 dBm is receiving one ten-millionth of a milliwatt. Radio receivers are extraordinarily sensitive, which is why small amounts of noise matter so much.

27.3.2 Signal, noise and the number that actually matters

Measure	What it is	Rule of thumb
RSSI	Received signal strength, in dBm. Always negative in practice	−67 dBm or better for voice and video; −70 for data; below −80 is unusable
Noise floor	The background RF energy on the channel, in dBm	Around −92 dBm is clean. −80 is a problem
SNR	The <i>gap</i> between signal and noise, in dB	25 dB or better for voice; 20 dB minimum for reliable data

⚠ Signal strength alone tells you almost nothing

A client at −65 dBm sounds healthy. If the noise floor on that channel is −70 dBm, the SNR is 5 dB and the link is unusable.

A client at -75 dBm sounds marginal. With a noise floor of -95 dBm the SNR is 20 dB and it works perfectly well.

SNR is the number that decides whether a link works. Users and helpdesk tools report bars and RSSI; you should ask for the noise floor before drawing any conclusion. This is the single most useful idea in the chapter and it comes up again in Chapter 28.

27.3.3 What happens to a wave on the way

Effect	What happens	Where you meet it
Absorption	Energy converted to heat by the material	Walls, concrete, and water — which means people. A room that works when empty and fails when full
Reflection	The wave bounces off a smooth surface	Metal, glass, lift shafts, warehouse racking. Causes multipath
Refraction	The wave bends passing between materials	Glass, water, atmospheric layers on long outdoor links
Diffraction	The wave bends around an obstacle, leaving a shadow behind it	Pillars, corners. Why coverage dies just around a corner
Scattering	The wave breaks up against an uneven surface or particles	Rough plaster, chain-link fence, dust, heavy rain
Free-space loss	Signal weakens with distance regardless of obstacles	Everywhere. Doubling the distance costs about 6 dB

💡 Antennas do not amplify

An antenna with 9 dBi of gain does not create energy. It *shapes* the same energy into a narrower pattern — more in the direction you want, less everywhere else.

- **Omnidirectional** — radiates in a doughnut around the antenna. Ceiling APs in offices. Low gain, wide coverage.
- **Directional** (patch, Yagi, parabolic) — radiates into a cone. Point-to-point links, long corridors, lecture theatres, warehouse aisles. High gain, narrow coverage.

A common failure at scale: replacing omnidirectional antennas with high-gain ones to “fix” coverage, and discovering the new pattern misses the floor directly below the AP entirely.

27.4 Security protocols

	Encryption	Key exchange	Status
WEP	RC4, static key	None worth the name	Broken since 2001. Recoverable in minutes. Never deploy
WPA	TKIP (RC4)	4-way handshake	Deprecated. A stopgap for WEP-era hardware
WPA2	AES-CCMP	4-way handshake, PSK or 802.1X	The long-standing default. Vulnerable to offline dictionary attack on weak pre-shared keys

	Encryption	Key exchange	Status
WPA3	AES-GCMP	SAE (Simultaneous Authentication of Equals)	Current. Resists offline attack, adds forward secrecy, and adds OWE for open networks

The two distinctions to get right

Personal against Enterprise is about *who* authenticates.

- **Personal (PSK)** — one shared passphrase for everyone. Simple. Nobody is individually identified, and changing it means changing it on every device.
- **Enterprise (802.1X/EAP)** — each user authenticates individually against a RADIUS server (Chapter 23). Revoke one account without touching anyone else. This is what a campus should run.

WPA2 against WPA3 is about *how the key is agreed*. WPA2-Personal's 4-way handshake can be captured and attacked offline, so a weak passphrase falls. WPA3 replaces it with SAE, where an attacker must guess against the live network each time — so offline attack is not possible, and each session has its own key even if the passphrase is later disclosed.

Open networks and OWE

An open network with no passphrase sends everything in clear. **Opportunistic Wireless Encryption**, part of the WPA3 family and marketed as “Enhanced Open”, encrypts the traffic anyway using an unauthenticated key agreement.

It stops passive eavesdropping on a guest network. It does **not** authenticate the AP, so it does not stop somebody standing up a fake one. Worth deploying on guest SSIDs; not worth mistaking for security.

27.5 Causes of interference

27.5.1 Wi-Fi interfering with Wi-Fi

	Co-channel (CCI)	Adjacent-channel (ACI)
Cause	Two APs on the <i>same</i> channel	Two APs on <i>partly overlapping</i> channels
What happens	They hear each other and take turns politely	They cannot decode each other, so they transmit over each other
Result	Reduced throughput. Everyone shares the airtime	Corruption and retransmission. Much worse
Verdict	Tolerable, and unavoidable at density	To be eliminated. This is why the plan is 1, 6, 11

Counter-intuitive, and a favourite question

Co-channel interference is better than adjacent-channel interference.

Two APs on channel 6 can hear one another, so CSMA/CA works: they defer to each other and share the channel. Throughput halves; nothing is corrupted.

Two APs on channels 6 and 8 cannot decode one another — each sees the other only as raised noise. Neither defers, both transmit, and frames are destroyed and retransmitted. Throughput collapses.

So a bad channel plan is worse than no channel plan. If you take one thing from this section: **overlap, do not straddle.**

27.5.2 Everything else

- **Microwave ovens** — 2.4 GHz, high power, intermittent. Classic symptom: wireless fails in the staff room at 12:30 every day.
- **Bluetooth and wireless headsets** — 2.4 GHz frequency hopping, low power, constant.
- **Cordless phones, video senders, wireless cameras, baby monitors** — often 2.4 GHz, often continuous, often invisible to Wi-Fi tools because they are not Wi-Fi.
- **Radar** — 5 GHz, and triggers DFS events rather than plain interference.
- **Too much of your own network** — APs at full transmit power in a dense deployment interfere with each other. Reducing power often improves a busy site.
- **Physical obstruction** — not interference strictly, but it presents identically to a user. Lift shafts, fire doors, foil-backed insulation, and fish tanks.

💡 Non-Wi-Fi interference needs a spectrum analyser

A Wi-Fi analyser on a laptop shows you Wi-Fi. A microwave oven does not appear in it, because it transmits no frames — it just raises the noise floor.

So a channel that looks empty in a Wi-Fi scan and performs terribly is the signature of a non-Wi-Fi source. Finding it needs a spectrum analyser, which looks at raw RF energy rather than at decoded frames. Knowing *that the distinction exists* is the examinable part.

❗ Common Pitfalls

- **Using any 2.4 GHz channel other than 1, 6 or 11.** Creates ACI for two networks instead of sharing with one.
- **Bonding channels at 2.4 GHz.** Consumes the band.
- **Reading RSSI without the noise floor.** SNR is the number that matters.
- **Turning transmit power up to fix coverage.** In a dense deployment this usually makes things worse, and it does nothing for the client's weaker transmitter on the return path.
- **Confusing SSID with BSSID.** A client roams between BSSIDs while the SSID never changes.
- **Assuming the AP controls roaming.** The client decides.
- **Treating WPA2-Personal as adequate for a campus.** Use Enterprise; a shared passphrase cannot be revoked for one person.
- **Expecting a Wi-Fi scanner to reveal a microwave oven.**

🔧 Break & Fix — One lecture theatre is unusable when full and perfect when empty.

A 200-seat theatre has four access points and tests clean every morning. During lectures, clients associate but throughput collapses and voice calls fail. The wired network behind the APs shows no errors and no congestion. A survey taken at 08:00 shows every seat at -58 dBm or better.

>_ Survey, taken during a full lecture — wifi scan -- summary

BSSID	SSID	Chan	Width	RSSI	Noise	SNR
00:11:22:aa:bb:01	CAMPUS	6	20	-61	-73	12
00:11:22:aa:bb:02	CAMPUS	6	20	-64	-73	9
00:11:22:aa:bb:03	CAMPUS	8	20	-59	-73	14
00:11:22:aa:bb:04	CAMPUS	11	20	-66	-73	7

Diagnosis. Three faults are visible in that one table, and they compound.

First, the noise floor is -73 dBm. A clean environment is around -92 . Something is raising it by nearly 20 dB, and the strong signal readings are therefore worthless — every SNR here is below the 20 dB minimum for reliable data.

Second, the channel plan is wrong. Four APs in one room are using channels 6, 6, 8 and 11. The pair on channel 6 is co-channel interference, which is merely wasteful. But **channel 8 overlaps both 6 and 11** — it is adjacent-channel interference against three other radios simultaneously, and that is corruption rather than sharing.

Third, the room is full of people. Bodies are largely water, which absorbs 2.4 GHz strongly. This is why it works when empty: the morning survey measured a room with no attenuation in it and no clients contending.

Fix, in order of value.

- Move the channel-8 radio to 1. That converts the worst interference in the room from ACI to CCI and costs nothing.
- Move as many clients as possible to 5 GHz, where there is enough spectrum to give all four APs non-overlapping channels. In a dense room this is the real fix.
- Reduce transmit power so the four cells are smaller and clients associate with the nearest AP instead of the loudest.
- Find the noise source. Nothing in a Wi-Fi scan explains a -73 dBm floor, so bring a spectrum analyser during a lecture.

And change the survey practice. A survey of an empty room measures a room that will never be used. Survey when occupied, or accept that the numbers are optimistic by 10 dB or more.

Lab 27.1 — A survey of somewhere you actually use *(Any laptop or phone with a Wi-Fi analyser)*

Platform note. This topic has no configuration on the blueprint, so this lab is measurement rather than building. Use any Wi-Fi analyser — built-in tools work: `netsh wlan show interfaces` on Windows, the option-click Wi-Fi menu on macOS, `iw dev wlan0 scan` on Linux.

Location. A room you can visit twice, ideally once empty and once busy.

Task 1. Record every visible SSID, its BSSIDs, band, channel and channel width. Draw the channel occupancy of 2.4 GHz as a diagram like the one in this chapter.

Task 2. Identify every case of adjacent-channel interference in what you found. For each, name the channel that should have been used instead.

Task 3. Record RSSI *and* noise for your own connection at five points: beside the AP, across the room, behind a wall, around a corner, and at the far edge of coverage. Calculate SNR at each.

Task 4. Plot RSSI against distance. Confirm the roughly 6 dB loss per doubling of distance, and identify where your measurements depart from it and why.

Task 5. Find one place where signal is strong and SNR is poor. Explain what that tells you.

- Task 6.** Identify the security protocol in use on each SSID you can see. Note any still offering WPA or WEP.
- Task 7.** Move between two APs on the same SSID while watching the BSSID. Record the RSSI at which your client chose to roam — and note that you did not choose it and the network did not either.
- Task 8. If you can:** repeat the RSSI and noise measurements with the room occupied. Compare, and quantify the absorption.
- Task 9. Extension.** Write the one-page recommendation you would give the building's network team, with the evidence for each point.

Checkpoint — Test Yourself

1. Why does the 2.4 GHz band have only three non-overlapping channels, and which are they?
2. Which is worse, co-channel or adjacent-channel interference, and why?
3. A client reports -62 dBm and cannot hold a call. What is the next number you ask for, and what would make this normal?
4. State the difference between WPA2-Personal and WPA2-Enterprise, and between WPA2 and WPA3.
5. Give two reasons a room might work when empty and fail when full.
6. What kind of interference is invisible to a Wi-Fi analyser, and what tool finds it?

Chapter Summary

- Wireless is half duplex, shared, and uses collision *avoidance*. The advertised rate is shared between everyone on the channel.
- SSID is the name, BSSID is the radio's MAC. The *client* decides when to roam.
- 2.4 GHz: three non-overlapping channels, 1, 6 and 11. 5 GHz: around 25, some DFS. 6 GHz: 59, newest clients only.
- Channel bonding doubles throughput, halves the channel count and raises the noise floor. Never at 2.4 GHz.
- $+3$ dB doubles power, $+10$ dB multiplies by ten. 0 dBm is 1 mW.
- **SNR decides whether a link works**, not RSSI. Aim for 25 dB for voice, 20 dB minimum for data.
- Radio waves are absorbed, reflected, refracted, diffracted and scattered; free-space loss costs about 6 dB per doubling of distance.
- WEP is broken, WPA deprecated, WPA2 uses AES-CCMP with a 4-way handshake, WPA3 uses SAE and resists offline attack. Personal shares a passphrase; Enterprise authenticates each user via RADIUS.
- Co-channel interference shares politely; adjacent-channel interference corrupts. A bad plan is worse than no plan.

Review Questions

1. (MCQ) Which set of 2.4 GHz channels does not overlap?
(a) 1, 5, 9 (b) 1, 6, 11 (c) 2, 7, 12 (d) 1, 4, 8
2. (MCQ) A client shows RSSI -68 dBm and a noise floor of -88 dBm. What is the SNR, and is it adequate for data?
(a) 20 dB, adequate (b) 20 dB, inadequate

- (c) 156 dB, adequate (d) -20 dB, inadequate
3. **(MCQ)** Which WPA3 mechanism replaces the WPA2 4-way handshake and prevents offline dictionary attack?
- (a) TKIP (b) CCMP (c) SAE (d) OWE
4. **(Short)** Explain why two APs on channel 6 perform better than two APs on channels 6 and 8.
5. **(Short)** An outdoor link works in dry weather and fails in heavy rain. Name the two RF effects responsible and explain each.
6. **(Applied)** You are given a floor with six APs, all on 2.4 GHz, on channels 1, 3, 6, 8, 11 and 11. Propose a corrected plan, justify each change, and state the one further recommendation that would help more than any channel change.
7. **(Applied)** A site reports that wireless drops for a few seconds at unpredictable times on 5 GHz only, and never on 2.4 GHz. Give the most likely cause, explain the mechanism, and state how you would confirm it.

Troubleshooting Client Connectivity

EXAM TOPICS **1.6**

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Read** a host's IP configuration on Windows, macOS and Linux, and recognise the four configurations that cannot work.
- **Apply** a reachability ladder that localises a fault in five commands.
- **Check** wireless security parameters on all three operating systems, including the modern default that breaks MAC-based authorisation.
- **Decide**, from the host alone, whether a fault is the client's, the access layer's, or the network's — before escalating it.

🔗 Before You Start

Chapter 4 and Chapter 5 for addressing. Chapter 20 for what an address means when it arrives. Chapter 21 for name resolution. Chapter 27 for RSSI, SNR and the security protocols this chapter checks the settings of.

📖 Key Terms in This Chapter

APIPA • default gateway • subnet mask • reachability ladder • `ipconfig` • `ip addr` • ARP cache • MAC randomisation • supplicant • EAP method

🎓 Three areas, three operating systems, named explicitly

Topic 1.6 is *troubleshoot wired and wireless client connectivity (IP configuration, network reachability, and wireless security parameters on Windows, MacOS, and Linux)*.

The three operating systems are named in the topic statement. That is a strong signal: you are expected to know the *equivalent command on each*, not just the one you happen to use. The table in Section 28.2 is the single most examinable page in this chapter, and it is worth memorising in all three columns.

Note also what is *not* here: no server-side configuration, no controller. Everything in this topic is done standing at the affected machine.

28.1 Start at the host

The instinct when a user cannot connect is to go and look at the switch. Resist it. Four commands at the host tell you whether there is any point.

💡 Why the order matters

Each step depends on every step above it. So the *first* failure is the fault, and everything below it is a symptom rather than a cause.

The commonest mistake in practice is starting at step 4 — “the website does not load” — and reasoning about DNS for twenty minutes on a machine that never had a valid address. Four commands, in order, cost ninety seconds and stop that happening.

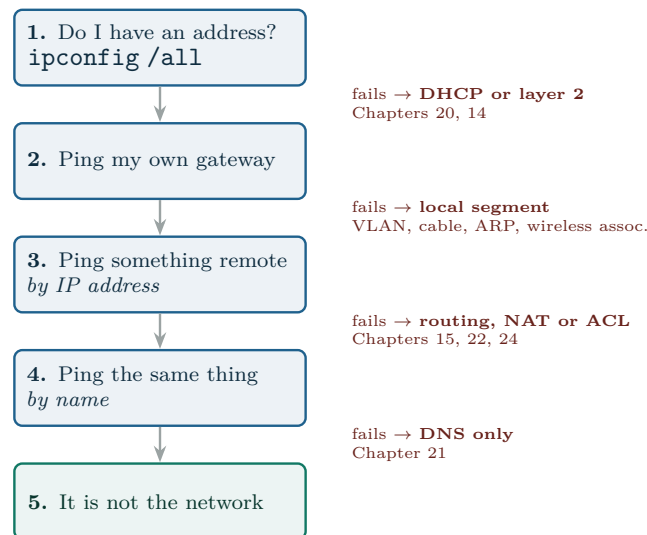


Figure 28.1: The reachability ladder. Each rung that passes eliminates everything below it; the first rung that fails names the area to investigate.

28.2 The same job on three operating systems

Task	Windows	macOS	Linux
Full IP configuration	ipconfig /all	ifconfig ipconfig getpacket en0	ip addr ip a
Routing table	route print	netstat -rn	ip route
ARP cache	arp -a	arp -a	ip neigh
Release and renew	ipconfig /release ipconfig /renew	ipconfig set en0 NONE ipconfig set en0 DHCP	dhclient -r dhclient
Name resolution	nslookup	dig scutil -dns	dig resolvectl status
Flush DNS cache	ipconfig /flushdns	killall -HUP mDNSResponder	resolvectl flush-caches
Path to a destination	tracert	traceroute	traceroute tracepath
Test a TCP port	Test-NetConnection -Port	nc -zv	nc -zv
Wireless status	netsh wlan show interfaces	option-click the Wi-Fi menu	iw dev wlan0 link nmcli dev wifi

⚠ Two differences worth knowing rather than guessing

ifconfig is deprecated on Linux and absent from many modern distributions. **ip** replaced it. On macOS **ifconfig** is still the normal tool. Same command name, opposite advice, depending on the platform in front of you.

macOS does not release and renew the way Windows does. Setting the interface to NONE and back to DHCP is the equivalent; there is no single “renew” verb. In the GUI it is the Renew DHCP Lease button in the TCP/IP pane.

28.3 Step 1: reading the IP configuration

>_ Windows — ipconfig /all

Wireless LAN adapter Wi-Fi:

```
Description . . . . . : Intel(R) Wi-Fi 6 AX201
Physical Address. . . . . : 8A-3C-D1-77-04-B2
DHCP Enabled. . . . . : Yes
IPv4 Address. . . . . : 192.168.10.34(Preferred)
Subnet Mask . . . . . : 255.255.255.0
Lease Obtained. . . . . : 27 August 2026 08:14:02
Lease Expires . . . . . : 27 August 2026 16:14:02
Default Gateway . . . . . : 192.168.10.1
DHCP Server . . . . . : 10.0.0.5
DNS Servers . . . . . : 192.168.20.10
                        192.168.20.11
```

>_ Linux — ip addr show wlan0

```
3: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 state UP
    link/ether 8a:3c:d1:77:04:b2 brd ff:ff:ff:ff:ff:ff
    inet 192.168.10.34/24 brd 192.168.10.255 scope global dynamic wlan0
        valid_lft 21847sec preferred_lft 21847sec
    inet6 2001:db8:acad:10:8a3c:d1ff:fe77:4b2/64 scope global dynamic
```

28.3.1 The four configurations that cannot work

What you see	What it means	Where to go next
169.254.x.x, or no address at all	The host asked for a lease and heard nothing	Layer 2 first: is the port in the right VLAN, is the cable good, did the wireless association actually complete? Then the relay (Chapter 20)
Address correct, gateway blank or wrong	The host can reach its own subnet and nothing else	The DHCP pool's default-router option, or a leftover static configuration
Address correct, mask wrong	The host misjudges which addresses are local. Some destinations work and others do not, unpredictably	The pool's network statement, or a static entry. This one is genuinely confusing to diagnose from the symptom
Address correct, no DNS servers	Everything works by IP address and nothing by name	The pool's dns-server option

💡 The wrong-mask fault, because it is worth recognising

A host configured 192.168.10.34/16 instead of /24 believes the whole of 192.168.0.0/16 is on its local segment. So for a destination in 192.168.20.0/24 it ARPs directly instead of sending to the gateway — gets no answer, and fails. For 8.8.8.8 it uses the gateway correctly, and works.

The symptom is therefore “the internet works but the file server does not”, which sounds like a server problem and is not. Check the mask whenever connectivity is selectively broken toward *internal* destinations.

28.4 Steps 2 to 4: the reachability ladder

>_ Windows — ping 192.168.10.1

```
Pinging 192.168.10.1 with 32 bytes of data:
Reply from 192.168.10.1: bytes=32 time=1ms TTL=255
Reply from 192.168.10.1: bytes=32 time=1ms TTL=255

Ping statistics for 192.168.10.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)
```

- **Gateway fails** → the problem is on this segment. Wrong VLAN, a bad cable, a wireless client associated but not authorised, or an ACL on the SVI. Check the ARP cache: if the gateway has no MAC entry, the host cannot even find it.
- **Gateway works, remote IP fails** → the host is fine. The fault is routing, NAT or a filter. Run a trace to find where it stops.
- **Remote IP works, remote name fails** → DNS, and only DNS. Go to Chapter 21 and stop looking at anything else.

>_ Linux — ip neigh

```
192.168.10.1 dev wlan0 lladdr 00:1a:2b:3c:4d:5e REACHABLE
192.168.10.22 dev wlan0 lladdr 00:50:79:66:68:22 STALE
192.168.10.99 dev wlan0 FAILED
```

FAILED against an address means the host ARPed for it and nothing replied — so that address is either absent or not on this segment after all. It is a fast way to confirm a wrong-mask fault.

⚠ A ping that fails does not always mean unreachable

Plenty of hosts and firewalls drop ICMP by policy. Before concluding that a server is down, test the port the application actually uses:

- Windows: `Test-NetConnection server.example.edu -Port 443`
- macOS and Linux: `nc -zv server.example.edu 443`

A successful TCP connection to port 443 on a host that ignores ping is a working server. Reporting it as down because it did not answer ICMP is a common and avoidable error.

28.5 Wireless security parameters

Wireless adds a step before the ladder even starts: the client must associate *and* authenticate. A client can be fully associated and still have no network access.

>_ Windows — netsh wlan show interfaces

```
Name                : Wi-Fi
State                : connected
SSID                 : CAMPUS-STAFF
BSSID                : 00:11:22:aa:bb:03
Network type        : Infrastructure
Radio type           : 802.11ax
Authentication       : WPA2-Enterprise
Cipher               : CCMP
Channel              : 44
Signal               : 78%
```

>_ Linux — iw dev wlan0 link

```
Connected to 00:11:22:aa:bb:03 (on wlan0)
    SSID: CAMPUS-STAFF
    freq: 5220
    signal: -58 dBm
```

```
rx bitrate: 780.0 MBit/s
tx bitrate: 585.0 MBit/s
```

💡 Windows reports signal as a percentage, and it is not one

That **Signal : 78%** is a rescaling of RSSI, not a physical quantity, and different drivers scale it differently. It also tells you nothing about the noise floor, which Chapter 27 established is the number that decides whether a link works.

Prefer a tool that reports dBm. On Linux `iw` gives it directly; on macOS the option-click menu shows RSSI and noise together; on Windows a third-party analyser is usually needed.

28.5.1 What actually goes wrong

Fault	Symptom	Check
Wrong pre-shared key	Repeated association attempts, never connects	Retype it. On WPA3 the failure is immediate rather than after a timeout
Wrong EAP method or identity	Associates, then drops. Prompts for credentials repeatedly	The saved 802.1X profile: method (PEAP, EAP-TLS), identity, and whether it uses machine or user credentials
Untrusted or expired server certificate	Connects on some devices and not others; a security warning	The RADIUS server's certificate chain, and whether the client trusts the issuer
Stale saved profile	Worked yesterday, fails today, works for colleagues	Forget the network and re-add it. This genuinely is the fix more often than it deserves to be
Band steering or 802.11r	Drops when moving between areas	Whether the client supports fast roaming; try locking to one band to confirm
MAC randomisation	Associates and authenticates, then gets no address or the wrong VLAN	See below — this is the modern one

⚠️ MAC randomisation, and why it breaks things that used to work

Windows, macOS, iOS, Android and most Linux distributions now generate a **random MAC address per network by default**, and some re-randomise periodically. It is a privacy feature and it is on unless somebody turned it off.

It breaks anything that identifies a device by MAC address:

- MAC-based authorisation or filtering — the device is unknown.
- DHCP reservations — the printer that always had .50 now does not.
- Per-device VLAN assignment.
- Any audit trail that assumed a MAC identified a machine.

The diagnostic tell: the MAC in `ipconfig /all` does not match the MAC on the device's label or in the network's records, and the second octet's low-order bits mark it as locally administered.

Where to turn it off — and note the topic names all three:

- **Windows** — Settings, Network, Wi-Fi, the network’s properties: “Random hardware addresses”.
- **macOS and iOS** — the network’s info panel: “Private Wi-Fi Address”.
- **Linux (NetworkManager)** — `wifi.cloned-mac-address` set to `permanent` on the connection profile.

The better long-term answer is to stop authenticating by MAC address at all and use 802.1X, which identifies the *user*. But the exam and the helpdesk both need the short answer.

! Common Pitfalls

- **Starting at DNS.** Run the ladder in order.
- **Concluding “down” from a failed ping.** Test the actual port.
- **Trusting a signal percentage.** Get dBm, and get the noise floor.
- **Ignoring the subnet mask** when connectivity is selectively broken toward internal hosts.
- **Forgetting that `ifconfig` is deprecated on Linux** and normal on macOS.
- **Assuming a MAC address identifies a device.** It has not been safe to assume that since roughly 2020.
- **Not checking whether it is one host or many.** One host is the host; a whole VLAN is the network. Ask before you start.
- **Fixing it without recording what it was.** A configuration you overwrote is evidence you destroyed.

🔧 Break & Fix — One laptop associates to the staff wireless, authenticates, and gets no address. Everyone else is fine.

A user’s laptop connects to **CAMPUS-STAFF**, shows as connected, and has 169.254.x.x. The same laptop works on the wired dock. Other laptops in the same room work on the same SSID. The wireless team confirms the client is associated and that 802.1X authentication succeeded.

>_ Windows — `ipconfig /all`

Wireless LAN adapter Wi-Fi:

```
Physical Address. . . . . : 8A-3C-D1-77-04-B2
DHCP Enabled. . . . . : Yes
Autoconfiguration IPv4 Address. . : 169.254.88.14(Preferred)
Subnet Mask . . . . . : 255.255.0.0
Default Gateway . . . . . :
```

🔍 DSW1# `show ip dhcp snooping binding | include 8a3c`

Diagnosis. Work through what has already been ruled out, because it is most of the problem.

Association and 802.1X both succeeded, so the passphrase, the EAP method, the certificate and the RSSI are all fine — the radio link is working and the user is authorised. The wired dock working proves the laptop’s TCP/IP stack and the user’s credentials are fine. Other laptops working on the same SSID in the same room proves the AP, the WLAN and the DHCP scope are fine.

That leaves something specific to *this device on wireless*. The MAC address is the clue: 8A-3C-D1-77-04-B2. The first octet, 8A, is 10001010 in binary — and the second-lowest bit is set, which marks the address as **locally administered** rather than assigned by a manufacturer. This is a randomised MAC.

The site assigns the staff VLAN by MAC address for devices in its asset register. The randomised address is not in the register, so the client is placed in a quarantine VLAN with no DHCP scope. Association succeeds, authentication succeeds, and no address is ever offered.

The confirming detail is the empty snooping binding query: the switch has no record of this MAC leasing anything, which is consistent with no offer having been made rather than with an offer being lost.

Immediate fix. Turn off random hardware addresses for this network in Windows Settings, then disconnect and reconnect. The laptop presents its burned-in address, matches the register, lands in the staff VLAN and leases normally.

The real fix is at the other end. MAC-based VLAN assignment stopped being viable when every major operating system began randomising by default. The site already runs 802.1X — authentication succeeded — so it should assign the VLAN from the RADIUS response based on the authenticated *user*, not from a MAC table. That works regardless of what address the client presents, and it is the answer to give when this is reported for the fifth time.

Lab 28.1 — The same fault on three operating systems (*Any three machines, or virtual machines*)

Platform note. This lab needs real operating systems rather than a network simulator, because the whole topic is about host-side tools. Virtual machines are fine. If you have only one machine, run Linux and Windows as VMs and use the host as the third.

- Task 1.** On each of Windows, macOS and Linux, produce the full IP configuration, the routing table and the ARP cache. Build your own version of the command table in Section 28.2 from what actually worked — including any command that was missing on your distribution.
- Task 2.** Run the reachability ladder on each machine while everything works, and record the normal output of each step. You cannot recognise abnormal output without this.
- Task 3. Break 1.** Set a static address with the correct address and a *wrong gateway*. Predict which ladder step fails, then test.
- Task 4. Break 2.** Set the mask to /16 on a /24 network. Find two destinations that behave differently and explain the difference using the ARP cache.
- Task 5. Break 3.** Remove the DNS servers. Confirm that steps 1 to 3 pass and step 4 fails, and that this is the *only* configuration that produces that pattern.
- Task 6. Break 4.** Disconnect the network and observe APIPA. Note the address and mask each operating system chooses.
- Task 7.** Release and renew the lease on all three, using the correct command for each. Note that macOS has no direct equivalent.
- Task 8. Wireless.** Report SSID, BSSID, channel, authentication method and signal on each operating system. Compare what each tool tells you and what it omits.
- Task 9. MAC randomisation.** Find the setting on each operating system. Record the MAC with it on and off, and confirm the locally-administered bit in the randomised one.
- Task 10. Extension.** Write the four-step diagnostic script you would give a first-line helpdesk, in the form of exact commands and what each result means. Keep it to one page.

🔍 Checkpoint — Test Yourself

1. Give the four rungs of the reachability ladder and the area each failure points to.
2. Name the command to show the full IP configuration on each of the three operating systems.
3. A host has a correct address and a blank default gateway. What works and what does not?
4. Why does a wrong subnet mask break some internal destinations and not others?
5. A device authenticates to wireless successfully and receives no address. Give two possible causes and how you would tell them apart.
6. How can you tell a randomised MAC address from a real one by inspection?

📋 Chapter Summary

- Diagnose at the host first. Address, gateway, remote by IP, remote by name — in that order. The first failure is the fault.
- `ipconfig /all` on Windows, `ifconfig` on macOS, `ip addr` on Linux. `ifconfig` is deprecated on Linux and normal on macOS.
- Four broken configurations: no address (169.254), no gateway, wrong mask, no DNS. Each has a distinct signature in the ladder.
- A wrong mask breaks internal destinations selectively, which is why it looks like a server fault.
- A failed ping is not proof of a dead host. Test the port with `Test-NetConnection` or `nc -zv`.
- Wireless clients can be associated and authenticated and still have no access. Check the security parameters separately from the radio.
- Windows reports signal as a meaningless percentage. Get dBm and the noise floor.
- MAC randomisation is on by default everywhere and breaks MAC-based authorisation, DHCP reservations and VLAN assignment. Identify it by the locally-administered bit.

🔧 Review Questions

1. (MCQ) A host pings its gateway successfully, pings 8.8.8.8 successfully, and cannot resolve any name. What is at fault?
(a) The default gateway (b) The subnet mask
(c) DNS configuration (d) The DHCP relay
2. (MCQ) Which command shows the ARP cache on Linux?
(a) `arp -a` (b) `ip neigh` (c) `netstat -rn` (d) `ip route`
3. (Short) Explain why a host with a /16 mask on a /24 network can reach the internet but not a server two VLANs away.
4. (Short) A user reports that a server is down because it does not respond to ping. Give the command you would run instead, on each of the three operating systems, and explain what a success would prove.
5. (Applied) A single laptop associates to an SSID, authenticates, and receives 169.254.x.x, while every other device on the same SSID works. List, in order, the checks you would make at the host, and state what each one rules out.

6. **(Applied)** A site has assigned VLANs by MAC address for ten years. Explain in three sentences why this has begun failing, and recommend a replacement with a justification the site's manager would accept.

Hypervisors, Virtual Machines and Containers

EXAM TOPICS **1.2**

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Distinguish** a type 1 from a type 2 hypervisor and say where each belongs.
- **Describe** what a virtual machine contains and how a virtual switch connects it to the physical network.
- **Compare** virtual machines with containers on isolation, start time, size and what they share.
- **Configure** the physical switch port a hypervisor host attaches to — which is the part of this topic that is actually your job.

🔗 Before You Start

Chapter 8 and Chapter 10 — a hypervisor uplink is a trunk, and this chapter will not make sense without knowing what one is. Chapter 25 for port security, which is the wrong tool on such a port. Chapter 3 for MAC learning.

📖 Key Terms in This Chapter

hypervisor • type 1 • type 2 • virtual machine • guest OS • vNIC • virtual switch • port group • container • image • kernel • orchestration • east-west traffic • live migration

🎓 Describe the role and function

Topic 1.2 is *describe the role and function of hypervisors, virtual machines, and containers*. No configuration of any hypervisor is required, and no vendor product is named. But it sits in domain 1, *Network Infrastructure and Connectivity*, and not by accident. A CCNA is not asked to administer a hypervisor; a CCNA is asked to **connect one correctly**, and to recognise the several things a virtualised server does to a network that a physical server never did. Section 29.4 is where those live, and it is the part of this chapter you will use at work next week.

29.1 Why any of this exists

A physical server running one application typically uses 5 to 15% of its capacity. The other 85% is bought, powered, cooled and wasted. Virtualisation puts many logical servers on one physical one, each believing it has the machine to itself.

📄 Hypervisor

Software that presents virtual hardware — CPU, memory, disk, network interface — to several guest operating systems at once, and schedules the real hardware between them. Also called a **virtual machine monitor**.

29.1.1 Type 1 and type 2

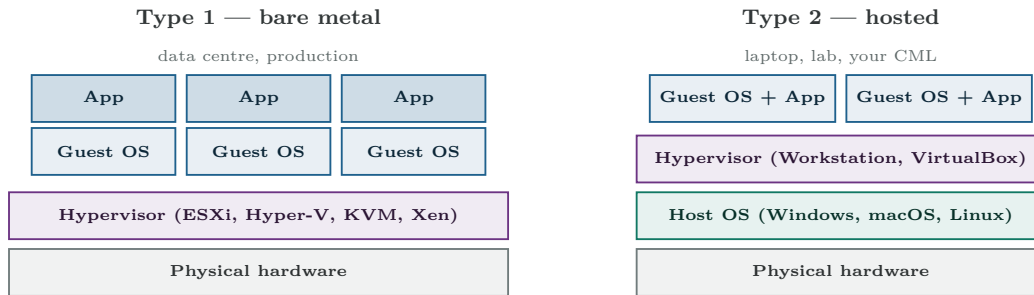


Figure 29.1: The difference is one layer. A type 1 hypervisor *is* the operating system; a type 2 hypervisor is an application running on one.

	Type 1 (bare metal, native)	Type 2 (hosted)
Runs on	The hardware directly	On top of a normal host operating system
Performance	Higher — no host OS in the path	Lower — every operation passes through the host OS
Examples	VMware ESXi, Microsoft Hyper-V, KVM, Xen	VMware Workstation and Fusion, Oracle VirtualBox, Parallels
Used for	Production servers, data centres, cloud	Development, testing, labs, running one OS inside another
Boots into	The hypervisor itself	The host OS; the hypervisor is launched afterwards

💡 You have almost certainly used both

The CML or GNS3 you run the labs in this book on is a type 2 hypervisor, or runs inside one. The cloud instances your university's systems run on are type 1.

If an exam question describes a hypervisor installed *onto* a server with no other operating system, it is type 1. If it describes one installed *into* Windows or macOS, it is type 2. That is the whole distinction.

29.2 What a virtual machine is

A VM is a set of files — a configuration file describing its virtual hardware, and one or more virtual disk files — plus the state the hypervisor keeps while it runs. It contains a complete guest operating system with its own kernel, drivers, and boot process.

Because it is files, it can be copied, snapshotted, moved between hosts while running (**live migration**, or vMotion), and restored. Those conveniences are also what makes it a networking problem, which is Section 29.4.

29.2.1 The virtual switch

Every VM has one or more **vNICs**, each with its own MAC address. Those vNICs plug into a **virtual switch** inside the hypervisor, and the virtual switch has one or more physical NICs as uplinks to the real network.

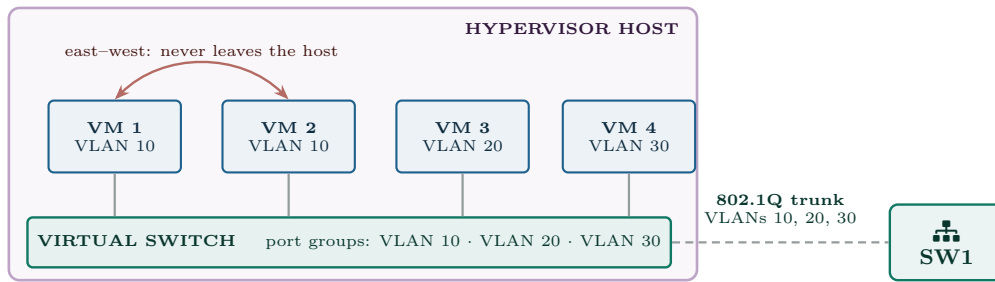


Figure 29.2: VM 1 talking to VM 2 never reaches SW1. Everything else does, over a trunk — a single physical port carrying three VLANs.

Port group

A named policy on the virtual switch — a VLAN ID, security settings and traffic shaping — that VMs are attached to. It is the virtual equivalent of deciding which VLAN a switch port belongs to, and the VLAN ID configured on it must match one the physical switch actually trunks.

29.3 Containers

A container does not virtualise the hardware. It virtualises the *operating system*: several isolated user-space environments share one kernel.

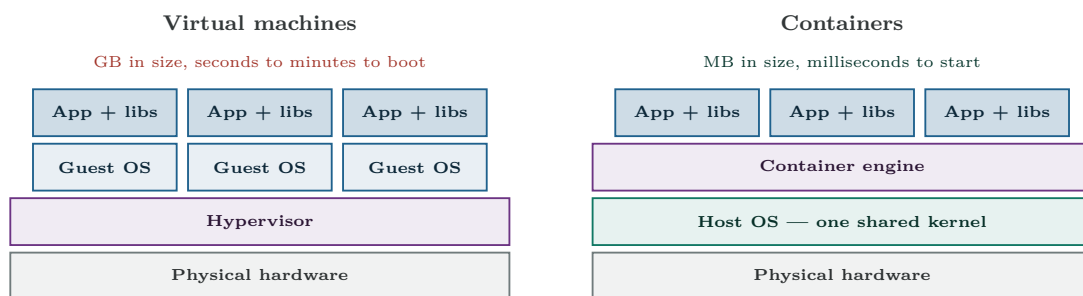


Figure 29.3: One layer removed. Containers share the host's kernel, which is why they are small and fast — and why the isolation between them is weaker.

	Virtual machine	Container
Contains	A full guest OS with its own kernel	An application and its libraries only
Shares	Nothing above the hypervisor	The host's kernel
Size	Gigabytes	Megabytes
Start time	Seconds to minutes — it boots	Milliseconds — there is nothing to boot
Isolation	Strong. A compromised guest is still inside its VM	Weaker. A kernel vulnerability is shared by every container on the host
Guest OS choice	Any, including a different family from the host	Must be compatible with the host kernel
Typical use	Consolidating servers; running different operating systems	Packaging and scaling one application; microservices

	Virtual machine	Container
Managed by	The hypervisor, and tools such as vCenter	A container engine, and an orchestrator such as Kubernetes

💡 They are not competitors

The common real deployment is containers *inside* virtual machines: the VM provides hard isolation and the ability to move workloads between hosts, and the container provides fast, repeatable application packaging inside it. Every major cloud provider runs some version of this.

An exam question asking you to choose between them is asking about the properties in the table — kernel sharing, size, start time, isolation — not about which is better.

📖 Orchestration

Running one container by hand is easy. Running four hundred across thirty hosts, restarting the ones that fail, scaling with demand and routing traffic to whichever are healthy is not. An **orchestrator** — Kubernetes is the dominant one — does that scheduling, health-checking and service discovery.

For this exam, know what the word means and what problem it solves.

29.4 What this does to your network

This is the section that matters to a network engineer, and it is where the blueprint's placement of this topic in domain 1 makes sense.

Consequence	What it means for you
The uplink is a trunk	One physical port carries every VLAN the host's VMs live in. Configuring it as an access port strands every VM outside that one VLAN
Many MACs on one port	Dozens, changing as VMs start and stop. Port security on that port is simply the wrong tool
MACs move without anyone touching a cable	Live migration relocates a running VM to another host, and its MAC appears on a different switch port seconds later. This looks exactly like a loop
East–west traffic is invisible	Two VMs on the same host and VLAN never reach the physical switch. You cannot see it, filter it with a physical ACL, or capture it from the network
The server team owns a switch now	The virtual switch's VLAN configuration is set by whoever administers the hypervisor. A mismatch between their port group and your trunk is a shared fault with two owners
Nothing has a fixed location	Documentation naming a physical port for a service goes stale immediately

>_ Configure — the physical switch port a hypervisor host attaches to

```
interface GigabitEthernet1/0/24
description ESXi host 3 -- vmnic0
switchport trunk encapsulation dot1q
switchport mode trunk
```

```

switchport trunk allowed vlan 10,20,30
switchport nonegotiate
!
! No port security: a hypervisor presents many MAC addresses and they
! change as machines start, stop and migrate.
!
! PortFast trunk, because the far side is a virtual switch that does
! not run spanning tree and will never send a BPDU.
spanning-tree portfast trunk
!
! BPDU guard is still correct: nothing on this port should ever send
! a BPDU, so one arriving means something is wrong.
spanning-tree bpduguard enable

```

⚠ The virtual switch does not run spanning tree

A vSwitch is not a learning switch in the ordinary sense: it does not forward traffic between its uplinks, so it cannot create a loop, and it does not participate in STP.

Two consequences. `spanning-tree portfast trunk` is appropriate on the physical port, because there is no risk of a loop and waiting 30 seconds on every host reboot is pure cost. And **BPDU guard should still be enabled** — precisely because a correctly configured vSwitch will never send a BPDU, one arriving means somebody has attached something else.

! Common Pitfalls

- **Access port on a hypervisor uplink.** Everything in one VLAN works, everything else is stranded, and the fault looks like a VM problem.
- **Port security on a hypervisor uplink.** Err-disables the port and takes down every VM on the host.
- **Reading migration MAC moves as a loop.** A MAC flap between two *access-layer uplinks* at the moment a VM migrated is expected behaviour.
- **Forgetting to add a VLAN to allowed** when the server team adds a port group. Their side is right, yours is not, and neither team sees both halves.
- **Expecting to capture east–west traffic** on the physical switch. It never gets there.
- **Assuming containers are just small VMs.** The shared kernel is the whole difference, and it is where the exam question will be.
- **Confusing type 1 with type 2.** Type 1 replaces the OS; type 2 runs on one.

🔧 **Break & Fix** — A new hypervisor host is racked. VMs in VLAN 10 work; VMs in VLANs 20 and 30 are dead.

The server team reports that of twelve VMs on the new host, the four in VLAN 10 have full connectivity and the eight in VLANs 20 and 30 cannot reach anything, including their own default gateways. The vSwitch configuration is identical to the three existing hosts, which all work. Later in the morning the port err-disables entirely and all twelve VMs drop.

🔍 SW1# show interfaces GigabitEthernet1/0/24 switchport

```

Name: Gi1/0/24
Switchport: Enabled
Administrative Mode: static access
Operational Mode: static access
Access Mode VLAN: 10 (STAFF)
Trunking Native Mode VLAN: 1 (default)

```

Q SW1# show interfaces status err-disabled

Port	Name	Status	Reason	Err-disabled Vlans
Gi1/0/24	ESXi host 4	err-disabled	psecure-violation	

Diagnosis. Two faults on one port, and the first explains the pattern while the second explains the timing.

The port is an access port in VLAN 10. A hypervisor uplink must be a trunk. VMs in the port group tagged VLAN 10 happen to work because the vSwitch sends their frames untagged and the access port puts them in VLAN 10 — the right answer by accident. Frames from VLANs 20 and 30 arrive tagged, and an access port discards tagged frames for other VLANs. Eight VMs are stranded.

The port also has port security. A hypervisor presents one MAC per vNIC plus the host's own management address — thirteen or more on this host. As the remaining VMs powered on through the morning, the maximum was exceeded and the port err-disabled, which is why the working four then failed too.

The tell that this is a switch-port fault and not a hypervisor fault is that the vSwitch configuration matches three working hosts. When one member of an identical set behaves differently, look at what is *not* identical — which here is the physical port it was patched into.

Fix.

>_ Configure — correcting the uplink

```
interface GigabitEthernet1/0/24
description ESXi host 4 -- vmnic0
no switchport port-security
no switchport access vlan 10
switchport trunk encapsulation dot1q
switchport mode trunk
switchport trunk allowed vlan 10,20,30
switchport nonegotiate
spanning-tree portfast trunk
spanning-tree bpduguard enable
!
shutdown
no shutdown
```

The `shutdown` then `no shutdown` is required: removing the port security configuration does not clear an existing err-disabled state.

The process lesson. This port was almost certainly configured from the site's standard access-port template, which is correct for a desk and wrong for a server. Sites that hit this repeatedly keep a separate documented template for hypervisor uplinks — trunk, explicit allowed list, no port security, `portfast trunk`, BPDU guard — and name it in the change request.

Lab 29.1 — Connecting a hypervisor properly (*VirtualBox, VMware Workstation, or CML*)

Platform note. You do not need a data centre. Any type 2 hypervisor on your own machine demonstrates every concept here; the switch-side tasks can be done in Packet Tracer or CML against a simulated host.

Task 1. Identify the hypervisor you already use for the labs in this book and classify it as type 1 or type 2. Justify the classification in one sentence.

Task 2. Create two VMs. Record the MAC address of each vNIC, and confirm they differ from the host's physical NIC.

- Task 3.** Put both VMs on a *host-only* or internal network. Confirm they can reach each other and that this traffic never appears on your physical network — capture on the physical interface to prove it. This is east–west traffic.
- Task 4.** Switch both to bridged networking. Confirm they now appear on the physical network with their own MAC addresses, and find them in your switch’s MAC address table.
- Task 5. Switch side.** Configure the port your host connects to as an access port in one VLAN. Put one VM in that VLAN and one in another. Reproduce the Break & Fix symptom, then convert the port to a trunk.
- Task 6.** Apply `switchport port-security maximum 1` to that trunk. Power on the second VM and observe. Recover the port.
- Task 7. Containers.** Install a container engine and run two containers. Compare their start time and image size against your VMs’ boot time and disk size. Record actual numbers.
- Task 8.** Inspect the container network. Find the bridge, the addresses assigned, and confirm that outbound traffic is being NATted by the host. Relate this to Chapter 22.
- Task 9.** Run `uname -r` inside two containers and on the host. Explain the result, and state what it implies for isolation.
- Task 10. Extension.** Write the switch-port template your site should use for hypervisor uplinks, with a one-line justification per command, and say what is different from your standard access-port template.

? Checkpoint — Test Yourself

1. What is the single structural difference between a type 1 and a type 2 hypervisor?
2. What does a container share with its host that a virtual machine does not, and what are the two main consequences?
3. Why must a hypervisor uplink be a trunk?
4. Why is port security inappropriate on that port?
5. What is east–west traffic, and why can you not capture it on the physical switch?
6. A MAC address moves between two switch ports with no cable change. Give a legitimate explanation.

✓ Chapter Summary

- A hypervisor presents virtual hardware to several guest operating systems. Type 1 runs on the bare metal; type 2 runs on a host OS.
- A VM is files: a configuration and virtual disks, containing a complete guest OS with its own kernel. It can be copied, snapshotted and migrated while running.
- Each vNIC has its own MAC and attaches to a virtual switch, whose port groups carry VLAN IDs that must match the physical trunk.
- Containers share the host kernel: megabytes rather than gigabytes, milliseconds rather than seconds, weaker isolation. Orchestrators such as Kubernetes schedule them at scale.
- The two are commonly combined — containers inside VMs.
- The physical port to a hypervisor is a **trunk** with an explicit allowed list, **no port security**, `portfast` **trunk** and BPDU guard.
- Live migration moves MAC addresses between ports legitimately. East–west traffic never reaches the physical network at all.

 Review Questions

1. **(MCQ)** Which describes a type 1 hypervisor?
 - (a) Runs as an application on Windows or macOS
 - (b) Runs directly on the physical hardware with no host OS
 - (c) Shares the host operating system's kernel with its guests
 - (d) Requires an orchestrator such as Kubernetes
2. **(MCQ)** What do containers share that virtual machines do not?
 - (a) The physical NIC
 - (b) The host's kernel
 - (c) The hypervisor
 - (d) A single MAC address
3. **(Short)** Explain why applying port security to a hypervisor uplink causes an outage, and what you should do instead.
4. **(Short)** A monitoring team asks why they cannot see traffic between two application servers that are known to communicate constantly. Give the explanation and one thing that would let them see it.
5. **(Applied)** A hypervisor host has VMs in VLANs 10, 20 and 40. Write the complete configuration for the switch port it attaches to, justify each line, and state which single line would strand two-thirds of the VMs if omitted.
6. **(Applied)** Compare virtual machines and containers across isolation, start time, size and guest OS flexibility, and recommend one for each of: running a legacy Windows application, and scaling a web front end from 3 to 300 instances during registration week.

Management Approaches, SNMP and Syslog

EXAM TOPICS 5.3 5.4 5.6

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Describe** the five network management approaches the blueprint names, and tell controller-based from automation-based.
- **Describe** how SNMP works, what its three versions differ on, and when to use a trap rather than an inform.
- **Interpret** any IOS syslog message: facility, severity, mnemonic and description.
- **Recite** the eight severity levels and explain what a logging threshold hides.

🔗 Before You Start

Chapter 23 for secure management access and for why credentials in clear are unacceptable. Chapter 6 for UDP. Chapter 14 — you have already read syslog output there; this chapter makes it systematic.

📖 Key Terms in This Chapter

device-based • controller-based • cloud-based • automation-based • infrastructure as code • SNMP manager • agent • MIB • OID • community string • trap • inform • facility • severity • mnemonic

🎓 Three topics, three different verbs

5.3 *describe network management approaches (device-based, cloud-based, controller-based, automation-based, and infrastructure as code)* — five named approaches, so expect a matching or scenario question.

5.4 *describe the function of SNMP in network operations* — function, not configuration. What it does and how, not how to type it.

5.6 *interpret syslog message content, severity levels, and facilities* — **interpret**. You will be shown a log line and asked what it means. Section 30.3 is built for exactly that and is the highest-value part of this chapter.

30.1 Five ways to manage a network

Approach	How it works	Trade-off
Device-based	You connect to each device individually by console or SSH and configure it	Complete control and no dependencies. Does not scale, and every device is a separate opportunity to make a different mistake

Approach	How it works	Trade-off
Controller-based	A controller holds the intended state and programs the devices to match (Catalyst Center, SD-WAN, SD-Access)	Consistency and central policy. The controller becomes critical infrastructure, and you manage the network its way
Cloud-based	The management plane is a service the vendor runs; devices call home to it (Meraki)	Nothing to host, upgrade or back up. You depend on the vendor's availability and on internet reachability from every site
Automation-based	Scripts and tools drive the devices' existing interfaces — Ansible, Python (Chapter 31)	Repeatable, and adds no new control plane. The devices are unchanged, so it works with what you already own
Infrastructure as code	The desired configuration lives in version-controlled files and is applied by a pipeline	Reviewable, testable, revertible; the repository is the source of truth. Requires discipline and tooling a small team may not have

⚠ Controller-based against automation-based: the distinction that gets tested

They sound similar and are structurally different.

Controller-based inserts a *new system* between you and the network. The controller holds the intended state, and it is in the path permanently. Take it away and you lose central management.

Automation-based adds *tooling* outside the network. Ansible logs in over SSH and types commands faster and more consistently than you can. Take it away and every device still works exactly as before, because nothing about them changed.

One-line test: **does the network still have a management system when the tool is switched off?** Controller, yes — and you have lost it. Automation, no — and you have lost nothing but convenience.

Infrastructure as code is a step beyond automation: not just *running* the change, but keeping the intended state in a repository that is reviewed, tested and versioned.

30.2 SNMP

📖 SNMP

The Simple Network Management Protocol. A **manager** polls **agents** running on network devices for values held in a **MIB** — a tree of variables, each identified by a numeric **OID**. Agents listen on UDP 161; the manager receives unsolicited notifications on UDP 162.

30.2.1 The operations

- **Get** — fetch one value. **GetNext** — fetch the next one in the tree, which is how a manager walks a whole table.
- **GetBulk** — fetch many at once. Added in v2c, and the reason polling a large switch is not painfully slow.
- **Set** — *write* a value, changing the device's configuration or state. Powerful, and the reason read-write access needs treating as an administrative credential.

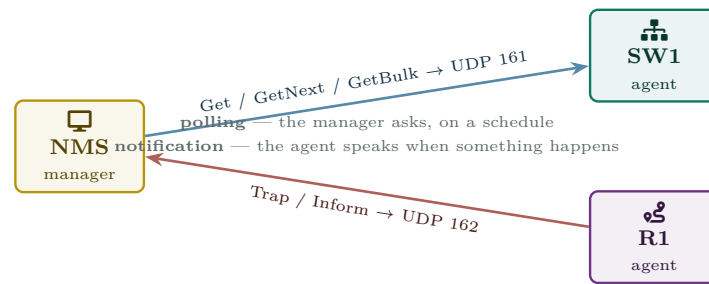


Figure 30.1: SNMP works in both directions: the manager polls on port 161, and agents report events unprompted to port 162.

- **Trap** — the agent notifies the manager of an event. Fire and forget: sent once, over UDP, never acknowledged.
- **Inform** — the same, but *acknowledged* and retransmitted until it is. Added in v2c.

💡 Trap or inform

A trap that is lost is lost silently, and the event it described never reaches anyone. An inform costs a round trip and some agent memory, and survives a dropped packet.

Use informs for anything you would be sorry to miss — a device losing power redundancy, a security event. Use traps for high-volume, low-consequence notifications where losing one does not matter. This is a fair exam question and a real design decision.

30.2.2 The three versions

Version	Authentication	Encryption	Verdict
v1	Community string, sent in clear	None	Obsolete. No GetBulk, no informs
v2c	Community string, sent in clear	None	Still widely deployed. Adds GetBulk and informs. The community string is a password on the wire in plain text
v3	Username with HMAC-SHA authentication	AES	The only version to use. Three levels: noAuthNoPriv, authNoPriv, authPriv

⚠️ “v2c is secure because the community string is not public”

It is not. The community string travels in clear text in every single polling packet, hundreds of times an hour, and anyone who can capture one packet has it. Renaming it from `public` to something obscure changes nothing.

And a read-write community string is equivalent to an enable password: an attacker holding one can reconfigure the device, and on many platforms can pull the entire running configuration. If SNMP must be v2c for a legacy tool, make it read-only, restrict it with an ACL to the management station’s address, and plan its removal.

➤ Configure — SNMPv3, read-only, for a monitoring station

```
snmp-server view OPSVIEW iso included
snmp-server group OPSGROUP v3 priv read OPSVIEW
!
! auth protects integrity and identity; priv adds encryption.
! "priv" implies "auth" -- you cannot encrypt without authenticating.
snmp-server user nms OPSGROUP v3 auth sha AuthPassPhrase priv aes 128 PrivPassPhrase
!
snmp-server host 10.0.0.45 version 3 priv nms
snmp-server enable traps
!
! These two are free and save an hour when somebody has to find the
! device in a building.
snmp-server location Campus Block C, Rack 4, RU 22
snmp-server contact netops@example.edu
```

💡 What SNMP is actually used for

Three jobs, and the exam's phrase "in network operations" points at them.

- **Performance monitoring** — polling interface counters every few minutes to produce the traffic graphs everybody recognises.
- **Fault notification** — traps and informs for events that cannot wait for the next poll.
- **Inventory** — serial numbers, models, software versions and uptime collected across the estate.

What it is *not* good at is configuration. Set exists, but nobody manages configuration by SNMP; that is what Chapters 31 and the automation approaches are for.

30.3 Syslog, and how to read a message

Syslog is where a device tells you what it did. Reading it fluently is the single most transferable skill in this chapter.

30.3.1 Anatomy of an IOS message

severity
↓
 Aug 27 09:14:22.331: %LINEPROTO-5-UPDOWN:
 Line protocol on Interface GigabitEthernet0/1, changed state to down
description — the part in English

timestamp facility mnemonic

Figure 30.2: Every IOS syslog message has the same four parts after the timestamp. The severity is the single digit between the two hyphens.

⚠ The severity is the digit in the middle, and it is not the number of anything else

In %LINEPROTO-5-UPDOWN the 5 is the severity level. In %LINK-3-UPDOWN the 3 is the severity level.

Those two messages describe the same interface going down and carry different severities because they concern different layers. Seeing both together is normal and tells you the physical link dropped, which took the line protocol with it. A %LINEPROTO-5-UPDOWN *without* a matching %LINK-3-UPDOWN means layer 1 stayed up and something above it failed — keepalives, an encapsulation mismatch, or a shutdown of a subinterface.

30.3.2 The eight severity levels

Level	Name	Meaning	Typical IOS message
0	Emergency	System unusable	Imminent thermal shutdown
1	Alert	Immediate action required	Redundant power supply failed
2	Critical	Critical condition	%SPANTREE-2-ROOTGUARD_BLOCK
3	Error	Error condition	%LINK-3-UPDOWN — interface down
4	Warning	Warning condition	%PM-4-ERR_DISABLE — port err-disabled
5	Notification	Normal but significant	%SYS-5-CONFIG_I — somebody saved a change
6	Informational	Informational message	%SEC-6-IPACCESSLOGP — an ACL matched
7	Debugging	Debug output	Anything from a debug command

💡 Remembering the order

Every Awesome Cisco Engineer Will Need Ice-cream Daily — Emergency, Alert, Critical, Error, Warning, Notification, Informational, Debugging.

Two facts that follow and are examined: **0 is the most severe and 7 the least**, which is the opposite of most people's instinct. And configuring a logging destination at level *n* sends everything from 0 *through* *n* — so logging console warnings (level 4) also delivers levels 0 to 3, and silently discards 5, 6 and 7.

30.3.3 Configuring logging

➤ Configure — a sensible logging setup

```

! Without this every message is stamped only with uptime, which is
! useless for correlating across devices. Do this first, everywhere.
service timestamps log datetime msec localtime show-timezone
service timestamps debug datetime msec localtime show-timezone
!
! Keep a generous local buffer at informational, so the evidence is
! on the device even if the syslog server was unreachable.
logging buffered 65536 informational
!
! Console logging on a busy device makes the CLI unusable. Turn it
! down; the buffer and the server still have everything.
logging console warnings
logging monitor informational
!
! The central server. "trap" here means the syslog severity
! threshold, NOT an SNMP trap -- an unfortunate collision of terms.
logging host 10.0.0.45
logging trap notifications
logging source-interface Loopback0
!
line con 0
  logging synchronous
line vty 0 15
  logging synchronous

```

⚠ logging trap has nothing to do with SNMP traps

It sets the *severity threshold for messages sent to a syslog server*. **logging trap notifications** means “send levels 0 to 5 to the host”. The word is a historical accident and it confuses people every year.

logging synchronous is unrelated to both: it stops a log message printed mid-command from scrambling what you were typing. It changes nothing about what is logged and it will save your temper.

Q SW1# show logging | include Console|Buffer|Trap

```
Console logging: level warnings, 2201 messages logged
Monitor logging: level informational, 0 messages logged
Buffer logging: level informational, 88214 messages logged
Trap logging: level notifications, 41022 message lines logged
```

! Common Pitfalls

- **Reading severity backwards.** 0 is worst, 7 is least important.
- **Setting a threshold too high and concluding nothing is wrong.** The message may exist and be filtered. Check the levels before trusting an empty log.
- **No service timestamps.** Messages stamped with uptime cannot be correlated across devices or with a user’s report.
- **Confusing logging trap with SNMP traps.**
- **Leaving console logging at debugging on a production device.** The CLI becomes unusable exactly when you need it.
- **SNMP v2c with a read-write community string.** That is an enable password sent in clear thousands of times a day.
- **No logging source-interface.** Messages arrive from whichever interface routing chose, and the server cannot reliably attribute them.
- **Relying on traps for events that matter.** Use informs.
- **Calling Ansible “controller-based”.** It adds no control plane.

🔧 Break & Fix — Ports keep going down overnight. The log “shows nothing”.

An access switch drops several user ports most nights. By morning they are back. The site’s monitoring shows the interfaces bouncing, but the network team reports that the device’s own log contains no explanation.

Q SW1# show logging | include level

```
Console logging: level errors, 14 messages logged
Monitor logging: level errors, 0 messages logged
Buffer logging: level errors, 14 messages logged
Trap logging: level errors, 14 message lines logged
```

Diagnosis. Every destination is set to **errors**, which is severity 3. That delivers levels 0 to 3 and **discards 4, 5, 6 and 7**.

Look at what lives in the discarded range:

- %PM-4-ERR_DISABLE — severity 4. The port being disabled.
- %LINEPROTO-5-UPDOWN — severity 5. The line protocol dropping.
- %SYS-5-CONFIG_I — severity 5. Somebody making a change.

Almost everything that explains why a port went down is severity 4 or 5, and none of it is being recorded anywhere — not the buffer, not the console, not the server.

The log does not “show nothing”. The log was told not to look.

The 14 messages that did survive are the severity-3 %LINK-3-UPDOWN events — the interfaces going down. So the team can see the effect and has filtered out every possible cause, which is precisely the worst configuration to have.

Fix.

>_ Configure — recording enough to diagnose

```
service timestamps log datetime msec localtime show-timezone
logging buffered 65536 informational
logging trap notifications
logging console warnings
```

Then wait for the next occurrence rather than guessing. With level 5 reaching the buffer and the server, the sequence around each drop becomes visible — and in this case it showed %PM-4-ERR_DISABLE for psecure-violation following a cleaner's floor polisher being plugged into a wall port each night.

The lesson. An empty log is a claim about the logging configuration, not about the network. Before concluding that nothing happened, run `show logging` and read the levels. This is the most useful habit in the whole chapter.

Lab 30.1 — Making a device tell you what it is doing (*Packet Tracer for syslog; CML for SNMPv3*)

Platform note. Packet Tracer has a syslog server and will carry tasks 1–6. SNMPv3 needs a real image and a real NMS, so tasks 7–9 want CML or GNS3 with an SNMP tool on a Linux host.

- Task 1.** On a switch with default logging, run `show logging` and record every threshold before changing anything.
- Task 2.** Enable `service timestamps` with date, time and milliseconds. Shut an interface and compare the message with one logged before the change.
- Task 3.** Shut and unshut an interface and capture both messages. Identify the facility, severity and mnemonic in each, and explain why the two severities differ.
- Task 4.** Set `logging buffered errors`. Repeat the shut and unshut. Record which of the two messages survives and explain why — this is the Break & Fix in miniature.
- Task 5.** Restore `logging buffered informational`, save a configuration change, and find the %SYS-5-CONFIG_I message. Note that it records *that* a change was made, not what it was.
- Task 6.** Point `logging host` at a syslog server, set `logging trap notifications`, and confirm messages arrive. Then set it to `emergencies` and confirm they stop.
- Task 7. SNMP.** Configure SNMPv3 with `authPriv` and poll the device's interface table from a Linux host. Record the OID you used.
- Task 8.** Capture the polling traffic. Confirm you cannot read it. Then reconfigure for `v2c`, capture again, and **find the community string in the packet**. Write down what you saw.
- Task 9.** Configure a trap receiver, generate an event, and confirm receipt. Change the trap to an `inform` and observe the acknowledgement.
- Task 10. Extension.** Write the logging and SNMP standard your site should apply to every device, with a one-line justification per command, and the one thing you would check first when someone says the log is empty.

🔍 Checkpoint — Test Yourself

1. Name the five management approaches and give the one-line test that separates controller-based from automation-based.
2. In %PM-4-ERR_DISABLE, identify the facility, the severity and the mnemonic, and say what the severity means.
3. Which severity levels does `logging buffered warnings` record, and which does it discard?
4. State the difference between an SNMP trap and an inform, and when you would choose each.
5. Why is SNMP v2c unsuitable regardless of how obscure the community string is?
6. What does `logging trap` configure, and what does it not?

📋 Chapter Summary

- Five approaches: device-based (per box), controller-based (a controller holds intent), cloud-based (vendor-hosted management plane), automation-based (tooling drives existing interfaces), infrastructure as code (state in a reviewed repository).
- Controller-based adds a system to the network; automation-based does not. That is the distinction to be able to state.
- SNMP: manager polls agents on UDP 161 with Get, GetNext, GetBulk and Set; agents send traps and informs to UDP 162. Informs are acknowledged, traps are not.
- v1 and v2c send community strings in clear. Only v3 authenticates and encrypts. Use `authPriv`.
- An IOS syslog message is %FACILITY-SEVERITY-MNEMONIC: description. The severity is the digit between the hyphens.
- Levels 0 to 7, emergency to debugging. **0 is worst**. A threshold of *n* records 0 through *n* and discards the rest.
- `service timestamps` first, always. `logging trap` is a syslog severity, not SNMP.
- An empty log describes the logging configuration, not the network.

🔧 Review Questions

1. (MCQ) In %LINEPROTO-5-UPDOWN, what does the 5 indicate?
 - (a) The interface number
 - (b) The severity level, notification
 - (c) The number of times the event occurred
 - (d) The facility code
2. (MCQ) Which severity level is the most severe?
 - (a) 7, debugging (b) 0, emergency (c) 1, alert (d) 4, warning
3. (MCQ) Which SNMP notification is acknowledged by the receiver?
 - (a) Trap (b) Inform (c) GetBulk (d) Set
4. (Short) Explain the difference between controller-based and automation-based management, and give one example of each.
5. (Short) A colleague reports that a switch's log contains no record of an outage that monitoring clearly shows. Give the first command you would run and the two things it might reveal.

6. **(Applied)** Interpret this line fully, naming every component and explaining what happened and what you would do next: Aug 27 02:41:09.882: %PM-4-ERR_DISABLE: psecure-violation error detected on Gi1/0/14, putting Gi1/0/14 in err-disable state
7. **(Applied)** Write the logging configuration for a production access switch that keeps full evidence locally, sends notifications and above to a central server, and does not make the console unusable. Justify each threshold you choose.

Ansible and Configuration Management

EXAM TOPICS **5.5**

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** what agentless means and why it matters for network devices.
- **Read** an inventory file and a playbook, and say what each line does.
- **Use** Ansible to run show commands across many devices and to push a configuration change.
- **Recognise** the three failures that account for most broken playbooks: YAML indentation, the wrong module, and a change that was never saved.

🔗 Before You Start

Chapter 23 for SSH and credentials — Ansible logs in exactly as you do. Chapter 30 for the five management approaches; this chapter is the automation-based one made concrete. Chapter 7 for the commands being pushed.

📖 Key Terms in This Chapter

agentless • idempotent • control node • managed node • inventory • play • task • module • playbook • facts • YAML • dry run • vault

🎓 The verb is *use*

Topic **5.5** is *use configuration management mechanisms such as Ansible to execute commands*. It is the only topic in domain 5 that is not *describe*, *select* or *interpret* — so you are expected to be able to read a playbook and say what it will do to which devices.

“Such as Ansible” leaves the door open to Puppet, Chef and SaltStack, but every published Cisco example uses Ansible, and Ansible is the one whose agentless model actually suits network hardware. Learn Ansible; know the other names and the one-line difference.

31.1 The problem

Forty access switches need one new VLAN and one new ACL line. Done by hand that is forty SSH sessions, forty opportunities to typo, and no record of which device got which version. Done a second time next month, it is forty more.

📑 Configuration management

Describing the state a set of machines should be in, and using a tool to make them match that description — repeatably, and with a record of what was done.

31.1.1 Agentless, and why that decides it

	Ansible	Puppet, Chef, SaltStack
Model	Agentless. Connects over SSH and issues commands	Agent-based. Software runs permanently on each managed machine
On the device	Nothing installed. Nothing to upgrade	An agent, which must be installed, updated and monitored
Works on a switch?	Yes — a switch already speaks SSH	Not really. You cannot install an agent on IOS
Direction	Push, when you run it	Pull, on the agent's schedule

⚠ This is the whole reason Ansible dominates network automation

A server can run an agent. A Catalyst switch cannot — it is an appliance, and you may not install arbitrary software on it. Any tool that requires an agent is therefore unavailable for the devices this exam is about.

Ansible needs nothing on the device except the SSH access you already configured in Chapter 23. That is the answer to “why Ansible for networks”, and it is a fair exam question.

📖 Idempotent

An operation that produces the same result whether it is applied once or twenty times. Ansible tasks are written to describe a desired state, so a task that has already been satisfied reports ok and changes nothing.

This is what makes it safe to run the same playbook every day. A script that blindly types commands is not idempotent; a playbook that says “this line should be present” is.

31.2 The pieces

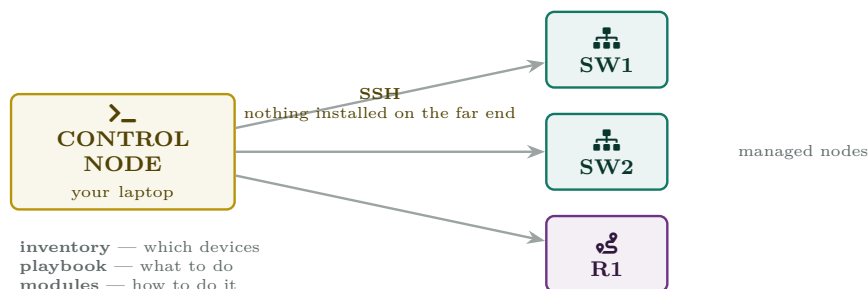


Figure 31.1: Everything lives on the control node. The devices need only the SSH access they already have.

31.2.1 YAML, in ninety seconds

Playbooks are YAML, and almost every first failure is a YAML failure. Three rules carry you:

- **Indentation is structure, and it must be spaces.** A tab character is a syntax error. Two spaces per level is the convention.
- **A leading - makes a list item.** Items at the same indentation belong to the same list.
- **key: value makes a mapping.** The space after the colon is required.

31.3 Inventory: which devices

 inventory.yml

```
all:
  children:
    access_switches:
      hosts:
        SW1: { ansible_host: 10.0.0.11 }
        SW2: { ansible_host: 10.0.0.12 }
        SW3: { ansible_host: 10.0.0.13 }
    routers:
      hosts:
        R1: { ansible_host: 10.0.0.1 }
  vars:
    # network_cli is essential. Without it Ansible assumes the far end
    # is a Linux host with a Python interpreter, and every task fails
    # with an error that does not mention the real problem.
    ansible_connection: ansible.netcommon.network_cli
    ansible_network_os: cisco.ios.ios
    ansible_user: netadmin
```

 Never put the password in the inventory

Use `ansible-vault` to encrypt a variables file, and supply the key at run time:

```
>_ Linux or macOS — ansible-vault create group_vars/all/vault.yml
```

```
New Vault password:
Confirm New Vault password:
```

Then run with `-ask-vault-pass`, or use SSH keys and no password at all. A plain-text credential in a file that is about to be committed to version control is how credentials leak, and “it is only the lab” is how the habit forms.

31.4 Executing commands

31.4.1 Ad-hoc, for one thing right now

```
>_ Linux or macOS — ansible access_switches -i inventory.yml -m cisco.ios.ios_command -a
"commands='show version'"
```

```
SW1 | SUCCESS => {
  "changed": false,
  "stdout_lines": ["Cisco IOS XE Software, Version 17.12.04", "..."]
}
SW2 | SUCCESS => {
  "changed": false,
  "stdout_lines": ["Cisco IOS XE Software, Version 17.09.05", "..."]
}
```

Note `"changed": false`. A `show` command reads and does not alter, so nothing changed — and Ansible says so explicitly. That field is the one to watch on every run.

31.4.2 A playbook, for something you will do again

 audit.yml — collect and report

```
---
- name: Audit access switches
  hosts: access_switches
  gather_facts: false
```

```

tasks:
  - name: Collect version and inventory facts
    cisco.ios.ios_facts:
      gather_subset:
        - hardware
    register: device_facts

  - name: Show what we found
    debug:
      msg: "{{ inventory_hostname }}" runs "{{ ansible_net_version }}"

  - name: Run a show command and keep the output
    cisco.ios.ios_command:
      commands:
        - show interfaces status
    register: intf

  - name: Write it to a file on the control node
    copy:
      content: "{{ intf.stdout[0] }}"
      dest: "./reports/{{ inventory_hostname }}-interfaces.txt"
      delegate_to: localhost

```

vlan.yml — make a change on every switch

```

---
- name: Add the guest VLAN everywhere
  hosts: access_switches
  gather_facts: false

  tasks:
    - name: Create VLAN 40
      cisco.ios.ios_config:
        lines:
          - name GUEST
        parents: vlan 40

    - name: Allow it on the uplink trunk
      cisco.ios.ios_config:
        lines:
          - switchport trunk allowed vlan add 40
        parents: interface GigabitEthernet1/0/48

    # Without this the change lives in the running configuration only
    # and is lost at the next reload. "modified" saves only when this
    # run actually changed something.
    - name: Save the configuration
      cisco.ios.ios_config:
        save_when: modified

```

ios_command reads, ios_config writes

This is the distinction the exam will test and the one beginners get wrong. `ios_command` runs commands in EXEC mode and returns their output. It **cannot make a configuration change**, and it will refuse configuration commands outright rather than silently failing.

`ios_config` enters configuration mode and applies lines, using `parents` to say which sub-mode they belong in. It compares against the running configuration first, so it is idempotent. A playbook that reports success and changes nothing is very often `ios_command` where `ios_config` was meant.

31.4.3 Check before you commit

```
>_ Linux or macOS — ansible-playbook -i inventory.yml vlan.yml -check -diff
```

```
TASK [Create VLAN 40] *****
changed: [SW1]
changed: [SW2]
ok: [SW3]

PLAY RECAP *****
SW1  : ok=2  changed=1  unreachable=0  failed=0
SW2  : ok=2  changed=1  unreachable=0  failed=0
SW3  : ok=3  changed=0  unreachable=0  failed=0
```

💡 Read the recap, every time

`-check` is a dry run: Ansible reports what it *would* do and touches nothing. `-diff` shows the exact lines. Run this before every change to production, without exception.

In that output, SW3 reports `ok` rather than `changed` because it already has VLAN 40. That is idempotency visible in the recap, and it is also a useful audit: the difference between the two tells you which devices had drifted.

`unreachable` is a different failure from `failed`: `unreachable` means Ansible could not log in at all, `failed` means it logged in and the task did not work.

❗ Common Pitfalls

- **Tabs in YAML.** A syntax error, and the message rarely says “tab”. Configure your editor to insert spaces.
- **Missing `ansible_connection: network_cli`.** Ansible tries to run Python on a switch and fails confusingly.
- **`ios_command` where `ios_config` was meant.** Reports success, changes nothing.
- **No `save_when`.** The change works until the device reloads.
- **Wrong parents.** Lines are applied in global configuration mode instead of under the interface, which either errors or does something you did not intend.
- **Skipping `-check`.** There is no undo across forty devices.
- **Credentials in the inventory file.** Use vault or SSH keys.
- **Running against all when you meant one group.** Check the `hosts:` line before every run. It is the single most expensive typo available to you.

🔧 **Break & Fix** — The playbook reports success on all forty switches. After a power cut, eleven have lost the change.

A new VLAN was added across the access layer with a playbook. Every device reported `changed`, spot checks confirmed the VLAN existed, and the change was signed off. Three weeks later a power event reboots part of the estate, and eleven switches come back without VLAN 40. The twenty-nine that did not lose power still have it.

vlan.yml — as it was run

```

---
- name: Add the guest VLAN everywhere
  hosts: access_switches
  gather_facts: false

  tasks:
    - name: Create VLAN 40
      cisco.ios.ios_config:
        lines:
          - name GUEST
        parents: vlan 40

```

Diagnosis. There is no `save_when` task. `ios_config` writes to the *running* configuration, exactly as typing the commands would — and like typing them, it does not save. Every switch was correctly configured and none of them wrote that configuration to *startup-config*. The evidence pattern confirms it precisely: the split is not between devices that the playbook succeeded on and devices it failed on. It is between devices that **rebooted** and devices that did not. Twenty-nine switches still have the VLAN because they have been up the whole time and are still running the configuration Ansible applied. The eleven that power-cycled reloaded *startup-config*, which never contained it.

Any fault that correlates with reboots rather than with the change itself points at the running-versus-startup distinction.

Fix. Add the `save` task and re-run.

vlan.yml — corrected

```

- name: Save the configuration
  cisco.ios.ios_config:
    save_when: modified

```

`save_when: modified` writes only when the run actually changed something, so re-running against the twenty-nine devices that are already correct costs nothing. The eleven get the VLAN and a save.

The verification lesson. The spot check confirmed the VLAN existed — in the running configuration. It should have compared running against startup:

```
Q SW1# show archive config differences nvram:startup-config system:running-config
```

```

!Contextual Config Diffs:
+vlan 40
+ name GUEST

```

Any output at all from that command means unsaved changes are present. Making it the last step of every change — by hand or as a final Ansible task — closes this class of fault permanently.

🔧 Lab 31.1 — Automating a change you would otherwise do forty times (CML or GNS3, with a Linux control node)

Platform note. Ansible needs real SSH and a real Python environment on the control node. Packet Tracer cannot do this. Use CML or GNS3 with three or more IOS devices and a Linux VM, or install Ansible on your own machine and point it at the lab.

Task 1. Install Ansible and the `cisco.ios` collection on the control node. Record the versions of both.

- Task 2.** Configure SSH and a local user on three devices as in Chapter 23. Confirm you can log in by hand before involving Ansible at all — if this does not work, nothing after it will.
- Task 3.** Write `inventory.yml` with two groups. Verify with `ansible-inventory -list`.
- Task 4.** Run an ad-hoc `ios_command` for `show version` against one group. Then deliberately remove `ansible_connection` and run it again. Record the error and explain why it does not mention the real cause.
- Task 5.** Store the password with `ansible-vault` and run using `-ask-vault-pass`. Confirm the file is unreadable without the key.
- Task 6.** Write `audit.yml` to gather facts and write a per-device report file on the control node. Confirm every task reports `changed: false` and explain why.
- Task 7.** Write `vlan.yml` to add a VLAN and allow it on a trunk. Run with `-check -diff` first. Record the diff, then run for real.
- Task 8.** Run it a second time with no changes. Confirm every device reports `ok` rather than `changed`. This is idempotency; write down what you would conclude if one device reported `changed` on the second run.
- Task 9. Break 1.** Reproduce the Break & Fix. Omit `save_when`, apply a change, reload a device, and confirm the loss. Then add the `save` task and repeat.
- Task 10. Break 2.** Replace `ios_config` with `ios_command` for the VLAN task. Record exactly what Ansible reports and what the device shows.
- Task 11. Break 3.** Introduce a tab character into the playbook. Record the error message and how long it takes you to find it.
- Task 12. Extension.** Add a final task that runs the `show archive config differences` command from this chapter and fails the play if it returns anything. You now cannot leave a change unsaved.

? Checkpoint — Test Yourself

1. What does `agentless` mean, and why does it decide the choice of tool for network devices?
2. What is idempotency, and where do you see it in a play recap?
3. State the difference between `ios_command` and `ios_config`.
4. What does `save_when: modified` do, and what fails without it?
5. What is the difference between `unreachable` and `failed` in the recap?
6. What do `-check` and `-diff` do, and when would you omit them?

📋 Chapter Summary

- Ansible is **agentless**: it connects over SSH and installs nothing. That is why it works on switches, where Puppet and Chef cannot.
- Tasks are **idempotent** — they describe desired state, so running twice changes nothing the second time.
- Playbooks are YAML: spaces not tabs, `-` for list items, `key: value` with a space after the colon.
- The inventory names devices and groups, and must set `ansible_connection: network_cli` and `ansible_network_os`.
- `ios_command` reads; `ios_config` writes, with `parents` to select the sub-mode.
- `save_when: modified` writes to startup-config. Without it the change survives only until the next reload.

- `-check -diff` is a dry run. Use it before every production change.
- In the recap, `changed` means it acted, `ok` means it was already correct, `unreachable` means it never logged in.

Review Questions

1. **(MCQ)** Why is Ansible, rather than Puppet, used for Cisco switches?
(a) It is faster (b) It is agentless
(c) It uses YAML (d) It supports idempotency
2. **(MCQ)** Which module applies configuration to an IOS device?
(a) `cisco.ios.ios_command`
(b) `cisco.ios.ios_facts`
(c) `cisco.ios.ios_config`
(d) `debug`
3. **(Short)** A playbook runs cleanly and reports `changed` on every device, but a colleague finds the devices unmodified after a maintenance reboot. Give the cause and the fix.
4. **(Short)** Explain idempotency using the `ok` and `changed` counts in a play recap, and say what a device reporting `changed` on every single run would tell you.
5. **(Applied)** Write a playbook that adds an ACL line to every router in a group, saves the configuration, and would be safe to run daily. State which single line makes it safe to re-run.
6. **(Applied)** You are handed a playbook whose `hosts:` line reads `all` and which contains an `ios_config` task with no `-check` in the runbook. Identify every risk and rewrite the runbook steps you would insist on before it is executed.

AI in Network Operations

EXAM TOPICS **5.1** **5.2**

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Distinguish** generative AI from agentic AI, and describe what makes a system agentic.
- **Describe** where AI is genuinely used in network operations, and where it should not be trusted.
- **Select** an effective prompt, using the four components the blueprint names: data classification, output format, persona and instructions.
- **Judge** what data may safely be sent to an AI system, and sanitise what may not.

🔗 Before You Start

Chapter 30 for the five management approaches — agentic systems sit on top of them, not instead of them. Chapter 31 for automation, which is what an agent actually uses to act. Chapter 23 for why credentials must not leave the organisation.

📖 Key Terms in This Chapter

generative AI • agentic AI • autonomy • tool use • human in the loop • prompt • persona • data classification • sanitisation • hallucination • blast radius

🎓 Two topics, and the second one is unusual

5.1 is *describe the role of agentic AI in network operations*.

5.2 is *select a prompt to send to a generative AI system to support network operations considering prompt components such as data classification, output format, persona, and instructions*.

Read **5.2** carefully, because it tells you the question format. *Select* means you will be shown several candidate prompts and asked which is best — not asked to write one. And the four components are **named**, which means the marking scheme is those four. A prompt that has all four beats one that does not.

Note the order Cisco lists them in: **data classification comes first**. That is not accidental, and Section 32.3.1 explains why.

32.1 Generative and agentic

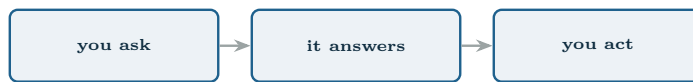
📄 Generative AI

A system that produces content — text, code, configuration — in response to a prompt. It answers and stops. Every action taken as a result is taken by you.

📄 Agentic AI

A system that pursues a *goal* rather than answering a question. It perceives the current state, plans, uses tools to act, observes the result, and iterates — choosing the intermediate steps itself.

Generative



Agentic

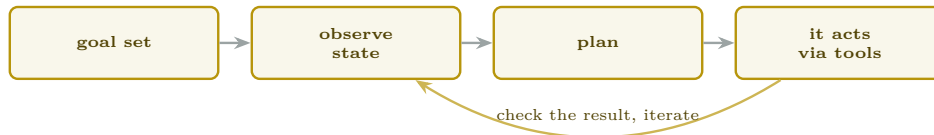


Figure 32.1: The difference is the loop and who closes it. A generative system hands you an answer; an agentic one acts and checks its own work.

	Generative	Agentic
Given	A prompt	A goal
Produces	An answer	A sequence of actions
Tools	None. It only writes	Calls APIs, runs show commands, opens tickets, applies configuration
Steps	One	As many as it decides it needs
Who acts	You	It — within whatever bounds you set
Risk if wrong	A bad suggestion you can reject	A change already made to a live network

⚠ The one-sentence answer for the exam

Generative AI produces content; agentic AI takes action toward a goal, using tools, and evaluates its own results. Autonomy and tool use are the two words that separate them.

32.2 Where it is actually used

Role	What it does	Why it is suited to this
Anomaly detection	Learns normal behaviour and flags departures from it	The volume of telemetry is far past what anyone can watch
Event correlation	Collapses a thousand alarms from one fault into one incident	Pattern matching across sources, at speed
Root-cause suggestion	Proposes the likely cause and the evidence for it	It has seen the whole log, and you have seen the last screen
Configuration drafting	Produces candidate configuration and reviews existing configuration	Syntax and boilerplate are exactly what it is good at

Role	What it does	Why it is suited to this
Documentation and summary	Turns a change record or a packet capture into readable prose	Tedious, low-risk, easily checked
Capacity forecasting	Projects growth from historical counters	A statistical problem with plenty of data
Guided remediation	Proposes a fix; with more autonomy, applies it	This is the agentic end, and where care is needed

32.2.1 The autonomy ladder

The useful question about an agentic system is not “is it intelligent” but **how much is it allowed to do without asking**.

- **Recommend.** It tells you what it would do. You do it. No risk beyond a wasted suggestion.
- **Approve then act.** It prepares the change and waits for a human to approve. This is where most production deployments sit, and sensibly.
- **Act then report.** It makes the change and tells you afterwards. Appropriate for narrow, reversible, well-understood actions — clearing an err-disabled port, restarting a stuck process.
- **Fully autonomous.** It acts within a defined scope with no notification. Appropriate for very little in a production network.

💡 Blast radius is the design question

Before granting autonomy, ask what the worst outcome is if the system is confidently wrong. Clearing one err-disabled port has a blast radius of one port. Changing a route map on a core router has a blast radius of the organisation.

The same technology can sit at different rungs for different actions, and a well-designed deployment does exactly that. “Human in the loop” is not a blanket policy; it is a decision made per action, based on reversibility and reach.

⚠️ What these systems are bad at, and you must know

- **Being confidently wrong.** A fluent, plausible, incorrect answer is the characteristic failure. It does not signal uncertainty the way a colleague would.
- **Your network specifically.** Training data is the internet, not your topology. It does not know that VLAN 40 is the guest network here.
- **Currency.** It may recommend commands deprecated years ago, or miss ones added last year.
- **Accountability.** When a change breaks a site, a person is answerable. Change control, approval and rollback do not become optional because a machine proposed the change.

32.3 Selecting a prompt

This is topic [5.2](#), and it is a practical skill with four named parts.

Component	What it does	Weak against strong
Data classification	Decides what you may include, and what must be removed or replaced first	Pasting a full running configuration with keys and public addresses, against pasting the four relevant lines with addresses re-written
Persona	Sets the expertise and viewpoint to answer from	“Explain VLANs” against “You are a senior network engineer working on Cisco IOS XE 17.12”
Instructions	The actual task, with its constraints and its scope	“Fix this” against “Identify why VLAN 30 clients get no address; do not suggest changes to VLANs 10 or 20”
Output format	The shape the answer must take, so it is usable	“Tell me about it” against “Reply as a table of Cause, Evidence, Command to confirm. No more than five rows”

Worked Example: The same question, asked twice

Weak:

Why is my DHCP not working? Here is my config: [entire running configuration pasted, including `snmp-server community, username ... secret` and the site's public addressing]

No persona, no output format, no scope — and it has just sent credentials and the organisation's real addressing to a third party. Even a good answer would not justify that.

Strong:

You are a senior network engineer working on Cisco IOS XE 17.12.

Clients in VLAN 30 receive APIPA addresses. VLANs 10 and 20 on the same distribution switch lease correctly. The DHCP server is central and reached across a routed core. Relevant configuration, with addresses substituted:

```
interface Vlan30 / ip address 10.30.0.1 255.255.255.0
```

List the three most likely causes, most likely first. For each give the single show command that would confirm or eliminate it. Reply as a table with columns Cause, Confirming command, Expected output if this is the fault. Do not propose configuration changes.

All four components are present, and each earns its place: the persona sets the platform, the classification decided that addresses would be substituted and credentials omitted, the instructions bound the scope, and the format makes the answer directly actionable.

How to answer a “select the best prompt” question

Score each candidate against the four components and pick the one with most. Then apply three tie-breakers:

- **Eliminate anything containing credentials, keys or real public addressing** immediately, however good the rest of it is. Data classification is listed first for a reason and a prompt that leaks is wrong regardless of its other qualities.
- **Prefer specific over polite.** “Please help me with my network” scores nothing. Length is not the measure — *specificity* is.
- **Prefer a stated output format.** It is the component most often missing from the distractors, so it frequently decides the answer.

32.3.1 Data classification, properly

⚠ Assume anything you send leaves your control

A prompt sent to an external service has been disclosed to a third party. It may be logged, retained, reviewed by humans, or used for training, depending on the contract — and deleting it later does not undo the disclosure.

Never send: passwords, hashes, SNMP community strings, pre-shared keys, certificates, API tokens; personal data; real public IP addressing; anything your organisation classifies as confidential or above.

Sanitise first: substitute addresses consistently (documentation ranges 192.0.2.0/24, 198.51.100.0/24, 203.0.113.0/24 exist for exactly this), replace hostnames, strip credential lines entirely, and include only the configuration relevant to the question.

And check the policy. Many organisations permit an approved internal or contracted tool and prohibit public ones. Knowing which applies to you is part of the job now.

! Common Pitfalls

- **Pasting a whole running configuration.** Slower, worse answers, and a disclosure.
- **Confusing generative with agentic.** Tool use and autonomy are the difference.
- **Omitting the output format.** You get an essay where you needed three commands.
- **No persona or platform.** You get generic advice, or IOS syntax for a platform you are not running.
- **Accepting a confident answer without verifying.** Check the command exists on your platform before typing it.
- **Granting autonomy without considering blast radius.**
- **Assuming it knows your network.** It knows the internet.
- **Treating an AI-proposed change as exempt from change control.**

🔧 Break & Fix — The suggested command was accepted and took the site off the air.

An engineer asks an assistant why a trunk is not carrying VLAN 50 and is told to run `switchport trunk allowed vlan 50` on the uplink. They apply it. Every VLAN except 50 immediately stops working across that link.

The prompt that was used: “VLAN 50 isn’t going over my trunk, what command fixes it?”

Diagnosis — of the answer. The command is real, correctly spelled and valid. It also *replaces* the allowed list rather than adding to it, so a trunk carrying 10, 20 and 30 now carries only 50. The correct command was `switchport trunk allowed vlan add 50` (Chapter 10).

Diagnosis — of the prompt, which is the examinable part. Measure it against the four components:

- **Persona** — absent. Nothing established the platform or the expected level of care.

- **Instructions** — absent. No mention that the trunk already carried other VLANs, which is the fact that makes the difference between the two commands. The model had no way to know it.
- **Output format** — absent. A request for “the command, plus what it changes, plus how to verify before and after” would have surfaced the replace-versus-add behaviour in the answer itself.
- **Data classification** — the one component that was, by accident, satisfied. Nothing sensitive was sent, because nothing was sent.

The prompt asked a question whose correct answer depends on context it did not supply. The response was reasonable given what it was told.

The better prompt.

*You are a senior network engineer working on Cisco IOS XE. An 802.1Q trunk between two switches currently carries VLANs 10, 20 and 30. I need to add VLAN 50 **without interrupting the existing three**. Give the exact command, state what it changes, and give the show command to verify the allowed list before and after. Warn me about any similar command that would be destructive here.*

The wider lesson, and it is the point of this chapter. The fault was not the tool’s. A prompt missing three of its four components produced an answer that was correct in general and wrong here, and it was applied to a production trunk without verification. The habit that prevents this is not distrust of AI — it is the same habit that should apply to a command copied from a forum: understand what it changes before you press enter, and have a way to check.

Lab 32.1 — Prompt engineering as an examinable skill *(Any generative AI assistant, plus a lab device)*

Platform note. This lab needs no particular tool. Use whatever assistant your institution permits — and **check that policy first**, which is task 1 and is not a formality.

- Task 1.** Find and read your institution’s or employer’s policy on sending data to AI services. Write down, in one sentence, what you are permitted to send.
- Task 2.** Take a real configuration from one of your lab devices. Produce a sanitised version fit to include in a prompt: substitute addressing into documentation ranges, rename hosts, remove every credential. List everything you removed and why.
- Task 3.** Write the weakest useful prompt you can for a real problem from Chapter 14. Record the answer.
- Task 4.** Rewrite it with all four components. Record the answer. Compare the two for specificity, correctness and usability.
- Task 5. Verify both answers on the device.** Note anything suggested that was wrong, deprecated, or not supported on your platform. This step is the one that matters.
- Task 6.** Ask a question whose correct answer depends on context you deliberately withhold — as in this chapter’s Break & Fix. Confirm you can produce a confidently wrong answer, and identify which missing component caused it.
- Task 7.** Write four candidate prompts for one task, one of which is clearly best, and swap with a colleague. Score each other’s against the four components. This is the exam question format.

Task 8. Autonomy. For each of these, place it on the four-rung ladder and justify: clearing an err-disabled port; adding a VLAN to an access switch; changing an OSPF cost on a core link; opening a ticket.

Task 9. Extension. Write the one-page guidance your team should follow for using AI assistants on network work — what may be sent, what must be verified, and which actions may ever be automatic.

Checkpoint — Test Yourself

1. State the difference between generative and agentic AI in one sentence.
2. Name the four prompt components the blueprint lists, and say which is listed first and why.
3. Give three categories of data that must never be sent to an external AI service.
4. What is blast radius, and how does it govern how much autonomy an agent should have?
5. An assistant gives a fluent, confident, wrong answer. What is this called and what is the defence?
6. Name three genuine uses of AI in network operations.

Chapter Summary

- Generative AI produces content in response to a prompt. **Agentic AI pursues a goal**, uses tools, acts, and evaluates its own results. Autonomy and tool use are the difference.
- Real roles: anomaly detection, event correlation, root-cause suggestion, configuration drafting, documentation, capacity forecasting, guided remediation.
- Autonomy is a ladder — recommend, approve then act, act then report, fully autonomous — chosen per action according to blast radius.
- The four prompt components: **data classification, output format, persona, instructions**. A prompt with all four beats one without.
- Data classification is listed first. Never send credentials, keys, personal data or real public addressing; sanitise into documentation ranges first; check your organisation's policy.
- In a “select the prompt” question: eliminate anything that leaks, prefer specificity over politeness, and look for the stated output format.
- These systems are confidently wrong, do not know your network, and may be out of date. Verify before you apply, and keep change control.

Review Questions

1. (MCQ) What most distinguishes agentic AI from generative AI?
 - (a) It produces longer answers
 - (b) It takes actions using tools and evaluates the results
 - (c) It runs on the network device itself
 - (d) It requires no prompt
2. (MCQ) Which prompt component decides what you are permitted to include in the first place?
 - (a) Persona (b) Output format
 - (c) Data classification (d) Instructions
3. (MCQ) Which of these should never appear in a prompt sent to an external service?
 - (a) A VLAN number (b) An SNMP community string

- (c) An interface name (d) A documentation-range address
4. **(Short)** Explain blast radius and use it to justify why clearing an err-disabled port may be automated while changing a core route map should not.
 5. **(Short)** An assistant recommends a command that is valid but destructive in your situation. Name the two prompt components whose absence caused this, and rewrite the prompt.
 6. **(Applied)** Write a prompt asking for help diagnosing an OSPF adjacency stuck in EXSTART, containing all four components, using only data that is safe to send. Then state how you would verify the answer before acting on it.
 7. **(Applied)** Your manager proposes letting an agentic system apply ACL changes automatically out of hours. Give your recommendation with three specific conditions you would require, referring to the autonomy ladder.

PART VI

Sitting the Exam

Everything up to here has been about networks. These last two chapters are about the exam — one on the method that the largest group of topics is really testing, and one on sitting the paper itself.

They are short, and they are not filler. The single most common way to fail an exam you know the material for is to mismanage it: to spend eleven minutes on a question worth the same as one that takes forty seconds, or to reach a configuration task with no time left, or to answer what you assumed was asked rather than what was.

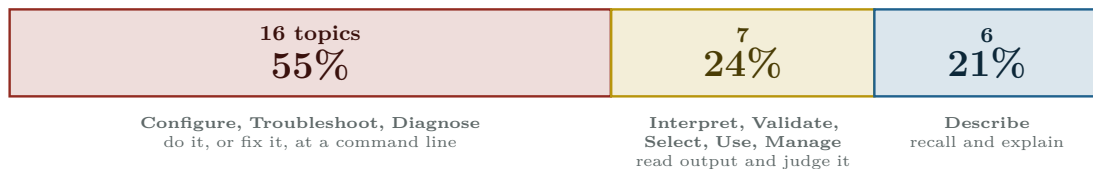


Figure P6.1: The twenty-nine topics by their opening verb. Only a fifth of the blueprint asks you to recall anything; the rest asks you to do something or to read something and decide.

The number that should shape your last month of revision

Count the verbs in Cisco's own topic list and the distribution above falls out: eight topics begin *Configure*, six *Troubleshoot*, two *Diagnose*. That is sixteen of twenty-nine — **55%** — where knowing the answer is not enough, because you have to produce it at a CLI or find it in someone else's broken configuration.

Revision that consists of re-reading notes prepares you for the 21%. The other 79% is prepared for by building things and breaking them, which is why every chapter of this volume has a lab and a Break & Fix rather than a summary and a quiz.

If you have four weeks left and have read everything once, spend three of them at a command line.

A Troubleshooting Method for the 28 Per Cent

EXAM TOPICS **2.4**

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Apply** a seven-step cycle to any fault, rather than guessing in a familiar order.
- **Choose** between the five structured methods and justify the choice from the symptom.
- **Scope** a fault before touching anything, using four questions that routinely halve the search.
- **Work** evidence-first: record before you change, change one thing, and undo what did not help.

🔗 Before You Start

Everything. This chapter is the method that the rest of the book supplies the knowledge for. Chapter 6 for the diagnostic tools and Chapter 14 for layer 2 in particular; the symptom index in Section 33.6 points at all the others.

📖 Key Terms in This Chapter

scoping • top-down • bottom-up • divide and conquer • follow the path • compare working to broken • hypothesis • blast radius • handover

🏠 What topic 2.4 actually says

Troubleshoot basic Layer 2/Layer 3 connectivity and device operations using show commands (including show logs), ping, extended ping, trace route, and packet capture output.

Every tool in that list has already been taught — Chapter 6 for ping, extended ping, traceroute and reading captures; Chapter 30 for show logging; Chapter 14 for layer 2. This chapter deliberately does not repeat them.

What it adds is the thing the tools cannot supply: **an order to use them in**. Eight of the twenty-nine topics begin *Troubleshoot* or *Diagnose*, which is 28% of the blueprint, and in every one of them the examiner already knows you can type `show ip interface brief`. What is being tested is whether you can decide to.

33.1 Why method beats knowledge under pressure

Two engineers with identical knowledge produce very different results on a fault neither has seen. The difference is almost never what they know; it is whether they have an order to work in when the obvious answer turns out to be wrong.

An unstructured approach checks the thing that was broken last time, then the thing the person before you suggested, then something at random. It works quickly when the fault is familiar and collapses when it is not — and the exam, like a bad morning, specialises in faults that are not.

⚠️ The failure mode to design against

The characteristic mistake is **finding a fault and stopping**. There is a misconfiguration on the path, you fix it, and the symptom persists — because it was not *the* fault, or not the only one.

A method protects against this by making you state what you expect the fix to do *before* you apply it. If the prediction fails, you have learned something; if you had no prediction, you have only changed something.

33.2 The cycle

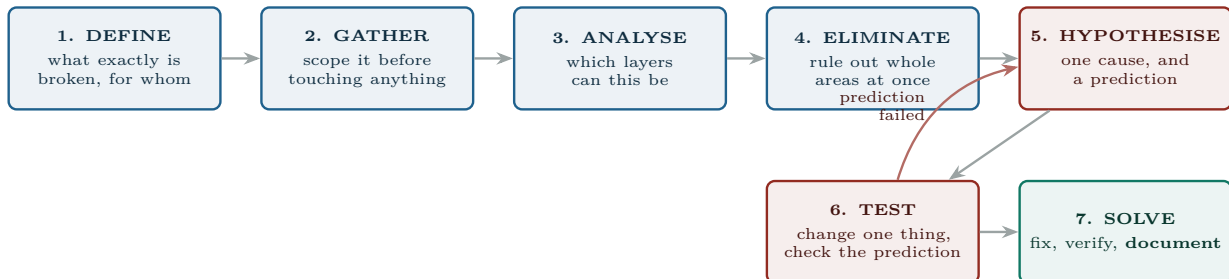


Figure 33.1: The loop is between 5 and 6, and it is the point. A hypothesis that does not predict correctly is discarded, not patched.

💡 Step 5 is where amateurs and professionals separate

“I think it is the trunk allowed list” is not a hypothesis. “**I think VLAN 30 is missing from the allowed list on the uplink, in which case show interfaces trunk will not list it, and VLANs 10 and 20 will be unaffected**” is one — because it can be wrong, and you will know immediately.

Say what you expect to see before you run the command. It takes three seconds and it converts every check into information whichever way it goes.

33.3 Five structured methods

Method	How it works	Use it when
Bottom-up	Start at layer 1 and work up. Cable, link light, duplex, VLAN, address, route	You suspect hardware or physical change; a new install; nothing works at all
Top-down	Start at the application and work down	Most of the network is fine and one application is not. Avoids inspecting a switch for a DNS fault
Divide and conquer	Start in the middle — usually layer 3. Ping the gateway; if it works, layers 1–3 are fine	You have no strong prior. This is the default , and the fastest on average
Follow the path	Trace the actual route the traffic takes, device by device, checking each hop	Connectivity fails between two specific points and works elsewhere
Compare working to broken	Diff a failing device or port against an identical working one	Some members of an identical set work. Frequently the fastest of all

⚠ Compare-to-working is under-used and enormously effective

Twenty-four ports configured the same way, one of them broken. Do not reason about it — `show running-config interface` on both and read the difference.

The same applies to a device: if three hypervisor hosts work and the fourth does not (Chapter 29), the fault is in what is *not* identical, and that is a much smaller search space than “what could be wrong with a hypervisor uplink”.

Whenever a working comparison exists, use it first. It converts an open-ended diagnosis into a diff.

33.4 Scope before you touch

Four questions, asked before any command, routinely eliminate most of the possible causes.

Question	What the answer eliminates
One user, or many?	One user: the host, its port, its cable. Many on one switch: that switch or its uplink. Many across sites: routing, DNS, DHCP, or the service itself. This single question sets the whole search space
When did it start, and has it ever worked?	“Never worked” is a configuration fault — something was built wrong. “Worked until Tuesday” is a change fault. They are searched completely differently
What changed?	Almost every fault in a previously working network follows a change. Ask about changes to anything, not just the network — a server move, a firmware update, a contractor in the comms room
Does it fail by name but work by address?	Answers “is this DNS” in one test, and DNS is a large fraction of reported network faults that are not network faults (Chapter 21)

💡 “What changed?” is the highest-yield question in operations

A network that worked on Monday and fails on Tuesday was changed. The change may be undocumented, may have been made by someone else, may not look related, and the person who made it may sincerely believe it could not have caused this.

Ask anyway, ask broadly, and check the evidence rather than the recollection: `show logging for %SYS-5-CONFIG-I` messages (Chapter 30), `show archive config differences`, and the change record.

The answer is often “nothing”, and it is often wrong.

33.5 Evidence discipline

Three rules. They are boring, they are what separates a twenty-minute fault from a three-hour one, and they are equally true in an exam simulation.

- **Record before you change.** Capture the current state — `show running-config`, the relevant `show output`, the exact symptom. A configuration you overwrote without reading is evidence you destroyed, and if the change makes things worse you cannot get back.
- **Change one thing at a time.** Two simultaneous changes that fix the fault leave you not knowing which one did, and one of them may have introduced tomorrow’s problem.
- **Undo what did not help.** An accumulation of speculative changes is how a network becomes undiagnosable. If your prediction was wrong, put it back before trying the next idea.

⚠ The exception, and know that you are taking it

Under a major outage there is pressure to restore service first and understand later. That is a legitimate decision — but take it deliberately, say out loud that you are taking it, and capture what you can as you go. The failure is not choosing speed; it is choosing speed by accident and having no idea afterwards what you did.

33.6 Symptom index

Where to go in this volume, given what the user actually said.

Symptom	Start here
No link light, or errors climbing	Chapter 2 — cable, duplex, speed, transceiver
Host has 169.254.x.x	Chapter 20, then Chapter 14. Nothing answered its request
Works within its VLAN, nothing beyond	Gateway or mask: Chapter 28, then Chapter 15
Works by address, fails by name	Chapter 21. Not a network fault
One whole VLAN dead, others fine	Chapter 8, Chapter 10 — allowed list, SVI, or a per-SVI setting
Intermittent, slow, MAC flapping	Chapter 12 — suspect a loop
Port <code>err-disabled</code>	Chapter 25 for port security or BPDU guard; Chapter 30 to read the reason
Some destinations reachable, others not	Chapter 15 for longest-prefix match, then Chapter 24
OSPF neighbours will not form	Chapter 17 — area, timers, MTU, network type, duplicate router ID
Internet works for a while then stops	Chapter 22 — pool exhaustion, or a missing overload
Two gateways both claim to be active	Chapter 19 — group numbers or subnet mismatch
Wireless: connects, no access	Chapter 28 — security parameters, or MAC randomisation
Wireless: strong signal, unusable	Chapter 27 — check the noise floor, not the signal
VMs on one host unreachable	Chapter 29 — the uplink is probably an access port
“The log shows nothing”	Chapter 30 — check the severity thresholds first

33.7 Handing it over

Faults outlive shifts. A handover that says “still looking at the VLAN issue” wastes the next person’s first hour.

- **The symptom**, precisely, and who is affected.
- **What has been ruled out**, and the evidence that ruled it out. This is the most valuable part and the most often omitted.
- **What has been changed**, including anything not yet undone.
- **The current hypothesis** and the next test.

! Common Pitfalls

- **Starting to type before scoping.** Four questions first.
- **Checking what broke last time.** Familiarity is not evidence.
- **No prediction before a change.** You learn nothing from the result.
- **Two changes at once.** Now you do not know which worked.
- **Leaving failed changes in place.** They become the next fault.
- **Fixing the first fault found and declaring victory** without re-testing the original symptom.
- **Ignoring an available working comparison.**
- **Believing “nothing changed”.** Check the logs.
- **Not writing it down.** You will meet this fault again.

🔧 Break & Fix — Two faults at once, and fixing the first one changes nothing.

Users on the second floor report that the file server is unreachable. You find that the uplink from SW2 has VLAN 20 missing from its trunk allowed list, which would certainly do it. You add it, confirm `show interfaces trunk` now lists VLAN 20 — and the users report no improvement at all.

The instinct is to assume the fix did not take, and to re-check the trunk. Resist it: you verified it, and it is correct. A correct fix that does not resolve the symptom means **your hypothesis explained a real fault, but not this one.**

Go back to scoping, which is where the answer was all along.

- **One user or many?** All of the second floor — consistent with the trunk.
- **Has anyone *else* lost the server?** Nobody asked. It turns out the third floor cannot reach it either, and they are on a different switch with a correct trunk.

That second answer eliminates the trunk as the cause of the reported symptom immediately: a fault common to two switches with different configurations is not in either switch.

```
R1# show ip route 10.20.0.0
```

```
% Network not in table
```

Diagnosis. The server’s subnet is not in the routing table. Its route was lost overnight — and *that* is why nobody can reach it, from any floor.

The missing VLAN on SW2’s trunk was a second, genuine, unrelated fault. It had been there for weeks without being noticed, because nothing in VLAN 20 on the second floor had needed the uplink until the server was investigated. Finding it was useful. Believing it was the cause was the error.

The lesson, and it is the one this chapter exists for. Two things went wrong here, and only one of them was reported:

- **The scope was never established.** One question — “is anyone else affected?” — would have ruled out the trunk before it was touched.
- **There was no prediction.** “Adding VLAN 20 will restore access for the second floor” is testable, and it failed. Because no prediction was stated, the failure read as “the fix did not work” rather than as “the hypothesis was wrong”, and the next twenty minutes went into re-checking a trunk that was already correct.

Fix the routing fault. Keep the trunk fix — it was a real defect — but record it separately, as a second issue found during the investigation, so the change record reflects what actually happened.

Lab 33.1 — Faults you did not create (*Packet Tracer or CML, in pairs*)

Platform note. The value here is entirely in *not* knowing the fault, so this lab must be done in pairs or by exchanging saved files. Working alone on a fault you injected yourself teaches nothing.

Topology. Any multi-VLAN, multi-router topology from Parts II and III that both of you have working.

- Task 1.** Each of you takes a working copy and injects **three** faults from different layers — one physical or layer 2, one layer 3, one a service (DHCP, DNS, NAT or an ACL). Record them privately. At least one should be silent, producing no log message.
- Task 2.** Exchange files. Before running a single command, write down your scoping questions and what each possible answer would eliminate.
- Task 3.** Diagnose. For every check, write the prediction *before* you run it, and whether it held. Keep this record — it is the deliverable.
- Task 4.** Fix one fault at a time and re-test the original symptom after each. Note any case where a correct fix did not change the symptom, and what you concluded.
- Task 5.** Time yourself. Compare against your partner on the same set of faults.
- Task 6.** Write the handover you would give if you ran out of time halfway.
- Task 7. Round 2.** Inject three more, but this time include **two faults on the same path** so that fixing one changes nothing. This is the Break & Fix above, and it is the most valuable version of this exercise.
- Task 8. Round 3.** Inject one fault into *one* of twenty-four identically configured ports. Time how long it takes with reasoning versus with a configuration diff.
- Task 9. Extension.** Build a one-page fault-finding card for your own use: the four scoping questions, the five methods, and the symptom index trimmed to what you personally forget.

Checkpoint — Test Yourself

1. Give the seven steps of the cycle and say which two form a loop.
2. Name the five structured methods and give the symptom that selects each.
3. What are the four scoping questions, and what does each eliminate?
4. Why must you state a prediction before making a change?
5. A correct fix does not resolve the reported symptom. What does this tell you, and what should you do next?
6. What are the four things a handover must contain?

Chapter Summary

- Define, gather, analyse, eliminate, hypothesise, test, solve — and loop between hypothesis and test until the prediction holds.
- A hypothesis must predict something that could be wrong. State it before you run the command.
- Five methods: bottom-up for physical, top-down for one application, divide and conquer as the default, follow the path between two points, and compare-to-working whenever an identical working case exists.
- Scope first: one user or many, has it ever worked, what changed, does it fail by name but work by address.

- “What changed” is the highest-yield question, and “nothing” is often wrong. Check `show logging` and the archive diff.
- Record before changing, change one thing, undo what did not help.
- A correct fix that does not resolve the symptom means the hypothesis was wrong — or that there is more than one fault.
- A handover states the symptom, what has been ruled out and how, what was changed, and the next test.

Review Questions

1. **(MCQ)** Which method should you reach for first when eight of twenty-four identically configured ports are failing?
(a) Bottom-up (b) Top-down
(c) Compare working to broken (d) Follow the path
2. **(MCQ)** A user reports the intranet is down. `ping` by IP address succeeds and by name fails. Which method has this already applied, and what does the result establish?
(a) Bottom-up; the cable is fine
(b) Divide and conquer; layers 1–3 work and the fault is DNS
(c) Follow the path; the route is correct
(d) Compare to working; the host is misconfigured
3. **(Short)** Explain why “I think it is the trunk” is not a hypothesis, and rewrite it as one.
4. **(Short)** You fix a genuine misconfiguration and the reported symptom is unchanged. Give the two possible explanations and the next step for each.
5. **(Applied)** A whole floor loses connectivity at 09:10 on a Monday. Write out your first five minutes: the questions you ask, the commands you run, and what each result would eliminate. Do not assume a cause.
6. **(Applied)** You must hand over an unresolved fault at the end of a shift. Write the handover, inventing plausible detail, and mark which part of it the next engineer will find most valuable and why.

Exam Strategy

🕒 Learning Outcomes

By the end of this chapter you will be able to:

- **Recognise** each question type and know what it demands of you.
- **Budget** 120 minutes across a paper you cannot navigate backwards through.
- **Read** a question for its qualifiers before answering it.
- **Plan** the final month of revision against the verb distribution rather than against the chapter list.

🔗 Before You Start

The rest of the book. This chapter assumes you have worked the labs; it is about delivering what you know inside two hours.

📖 Key Terms in This Chapter

multiple choice • drag and drop • simlet • simulation • qualifier • time budget • no back-navigation

⚠️ What this chapter can and cannot tell you

The 120-minute duration and the domain weights come from Cisco's published exam description and are reliable.

Cisco does not publish the number of questions, and does not publish a passing score for this exam. Scores are scaled, and the cut score can change between forms. Anyone quoting you an exact question count or an exact percentage is repeating hearsay, and this book will not add to it.

What follows is therefore built on the facts Cisco does publish plus the arithmetic of a fixed two hours. Before you book, work through Cisco's own free exam tutorial so that the interface is not new on the day, and re-read the current exam policies — they change, and this chapter does not update itself.

34.1 The constraint that shapes everything

⚠️ You cannot go back

Cisco's exam engine does not let you return to a question once you have moved past it. There is no review screen, no flag-for-later, no second pass.

Every habit you built sitting university papers — skim first, do the easy ones, come back to the hard ones — is unavailable. Sitting this exam as though it were a normal paper is the most expensive mistake available to you, and it is entirely avoidable.

Three consequences follow, and they are the whole of exam strategy here:

- **Every question must be answered when you meet it.** Never leave one blank intending to return. You cannot.
- **Never leave one blank at all.** There is no negative marking. An educated guess is worth strictly more than a blank.

- A question you dwell on is stealing time from questions you have not seen yet, and you have no way to know what they are worth.

34.2 Question types

Type	What it is	How to handle it
Multiple choice, one answer	Four or five options, one correct	Eliminate first. Two options are usually wrong on their face, which turns a guess from 25% to 50%
Multiple choice, several answers	“Choose two”, “choose three”	The count is part of the question. Selecting the right number is required; partial credit is not something to rely on
Drag and drop	Match terms to definitions, or order a sequence	Place the ones you are sure of first; the remainder are constrained by what is left
Exhibit-based	A topology, a configuration or show output, then a question	Read the question before the exhibit. Otherwise you study a whole configuration to answer something about one line
Simlet	A working simulated network you inspect with <code>show</code> commands, then several questions about it	Read all the questions first, then gather everything in one pass
Simulation	A network you must actually configure or repair	The expensive ones. See below
AI output evaluation	Output or a recommendation from an AI assistant, which you must assess	New in v2.0 and named in Cisco’s exam description. See Chapter 32

The AI-evaluation questions are not confined to domain 5

Cisco’s exam description says you “may be required to evaluate output and recommendations from agentic AI and digital network assistants to support network operations and troubleshooting activities.”

Read that as: a question anywhere in the paper may show you an assistant’s suggested diagnosis or configuration and ask whether it is right. The skill is the one from Chapter 32 — a plausible, fluent, confidently wrong answer is the characteristic failure — combined with the actual technical knowledge to spot it. Treat such output exactly as you would a suggestion from a colleague you do not know: check it against what the exhibit shows.

34.3 Budgeting 120 minutes

You do not know the question count, but you do know the shape: a majority of short questions and a small number of simulations that cost ten to fifteen minutes each.

Rule	Why
60 to 75 seconds for a standard question	Enough to read carefully and eliminate. If you are past 90 seconds you are already losing
Answer and move at 90 seconds	Pick the best remaining option and go. The question you have not seen may be one you would have answered in twenty seconds
10 to 15 minutes for a simulation	They involve typing, and typing is slow. Budget for them or they take the time from everything else
Check the clock at fixed points , not continuously	Roughly every twenty minutes. Continuous clock-watching costs both time and composure

💡 Simulations: do them properly when you meet them

Because you cannot come back, a simulation cannot be deferred. Skimping on it to save time is a false economy — it is worth several ordinary questions.

- **Read the whole task first.** All of it. Then plan the commands before typing any.
- **Configure exactly what is asked and nothing else.** Extra work risks breaking a check.
- **Verify with show before you leave.** The habit from every chapter of this book applies here and takes thirty seconds.
- **Save the configuration if the task says to** — and consider it even when it does not. `copy running-config startup-config` costs nothing (Chapter 31 makes the same point about `save_when`).
- **If you cannot complete it, complete part of it.** A partially configured task scores more than an untouched one.

34.4 Reading the question

Most wrong answers on questions candidates *knew* come from misreading, and the misreadings are predictable.

- **Qualifiers.** “BEST”, “MOST likely”, “FIRST”, “LEAST”. More than one option is often defensible; the qualifier picks between them. “What should you check first” has a different answer from “what is the cause”.
- **Negatives.** “Which is *not* true”, “which would *fail*”. Slow down; these are where a fluent reader loses marks.
- **The count.** “Choose two” is an instruction, not a hint.
- **The scenario’s constraints.** “Without interrupting existing traffic” or “on IOS XE” is there to eliminate options, and it is usually the whole question — `switchport trunk allowed vlan 50` against ... `vlan add 50` is exactly this (Chapter 10).
- **Exhibit first or question first? Question first**, always. Then read the exhibit for what you now know you need.

⚠️ Answer the question that was asked

The commonest self-inflicted error is answering a slightly different, more interesting question — explaining the cause when asked for the next diagnostic step, or giving the best long-term design when asked what restores service now.

Before selecting, re-read the interrogative. It takes two seconds.

34.5 The last month

Part VI's opener has the number that should organise this: **55% of the blueprint asks you to configure, troubleshoot or diagnose**, and only 21% asks you to describe. Revision that is re-reading prepares you for the smallest portion of the paper.

When	What
Weeks 1–2	Rebuild the labs from Parts II, III and IV from a blank configuration , without looking at the book. Then look, and note every command you had to check. That list is your real revision material
Week 3	Break & Fix only. Work every one in the book with the answer covered, plus the paired fault-injection lab from Chapter 33. Time yourself
Week 4	The <i>describe</i> topics — wireless principles, virtualisation, VPNs, management approaches, AI — because they are recall-heavy and recall decays. Do them last, deliberately
Final days	Appendix A and Appendix B. Subnetting drills until they are automatic (Appendix C). Cisco's exam tutorial. Sleep

💡 Subnetting must be automatic, not calculable

It appears inside other questions — an ACL wildcard, a summary route, whether two addresses are on the same subnet — where it is not the point of the question but stands between you and the answer.

If you are working it out on paper, it is costing you thirty seconds every time it appears. Drill it until the common masks are recall rather than arithmetic. Appendix C exists for this.

34.6 On the day

- Arrive early with the identification the test centre requires; check their rules in advance, and if sitting online, test the machine and the room beforehand.
- Expect a whiteboard or its digital equivalent. Before the clock matters, consider writing down what you know you forget — the subnet boundary table, the eight syslog severities, the STP port states.
- Do the first question properly. Settling into a rhythm early is worth more than the two minutes it costs.
- If a question is genuinely unfamiliar: eliminate, choose, move. You will not think of it later, and later does not exist here.
- If you finish with time in hand, you cannot revisit anything — so there is nothing to do but finish. Spend the time on the questions in front of you instead of hurrying to that point.

⚠ Common Pitfalls

- **Planning to come back to it.** You cannot.
- **Leaving anything blank.** No negative marking.
- **Reading the exhibit before the question.**
- **Missing “choose two”.**
- **Rushing a simulation** to protect time for questions you have not seen.
- **Not verifying a simulation** with a `show` command.
- **Revising by re-reading** when 55% of the paper is hands-on.

- **Learning the interface on the day.** Take the tutorial.
- **Trusting a quoted pass mark or question count.** Cisco publishes neither.

🔧 Break & Fix — A candidate who knew the material and ran out of time.

A candidate rebuilds every lab in this book, works every Break & Fix, and scores well on practice tests. They fail. Their time log, reconstructed afterwards, looks like this.

🔍 Reconstructed# time log

```
00:00 - 00:34   First 18 questions. Comfortable, roughly 110 sec each.
00:34 - 00:48   One OSPF question. Recognised it, wanted to be sure,
                worked the DR election through on the whiteboard.
00:48 - 01:12   Next 12 questions, hurrying now, roughly 120 sec each.
01:12 - 01:31   First simulation. Completed, not verified.
01:31 - 01:52   Next 9 questions. Guessing on several.
01:52 - 02:00   Second simulation reached with 8 minutes left.
                Partially configured. Not saved.
```

Diagnosis. The knowledge was there. Three decisions spent it.

The 14 minutes at 00:34. One question, already answered correctly in their head, verified at the cost of roughly twelve other questions' worth of time. The instinct — be certain before committing, because you cannot come back — is understandable and exactly backwards: you cannot come back, so the cost of over-investing is unrecoverable too.

The hurrying that followed. Every question from 00:48 onward was answered under self-inflicted pressure created fourteen minutes earlier. Time lost early is not repaid; it is paid for by everything after it.

The second simulation. Reached with eight minutes left because of the first two decisions, and worth several times an ordinary question. Partially configured and not saved — so much of the work that was done scored nothing.

What should have happened. At 00:44 — ten minutes into that OSPF question — the correct move was to select the best answer available and go. It was probably right. Even if it was wrong, one mark is cheaper than twelve.

The rule to take in with you. No single question is worth more than about ninety seconds. Not the one you nearly know, not the one you are annoyed by, not the one you could definitely get with a little longer. Set that limit before you sit down, because you will not set it fairly at 00:40 with a half-solved problem in front of you.

🏠 Lab 34.1 — A full-length timed rehearsal (*Your own material, a timer, and no interruptions*)

Why this is the last lab in the book. You have practised the material. You have not practised the *conditions* — two hours, no going back, no notes, no pausing. Those are a separate skill and they are what the Break & Fix above is about.

Task 1. Assemble a paper: about 90 questions from the review sections of this book, weighted to the domain percentages — roughly a quarter each from Parts I and II, a fifth each from Parts III and IV, a tenth from Part V.

Task 2. Add two configuration tasks from the labs, to stand in for simulations.

Task 3. Set the rules and keep them. 120 minutes. No notes. **No returning to a previous question** — cover them, or have someone else administer it. This constraint is the entire point of the exercise.

Task 4. Sit it in one block, at the time of day you have booked the real exam.

Task 5. Record a timestamp every ten questions. Do not analyse during.

Task 6. Afterwards, mark the timing before you mark the answers. Find every question that took more than 90 seconds and count what they cost collectively. This number is usually a surprise.

Task 7. Now mark the answers. Separate the wrong ones into *did not know* and *knew but misread or rushed*. They need completely different remedies, and the second pile is usually bigger than expected.

Task 8. For every “did not know”, find the chapter and redo its lab — not its summary.

Task 9. Repeat two weeks later with a different selection. Compare the timing profile, not just the score. Improvement in timing is the better predictor.

Checkpoint — Test Yourself

1. What is the single most important structural fact about the exam interface, and name three habits it invalidates?
2. How long should a standard question take, and what do you do at the limit?
3. Why can a simulation not be deferred, and what four things should you do in one?
4. Read the exhibit or the question first? Why?
5. Which topics should be revised last, and why?
6. What does Cisco publish about this exam, and what does it not?

Chapter Summary

- **You cannot return to a question.** Answer everything when you meet it, never leave a blank, and never plan to come back.
- Roughly 60–75 seconds per standard question; hard stop at 90. Budget 10–15 minutes for each simulation.
- No single question is worth more than ninety seconds. Time lost early is never repaid.
- Question before exhibit. Watch for qualifiers, negatives and the required count.
- In a simulation: read it all, plan, configure only what is asked, verify with **show**, save. Partial completion beats none.
- v2.0 may ask you to evaluate an AI assistant’s output anywhere in the paper, not only in domain 5.
- Revise against the verbs: 55% hands-on, so spend the time at a command line. Leave the *describe* topics until last, and make subnetting automatic.
- Cisco publishes 120 minutes and the domain weights. It does not publish the question count or the pass mark. Take the official tutorial before the day.

Review Questions

1. (MCQ) You are unsure of a question and have spent 90 seconds on it. What should you do?
 - (a) Flag it and return at the end
 - (b) Eliminate what you can, choose the best remaining option, and move on
 - (c) Leave it blank to avoid a wrong answer
 - (d) Skip ahead and come back if time allows
2. (MCQ) You reach a simulation with 25 minutes remaining and several questions still to come. What is the best approach?
 - (a) Skip it to protect time for the remaining questions
 - (b) Complete it properly, verify, and accept the time cost

- (c) Configure quickly without verifying
 - (d) Answer the questions after it first, then return
3. **(Short)** Explain why “I will come back to it” is the most expensive assumption you can carry into this exam.
4. **(Short)** Give three question-wording traps and an example of each.
5. **(Applied)** Write your own revision plan for the four weeks before your exam date, justified against the verb distribution rather than the chapter order. State what you will *not* spend time on and why.
6. **(Applied)** Set your personal per-question time limit and write down what you will do when you hit it on a question you believe you can solve. Be specific — this is the decision the Break & Fix in this chapter turns on, and it must be made in advance.

Appendices

Command Reference

Grouped by the job you are doing, not alphabetically, because that is how you will look them up. Commands are shown in the mode they are entered in; the chapter that teaches each group is named beside it.

Two conventions used throughout

Angle brackets mark something you supply: `<vlan-id>`. A command shown indented belongs in the sub-mode above it.

These are written for IOS and IOS XE and have not been verified on live hardware. Run each one on your own platform before relying on it, and check interface naming — IOS XE uses `GigabitEthernet0/0/0`, classic IOS `GigabitEthernet0/0`, and a Catalyst switch `GigabitEthernet1/0/1`.

A.1 Getting on the box (Chapter 7, Chapter 23)

Command	What it does
<code>enable</code>	Privileged EXEC mode
<code>configure terminal</code>	Global configuration mode
<code>hostname <name></code>	Sets the name, and is required before RSA keys
<code>ip domain name <domain></code>	Also required before RSA keys
<code>crypto key generate rsa modulus 2048</code>	Generates the SSH host key
<code>ip ssh version 2</code>	Refuses SSHv1
<code>username <u> privilege 15 algorithm-type</code> <code>script secret <p></code>	A local user with a type-9 hash
<code>enable algorithm-type script secret <p></code>	The enable secret, hashed
<code>line console 0</code> <code>login local</code> <code>exec-timeout 10 0</code> <code>logging synchronous</code>	Console: authenticate locally, time out after ten minutes, and stop log messages scrambling what you type
<code>line vty 0 15</code> <code>login local</code> <code>transport input ssh</code> <code>access-class <acl> in</code>	Remote access by SSH only. Without the last line Telnet still works
<code>no ip domain lookup</code>	Restricts which hosts may connect Stops a mistyped command hanging the CLI
<code>copy running-config startup-config</code>	Saves. Also write memory

A.2 Interfaces (Chapter 2, Chapter 10)

Command	What it does
<code>interface <type><n></code>	Enter interface configuration
<code>interface range <type><a> - </code>	Several at once
<code>description <text></code>	Free text. Costs nothing, saves hours
<code>ip address <addr> <mask></code>	IPv4 address
<code>ipv6 address <addr>/<len></code>	IPv6 address
<code>ipv6 address <prefix>/64 eui-64</code>	Host portion from the MAC
<code>no shutdown</code>	Enables it. Router interfaces are down by default
<code>duplex auto / speed auto</code>	Negotiate. Set both ends the same way or neither
<code>no switchport</code>	Makes a switch port a routed port
<code>interface vlan <id></code>	An SVI
<code>ip routing</code>	Enables routing on a layer 3 switch. Without it, SVIs do not route

A.3 VLANs, trunks and edge ports (Chapter 8–Chapter 10)

Command	What it does
<code>vlan <id></code>	Creates and names a VLAN
<code>name <name></code>	
<code>switchport mode access</code>	Fixes the port as an access port. Before assigning a VLAN
<code>switchport access vlan <id></code>	Puts it in that VLAN
<code>switchport voice vlan <id></code>	Adds a tagged voice VLAN for a phone
<code>switchport trunk encapsulation dot1q</code>	Required on platforms that support ISL
<code>switchport mode trunk</code>	Fixes the port as a trunk
<code>switchport trunk allowed vlan <list></code>	Replaces the list. This is an outage waiting to happen
<code>switchport trunk allowed vlan add <list></code>	Adds to it. Almost always what you meant
<code>switchport trunk native vlan <id></code>	Sets the untagged VLAN. Must match both ends
<code>switchport nonegotiate</code>	Stops DTP frames
<code>power inline auto / never</code>	PoE on or off for the port

A.4 EtherChannel and spanning tree (Chapter 11, Chapter 12)

Command	What it does
<code>channel-group <n> mode active</code>	LACP, actively negotiating
<code>channel-group <n> mode passive</code>	LACP, responding only. Two passive ends never form

Command	What it does
<code>channel-group <n> mode on</code>	No negotiation. Both ends must be on
<code>interface port-channel <n></code>	Configure the bundle, not its members
<code>spanning-tree mode rapid-pvst</code>	Rapid PVST+
<code>spanning-tree vlan <id> root primary</code>	Lowers the priority enough to win
<code>spanning-tree vlan <id> priority <n></code>	Explicit priority, in steps of 4096
<code>spanning-tree portfast</code>	Access port goes straight to forwarding
<code>spanning-tree portfast trunk</code>	The same on a trunk to a host or hypervisor
<code>spanning-tree bpduguard enable</code>	Err-disables the port if a BPDU arrives
<code>spanning-tree guard root</code>	Refuses a superior BPDU on this port
<code>spanning-tree guard loop</code>	Protects against a link that stops sending BPDUs

A.5 Routing (Chapter 16–Chapter 19)

Command	What it does
<code>ip route <net> <mask> <next-hop></code>	Static route
<code>ip route 0.0.0.0 0.0.0.0 <next-hop></code>	Default route
<code>ip route <net> <mask> <next-hop> <ad></code>	Floating static, for backup
<code>ipv6 unicast-routing</code>	Enables IPv6 forwarding. Nothing routes without it
<code>ipv6 route <prefix>/<len> <interface> <link-local></code>	IPv6 static via a link-local next hop — the interface is required
<code>router ospf <pid></code> <code>router-id <a.b.c.d></code> <code>network <addr> <wildcard> area <n></code> <code>passive-interface <int></code>	OSPFv2. The wildcard is inverted; passive advertises without sending hellos
<code>ipv6 router ospf <pid></code> <code>router-id <a.b.c.d></code>	OSPFv3. The router ID is mandatory on an IPv6-only router
<code>ipv6 ospf <pid> area <n></code>	Enables OSPFv3 <i>on the interface</i> . There are no network statements
<code>ip ospf network point-to-point</code>	No DR election on a two-router link
<code>standby <grp> ip <vip></code> <code>standby <grp> priority <n></code> <code>standby <grp> preempt</code>	HSRP. Without preempt a recovered router stays standby
<code>vrrp <grp> ip <vip></code>	VRRP. Preempts by default

A.6 Services (Chapter 20, Chapter 22)

Command	What it does
<code>ip dhcp excluded-address <first> <last></code>	Reserves addresses. Enter this before the pool
<code>ip dhcp pool <name></code> <code>network <net> <mask></code> <code>default-router <addr></code> <code>dns-server <addr></code> <code>lease <days> <hours> <mins></code> <code>ip helper-address <server></code>	A pool. The network statement must match the serving interface's subnet exactly
<code>ip address dhcp</code> <code>ip nat inside / ip nat outside</code>	Relay. Goes on the interface facing the clients The router leases its own address Marks the NAT boundary. Both are required or nothing is translated
<code>ip nat inside source static <local> <global></code> <code>ip nat inside source static tcp <local> <port> <global> <port></code>	Static one-to-one NAT Port forwarding
<code>ip nat inside source list <acl> interface <int> overload</code> <code>clear ip nat translation *</code>	PAT onto the interface address. overload is the keyword that matters After every change, or you test stale entries

A.7 Security (Chapter 24, Chapter 25, Chapter 23)

Command	What it does
<code>access-list <1-99> {permit deny} <src> <wildcard></code>	Numbered standard ACL
<code>access-list <100-199> {permit deny} <proto> <src> <sw> <dst> <dw> eq <port></code>	Numbered extended ACL
<code>ip access-list {standard extended} <name></code>	Named ACL, with sequence numbers you can edit one at a time
<code>ip access-group <acl> {in out}</code> <code>switchport port-security</code> <code>switchport port-security maximum <n></code> <code>switchport port-security mac-address sticky</code> <code>switchport port-security violation {protect restrict shutdown}</code>	Applies it to an interface Port security. Needs switchport mode access first. Use 3 for a phone plus PC
<code>errdisable recovery cause psecure-violation</code> <code>errdisable recovery interval <sec></code>	Automatic recovery. Manual is shutdown then no shutdown
<code>ip dhcp snooping</code> <code>ip dhcp snooping vlan <list></code> <code>no ip dhcp snooping information option</code> <code>ip dhcp snooping trust</code>	Snooping. Trust the uplink, and expect to disable option 82
<code>ip arp inspection vlan <list></code> <code>ip arp inspection trust</code>	DAI. Requires snooping first

Command	What it does
arp access-list <name> permit ip host <addr> mac host <mac> ip arp inspection filter <name> vlan <id> storm-control broadcast level <rise> <fall> storm-control action {trap shutdown}	Permits statically addressed hosts, which have no snooping binding Storm control
ipv6 nd raguard policy <name> device-role {host router} ipv6 nd raguard attach-policy <name>	RA guard
aaa new-model aaa authentication login default group <grp> local	AAA with a local fallback. Never omit the trailing local
tacacs server <name> address ipv4 <addr> key <secret>	A TACACS+ server
ip tacacs source-interface Loopback0	A stable source address

A.8 Management and files (Chapter 30, Chapter 23)

Command	What it does
service timestamps log datetime msec localtime show-timezone	Do this first, everywhere. Otherwise messages carry only uptime
logging buffered <size> informational	Local buffer
logging console warnings	Keeps the CLI usable
logging host <addr> logging trap notifications	Central server, and its severity threshold. Nothing to do with SNMP traps
logging source-interface Loopback0	A stable source address
snmp-server user <u> <grp> v3 auth sha <p> priv aes 128 <p>	SNMPv3 authPriv
snmp-server host <addr> version 3 priv <user>	Where notifications go
cdp run / no cdp run	CDP globally
lldp run	LLDP is off by default
ip scp server enable	SCP server. Also needs <code>aaa new-model</code> , an authentication method and <code>aaa authorization exec</code>
copy sftp: flash:	Fetch a file. IOS is an SFTP client, not a server
verify /md5 flash:<image>	Check an image before reloading onto it
archive path scp://<user>@<host>/<prefix> write-memory	Off-box backup on every save

The show Command Cookbook

Eight of the twenty-nine blueprint topics begin *Troubleshoot* or *Diagnose*, and topic 2.4 names `show` commands explicitly. This appendix is organised by **the question you are asking**, because that is what you have when a fault starts — not by command name, which is what you have afterwards.

Each entry gives the question, the command, and **the specific field to read**. Running a command and skimming its output is not diagnosis; knowing which line answers your question is.

💡 Four modifiers that make all of these faster

- `| include <text>` — only lines containing it.
- `| exclude <text>` — everything except those lines.
- `| begin <text>` — start output at the first match, and continue.
- `| section <text>` — the whole configuration block matching it.

`show running-config | section interface GigabitEthernet1/0/5` is the fastest way to see one port's configuration, and `show interfaces status | exclude connected` shows only the ports that are not working — which on a 48-port switch is the difference between reading four lines and reading forty-eight.

B.1 Is the interface healthy? (Chapter 2)

Question	Command	Read this
Which ports are up, and with what address?	<code>show ip interface brief</code>	Status and Protocol. Method says DHCP or manual
Which ports are <i>not</i> working?	<code>show interfaces status exclude connected</code>	<code>notconnect</code> , <code>err-disabled</code> , <code>disabled</code>
Is the physical link clean?	<code>show interfaces <int></code>	input errors, CRC, late collision, runts. CRC with no collisions suggests cable or duplex
Is duplex mismatched?	<code>show interfaces <int> include duplex</code>	Half duplex on a switch-to-switch link is almost always wrong
Why is the port err-disabled?	<code>show interfaces status err-disabled</code>	The Reason column names the feature that did it
Is the optic within spec?	<code>show interfaces transceiver</code>	Rx power against the stated range

B.2 Is layer 2 right? (Chapter 8–Chapter 12)

Question	Command	Read this
Which VLAN is this port in?	<code>show vlan brief</code>	The port should be listed against the VLAN you expect
Does the VLAN exist at all?	<code>show vlan brief</code>	A missing VLAN is a common cause of a dead access port
Is the trunk carrying what I think?	<code>show interfaces trunk</code>	Vlans allowed and active — not the configured list. This is the field that matters
Do the native VLANs match?	<code>show interfaces trunk</code>	Native vlan, on both ends. A mismatch is logged by CDP
Is the EtherChannel up?	<code>show etherchannel summary</code>	(P) in the bundle, (I) independent, (s) suspended
Who is the root bridge?	<code>show spanning-tree vlan <id></code>	This bridge is the root, or the root's ID and the port cost to it
Which ports are blocking?	<code>show spanning-tree vlan <id> include BLK</code>	Any unexpected blocking port
Is there a loop?	<code>show logging include MACFLAP</code>	A MAC flapping between two ports
Where is this MAC?	<code>show mac address-table address <mac></code>	The port it was last seen on

B.3 What is attached? (Chapter 13)

Question	Command	Read this
What Cisco device is on this port?	<code>show cdp neighbors</code>	Local port, remote device, remote port
What is its address and version?	<code>show cdp neighbors detail</code>	Management address, platform, IOS version
What non-Cisco device is attached?	<code>show lldp neighbors</code>	LLDP must be enabled first — it is off by default

B.4 Where does traffic go? (Chapter 15–Chapter 19)

Question	Command	Read this
How will this packet be routed?	<code>show ip route <destination></code>	The single entry that would be used — longest prefix match already applied
What does the table hold?	<code>show ip route</code>	Codes, then [AD/metric] on each entry
Is there a default route?	<code>show ip route include 0.0.0.0</code>	Gateway of last resort
Are OSPF neighbours up?	<code>show ip ospf neighbor</code>	FULL is healthy. EXSTART means MTU mismatch

Question	Command	Read this
Why will they not form?	<code>show ip ospf interface <int></code>	Area, hello and dead timers, network type — all must match
Which OSPF interfaces are running?	<code>show ip ospf interface brief</code>	Cost, state, neighbour count
Is IPv6 routing working?	<code>show ipv6 route ospf</code>	Next hops are fe80:: link-local, and that is correct
Which router is the active gateway?	<code>show standby brief</code>	Active, Standby, priority, P for preempt
Same for VRRP?	<code>show vrrp brief</code>	Master or Backup

B.5 Are the services working? (Chapter 20, Chapter 22, Chapter 24)

Question	Command	Read this
Did anyone get a lease?	<code>show ip dhcp binding</code>	Address, MAC, expiry
Is the pool exhausted?	<code>show ip dhcp pool</code>	Leased against Total addresses
Is something using a pool address statically?	<code>show ip dhcp conflict</code>	Conflicts stay out of the pool until cleared
Is NAT translating?	<code>show ip nat translations</code>	Pro column: tcp means PAT; -- means static or dynamic
Is NAT even being attempted?	<code>show ip nat statistics</code>	Inside interfaces and Outside interfaces. Empty means the markers are missing. Hits: 0, Misses: 0 means no packet was considered
Is the ACL dropping things?	<code>show access-lists</code>	The match counters. Zero on the last line means everything is being dropped; zero in the middle means a shadowed rule
Is the ACL even applied?	<code>show ip interface <int> include access list</code>	Inbound and outbound, named separately

B.6 What is the security state? (Chapter 25)

Question	Command	Read this
Is port security refusing something?	<code>show port-security interface <int></code>	Security Violation Count on a Secure-up port usually means the maximum is too low
Which ports have violations?	<code>show port-security</code>	Violation counts across the switch

Question	Command	Read this
Is snooping building bindings?	<code>show ip dhcp snooping binding</code>	Empty for a device means it is statically addressed — which breaks DAI
Is DAI dropping ARP?	<code>show ip arp inspection statistics vlan <id></code>	DHCP Drops means no binding; ACL Drops means an ARP ACL denied it
Is a port suppressing a storm?	<code>show storm-control broadcast</code>	Blocking means it is suppressing now

B.7 What happened? (Chapter 30)

Question	Command	Read this
Why does the log show nothing?	<code>show logging include level</code>	Run this before believing an empty log. A threshold of errors discards severity 4 to 7, which is most of what explains a fault
What happened recently?	<code>show logging</code>	Newest at the bottom
Did somebody change something?	<code>show logging include CONFIG</code>	%SYS-5-CONFIG_I — somebody saved a change
Are there unsaved changes?	<code>show archive config differences</code> <code>nvram:startup-config</code> <code>system:running-config</code>	Any output at all means unsaved changes. Make this the last step of every change
How long has it been up?	<code>show version</code>	uptime, and the reason for the last reload
Is the SNMP server reachable?	<code>show aaa servers</code> <code>show snmp</code>	State and duration

B.8 The ten to know cold

If you learn nothing else in this appendix, learn these. Between them they answer the first question in almost every fault in this book.

Command	Answers
<code>show ip interface brief</code>	Is it up, and what address does it have?
<code>show interfaces status</code>	What is plugged in, and what is broken?
<code>show vlan brief</code>	Is this port in the VLAN I think?
<code>show interfaces trunk</code>	Is the trunk carrying the VLAN?
<code>show ip route <destination></code>	Where would this packet actually go?
<code>show ip ospf neighbor</code>	Is the routing protocol healthy?
<code>show cdp neighbors</code>	What is on the other end of this cable?
<code>show logging</code>	What happened, and when?

Command	Answers
<code>show running-config section <thing></code>	What is this one thing configured as?
<code>show access-lists</code>	Is a filter responsible?

⚠ The habit that makes all of this work

Run these on a *working* network first, and read the output until it is familiar. You cannot recognise abnormal output if you have never looked at normal output, and the exam will show you a `show` dump and expect you to spot the one wrong field in it.

Every lab in this volume asks you to record normal output before breaking anything. That instruction is what this appendix is for.

Subnetting and IPv6 Drills

Subnetting appears on the exam mostly *inside* other questions — an ACL wildcard, a summary route, whether two hosts share a subnet. It is rarely the point of the question, which means every second you spend calculating is a second taken from the question you were actually asked.

The standard to aim for

Any /24 to /30 question, answered in **under fifteen seconds, without writing anything down**. That is achievable by almost anyone with about two hours of drilling, and it is worth more marks than any single chapter of this book.

The drills in this appendix have their answers computed rather than typed. If your answer differs from one here, work it through again — the table is right.

C.1 The method

Forget binary conversion. There is one number to find.

The block size, or “magic number”

256 minus the interesting octet of the mask. Subnets begin at multiples of that number, in that octet.

Worked Example: 192.168.10.77/26

Step 1 — the mask. /26 is 24 bits plus 2, so the fourth octet has 2 bits: $128 + 64 = 192$. Mask 255.255.255.192. The fourth octet is the interesting one.

Step 2 — the block size. $256 - 192 = 64$.

Step 3 — count in blocks until you pass the address. 0, 64, 128, 192. The address ends in 77, which falls between 64 and 128, so the network is **192.168.10.64**.

Step 4 — the rest follows. The next block starts at 128, so the broadcast is one below it: **192.168.10.127**. Usable hosts are everything between: **.65 to .126**, which is $64 - 2 = 62$. Four steps, no binary, and with the two tables below memorised it is arithmetic you can do while reading the question.

C.2 The two tables to memorise

Mask octet	Block size	Bits	Subnets it makes in that octet
128	128	1	0, 128
192	64	2	0, 64, 128, 192
224	32	3	0, 32, 64, 96, 128, 160, 192, 224
240	16	4	0, 16, 32, 48, ...
248	8	5	0, 8, 16, 24, ...
252	4	6	0, 4, 8, 12, ...
254	2	7	0, 2, 4, 6, ...
255	1	8	every value

Prefix	Mask	Block	Usable	Typical use
/30	255.255.255.252	4	2	A point-to-point link
/29	255.255.255.248	8	6	A handful of servers
/28	255.255.255.240	16	14	A small office
/27	255.255.255.224	32	30	A classroom
/26	255.255.255.192	64	62	A floor
/25	255.255.255.128	128	126	Half a /24
/24	255.255.255.0	256	254	The default assumption
/23	255.255.254.0	2 (3rd)	510	Two /24s
/22	255.255.252.0	4 (3rd)	1022	A building
/21	255.255.248.0	8 (3rd)	2046	A campus block
/20	255.255.240.0	16 (3rd)	4094	A site

💡 Wildcard masks are the same table, inverted

Subtract each octet of the mask from 255 (Chapter 24). /26 is 255.255.255.192, so the wildcard is 0.0.0.63 — and 63 is one less than the block size of 64, which is the quickest way to produce it.

Check it every time: network address plus wildcard should equal the broadcast address. $192.168.10.64 + 0.0.0.63 = 192.168.10.127$. If it does not, the wildcard is wrong.

C.3 Drill 1 — network, broadcast and range

For each, give the mask in dotted decimal, the network address, the broadcast address, the usable range and the host count. Answers in Section C.7.

	Given	Network	Broadcast	Usable hosts
1	192.168.10.77/26			
2	10.4.19.200/21			
3	172.16.35.14/20			
4	192.168.1.130/25			
5	10.0.0.66/30			
6	172.20.140.9/22			
7	192.168.200.35/27			
8	10.55.12.3/19			
9	172.31.255.200/18			
10	192.168.16.62/28			
11	10.128.64.31/27			
12	172.16.0.129/29			
13	203.0.113.45/29			
14	198.51.100.17/28			
15	192.0.2.200/26			
16	10.10.10.10/23			
17	172.18.6.250/21			

	Given	Network	Broadcast	Usable hosts
18	192.168.99.99/24			

C.4 Drill 2 — sizing a subnet

The other direction, and the one VLSM questions need: given a host requirement, find the smallest prefix that fits. Work each out before reading the row.

Hosts needed	Prefix	Mask	Usable / wasted
2	/30	255.255.255.252	2 / 0
6	/29	255.255.255.248	6 / 0
12	/28	255.255.255.240	14 / 2
25	/27	255.255.255.224	30 / 5
50	/26	255.255.255.192	62 / 12
100	/25	255.255.255.128	126 / 26
200	/24	255.255.255.0	254 / 54
500	/23	255.255.254.0	510 / 10
1000	/22	255.255.252.0	1022 / 22

⚠ The off-by-one that costs marks

A requirement for 30 hosts needs a /26, not a /27. A /27 has 32 addresses and **30 usable** — so it fits exactly, with no room for the gateway you forgot to count.

Count the gateway, and count any HSRP virtual address (Chapter 19) as well: a subnet with two routers running HSRP consumes three addresses before a single user appears.

C.5 Drill 3 — IPv6 compression

Apply both rules: drop leading zeros in each group, and replace *one* run of consecutive all-zero groups with ::.

	Given	Compressed
1	2001:0db8:0000:0000:0000:ff00:0042:8329	
2	2001:0db8:acad:0010:0000:0000:0000:0001	
3	fe80:0000:0000:0000:0204:61ff:fe9d:f156	
4	2001:0db8:0000:0001:0000:0000:0000:0100	
5	0000:0000:0000:0000:0000:0000:0000:0001	
6	2001:0db8:aaaa:0001:0000:0000:0000:0000	

💡 The rule people get wrong

:: may appear **once**. 2001:db8::1::5 is not a valid address, because a parser cannot tell how many zero groups each :: stands for.

Where there are two runs of zeros, compress the *longer* one and write the shorter out in full. If they are equal in length, compress the first.

C.6 Drill 4 — EUI-64

Build the full address from the prefix and the MAC. The procedure is: split the MAC in half, insert `fffe` in the middle, and **flip the seventh bit of the first octet**.

	Prefix	MAC	EUI-64 address
1	2001:db8:acad:10::/64	00:1a:2b:3c:4d:5e	
2	2001:db8:1:1::/64	aa:bb:cc:dd:ee:ff	
3	fe80::/64	0c:1d:2e:3f:40:51	
4	2001:db8:99::/64	52:54:00:12:34:56	

Worked Example: Why the seventh bit flips, and how to do it fast

That bit is the universal/local flag. In a MAC address it is 0 for a manufacturer-assigned address; EUI-64 inverts it, so a burned-in address becomes 1.

In practice you are flipping the value 2 in the second hex digit of the first octet. 00 becomes 02. aa becomes a8. 52 becomes 50. 0c becomes 0e.

If the second hex digit is even, add 2; if it is odd... it will not be, for a normal MAC. Learn the four cases above and you have covered nearly everything you will be shown.

C.7 Answers

C.7.1 Drill 1

	Mask	Network	Broadcast	Usable range (count)
1	255.255.255.192	192.168.10.64	192.168.10.127	192.168.10.65 to 192.168.10.126 (62)
2	255.255.248.0	10.4.16.0	10.4.23.255	10.4.16.1 to 10.4.23.254 (2046)
3	255.255.240.0	172.16.32.0	172.16.47.255	172.16.32.1 to 172.16.47.254 (4094)
4	255.255.255.128	192.168.1.128	192.168.1.255	192.168.1.129 to 192.168.1.254 (126)
5	255.255.255.252	10.0.0.64	10.0.0.67	10.0.0.65 to 10.0.0.66 (2)
6	255.255.252.0	172.20.140.0	172.20.143.255	172.20.140.1 to 172.20.143.254 (1022)
7	255.255.255.224	192.168.200.32	192.168.200.63	192.168.200.33 to 192.168.200.62 (30)
8	255.255.224.0	10.55.0.0	10.55.31.255	10.55.0.1 to 10.55.31.254 (8190)
9	255.255.192.0	172.31.192.0	172.31.255.255	172.31.192.1 to 172.31.255.254 (16382)
10	255.255.255.240	192.168.16.48	192.168.16.63	192.168.16.49 to 192.168.16.62 (14)

	Mask	Network	Broadcast	Usable range (count)
11	255.255.255.224	10.128.64.0	10.128.64.31	10.128.64.1 to 10.128.64.30 (30)
12	255.255.255.248	172.16.0.128	172.16.0.135	172.16.0.129 to 172.16.0.134 (6)
13	255.255.255.248	203.0.113.40	203.0.113.47	203.0.113.41 to 203.0.113.46 (6)
14	255.255.255.240	198.51.100.16	198.51.100.31	198.51.100.17 to 198.51.100.30 (14)
15	255.255.255.192	192.0.2.192	192.0.2.255	192.0.2.193 to 192.0.2.254 (62)
16	255.255.254.0	10.10.10.0	10.10.11.255	10.10.10.1 to 10.10.11.254 (510)
17	255.255.248.0	172.18.0.0	172.18.7.255	172.18.0.1 to 172.18.7.254 (2046)
18	255.255.255.0	192.168.99.0	192.168.99.255	192.168.99.1 to 192.168.99.254 (254)

C.7.2 Drill 3

	Given	Compressed
1	2001:0db8:0000:0000:0000:ff00:0042:8329	2001:db8::ff00:42:8329
2	2001:0db8:acad:0010:0000:0000:0000:0001	2001:db8:acad:10::1
3	fe80:0000:0000:0000:0204:61ff:fe9d:f156	fe80::204:61ff:fe9d:f156
4	2001:0db8:0000:0001:0000:0000:0000:0100	2001:db8:0:1::100
5	0000:0000:0000:0000:0000:0000:0000:0001	::1
6	2001:0db8:aaaa:0001:0000:0000:0000:0000	2001:db8:aaaa:1::

C.7.3 Drill 4

	Prefix	MAC	EUI-64 address
1	2001:db8:acad:10::/64	00:1a:2b:3c:4d:5e	2001:db8:acad:10:21a:2bff:fe3c:4d5e
2	2001:db8:1:1::/64	aa:bb:cc:dd:ee:ff	2001:db8:1:1:a8bb:ccff:fedd:eeff
3	fe80::/64	0c:1d:2e:3f:40:51	fe80::e1d:2eff:fe3f:4051
4	2001:db8:99::/64	52:54:00:12:34:56	2001:db8:99:0:5054:ff:fe12:3456

⚠ Regenerating these

Every answer above is computed by `tools/gen_drills.py` using Python's `ipaddress` module, not worked out by hand. To change or extend the drills, edit the lists at the top of that script and re-run it — do not edit the tables here, because a hand-edited answer is exactly the error this arrangement exists to prevent.

Blueprint Coverage Matrix

Every topic in Cisco’s published exam-topics document for CCNA 200-301 version 2.0, and the chapter of this volume that teaches it. The table is generated from the `\bp{}` tags in the chapter sources and from the PDF itself, so it cannot drift from either.

Topic	Cisco’s wording	Chapter
Domain 1.0 — Network Infrastructure and Connectivity		25%
1.1	Diagnose interface and cable (copper and fiber) issues such as collisions, errors, mismatched duplex, speed, distance, interface, signal levels, pin out, and cable types	2, 3
1.2	Describe the role and function of hypervisors, virtual machines, and containers	29
1.3	Troubleshoot IPv4 address configuration, assignment, and subnetting (public and private)	4
1.4	Troubleshoot IPv6 address configuration, assignment, and prefix sizing (unicast and modified EUI 64)	5
1.5	Describe wireless principles	27
1.6	Troubleshoot wired and wireless client connectivity (IP configuration, network reachability, and wireless security parameters on Windows, MacOS, and Linux)	28
1.7	Troubleshoot DHCPv4 client, server, and relay on IOS devices	20
Domain 2.0 — Switching and Network Access		25%
2.1	Configure network infrastructure connectivity (switch-to-switch and switch-to-router)	10, 11
2.2	Configure Layer 2 switch port attributes for edge-host connectivity (VLAN, PoE, port channel, and LACP)	8, 9
2.3	Validate the accuracy of network documentation using CDP and LLDP	13
2.4	Troubleshoot basic Layer 2/Layer 3 connectivity and device operations using show commands (including show logs), ping, extended ping, trace route, and packet capture output	6, 7, 14, 33
2.5	Configure operations of the Rapid Per VLAN Spanning Tree Protocol (Rapid PVST+)	12
Domain 3.0 — IP Routing		20%
3.1	Interpret a routing table to identify the next hop for a packet (routing protocol, prefix/mask, administrative distance, metric, and default route)	15
3.2	Troubleshoot IPv4 and IPv6 static routing	16

Topic	Cisco's wording	Chapter
3.3	Configure single area OSPFv2 for IPv4 and OSPFv3 for IPv6	17, 18
3.4	Interpret the operational status of First Hop Redundancy Protocols (HSRP and VRRP)	19
Domain 4.0 — Network Services and Security 20%		
4.1	Configure network devices with local usernames and as an AAA client (TACACS+ and RADIUS) for management	23
4.2	Manage device configuration and software files using secure file transfer operations with SFTP/SCP	23
4.3	Configure NAT/PAT on IOS XE routers	22
4.4	Diagnose issues with DNS records (A, AAAA, CNAME, MX, NS, and PTR) to support host, web application, and mail server access by name	21
4.5	Describe IPsec remote access and site-to-site VPNs (protocols and transport modes)	26
4.6	Configure IPv4 access control lists (standard, extended, numbered, and named ACLs)	24
4.7	Configure Layer 2 security features	25
Domain 5.0 — AI, and Network Operations and Management 10%		
5.1	Describe the role of agentic AI in network operations	32
5.2	Select a prompt to send to a generative AI system to support network operations considering prompt components such as data classification, output format, persona, and instructions	32
5.3	Describe network management approaches (device-based, cloud-based, controller-based, automation-based, and infrastructure as code)	30
5.4	Describe the function of SNMP in network operations	30
5.5	Use configuration management mechanisms such as Ansible to execute commands	31
5.6	Interpret syslog message content, severity levels, and facilities	30

Checkpoint Answers

Answers to the **Checkpoint** questions in each chapter. The Review Questions at the end of each chapter are deliberately not answered here — they are for assessment, and several have more than one defensible answer.

Use these to check, not to learn

A checkpoint question you read the answer to teaches almost nothing. Write your answer down first, then compare. Where yours differs, the useful question is not “was I right” but “what did I not consider”.

Part I — Foundations

Chapter 1 — Networks, Models and This Exam

1. Layers 1 to 3 are ruled out: something reached the server and came back. Suspect layer 7 first — specifically name resolution or the web service itself. Test the name (Chapter 21) and the port (Chapter 6).
2. Layer 1. Rising collisions on a link that reports full duplex means a duplex mismatch: one end is half duplex and is correctly reporting collisions the other end does not see.
3. Because they are different jobs. A switch’s flooding is a *recovery* mechanism — the destination is probably on one of those ports and will answer, teaching the switch where it is. A router’s unknown network has no such prospect; flooding a packet to every interface would loop and multiply. So a router drops and reports.

Chapter 2 — The Physical Layer and Cable Faults

1. That the two ends disagree about duplex. It is certain because a full duplex link *cannot* have collisions — there is no shared medium to collide on. The counter can only be incrementing because the local end is actually running half duplex, or because the far end is and is causing them.
2. In order: a dirty or damaged connector; the wrong optic or wavelength for the distance; and excessive length or a bad splice. Check the cheapest first — clean the connector before ordering hardware.
3. Because it is unambiguous. Administratively down is a decision somebody made, recorded in the configuration, and reversible with one command. A port that is up with rising CRC errors is intermittently working, so traffic is partly passing, users report “slow” rather than “down”, and the cause could be cable, connector, optic, EMI or duplex.

Chapter 3 — Ethernet, Frames and Switching

1. It drops it. The behaviour is called **filtering**: the destination is known to be reachable through the same port the frame arrived on, so forwarding it would send it back where it came from.
2. Because the first one triggers an ARP request and possibly a MAC flood, and the round trip includes that resolution. Later pings use the cached entries.
3. The MAC address table, on the switch the laptop is connected to. You need the laptop’s MAC address first — and note that on wireless it may be randomised (Chapter 28), so the MAC on the label may not be the one on the network.

Chapter 4 — IPv4 Addressing and Subnetting

1. /27 gives a block size of 32 in the fourth octet. The address ends in **37**, which falls in the block starting at 32. Network 10.19.200.32, broadcast 10.19.200.63, first usable 10.19.200.33, last usable 10.19.200.62.
2. **No.** /25 has a block size of 128, so the host is in 192.168.1.128/25, whose range is .129 to .254. The gateway 192.168.1.126 is in the *other* half, 192.168.1.0/25. The host will ARP for a gateway that is not on its segment and reach nothing beyond its own subnet.
3. Two. A /31 is used on a router-to-router link because there are only ever two devices and neither needs a broadcast address, so it provides both addresses usefully and halves the address consumption of every point-to-point link on a network.

Chapter 5 — IPv6 Addressing

1. 2001:db8:0:12::ab. Leading zeros drop in every group; the single run of three all-zero groups becomes ::. Note the fourth group is 0, not part of the :: — only one run may be compressed, and the longer one wins.
2. A global unicast address, which means no router advertisement is being received. Look at the router or layer 3 switch for the segment: is `ipv6 unicast-routing` enabled, and does the interface have a global address to advertise a prefix from?
3. Split the MAC, insert `fffe`, and flip the seventh bit: `aa` becomes `a8`. The result is `a8bb:ccff:fe00:0100`.

Chapter 6 — TCP, UDP and the Diagnostic Toolkit

1. **U** is an ICMP destination-unreachable reply. Something on the path actively told you it could not deliver — so the packet was routed, and a device took responsibility. Five dots means silence, which could be anything from a dropped packet to an ACL to a dead host. A reply is always more informative than a timeout.
2. Because a plain ping is sourced from the exit interface's address, which the far end may have no route back to. Sourcing from a LAN interface or loopback uses an address the far end's routing table knows about.
3. Any of: an ACL blocking the probes; a device configured not to send ICMP time-exceeded messages; asymmetric return path with no route back; a firewall dropping the traffic; or the destination genuinely unreachable beyond that hop.

Chapter 7 — The IOS Command Line

1. `do show ip route`. The `do` prefix runs a privileged EXEC command from any configuration mode without leaving it.
2. It boots from `startup-config` in NVRAM. You are editing `running-config` in RAM. Anything not saved is lost at the next reload — see Chapter 31 for how this catches out automation too.
3. Because the RSA key pair is named after the fully qualified domain name, `hostname.domain`. Without both, the device has no name to label the key with and refuses to generate it.

Part II — Switching and Network Access

Chapter 8 — VLANs and Access Ports

1. No. VLANs are separate broadcast domains, so the ARP request never reaches the other host. Sharing an IP subnet is irrelevant at layer 2 — and because both believe the destination is local, neither will send the traffic to a router either. This configuration cannot be fixed by routing; the VLANs must be corrected.

2. `show interfaces <int> switchport`, which reports both Administrative Mode (what you configured) and Operational Mode (what it actually became).
3. Either the VLAN does not exist in the VLAN database, or it exists and is shut down. `show vlan brief` distinguishes them.

Chapter 9 — Edge-Host Connectivity: PoE, Voice, APs and IoT

1. The voice VLAN, tagged. The phone learned its VLAN ID from CDP or LLDP-MED advertised by the switch. The PC's traffic stays untagged in the data VLAN, which is why one cable carries both.
2. The switch has only 4.2 W of power budget left, which is not enough for the AP. This is a *budget* problem, not a port problem — the port is willing (auto) and the supply cannot meet the request. Add a power supply, move the AP, or free capacity elsewhere.
3. Because it tunnels all client traffic to the wireless controller inside CAPWAP. The SSID-to-VLAN mapping is done by the controller, so the switch port only needs to reach the controller. A *standalone* AP bridges locally and does need a trunk.

Chapter 10 — Trunking, 802.1Q and SVIs

1. An access port. `dynamic auto` will accept a trunk if asked but never asks, so with neither side initiating, no trunk forms.
2. No access port in that VLAN is up, so the SVI has nothing to be up for. Confirm with `show vlan brief` — an SVI needs the VLAN to exist and at least one active port in it (or no autostate).
3. Because a trunk that fails to come up is obvious and fails safe — no traffic passes and somebody investigates. A native VLAN mismatch leaves the trunk up and quietly bridges two different VLANs together, merging broadcast domains that were meant to be separate. It is a security problem that looks like a working link.

Chapter 11 — EtherChannel and LACP

1. No channel forms — both ends wait to be asked. The individual links stay up, so spanning tree sees parallel paths and blocks all but one. The result is a working but single-link connection, which is why the symptom is “it works, but it is not four gigabits”.
2. (s) is suspended: a parameter mismatch. Compare speed, duplex, and the allowed VLAN list and native VLAN on each member. `show interfaces <int> switchport` and `show interfaces status` show them.
3. Because load balancing is per-*flow*, not per-packet. A hash of addresses and ports selects one member for a conversation, and one file copy is one conversation. Bundling raises aggregate capacity across many flows, not the speed of any single one.

Chapter 12 — Rapid PVST+ and the Protection Features

1. The one ending ...0050. With equal priorities the lowest MAC address wins the tie-break, and 0050 is numerically lowest.
2. No, it is correct and necessary. An alternate port blocking is spanning tree doing exactly its job — it has found a redundant path and is keeping it available without creating a loop.
3. Because PortFast removes the delay that would otherwise let spanning tree detect a loop before the port forwards. If somebody plugs a switch into that port, a loop forms immediately. BPDU guard restores the safety by err-disabling the port the instant a BPDU appears — which is the only thing that should never arrive on an access port.

Chapter 13 — CDP, LLDP and Documenting a Network

1. There is no switch between you and it that processes CDP — so the intervening devices are hubs, media converters, or unmanaged switches that pass the frames through. Neighbour discovery only shows *directly connected* devices, so a distant one appearing means the path is not what the diagram says.
2. Because entries persist for the advertised holdtime, 180 seconds by default for CDP. Until it expires, the table describes a neighbour that is no longer there.
3. `show cdp neighbors detail`. The summary form gives neither.

Chapter 14 — Troubleshooting Layer 2

1. Either the VLAN is missing from a trunk's allowed list, or it has been deleted or shut down on one switch. `show interfaces trunk` separates them — read `Vlans allowed` and `active`, which reflects both the allowed list and whether the VLAN exists.
2. To fix the cause. Err-disable is a symptom; whatever triggered it — a port security violation, a BPDU on a PortFast port — is still there and will trigger again the moment the port comes up.
3. Because everything that still works is evidence you can trust, and it eliminates whole layers and paths at once. A user's report tells you one thing is broken; knowing that four similar things are fine tells you where the fault *cannot* be, which is a much larger reduction in the search space.

Part III — IP Routing

Chapter 15 — The Routing Table and How a Router Decides

1. The static 10.5.0.0/16, because it is the longer prefix match. Administrative distance never enters into it — longest prefix match is applied first and decides between *entries*.
2. The static route, distance 1 against OSPF's 110. The OSPF route is not installed in the routing table but remains in the OSPF database, so it will appear if the static route is removed or its next hop becomes unreachable.
3. A layer 2 problem on the path; an ACL blocking the traffic; or no return route at the far end. `tracert` distinguishes the most common by showing where the packets stop, and whether they get there and cannot get back.

Chapter 16 — Static Routing, IPv4 and IPv6

1. The next hop is not reachable. A router will not install a static route whose next-hop address it cannot resolve to an interface. Confirm with `show ip route <next-hop>` — if that returns nothing, the static route has nowhere to point.
2. Nothing will happen: 100 is *lower* than OSPF's 110, so the “backup” route wins permanently and the OSPF route is never installed. A floating static must have a distance *higher* than the protocol it backs up.
3. Because a link-local address is only unique within a link, and a router with several interfaces has no way to tell which one fe80::2 refers to. Naming the exit interface removes the ambiguity.

Chapter 17 — OSPFv2, Single Area

1. When both routers are DROTHERs. Two non-designated routers on a broadcast segment deliberately stop at 2-WAY — they exchange databases with the DR and BDR, not with each other. It is a fault only if one of them should have been the DR or BDR.
2. 0.0.3.255. The mask for /22 is 255.255.252.0, and $255 - 252 = 3$.

3. The router ID was derived from that interface's address, so when the interface flaps the router ID changes and every adjacency rebuilds. Fix it with an explicit `router-id`, or a loopback, which never flaps.

Chapter 18 — OSPFv3 for IPv6

1. Because the router ID is a 32-bit value written in IPv4 format regardless of the address family, and it is normally derived from an IPv4 address on the device. With no IPv4 anywhere, none can be derived, and the process will not start at all — the interface commands are accepted and do nothing.
2. Nothing. They are separate processes with separate databases, neighbours and adjacencies. A working v2 adjacency proves the link and layer 2 are fine, which is useful, but says nothing about v3's configuration.
3. Per interface, with `ipv6 ospf <pid> area <n>`. *Every* IPv6 prefix on that interface is advertised — there are no network statements and no wildcard masks, which is why `passive-interface` matters more in OSPFv3 than in OSPFv2.

Chapter 19 — First Hop Redundancy: HSRP and VRRP

1. Because the virtual MAC address moves with the virtual IP. The host's ARP cache still maps the gateway IP to the same MAC; the new active router simply starts answering to it. The host never learns anything happened, which is the entire point.
2. That the two routers are not seeing each other's hellos — so each believes it is alone. Causes: mismatched group numbers, different subnets, a VLAN or trunk fault between them, or an ACL blocking the multicast. Both then claim active, and two devices answer for the same IP.
3. The priority drops to 140, which is still above the peer's 100, so **nothing happens** — the router keeps the active role despite having lost the tracked interface. That is almost certainly not what was intended: the decrement must be large enough to take the priority *below* the peer's, so 51 or more.

Part IV — Network Services and Security

Chapter 20 — DHCP: Client, Server and Relay

1. Discover, Offer and Request are broadcast; only the Ack is unicast. If the Request were unicast, a second server that had also made an offer would never learn that its offer was declined, and would hold that address reserved for nothing until it timed out.
2. It establishes only that the host asked and **nothing answered**. It does not establish where the failure is — it could be the switch port, the VLAN, the SVI, the relay, or the server, and it says nothing about whether the server is healthy.
3. On the interface facing the **clients** — the SVI in the subnet where the broadcasts are. The relay writes its own address on that interface into `giaddr`, which tells the server which pool to use and gives it a unicast address to reply to.
4. A relay or broadcast-path fault. Existing hosts renew at T1 by *unicast* directly to the server, which needs no relay, so they survive. Only newly booting hosts need the broadcast path, so only they fail.
5. No. DHCP delivered a correct address and a correct gateway, which is its entire job. The fault is elsewhere — routing, NAT, an ACL, or DNS. Check the DNS servers the lease carried before looking further afield.

Chapter 21 — DNS, and Diagnosing Records

1. That the network path and the server are fine and the fault is name resolution. Next, query the record explicitly and against a known server: `nslookup -type=A www.example.edu 8.8.8.8`, and compare with what your own resolver returns.
2. NXDOMAIN means the *name* does not exist. NOERROR with an empty answer means the name exists but has no record *of the type you asked for*. They look identical to a user and mean entirely different things.
3. Because a CNAME means “this name is an alias for that name”, and a name that is an alias cannot also carry data of its own. Standards prohibit other records alongside it, and mail servers differ in how they cope — which produces the worst symptom, where mail from some senders arrives and from others does not.
4. AAAA. A dual-stack client prefers IPv6 and will try the AAAA address first; if it is wrong or unreachable the client waits for a timeout before falling back to IPv4. IPv4-only clients never see it, so the site is slow for some and instant for others.
5. Usually whoever owns the address block — your ISP or cloud provider, not you, because reverse DNS is delegated separately in `in-addr.arpa`. Without it, outbound mail is commonly rejected or scored as spam, because receiving servers check that the connecting address reverse-resolves and forward-resolves back.

Chapter 22 — NAT and PAT on IOS XE

1. Inside local 10.1.1.9; inside global 198.51.100.4; outside global 203.0.113.80; outside local 203.0.113.80 — the same, because plain NAT does not translate the outside host’s address.
2. **overload**. It translates the port as well as the address, so one public address serves many hosts. Without it, dynamic NAT lends addresses one-to-one and **silently fails** when the pool is exhausted.
3. It rules out a pool or ACL problem — misses would be non-zero if packets had been considered and refused. Both counters at zero means no packet was ever evaluated for translation, which points at the missing `ip nat inside` or `ip nat outside` interface markers.
4. The **inside global** address, the translated one. An inbound ACL on the outside interface is evaluated *before* the packet is un-translated, so it sees the address that is on the wire at that interface.
5. Because it is an address-conservation mechanism. It happens to make unsolicited inbound connections difficult, since no translation entry matches them, but that is a side effect rather than a policy — it cannot be audited, it states no intent, and it is trivially bypassed by anything the inside host initiates. Use an ACL or firewall for policy.

Chapter 23 — Device Access, AAA and Secure File Transfer

1. `local` is used only when the TACACS+ group *does not respond at all*. If a server answers and rejects the credentials, that is an answer, the list ends, and the local database is never consulted. Fallback protects against a dead server, not a forgotten password.
2. TACACS+ is TCP 49 and encrypts the entire packet body; RADIUS is UDP 1812/1813 and encrypts only the password field. Choose **TACACS+** for switch administrators — it separates authorization fully and supports per-command control.
3. Because the RSA key pair is named `hostname.domain`. Without a domain name the device has no full name to label the key with and refuses.
4. `aaa new-model`, an `aaa authentication login` method, and `aaa authorization exec` — plus `ip scp server enable` itself. Without the authorization line, users authenticate successfully and every transfer is then refused.

5. TFTP has no user authentication, sends everything in clear text, and runs over UDP with no integrity protection. A configuration file in flight — including its shared secrets — is readable by anyone on the path, and anyone who can reach the server can fetch any file whose name they guess.

Chapter 24 — Access Control Lists

1. Mask 255.255.240.0, so the wildcard is 0.0.15.255. It matches 10.20.16.0 through 10.20.31.255 — sixteen consecutive /24s.
2. Because a standard ACL can only match the *source*. Placed near the source it would block that host from everything, not just the one destination. An extended ACL can name the destination too, so it is precise enough to sit near the source, which saves carrying doomed traffic across the network.
3. That everything is being dropped by an earlier line — no packet from anywhere reached the bottom of the list. An ACL whose final permit never matches is denying all traffic it sees.
4. TCP segments with the ACK or RST flag set — replies within a conversation somebody inside already started. It is not stateful because it inspects *flags* on each packet independently and keeps no record of connections; a crafted packet with ACK set will match.
5. **DHCP** — without it hosts get no address and the VLAN looks completely dead. **DNS** — everything works by IP and nothing by name. **ICMP** — users and you lose the ability to test anything, and troubleshooting becomes guesswork.

Chapter 25 — Layer 2 Security

1. Port security — one device claiming many MAC addresses or an unauthorised device. DHCP snooping — a rogue DHCP server. Dynamic ARP inspection — ARP poisoning. Storm control — flooding. RA guard — a rogue IPv6 router.
2. Because DAI validates ARP against the DHCP snooping **binding table**, which records which MAC legitimately holds which IP on which port. Without snooping running there is no table, and DAI has nothing to validate against.
3. It drops offending traffic silently — no counter, no log, no trap. A user whose traffic is being discarded reports “the network is slow” and nothing anywhere records why. Prefer **restrict**, which drops and reports.
4. Because the phone appears in the voice VLAN, the PC in the data VLAN, and the phone commonly registers a second address of its own. A maximum of 2 produces violations that come and go with phone firmware.
5. It is statically addressed, so it never took a DHCP lease and has no entry in the snooping binding table. DAI therefore drops every ARP reply it sends, so nothing can resolve its MAC address and it becomes unreachable while remaining healthy. Fix with an ARP ACL naming its address and MAC, applied with `ip arp inspection filter`.
6. The uplink toward the distribution layer, for **DHCP snooping** and for **dynamic ARP inspection**. Untrusted uplinks take out the whole switch in both cases.

Chapter 26 — IPsec VPNs

1. ESP, IP protocol **50**. AH provides integrity and authentication but no encryption.
2. Tunnel mode encrypts the entire original packet and adds a new IP header; transport mode keeps the original header and encrypts only the payload. A site-to-site VPN uses **tunnel mode**.
3. Because AH authenticates the entire packet including the source and destination addresses in the IP header, and NAT’s whole function is to rewrite them. The integrity check therefore fails by definition and the far end discards the packet. No configuration fixes this.

4. Phase 1 authenticates the peers and builds a secure channel between them — the IKE security association. Phase 2 uses that channel to negotiate the security associations that protect user traffic, including which traffic is protected.
5. Because plain IPsec tunnel mode carries unicast IP only, and OSPF depends on multicast. GRE can carry multicast, so the routing protocol runs inside a GRE tunnel, which IPsec then protects.
6. Path MTU. The added headers push large packets past the MTU somewhere on the path, so small packets pass and large ones are dropped or fragmented badly. A good reminder that “it pings” is not “it works”.

Part V — Wireless, Virtualisation and Operations

Chapter 27 — Wireless Principles

1. Because channels are numbered 5 MHz apart but each is 20 MHz wide, so five channel numbers of separation are needed to avoid overlap. In a band of eleven usable channels that gives exactly three: **1, 6 and 11**.
2. **Adjacent-channel** is worse. Two APs on the same channel can hear each other, so CSMA/CA works and they share the airtime — throughput halves but nothing is corrupted. On partly overlapping channels neither can decode the other, so neither defers, both transmit, and frames are destroyed and retransmitted.
3. The **noise floor**, so you can calculate SNR. A signal of -62 is strong; if the noise floor is around -70 the SNR is only 8 dB, well below the 25 dB voice needs. That would make the report entirely normal.
4. Personal shares one passphrase among everyone and identifies nobody; Enterprise authenticates each user individually via 802.1X against a RADIUS server, so one account can be revoked without touching anyone else. WPA2 uses AES-CCMP with a 4-way handshake that can be captured and attacked offline; WPA3 replaces it with SAE, which resists offline attack and adds forward secrecy.
5. Bodies are largely water, which absorbs 2.4 GHz strongly — so a full room attenuates far more than an empty one. And a full room means many clients contending for the same airtime on a half-duplex shared medium.
6. Non-Wi-Fi interference — a microwave oven, a video sender, a cordless phone. It transmits no frames, so it never appears in a Wi-Fi scan; it simply raises the noise floor. Finding it needs a **spectrum analyser**, which looks at raw RF energy rather than decoded frames.

Chapter 28 — Troubleshooting Client Connectivity

1. (1) Do I have an address? Failure points to DHCP or layer 2. (2) Ping my gateway — failure points to the local segment. (3) Ping something remote by IP — failure points to routing, NAT or an ACL. (4) Ping the same thing by name — failure points to DNS and nothing else.
2. `ipconfig /all` on Windows, `ifconfig` on macOS, `ip addr` on Linux. Note that `ifconfig` is deprecated on Linux and normal on macOS.
3. It can reach hosts on its own subnet and nothing else. Everything local works, so the user may report it as “the internet is down” rather than as a network fault.
4. Because the mask decides which destinations the host believes are local. With a mask that is too short, the host ARPs directly for addresses that are actually on other subnets, gets no answer, and fails — while destinations genuinely outside the over-wide range still use the gateway correctly and work.
5. Either it is being placed in a VLAN with no DHCP scope — often because MAC randomisation means it is not recognised by a MAC-based authorisation list — or the relay or scope for its

VLAN is broken. Tell them apart by checking whether *other* clients on the same SSID and VLAN are leasing normally; if they are, it is specific to this device.

6. The locally-administered bit is set — the second-lowest bit of the first octet. In practice the first octet's second hex digit will be 2, 6, a or e (as in 8A), and the address will not match the device's label or the manufacturer OUI.

Chapter 29 — Hypervisors, Virtual Machines and Containers

1. One layer. A type 1 hypervisor runs directly on the hardware and *is* the operating system; a type 2 runs as an application on top of a host operating system.
2. The **kernel**. Consequences: containers are far smaller and start in milliseconds because there is no operating system to boot; and isolation is weaker, because a kernel vulnerability is shared by every container on the host.
3. Because the VMs on it live in several VLANs, and their frames arrive at the physical switch tagged. An access port would carry one VLAN and discard the rest, stranding every VM outside it.
4. Because a hypervisor presents many MAC addresses on one port — one per vNIC plus the host's own — and they change as machines start, stop and migrate. Port security will exceed its maximum and err-disable the port, taking down every VM on the host.
5. Traffic between two VMs on the same host and VLAN. It is switched inside the hypervisor's virtual switch and never leaves the server, so it never reaches the physical switch to be seen, filtered or captured there.
6. Live migration. A running VM was moved to another host, so its MAC address legitimately appears on a different switch port seconds later. It looks exactly like a loop and is not.

Chapter 30 — Management Approaches, SNMP and Syslog

1. Device-based, controller-based, cloud-based, automation-based, and infrastructure as code. The test: **does the network still have a management system when the tool is switched off?** For controller-based, yes — and you have just lost it. For automation-based, no — nothing about the devices changed.
2. Facility %PM, severity 4, mnemonic ERR_DISABLE. Severity 4 is *warning*.
3. **warnings** is severity 4, so it records levels 0 to 4 and **discards** 5, 6 and 7 — notification, informational and debugging.
4. A trap is sent once over UDP and never acknowledged, so it can be lost silently. An inform is acknowledged and retransmitted until it is. Use informs for anything you would be sorry to miss, traps for high-volume low-consequence events.
5. Because the community string travels in **clear text** in every polling packet, hundreds of times an hour. Anyone who can capture one packet has it, so obscurity is irrelevant. A read-write string is equivalent to an enable password.
6. It sets the *severity threshold for messages sent to a syslog server*. It has nothing whatever to do with SNMP traps, despite the name.

Chapter 31 — Ansible and Configuration Management

1. It installs nothing on the managed device and connects over SSH. It decides the choice because you cannot install an agent on a Catalyst switch — so any agent-based tool (Puppet, Chef) is simply unavailable for network hardware, while Ansible needs only the SSH access already configured.
2. An operation produces the same result whether applied once or many times. In the recap you see it as **ok** rather than **changed** on a device that is already in the desired state — so a second run of the same playbook reports **changed=0**.

3. `ios_command` runs EXEC commands and returns output; it cannot make a configuration change. `ios_config` enters configuration mode and applies lines. A playbook reporting success while changing nothing is very often the former where the latter was meant.
4. It writes the running configuration to startup-config, but only when that run actually changed something. Without it the change lives in the running configuration only and is lost at the next reload — which shows up weeks later as a fault correlating with reboots rather than with the change.
5. `unreachable` means Ansible could not log in to the device at all. `failed` means it logged in and the task did not work. They point at completely different problems.
6. `-check` is a dry run — it reports what it would do and touches nothing. `-diff` shows the exact lines. Omit them only when you have already run with them and are applying the change you just reviewed.

Chapter 32 — AI in Network Operations

1. Generative AI produces content in response to a prompt and stops; agentic AI pursues a goal, uses tools to act, and evaluates its own results.
2. Data classification, output format, persona and instructions. **Data classification** is listed first, because a prompt that discloses credentials or real addressing is wrong regardless of how good the rest of it is — and the disclosure cannot be undone.
3. Credentials of any kind (passwords, hashes, community strings, pre-shared keys, tokens); personal data; and real public IP addressing or anything the organisation classifies as confidential.
4. How much breaks if the action is wrong. It governs autonomy because the question is not how capable the system is but how bad a confident mistake would be — clearing one err-disabled port is reversible and local, while changing a core route map reaches the whole organisation.
5. A hallucination. The defence is verification: check the command exists on your platform, check it against the exhibit or the device, and keep change control. Fluency is not evidence.
6. Any three of: anomaly detection, event correlation and noise reduction, root-cause suggestion, configuration drafting and review, documentation and summarisation, capacity forecasting, guided remediation.

Part VI — Sitting the Exam

Chapter 33 — A Troubleshooting Method for the 28 Per Cent

1. Define, gather, analyse, eliminate, hypothesise, test, solve. Steps **5 and 6** form the loop — a hypothesis whose prediction fails is discarded and replaced, not patched.
2. Bottom-up for a suspected physical or hardware fault, or when nothing works. Top-down when one application fails and everything else is fine. Divide and conquer as the default when you have no strong prior. Follow the path when two specific points cannot reach each other. Compare working to broken whenever an identical working case exists.
3. *One user or many?* — sets the whole search space. *Has it ever worked?* — separates a build fault from a change fault. *What changed?* — finds the cause of most faults in previously working networks. *Does it fail by name but work by address?* — answers “is this DNS” in one test.
4. Because it converts a check into information whichever way it goes. Without a prediction, a fix that does not resolve the symptom reads as “the fix did not work” rather than “the hypothesis was wrong”, and you re-check something already correct.
5. Either the hypothesis was wrong, or there is more than one fault. Go back to scoping rather than re-checking the thing you just verified — and keep the fix if it corrected a genuine defect, but record it separately.

6. The precise symptom and who is affected; what has been ruled out and the evidence for it; what has been changed, including anything not yet undone; and the current hypothesis with the next test.

Chapter 34 — Exam Strategy

1. **You cannot return to a previous question.** It invalidates skimming the paper first, leaving hard questions to come back to, and flagging anything for review.
2. 60 to 75 seconds. At 90 seconds, eliminate what you can, choose the best remaining option, and move on — the question you have not seen yet may be one you would answer in twenty seconds.
3. Because you cannot come back to it, so a deferred simulation is an abandoned one. Read the whole task first; plan before typing; configure only what is asked; verify with a `show` command — and save if the task says to.
4. The **question** first. Otherwise you study an entire configuration or topology to answer something about one line of it.
5. The *describe* topics — wireless principles, virtualisation, VPNs, management approaches, AI — because they are recall-heavy and recall decays fastest. Configuration skill does not.
6. Cisco publishes the 120-minute duration, the five domains and their weights, and the 29 topics with their wording. It does **not** publish the number of questions or the passing score.

Glossary

Terms marked as key terms in the chapters, defined in one or two lines. Where a term is easy to confuse with another, the contrast is given rather than just the definition — those pairs are where the exam questions live.

Term	Meaning
A–B	
AAA	Authentication (who are you), authorization (what may you do), accounting (what did you do). Three separate questions answered by ordered method lists
A record	DNS: name to IPv4 address. AAAA is name to IPv6 address
Access port	A switch port carrying one untagged VLAN, for an end device
Administrative distance	How much a router trusts a <i>source</i> of routing information. Connected 0, static 1, OSPF 110. Decides between sources; longest prefix match decides between entries
Agentic AI	A system that pursues a goal, uses tools to act, and evaluates its own results. Contrast generative AI , which answers and stops
Agentless	Needs no software installed on the managed device. Why Ansible works on switches and Puppet does not
AH	IPsec Authentication Header, IP protocol 51. Integrity and authentication, no encryption , and incompatible with NAT
APIPA	A host's self-assigned 169.254.x.x address. Means it asked for a lease and nothing answered
ARP	Resolves an IP address to a MAC address on the local segment. Has no authentication, which is why DAI exists
Attenuation	Loss of signal strength with distance or through material
BPDU	Bridge Protocol Data Unit — the frame spanning tree uses to elect a root and detect loops
BPDU guard	Err-disables a port if a BPDU arrives on it. Applied to access ports, where none should ever appear
Blast radius	How much breaks if an action is wrong. Governs how much autonomy an automated system should be given
Block size	256 minus the interesting octet of a subnet mask. Subnets begin at multiples of it
Broadcast domain	The set of devices a broadcast reaches. One per VLAN
BSSID	The MAC address of one radio on one access point — what a client is actually associated to. Contrast SSID , the network's name
C–D	
CAM table	A switch's MAC address table: which MAC was last seen on which port
CDP	Cisco Discovery Protocol. Reveals directly connected Cisco devices. LLDP is the vendor-neutral equivalent, and is off by default

Term	Meaning
Channel bonding	Combining adjacent channels for more throughput. Halves the number of non-overlapping channels and raises the noise floor
CNAME	DNS: name to another <i>name</i> . A name that is a CNAME cannot hold any other record, which is why an MX must not point at one
Co-channel interference	Two APs on the <i>same</i> channel. They hear each other and share airtime. Better than adjacent-channel interference, where they cannot hear each other and corrupt each other's frames
Collision domain	The set of devices that can collide with one another. One per switch port on a modern network
Community string	SNMP v1 and v2c's credential, sent in clear text in every packet. Renaming it from public protects nothing
Container	An isolated user-space environment sharing the host's kernel . Megabytes, milliseconds to start, weaker isolation than a VM
Controller-based	A controller holds intended state and programs devices. Contrast automation-based , which adds tooling but no control plane
DAI	Dynamic ARP Inspection. Validates ARP against the DHCP snooping binding table, so statically addressed hosts need an ARP ACL
Data classification	Deciding what data may be included in a prompt. Listed first among the four prompt components, and the one that can disqualify a prompt outright
DHCP snooping	Drops DHCP <i>server</i> messages on untrusted ports, and builds the binding table that DAI depends on
DORA	Discover, Offer, Request, Ack. The first three are broadcast, which is why relays exist
DR / BDR	OSPF's designated and backup designated router on a multi-access segment. Elected by priority, then highest router ID. Not pre-emptive
DTP	Dynamic Trunking Protocol. Negotiates trunks automatically; disable with switchport nonegotiate
E–H	
EAP	The authentication framework behind WPA-Enterprise and 802.1X
Err-disabled	A port shut down by a protection feature. Recover with shutdown then no shutdown , or errdisable recovery
ESP	IPsec Encapsulating Security Payload, IP protocol 50. The one that encrypts , and the one everyone uses
Established	An ACL keyword matching TCP segments with ACK or RST set — replies to conversations already started. Inspects flags, so it is <i>not</i> stateful
EtherChannel	Several physical links treated as one logical link. Negotiated by LACP, or forced with mode on
EUI-64	Building an IPv6 interface ID from a MAC: split it, insert fffe , flip the seventh bit
Extended ping	A ping with a chosen source, size, count or DF bit. Finds faults a plain ping cannot

Term	Meaning
Facility	The subsystem in a syslog message — %LINEPROTO in %LINEPROTO-5-UPDOWN
Floating static	A static route with a raised administrative distance, so it installs only when the preferred route disappears
Giaddr	The gateway IP address field a DHCP relay fills in. Selects the pool and carries the reply back
Global unicast	A routable IPv6 address, currently from 2000::/3. Contrast link-local (fe80::/10), which exists on every interface
HSRP	Cisco's first hop redundancy protocol. Virtual IP and virtual MAC shared between routers; not pre-emptive unless configured

I–M

Idempotent	Produces the same result applied once or many times. Why an Ansible playbook is safe to re-run
IKE	Internet Key Exchange, UDP 500 (4500 with NAT-T). Phase 1 builds a secure channel between peers; phase 2 protects user traffic
Implicit deny	The invisible deny any at the end of every ACL. Permit what you need and everything else stops
Inform	An SNMP notification that is acknowledged and retransmitted. Contrast a trap , which is sent once over UDP and may be lost
Inside local / global	The internal host's private address / the public address it is translated to. Outside local and outside global are usually the same address
Link-local next hop	An OSPFv3 or IPv6 static route's fe80:: next hop. It is correct, and it requires the interface to be named alongside it
LLDP	Vendor-neutral neighbour discovery. Off by default
Longest prefix match	The most specific matching route wins, regardless of administrative distance or metric. The first rule of forwarding
Loop guard	Blocks a port that stops receiving BPDUs, preventing a unidirectional link from opening a loop
MAC randomisation	Operating systems presenting a random per-network MAC. On by default everywhere, and it breaks MAC-based authorisation and DHCP reservations
Method list	An ordered list of AAA methods. A method is skipped only when it <i>does not respond</i> ; a rejection ends the list
MIB / OID	The tree of variables an SNMP agent exposes, and the numeric identifier of one of them
Mnemonic	The event name in a syslog message — UPDOWN in %LINEPROTO-5-UPDOWN
MX	DNS: domain to mail exchanger, with a preference number. Must name a host with an A record, and must not name a CNAME

N–R

Native VLAN	The untagged VLAN on an 802.1Q trunk. Must match at both ends
NAT-T	NAT traversal. Wraps ESP in UDP 4500 so PAT can handle it

Term	Meaning
Noise floor	Background RF energy on a channel. Signal strength without it tells you nothing — SNR is what decides whether a link works
NS record	DNS: the servers authoritative for a zone. A fault here takes out everything in the domain at once
NXDOMAIN	The name does not exist. Contrast NOERROR with an empty answer section, which means the name exists but not with that record type
Orchestration	Scheduling, health-checking and scaling containers across hosts. Kubernetes is the dominant one
Overload	The keyword that turns dynamic NAT into PAT by translating ports as well as addresses. Its absence is why NAT “stops working after lunch”
PAT	Port Address Translation. Many internal hosts behind one public address
Persona	The prompt component setting the expertise and platform to answer from
PoE	Power over Ethernet. Budget matters: a switch’s total power is finite
PortFast	Moves an access port straight to forwarding, skipping listening and learning. Use with BPDU guard
Port group	A named VLAN and policy on a virtual switch, that VMs attach to
Port security	Limits which and how many MAC addresses a port accepts. Violation modes protect (silent — avoid), restrict (logs), shutdown (default)
PTR	DNS: address to name, in <code>in-addr.arpa</code> . Usually controlled by whoever owns the address block, and its absence gets outbound mail rejected
RA guard	Drops IPv6 router advertisements on ports classified as hosts. The IPv6 counterpart of DHCP snooping
RADIUS	UDP 1812/1813. Encrypts only the password; combines authentication and authorization. For network access . Contrast TACACS+
Rapid PVST+	Cisco’s per-VLAN rapid spanning tree
Root guard	Refuses a superior BPDU on a port, so a rogue switch cannot become root
RSSI	Received signal strength in dBm, always negative. -67 or better for voice, -70 for data

S–Z

SAE	WPA3’s key exchange, replacing WPA2’s 4-way handshake. Resists offline dictionary attack and gives forward secrecy
Save_when	The Ansible <code>ios_config</code> option that writes to startup-config. Without it, a change survives only until the next reload
SCP / SFTP	File transfer inside SSH. IOS can be an SCP <i>server</i> and an SFTP <i>client</i> . Both replace TFTP, which authenticates nobody
Severity	The digit in a syslog message, 0 (emergency) to 7 (debugging). 0 is worst . A threshold of n records 0 through n

Term	Meaning
SLAAC	IPv6 hosts building their own address from a router advertisement
SNR	The gap in dB between signal and noise. The number that decides whether a wireless link works. 25 dB for voice, 20 dB minimum for data
Sticky MAC	Port security converting learned addresses into configuration — which is not saved until you write memory
Storm control	Caps broadcast, multicast or unknown unicast as a share of a port's bandwidth. Rising and falling thresholds prevent oscillation
SVI	Switch Virtual Interface. A VLAN's layer 3 interface on a switch
TACACS+	TCP 49. Encrypts the entire packet, separates AAA fully, supports per-command authorization. For device administration
Trunk	A link carrying several VLANs, tagged with 802.1Q. allowed vlan replaces the list; allowed vlan add extends it
Trusted port	A port on which DHCP server messages, or ARP, are accepted without validation. Uplinks must be trusted or a whole switch fails
TTL	Two meanings. In DNS, how long a record may be cached — which is why a correct fix takes time to appear. In IP, a hop counter
Tunnel mode	IPsec encrypting the entire original packet and adding a new IP header. The normal case. Contrast transport mode , which keeps the original header
Type 1 / type 2	A hypervisor running on the bare metal / running on a host operating system
Virtual switch	The switch inside a hypervisor. Does not run spanning tree, and its uplink to the physical network is a trunk
VRRP	The standards-based first hop redundancy protocol. Pre-empts by default, unlike HSRP
Wildcard mask	The inverse of a subnet mask: 0 means must match, 1 means ignore. Network address plus wildcard equals the broadcast address
WPA2 / WPA3	AES-CCMP with a 4-way handshake / AES-GCMP with SAE. Personal shares a passphrase; Enterprise authenticates each user against RADIUS
YAML	The format Ansible playbooks are written in. Spaces not tabs; - for list items; a space after every colon

Schedule, Lab Index and Exam Facts

G.1 A sixteen-week schedule

Two sessions a week — one of theory, one at a keyboard — with the lab session treated as the more important of the two. The blueprint is 55% configure, troubleshoot and diagnose, and a course that teaches this material without hardware or a simulator is teaching the smaller fraction of the exam.

Week	Chapters	Lab session	Assessment
1	1 Networks and this exam; 2 Physical layer	Cabling, <code>show interfaces</code> , injected cable faults	Quiz 1 — subnetting, timed
2	3 Ethernet and switching; 7 The IOS command line	First switch: hostname, SSH, save	
3	4 IPv4 addressing and subnetting	Subnetting drills (Appendix C)	
4	5 IPv6 addressing; 6 TCP/UDP and the toolkit	Dual-stack a small network; extended ping and traceroute	
5	8 VLANs; 9 Edge-host connectivity	VLANs, access ports, voice and PoE	Lab exam 1 — build a switched network
6	10 Trunking and SVIs; 11 EtherChannel	Trunk two switches, route between VLANs, bundle links	
7	12 Rapid PVST+; 13 CDP and LLDP	Steer the root bridge; document a network from the devices	
8	14 Troubleshooting layer 2	Five injected faults against the clock	
9	15 The routing table; 16 Static routing	Read tables, predict paths, four kinds of static route	Mid-term — Parts I and II
10	17 OSPFv2; 18 OSPFv3	Single-area OSPF, then dual-stack	
11	19 HSRP and VRRP; 20 DHCP	Fail a gateway; DHCP as client, server and relay	
12	21 DNS; 22 NAT and PAT	Diagnose a broken zone; PAT and port forwarding	
13	23 AAA and secure file transfer; 24 ACLs	AAA with a fallback; write, read and repair ACLs	Lab exam 2 — routing and first-hop redundancy
14	25 Layer 2 security; 26 IPsec VPNs	Port security, DHCP snooping, DAI, storm control	
			Quiz 2 — Part IV

Week	Chapters	Lab session	Assessment
15	27 Wireless; 28 Client troubleshooting; 29 Virtualisation	A real site survey; the same fault on three operating systems	
16	30 Management, SNMP, syslog; 31 Ansible; 32 AI	Syslog severities; a playbook across three devices	Final — full mock paper, 120 minutes

💡 Where chapters 33 and 34 go

Not in a week of their own. Chapter 33 should be introduced in week 8 alongside Chapter 14 and then *used* in every lab after it — the method is worthless as a lecture and valuable as a habit. Chapter 34 belongs in the last session before the mock, when the constraints it describes are about to become real.

⚠️ If you have twelve weeks, not sixteen

Do not cut the labs. Cut in this order, and be honest with students about what they are getting:

- Merge weeks 15 and 16 into one, and set Chapter 27, Chapter 29 and Chapter 32 as directed reading with a quiz. They are *describe* topics, which is the only part of this blueprint that survives being read rather than done.
- Merge Chapter 18 into Chapter 17's session — it is written as a delta and takes forty minutes.
- Merge Chapter 26 into Chapter 24's week. It has no lab by design.

What must not be cut: Chapter 14, Chapter 33, and the fault injection in every lab. That is the 28% of the blueprint that begins *troubleshoot* or *diagnose*, and it cannot be delivered any other way.

G.2 Lab index

Thirty-two labs. Every one states its platform in its header; Parts I to III run in Packet Tracer, and from Part IV the labs move to CML or GNS3 where real platform behaviour matters. Chapter 1 has no lab because it has nothing to configure yet, and Chapter 26 has none because topic 4.5 is *describe*.

Lab	Title	Platform
2.1	Reading a broken link	Packet Tracer
3.1	Watching a switch learn	Packet Tracer
4.1	Subnet, configure, then break it	Packet Tracer
5.1	Dual-stack a small network	Packet Tracer
6.1	Diagnose with intent	Packet Tracer
7.1	Bring up a switch you can reach from your desk	Packet Tracer
8.1	Divide one switch into three networks	Packet Tracer
9.1	Wire an edge properly	Packet Tracer
10.1	Trunk two switches and route between VLANs	Packet Tracer
11.1	Bundle four links and break them	Packet Tracer
12.1	Steer the tree, then protect it	Packet Tracer

Lab	Title	Platform
13.1	Draw the network from the devices	Packet Tracer
14.1	Fix five faults against the clock	Packet Tracer
15.1	Read the table, then predict the path	Packet Tracer
16.1	Four routes, one failure	Packet Tracer
17.1	Three routers, one area	Packet Tracer
18.1	Dual-stack routing, both protocols side by side	Packet Tracer
19.1	Fail a gateway and watch it survive	Packet Tracer
20.1	DHCP in all three roles, then broken deliberately	Packet Tracer
21.1	Diagnosing a zone you did not build	CML or GNS3, with a BIND or Windows DNS server
22.1	PAT for a branch, then a published server	CML or GNS3 with IOS XE
23.1	Secure access, AAA, and a backup you can restore	CML or GNS3, with a TACACS+ or RADIUS server
24.1	Writing, reading and repairing ACLs	Packet Tracer, or CML
25.1	Securing an access switch, one feature at a time	Packet Tracer for tasks 1–6; CML for RA guard
27.1	A survey of somewhere you actually use	Any laptop or phone with a Wi-Fi analyser
28.1	The same fault on three operating systems	Any three machines, or virtual machines
29.1	Connecting a hypervisor properly	VirtualBox, VMware Workstation, or CML
30.1	Making a device tell you what it is doing	Packet Tracer for syslog; CML for SNMPv3
31.1	Automating a change you would otherwise do forty times	CML or GNS3, with a Linux control node
32.1	Prompt engineering as an examinable skill	Any generative AI assistant, plus a lab device
33.1	Faults you did not create	Packet Tracer or CML, in pairs
34.1	A full-length timed rehearsal	Your own material, a timer, and no interruptions

💡 Regenerating this table

The lab index and Appendix D are both produced by `tools/gen_appendices.py`, which reads the chapter sources. Re-run it after adding, moving or retagging a chapter, and paste the table back here. Nothing in either table is typed by hand, so neither can drift from the chapters they describe.

G.3 Exam facts, on one page

Exam	200-301 CCNA, <i>Implementing and Administering Cisco Solutions</i> , blueprint v2.0
Duration	120 minutes
Live from	3 February 2027; v1.1 retires 2 February 2027. The exam number does not change
Domains	1.0 Network Infrastructure and Connectivity 25%; 2.0 Switching and Network Access 25%; 3.0 IP Routing 20%; 4.0 Network Services and Security 20%; 5.0 AI, and Network Operations and Management 10%
Topics	29, numbered 1.1–5.6. All covered — Appendix D
Verb split	8 <i>Configure</i> , 6 <i>Troubleshoot</i> , 2 <i>Diagnose</i> (55% hands-on); 6 <i>Describe</i> ; 3 <i>Interpret</i> ; one each of <i>Validate</i> , <i>Manage</i> , <i>Select</i> , <i>Use</i>
Navigation	You cannot return to a previous question
Question count	Not published by Cisco
Passing score	Not published by Cisco. Scores are scaled
New in v2.0	You may be asked to evaluate output and recommendations from agentic AI and digital network assistants — anywhere in the paper
Before you book	Take Cisco’s free exam tutorial, and re-read the current exam policies. Both change

Verify before you teach from this

Duration, domain weights, topic wording and the AI statement are taken from Cisco’s published exam-topics document, which is included in this folder as the source of record. Everything Cisco does not publish is marked as such above, rather than filled in with a plausible number.

Cisco revises blueprints without renaming exams. Check cisco.com/go/certroadmap before each cohort.